
DATA ANALYST

Série 2 — Volume I

Fondations

Statistique · Excel · SQL
Python et R · Collecte de données

De la question à la donnée,
avec une boutique pour fil rouge.

Table des matières

Introduction à la série et mode d'emploi	1
Bienvenue	1
Le métier de data analyst : question, données, enseignement, décision	1
La gérante et ses questions	1
De la question à la décision : la boucle de l'analyse	2
Ce que fait un analyste, et ce qu'il ne fait pas	4
Analyste, data scientist, ingénieur de données	4
Les trois compétences de l'analyste	5
Honnêteté et éthique	5
Comment lire la série	5
Parcours essentiel et ⊕ pour aller plus loin	5
Les six volumes	6
Le livre et son cahier	6
Cette série et la série 1	6
Mode d'emploi de ce volume	7
Ce que le volume suppose	7
Une habitude à prendre : la vérification croisée	7
Carte du volume, données et environnement	9
Carte du volume	9
Les jeux de données du volume	10
Les clients et les produits	11
Les commandes, les lignes et les retours	11
Les jours d'exploitation	12
L'enquête de satisfaction	13
Les fichiers « de bureau » : un classeur et un export de caisse	13
La base de données boutique.db	14
La vérité programmée	15
L'environnement de travail	15
Python, R et SQL	15
Le tableur : Excel, LibreOffice et nos maquettes	16
Régénérer les données et refaire les calculs	16
Conventions du volume	16
Chapitre 1 : Les essentiels de la statistique	17
Le chemin de ce chapitre	17
Les données du chapitre	18
1.1 Statistique descriptive : centre, dispersion, forme	18
1.1.1 Résumer un panier : le centre	19

1.1.2 Mesurer la dispersion	20
1.1.3 La forme : asymétrie, queues et valeurs aberrantes	22
1.1.4 Quand les moyennes mentent	23
1.1.5 Les mêmes chiffres dans Excel et dans R	24
1.2 Distributions et courbe normale	26
1.2.1 Qu'est-ce qu'une distribution ?	26
1.2.2 Compter : les lois binomiale et de Poisson	26
1.2.3 La courbe normale	28
1.2.4 Quand la normale ne convient pas	29
1.3 Échantillonnage et erreur d'échantillonnage	31
1.3.1 Population, échantillon, paramètre, statistique	31
1.3.2 L'erreur d'échantillonnage en action	31
1.3.3 L'intervalle de confiance d'une moyenne	33
1.3.4 L'intervalle de confiance d'une proportion	34
1.3.5 Combien d'observations faut-il ?	35
1.3.6 Les biais, que la taille ne corrige pas	36
1.4 Corrélation et causalité	37
1.4.1 Mesurer une liaison : covariance et corrélation	37
1.4.2 Toujours regarder le nuage de points	38
1.4.3 La publicité et les ventes : une corrélation trompeuse	39
1.4.4 La température et les ventes de jardin	40
1.4.5 Les corrélations fortuites	40
1.4.6 De la corrélation à la causalité	41
1.5 ⊕ Pour aller plus loin : mathématiques du quotidien en entreprise	42
1.5.1 Pourcentages : part, variation, points	43
1.5.2 Taux de croissance	43
1.5.3 Moyennes pondérées et décomposition prix-volume	44
1.5.4 Marge, marque et TVA	45
1.5.5 Arrondis : où et quand	46
Bilan du chapitre 1	47
Chapitre 2 : Excel	49
Le chemin de ce chapitre	50
Les données du chapitre	50
2.1 Formules et fonctions	51
2.1.1 Anatomie d'un classeur et d'une formule	51
2.1.2 Références relatives, absolues et mixtes	52
2.1.3 Plages nommées et tableaux structurés	53
2.1.4 Agréger sous conditions : SOMME.SI.ENS, NB.SI.ENS, MOYENNE.SI.ENS	53
2.1.5 Décider : SI, SI.CONDITIONS, ET, OU, SIERREUR	54
2.1.6 Chercher : RECHERCHEV, INDEX+EQUIV, RECHERCHEX	55
2.1.7 Nettoyer du texte	57
2.1.8 Manipuler des dates	58
2.1.9 Les formules à résultats multiples : FILTRE, TRIER, UNIQUE, LET	59
2.1.10 Les erreurs, et la correspondance anglais-français	59
2.2 Tableaux croisés dynamiques et graphiques croisés dynamiques	61
2.2.1 Le principe : lignes, colonnes, valeurs, filtres	61
2.2.2 Construire un tableau croisé pas à pas	62
2.2.3 Regrouper les dates : par mois, par trimestre, par année	63
2.2.4 Afficher des proportions, calculer dans un TCD	65
2.2.5 Segments, chronologies et pièges	65
2.3 Power Query pour l'import et la transformation	66

2.3.1 L'idée : une recette d'étapes enregistrées	66
2.3.2 Importer : formats, encodage, paramètres régionaux	67
2.3.3 Les étapes de nettoyage, une à une	68
2.3.4 Scinder, fusionner des colonnes, dépivoter	69
2.3.5 Fusionner et ajouter des requêtes	70
2.3.6 Power Query, formules ou Python ?	72
2.4 Bonnes pratiques de tableur	72
2.4.1 Une structure en trois étages	72
2.4.2 Des données propres : une ligne, une observation	73
2.4.3 Séparer les hypothèses des formules	75
2.4.4 Se contrôler : validation, mise en forme conditionnelle, feuille de contrôles	75
2.4.5 Les erreurs de tableur que tout le monde commet	76
2.4.6 Documenter, protéger, versionner, partager	77
2.5 ⊕ Pour aller plus loin : Excel avancé	78
2.5.1 Power Pivot et le modèle de données	78
2.5.2 Les mesures DAX	78
2.5.3 Macros VBA et scripts Office	80
2.5.4 Quand quitter Excel : SQL ou Python ?	80
2.6 ⊕ Pour aller plus loin : Google Sheets et Looker Studio	81
2.6.1 Google Sheets : ce qui ressemble, ce qui diffère	82
2.6.2 Looker Studio : des tableaux de bord reliés à des sources	83
2.6.3 Tableau d'équivalences	83
Bilan du chapitre 2	84
Chapitre 3 : SQL	86
Pourquoi un analyste a besoin du SQL	86
Le chemin de ce chapitre	87
Les données du chapitre	87
3.1 SELECT, filtrage, tri, agrégation	88
3.1.1 Une base, des tables, des clés	88
3.1.2 SELECT et FROM : choisir des colonnes	90
3.1.3 WHERE : ne garder que les lignes utiles	91
3.1.4 ORDER BY, LIMIT, DISTINCT	92
3.1.5 Calculer des colonnes : expressions, CASE et dates	93
3.1.6 Agréger : COUNT, SUM, AVG, MIN, MAX et GROUP BY	94
3.1.7 HAVING et l'ordre logique d'exécution	95
3.1.8 NULL : la valeur absente	97
3.1.9 Quatre pièges classiques	98
3.2 Jointures	99
3.2.1 Pourquoi joindre	99
3.2.2 INNER JOIN : ne garder que les correspondances	100
3.2.3 LEFT JOIN : garder tout le monde	101
3.2.4 RIGHT JOIN et FULL JOIN	102
3.2.5 CROSS JOIN et auto-jointure	103
3.2.6 Le piège de la multiplication des lignes	104
3.2.7 Empiler et comparer des ensembles	106
3.2.8 Joindre sur plusieurs colonnes	107
3.3 Fonctions fenêtres	107
3.3.1 Agréger sans perdre le détail	108
3.3.2 Anatomie d'une fonction fenêtre	108
3.3.3 Classer : ROW_NUMBER, RANK, DENSE_RANK	109
3.3.4 Comparer à la ligne précédente : LAG et LEAD	111

3.3.5 Cumuls, moyennes mobiles et cadres	111
3.3.6 Découper en tranches et mesurer la concentration : NTILE	113
3.3.7 Les clients qui ne reviennent plus	113
3.4 CTE et structuration de requêtes complexes	115
3.4.1 Nommer ses étapes avec WITH	115
3.4.2 Vérifier par un autre outil	116
3.4.3 Enchaîner plusieurs étapes : le taux de retour	117
3.4.4 Une méthode pour construire une requête complexe	118
3.4.5 CTE récursives : fabriquer un calendrier	118
3.4.6 Écrire des requêtes que l'on relit	119
3.5 ⊕ Pour aller plus loin : SQL avancé	120
3.5.1 Sous-requêtes : une requête dans une requête	120
3.5.2 Index et plans d'exécution	122
3.5.3 Quelques règles d'optimisation	123
3.5.4 Vues : donner un nom à une requête	123
3.5.5 Transactions : tout ou rien	124
3.5.6 Déclencheurs et procédures stockées	124
3.5.7 Sécurité : l'injection SQL	125
3.6 ⊕ Pour aller plus loin : PostgreSQL, MySQL, SQL Server, Oracle	126
3.6.1 Un standard, plusieurs dialectes	126
3.6.2 La table de correspondance	127
3.6.3 Les mêmes requêtes, ailleurs	127
3.6.4 DuckDB : un dialecte voisin de PostgreSQL, que l'on peut exécuter	128
3.6.5 Écrire du SQL portable	129
Bilan du chapitre 3	130
Chapitre 4 : Python et R pour l'analyse	132
Le chemin de ce chapitre	132
Pourquoi un langage, alors qu'Excel et SQL existent ?	133
Python ou R ?	133
Comment lire les blocs de ce chapitre	134
4.1 Les fondamentaux de pandas	135
4.1.1 Python en dix minutes	135
4.1.2 Le DataFrame et la Series	136
4.1.3 Lire un fichier : les arguments qui comptent	136
4.1.4 Sélectionner : colonnes, lignes, cellules	137
4.1.5 Filtrer : garder les lignes qui répondent à une condition	138
4.1.6 Colonnes calculées et tri	138
4.1.7 Valeurs manquantes et comptages	139
4.1.8 La méthode chaînée	139
4.2 Les fondamentaux du tidyverse	140
4.2.1 R, les paquets et le tube	140
4.2.2 Lire avec readr	141
4.2.3 Les verbes de dplyr	142
4.2.4 Dates, texte et catégories : lubridate, stringr, forcats	143
4.2.5 L'équivalence entre pandas et le tidyverse	144
4.2.6 Ce qui surprend quand on passe de l'un à l'autre	144
4.3 Lire, filtrer, regrouper, restructurer	145
4.3.1 Lire un export désordonné	145
4.3.2 Regrouper : une question, un groupe, un calcul	147
4.3.3 Joindre : relier deux tables par une clé	149
4.3.4 Restructurer : le format large et le format long	151

4.3.5 Fenêtres glissantes et comparaisons dans le temps	153
4.3.6 Le tableau du lundi	154
4.4 Notebooks d'analyse	156
4.4.1 Qu'est-ce qu'un notebook ?	156
4.4.2 Le piège de l'état caché	156
4.4.3 Les habitudes d'un notebook fiable	157
4.4.4 R Markdown et Quarto : écrire le rapport comme un programme	158
4.4.5 Quand quitter le notebook ?	158
4.5 ⊕ R : ggplot2 et Shiny	159
4.5.1 La grammaire des graphiques	159
4.5.2 Facettes : plusieurs petits graphiques	160
4.5.3 Shiny : une application interactive	161
4.6 ⊕ Python : NumPy, polars, rapports avec Jupyter	162
4.6.1 NumPy : les tableaux et la vectorisation	162
4.6.2 polars : une alternative rapide à pandas	164
4.6.3 Des rapports avec Jupyter	165
Bilan du chapitre 4	166
Chapitre 5 : Types de données, collecte et conception d'enquêtes	168
Le chemin de ce chapitre	169
Les données du chapitre	169
5.1 Types de données et niveaux de mesure	170
5.1.1 Ce que contient une colonne	170
5.1.2 Quatre niveaux de mesure, et ce que l'on a le droit de calculer	171
5.1.3 Types techniques et types statistiques	173
5.1.4 Un tableau propre, et son grain	174
5.2 Sources de données	176
5.2.1 Les sources d'une entreprise	176
5.2.2 La carte d'identité d'une source	177
5.2.3 Droits, licences et vie privée	178
5.2.4 Le biais de la source	179
5.2.5 Choisir une source	180
5.3 Bases de la conception d'enquêtes	180
5.3.1 De l'objectif au questionnaire	181
5.3.2 Les types de questions	181
5.3.3 Le taux de réponse et la non-réponse	182
5.3.4 Peut-on corriger la non-réponse ?	183
5.3.5 Nettoyer l'enquête avant de calculer	185
5.3.6 Le NPS et son intervalle de confiance	185
5.4 ⊕ Pour aller plus loin : questionnaires, plans de sondage et biais d'enquête	187
5.4.1 Bien poser une question	187
5.4.2 Plans de sondage : qui interroger ?	188
5.4.3 Combien de personnes faut-il interroger ?	189
5.4.4 Les biais d'enquête	190
5.5 ⊕ Pour aller plus loin : sources ouvertes, API et web scraping	192
5.5.1 Les données ouvertes	192
5.5.2 Interroger une API	193
5.5.3 Le web scraping	195
5.5.4 Éthique et droit	197
Bilan du chapitre 5	198
Points clés	200

Les points clés, chapitre par chapitre	200
Six idées qui traversent tout le volume	201
Et maintenant ?	202

Introduction à la série et mode d'emploi

« Un chiffre n'est pas une réponse : c'est le début d'une conversation entre une question et une décision. »

Bienvenue

Cette série de six volumes vous apprend un métier : celui de **data analyst**, l'analyste de données. Son travail tient en une phrase : **aider quelqu'un à prendre une meilleure décision, en s'appuyant sur des données que l'on a su rendre fiables et que l'on sait expliquer**. Ce n'est ni un métier de calcul pur, ni un métier d'informatique pure. C'est un métier de **traduction** : on reçoit une question posée avec les mots de l'activité (« Le Site nous fait-il perdre des clients en boutique ? »), on la transforme en une question que les données peuvent éclairer, on fait les calculs, puis on traduit le résultat en une réponse que la personne peut utiliser (« Oui, un peu, mais pas pour la raison que vous croyez, et voici ce que je ferais »).

Au bout des six volumes, vous saurez **collecter et nettoyer** des données, les **explorer et les tester**, les **représenter** par des graphiques et des tableaux de bord, **automatiser** vos rapports et **présenter** votre travail. Ce premier volume pose les **fondations** : de la statistique pour lire les chiffres sans se tromper, trois outils que tout analyste rencontre (le **tableur**, le langage de requête **SQL**, et les langages de programmation **Python** et **R**), et les bases de la **collecte de données** et des enquêtes.

Cette série est **indépendante de la série 1** (« Data Science »), qui forme à la modélisation prédictive. Vous n'avez besoin d'avoir lu aucun autre livre : si une idée est utile, nous l'expliquons ici. Ici, les mêmes outils servent un autre métier, plus proche de l'activité, plus soucieux de **clarté** que de sophistication.

Le métier de data analyst : question, données, enseignement, décision

La gérante et ses questions

Tout au long de la série, un même cadre : **la boutique**, une petite enseigne de maison et de décoration. Elle vend sur trois **canaux** (en Boutique, sur le Site, par les Réseaux sociaux) et vous venez d'y être embauchée comme analyste. **La gérante**, qui a créé l'enseigne, n'est pas statisticienne ; elle connaît ses clients, ses produits, ses fournisseurs et son équipe. Elle vous accueille un lundi matin avec trois questions, que ce volume vous apprendra à traiter :

1. « Le Site nous fait-il perdre des clients en boutique ? »

2. « **Combien nous coûtent les retours ?** »

3. « **Un client sur quatre a répondu à notre enquête de satisfaction : peut-on la croire ?** »

Pour y répondre, vous disposez de trois années de données (2023 à 2025). Voici de quoi il s'agit, en chiffres.

La boutique compte **6 000 clients** inscrits, **120 produits** répartis en six catégories, et a enregistré **36 395 commandes** sur trois ans, soit **84 000 lignes de commande** et environ **3,65 millions d'euros** de chiffre d'affaires. Le **panier moyen** (le montant moyen d'une commande) est d'environ 100 €. Les ventes progressent : 1,14 million d'euros en 2023, 1,19 en 2024, 1,32 en 2025. Un peu moins de 6 % des lignes ont donné lieu à un retour, pour 221 000 € remboursés. Enfin, 958 clients ont répondu à l'enquête de satisfaction, parmi les 3 875 qui avaient commandé en 2025.

Les données sont simulées. Aucune de ces données n'est réelle : elles ont été fabriquées par un programme, avec un mécanisme connu et des graines fixes (la section suivante les décrit). C'est un atout pédagogique : nous pourrions comparer ce que l'analyse trouve à ce que nous avons programmé. Dans la vie réelle, on ne connaît jamais la vérité, et c'est ce qui rend le métier à la fois difficile et passionnant.

De la question à la décision : la boucle de l'analyse

Une analyse se déroule toujours selon la même boucle en quatre temps, que ce volume vous fera parcourir plusieurs fois.

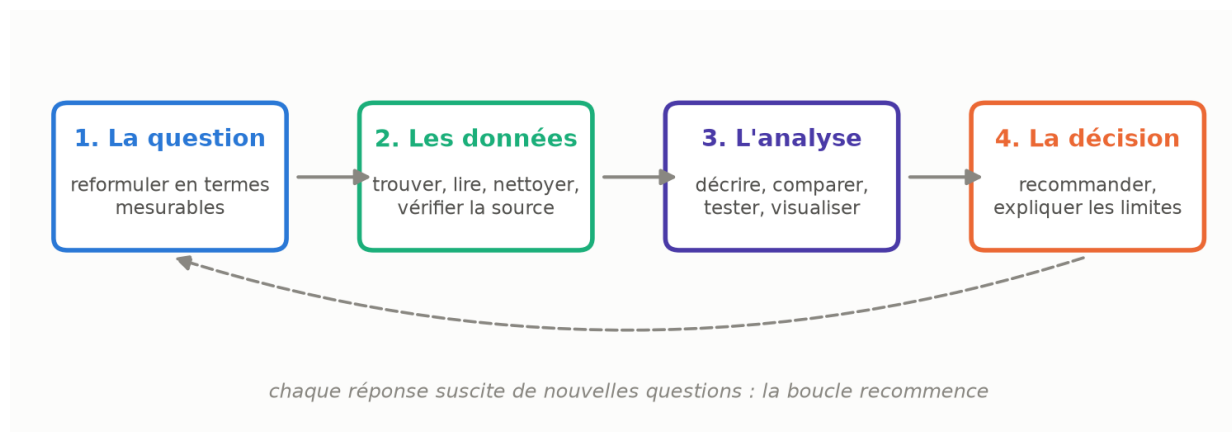


Figure 1: Les quatre temps d'une analyse : on part d'une question de l'activité, on cherche et on prépare les données, on analyse, puis on recommande une décision en disant les limites ; la décision fait naître de nouvelles questions.

Prenons la première question de la gérante pour voir la boucle à l'œuvre.

Temps 1 : préciser la question

« Le Site nous fait-il perdre des clients en boutique ? » est une question d'activité, pas encore une question de données. L'analyste commence par **la rendre précise**. Que veut dire « perdre » ? Moins de commandes en boutique ? Moins de clients différents ? Moins de chiffre d'affaires ? Par rapport à quand ? Et « le Site » : son existence, ou sa croissance récente ? En discutant avec la gérante, vous convenez d'une première version, volontairement simple : **le nombre de commandes passées en boutique a-t-il diminué pendant que celui du Site augmentait ?**

Temps 2 : trouver et vérifier les données

La table des commandes porte pour chaque commande une date et un **canal** (Boutique, Site, Réseaux). Il faut vérifier qu'elle est exploitable : pas de doublon d'identifiant, des dates cohérentes, des canaux connus. Ce temps de vérification est souvent le plus long d'une analyse ; le volume II de la série lui est entièrement consacré.

Temps 3 : analyser

On compte les commandes par année et par canal.

canal	Boutique	Réseaux	Site	Total
annee				
2023	5916	1230	4272	11418
2024	5617	1301	5113	12031
2025	5442	1426	6078	12946
variation 2023 -> 2025 (%) : {'Boutique': -8.0, 'Réseaux': 15.9, 'Site': 42.3, 'Total': 13.4}				
part du Site (%) : {2023: 37.4, 2024: 42.5, 2025: 46.9}				

La variation s'obtient par un calcul que vous maîtriserez au chapitre 1 (section 1.5) : pour la boutique, $(5442 - 5916)/5916 \approx -8,0\%$; pour le Site, $(6078 - 4272)/4272 \approx +42,3\%$. En deux ans, les commandes en boutique ont baissé de 8 % et celles du Site ont augmenté de 42 %, de sorte que le Site pèse 46,9 % des commandes en 2025 contre 37,4 % en 2023.

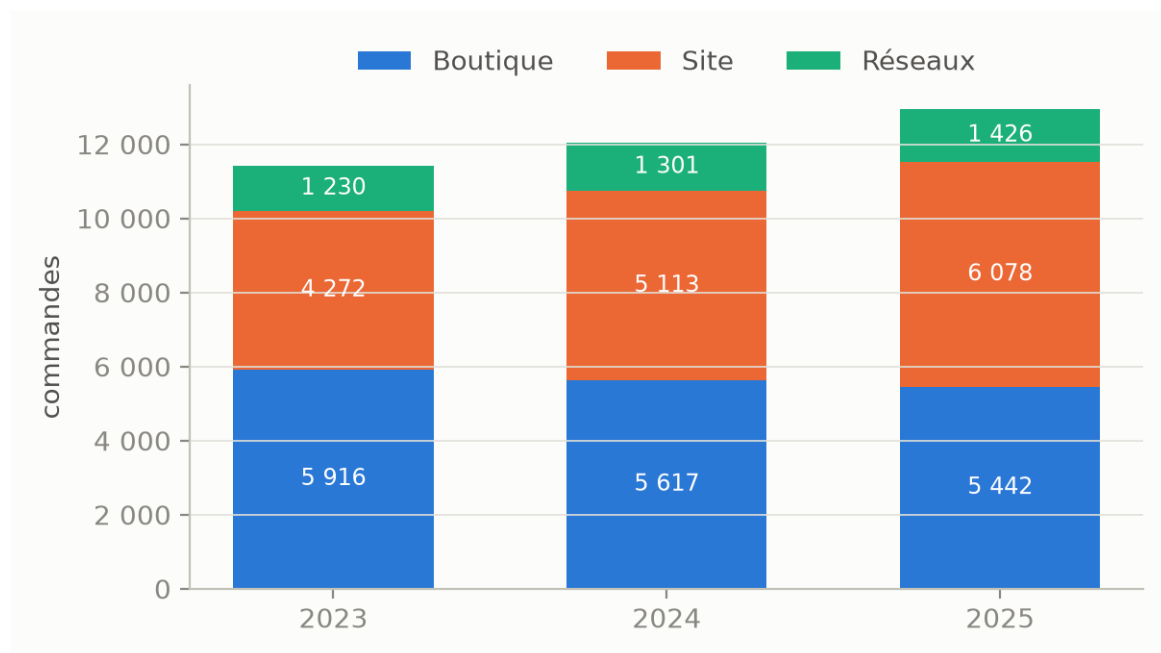


Figure 2: Nombre de commandes par année et par canal : la boutique recule régulièrement, le Site progresse beaucoup, les réseaux sociaux progressent aussi mais restent petits.

Temps 4 : décider, et dire ce que le chiffre ne dit pas

Reste la partie que l'on oublie le plus souvent : **la réponse à la gérante**. Une réponse honnête serait : « Les commandes en boutique ont baissé de 8 % en deux ans pendant que celles du Site augmentaient de 42 %. Cela *ressemble* à un transfert d'une partie de la clientèle de la boutique vers le Site. Mais ce chiffre ne prouve pas que le Site ait *causé* la baisse : la fréquentation de la boutique a pu baisser pour d'autres raisons, et le Site a aussi attiré de nouveaux clients. Pour trancher, il faudrait suivre les mêmes clients d'un canal à l'autre. »

Cette dernière phrase est celle qui fait la différence entre un compte rendu et une analyse. Elle ouvre aussi la question suivante : **cette personne a-t-elle remplacé ses achats en boutique par**

des achats sur le Site ? Vous saurez y répondre à la fin de la série.

Intuition. Une analyse n'est pas un calcul : c'est une chaîne de décisions (quelle question, quelles données, quelle comparaison, quel résultat mis en avant). Chaque maillon peut introduire une erreur ; l'objectif de ce volume est de vous donner les outils pour que **chaque maillon soit solide et vérifiable**.

Ce que fait un analyste, et ce qu'il ne fait pas

Voici, concrètement, les gestes d'un analyste, du plus fréquent au plus rare.

- **Questionner** : clarifier la demande, la reformuler, proposer une version mesurable.
- **Collecter et préparer** : trouver les bonnes sources, lire les fichiers, nettoyer, relier les tables.
- **Décrire** : moyennes, proportions, évolutions, tableaux de synthèse.
- **Comparer et tester** : un groupe contre un autre, une période contre une autre, avec une idée de l'incertitude.
- **Visualiser** : un graphique ou un tableau de bord qui se lit en quelques secondes.
- **Recommander et expliquer** : transformer un résultat en décision possible, avec ses limites.
- **Documenter et automatiser** : que quelqu'un d'autre (ou vous dans six mois) puisse refaire le calcul.

Il y a aussi ce qu'un analyste **ne fait pas** seul, ou ne devrait pas prétendre faire :

- **décider à la place de l'organisation** : il éclaire, il ne tranche pas ;
- **prouver une causalité** à partir de simples observations : il sait la suggérer, et dire quelle expérience la prouverait (chapitres 1.4 et, plus tard, volume III) ;
- **construire seul des systèmes de production** (entrepôts, flux de données robustes) ou **des modèles prédictifs complexes** : ce sont d'autres métiers, voisins, avec lesquels il travaille.

Analyste, data scientist, ingénieur de données

Les trois métiers se recoupent et leurs frontières varient d'une organisation à l'autre. Cette table donne un repère.

	Data analyst	Data scientist	Ingénieur de données
Question type	« Que s'est-il passé, pourquoi, que faire ? »	« Peut-on prédire ou automatiser cela ? »	« Comment livrer les données, vite et fiables ? »
Production	analyses, rapports, tableaux de bord, recommandations	modèles prédictifs, expériences, algorithmes	pipelines, bases, entrepôts de données
Outils centraux	tableur, SQL, outil de visualisation, un peu de Python ou de R	Python ou R, statistique, apprentissage automatique	SQL, Python, orchestration, bases de données
Plus proche de	l'activité et de ses décideurs	la recherche et le produit	l'informatique

Les trois partagent des outils (SQL, Python) et des exigences (reproductibilité, honnêteté). Un analyste qui sait lire un modèle de data scientist et dialoguer avec un ingénieur de données gagne beaucoup en efficacité.

Les trois compétences de l'analyste

On distingue trois familles de compétences. Aucune ne suffit seule.

- **La compétence technique** : manipuler des données (tableur, SQL, Python ou R), calculer correctement, vérifier. C'est l'objet principal de ce volume et du suivant.
- **La compétence statistique** : savoir ce qu'un chiffre mesure, ce qu'il ne mesure pas, et quelle confiance lui accorder. C'est le chapitre 1, et le fil du volume III.
- **La compétence d'activité et de communication** : comprendre le métier de la personne qui pose la question, choisir le bon indicateur, raconter le résultat clairement. Elle se travaille tout au long de la série, et particulièrement dans le volume IV.

L'expérience montre qu'un analyste moyennement technique mais qui comprend l'activité et sait expliquer est plus utile qu'un virtuose du code qui ne communique pas. Cela ne veut pas dire que la technique est secondaire : une erreur de calcul ruine toute la confiance. Cela veut dire qu'il faut travailler les deux.

Honnêteté et éthique

Un analyste a un pouvoir réel : ses chiffres orientent des décisions, parfois sur des personnes. Trois règles simples en découlent, que nous répéterons.

1. **Reproductible** : une autre personne doit pouvoir refaire votre calcul à partir des mêmes données et obtenir le même résultat. D'où les graines fixes, les requêtes conservées, les notebooks documentés.
2. **Sourcé** : chaque chiffre porte son origine (quel fichier, quelle date, quelles exclusions). Un chiffre sans source n'est qu'une opinion.
3. **Borné** : chaque résultat dit ce qu'il ne sait pas (période, population, incertitude, causalité non établie). Choisir la période ou le groupe qui donne le plus beau graphique sans le dire est une **tromperie**, même si les calculs sont justes.

S'y ajoute le **respect des personnes** : les données de clients sont des données personnelles ; on ne les partage pas, on n'en dit que ce qui est nécessaire, on les agrège ou on les anonymise quand c'est possible (le volume II y consacre un chapitre complémentaire).

Piège. La tentation la plus fréquente n'est pas de falsifier un chiffre, mais de **ne montrer que ceux qui plaisent**. La parade est simple : écrire à l'avance la question et la façon de la mesurer, puis rapporter le résultat quel qu'il soit.

Comment lire la série

Parcours essentiel et ☉ pour aller plus loin

Chaque chapitre a **deux niveaux**. Le **parcours essentiel** regroupe les idées principales : il suffit à comprendre le sujet et à l'utiliser en pratique, et un lecteur qui veut la vue d'ensemble peut ne suivre que lui. Les sections marquées ☉ **Pour aller plus loin** sont des approfondissements facultatifs : méthodes plus précises, outils connexes, sujets voisins. Le reste du livre ne les suppose jamais. Un chapitre entièrement facultatif est marqué ☉ **Chapitre complémentaire**.

Les six volumes

Volume	Titre	Ce que vous y apprenez
I	Fondations	la statistique essentielle, Excel, SQL, Python et R, la collecte de données et les enquêtes
II	Préparation des données	nettoyer, transformer, fusionner, contrôler la qualité, réconcilier, documenter
III	Analyse	explorer les données, tests d'hypothèses et tests A/B, régression, segmentation, séries temporelles, indicateurs
IV	Visualisation et communication	choisir et construire un graphique, tableaux de bord, raconter les données, présenter
V	Analytique avancée et automatisation	entrepôts de données, flux de chargement, automatisation, introduction au prédictif, reporting de risque
VI	Travaux appliqués et portfolio	projets de reporting, études de cas, comment présenter un travail réel

L'ordre suit la vie d'une analyse : on **apprend les outils** (I), on **prépare** les données (II), on les **analyse** (III), on **montre** le résultat (IV), on **industrialise** (V), puis on **met en pratique** (VI).

Le livre et son cahier

Chaque volume existe en **deux ouvrages**. Le **livre** que vous lisez explique : intuition, exemples calculés à la main, pièges, résumé de chaque chapitre. Il ne contient **ni exercices ni corrigés**. Le **cahier d'exercices et d'applications** est son compagnon : applications guidées (de petites études sur les données de la boutique), exercices classés par difficulté avec leurs corrigés, et en fin de volume un projet complet et une auto-évaluation.

Le livre renvoie au cahier par une ligne de ce genre :

Pour s'entraîner. Cahier, chapitre 1 : application 1.1, exercices 1.1 à 1.4.

Lisez la section, puis faites l'application correspondante : **on comprend en lisant, on retient en calculant**.

Cette série et la série 1

La série 1 (« Data Science ») utilise les mêmes outils mais poursuit un autre but : construire des modèles prédictifs rigoureux. La série 2 que vous lisez s'adresse à celles et ceux qui veulent **analyser, expliquer et communiquer**. Elles sont indépendantes ; si l'envie vous prend de creuser un sujet, nous indiquons parfois dans un encadré où la série 1 l'approfondit.

Mode d'emploi de ce volume

Ce que le volume suppose

Presque rien. Il suppose que vous savez faire une règle de trois et calculer un pourcentage, que vous avez une idée de ce qu'est un tableau de nombres, et que vous êtes prêt à taper quelques lignes dans un langage de programmation. Vous n'avez **pas** besoin d'avoir programmé auparavant, ni de posséder Excel (le chapitre 2 est lisible sans, et ses formules sont aussi vérifiées avec un autre tableur). Les notions de statistique sont reprises depuis la base au chapitre 1.

Si vous...	Alors...
découvrez tout	lisez dans l'ordre, en faisant les applications du cahier
connaissez déjà un outil (Excel, SQL...)	prenez le chapitre correspondant en diagonale, puis faites ses exercices ★★ et ★★★
voulez seulement la vue d'ensemble	suivez le parcours essentiel, lisez les encadrés ✓ À retenir et les bilans
préparez un entretien ou une certification	faites le projet du volume et l'auto-évaluation, puis relisez les sections indiquées par les corrigés

Une habitude à prendre : la vérification croisée

Au fil du volume, nous obtiendrons plusieurs fois **le même chiffre par deux chemins différents** : un tableur, une requête SQL, un programme Python, un programme R. Ce n'est pas du zèle : c'est le meilleur moyen de détecter une erreur, parce que deux méthodes indépendantes ne se trompent presque jamais de la même façon. Voici la première vérification croisée. Le chiffre d'affaires de 2025 se calcule en additionnant les montants des lignes de commande de 2025.

```
import sqlite3
import pandas as pd

lignes = pd.read_csv("donnees/lignes_commande.csv")
commandes = pd.read_csv("donnees/commandes.csv")
l25 = lignes.merge(commandes[["id_commande", "date_commande"]],
    ↪ on="id_commande").query("date_commande >= '2025-01-01'")
print("pandas :", round(l25["montant"].sum(), 2))

con = sqlite3.connect("donnees/boutique.db")
print("SQL      :", con.execute("SELECT ROUND(SUM(l.montant), 2) FROM lignes_commande l JOIN commandes c
    ↪ USING (id_commande) WHERE c.date_commande >= '2025-01-01'").fetchone()[0])
```

```
pandas : 1324763.72
SQL      : 1324763.72
```

Les deux chemins donnent le même chiffre d'affaires, **1 324 763,72 €**. Et si l'on compare au chiffre de la table `jours_exploitation`, qui contient les totaux journaliers, on retrouve encore la même somme sur trois ans (3 653 157,28 €) : un troisième chemin, qui confirme.

À retenir. Un analyste traduit une **question d'activité** en question mesurable, trouve et vérifie les **données**, **analyse**, puis **recommande** en disant les **limites**. Il fait des chiffres **reproductibles**, **sourcés et bornés**, et il les vérifie par **deux chemins**. Les trois questions de la gérante (le Site contre la boutique, le coût des retours, la fiabilité de l'enquête) guideront les chapitres de ce volume ; la section suivante décrit les données et l'environnement.

Pour s'entraîner. Cahier, mode d'emploi : un mini-diagnostic de huit questions (pourcentages, moyennes, filtres, jointures, lecture de tableaux, esprit critique), avec corrigés, pour savoir où revenir avant de commencer.

Carte du volume, données et environnement

Cette section ouvre le volume par trois choses : la **carte des chapitres**, le **catalogue des jeux de données** (avec, pour chacun, ce qu'il contient et ce qu'on y a « programmé »), et l'**environnement** nécessaire pour refaire tous les calculs.

Carte du volume

Chaque chapitre répond à l'une des questions de la gérante, ou à une question que l'on se pose en chemin. Les chapitres sont lisibles dans l'ordre, mais on peut les prendre séparément ; les sections marquées ⊕ sont facultatives.

Chapitre	Question posée	Contenu
1. Les essentiels de la statistique	Comment résumer 36 000 commandes ? Peut-on croire un échantillon ? Ce lien est-il une cause ?	statistique descriptive, distributions et courbe normale, échantillonnage, corrélation et causalité ; ⊕ pourcentages, taux de croissance, moyennes pondérées
2. Excel	Comment construire un suivi des ventes que d'autres comprennent ?	formules et fonctions, tableaux croisés dynamiques, Power Query, bonnes pratiques ; ⊕ Power Pivot et DAX, macros, Google Sheets et Looker Studio
3. SQL	Comment interroger la base de la boutique ?	SELECT, jointures, fonctions fenêtres, requêtes structurées (CTE) ; ⊕ optimisation, différences entre moteurs
4. Python et R pour l'analyse	Comment automatiser et reproduire un calcul ?	pandas, tidyverse, lecture et restructuration, notebooks ; ⊕ ggplot2 et Shiny, NumPy, polars
5. Types de données, collecte et enquêtes	D'où viennent les données, et comment en produire de fiables ?	types et niveaux de mesure, sources, conception d'enquêtes ; ⊕ questionnaires, plans de sondage, API et web scraping
Projet du volume (cahier)	Une première analyse de bout en bout	du fichier brut au tableau de synthèse, par trois outils

Les jeux de données du volume

Tout est **simulé**, avec des graines fixes, par le script `build/donnees_a1.py`. Chaque jeu suit un mécanisme que le script décrit en tête de fichier : c'est la **vérité programmée**. Nous la révélerons dans les chapitres où elle est instructive, et le cahier permet de la retrouver. Aucune donnée ne vient d'une entreprise réelle.

Chargeons les sept tables principales et regardons leur forme.

clients	6000 lignes	8 colonnes	0 manquants
produits	120 lignes	7 colonnes	0 manquants
commandes	36395 lignes	7 colonnes	30652 manquants
lignes_commande	83905 lignes	7 colonnes	0 manquants
retours	5002 lignes	5 colonnes	0 manquants
jours_exploitation	1096 lignes	8 colonnes	0 manquants
enquete_satisfaction	958 lignes	12 colonnes	1295 manquants

Les valeurs manquantes sont **voulues** : un code promo absent signifie « pas de promotion » (30 652 commandes sur 36 395), et l'enquête a des questions sans réponse (le conseil ne concerne que la boutique, certains clients restent anonymes, beaucoup ne laissent pas de commentaire).

Jeu	Contenu	Nature	Chapitres
clients.csv	6 000 clients : ville, canal d'acquisition, carte de fidélité	simulé	tous
produits.csv	120 produits : catégorie, prix de vente, coût d'achat, fournisseur	simulé	1 à 4
commandes.csv	36 395 commandes (2023-2025) : date, heure, client, canal, livraison, code promo	simulé	1 à 4
lignes_commande.csv	83 905 lignes : produit, quantité, prix unitaire, remise, montant	simulé	1 à 4
retours.csv	5 002 retours : ligne, date, motif, montant remboursé	simulé	3, 4, projet
jours_exploitation.csv	1 096 jours : commandes, chiffre d'affaires, météo, promotion, publicité	simulé	1, 2, 4
enquete_satisfaction.csv	958 réponses d'une enquête de satisfaction	simulé	1, 5, projet
ventes_2025.xlsx	classeur Excel de 2025 : feuilles Lignes, Produits, Clients	simulé	2
export_caisse_brut.csv	export de caisse désordonné (une semaine, boutique)	simulé	2, 4, projet
boutique.db	base SQLite contenant les six premières tables	simulé	3, 4

Les clients et les produits

La table clients décrit les 6 000 clients de la boutique ; la table produits décrit le catalogue.

Colonne	Signification
id_client	identifiant unique du client
date_inscription	date d'inscription (de 2018 à 2025)
annee_naissance	année de naissance
ville	une des 20 villes (« Ville A » à « Ville T »)
canal_acquisition	canal par lequel le client est arrivé : Boutique, Site, Réseaux
fidelite	1 si le client a la carte de fidélité
email_valide, consentement_marketing	1 si l'adresse est valide ; 1 si le client accepte les messages commerciaux

Colonne	Signification
id_produit, nom_produit	identifiant et nom (« Bougie classique », « Plaid nordique »...)
categorie	Cuisine, Maison, Décoration, Papeterie, Jardin, Bien-être (20 produits chacune)
prix_vente, cout_achat	prix de vente TTC et coût d'achat, en €
fournisseur	un des huit fournisseurs (« Fournisseur A » à « Fournisseur H »)
date_lancement	date de mise au catalogue

```
clients : inscrits avant 2023 : 4000 | depuis : 2000
  âge moyen en 2025 : 43.3 | ville la plus fréquente : Ville A 0.144
  canal d'acquisition : {'Boutique': 0.493, 'Site': 0.388, 'Réseaux': 0.119}
  carte de fidélité : 0.351 | email valide : 0.921 | consentement : 0.605
produits : prix de 2.9 à 152.9 € | marge moyenne 0.478 | de 0.352 à 0.6
  prix moyen par catégorie : {'Bien-être': 27.5, 'Cuisine': 34.8, 'Décoration': 38.0, 'Jardin': 51.8,
  ↵ 'Maison': 51.8, 'Papeterie': 11.4}
```

Les deux tiers des clients (4 000 sur 6 000) étaient déjà inscrits au 1er janvier 2023 ; les 2 000 autres se sont inscrits pendant la période. L'âge moyen est d'environ 43 ans, et la ville la plus fréquente, la Ville A, regroupe 14 % des clients. Près de la moitié (49 %) sont arrivés par la boutique, 39 % par le Site, 12 % par les réseaux. Un client sur trois (35 %) a la carte de fidélité. La marge moyenne des produits est de 48 % du prix de vente (de 35 % à 60 %), et le prix va de 2,90 € à 152,90 € : les catégories Jardin et Maison contiennent les articles les plus chers (prix moyen de 51,80 € chacune, contre 11,40 € en papeterie).

Les commandes, les lignes et les retours

Une **commande** est un achat d'un client, à une date et par un canal ; elle contient une ou plusieurs **lignes** (une ligne = un produit en une certaine quantité) ; une ligne peut donner lieu à un **retour**.

Table	Colonnes principales
commandes	id_commande, date_commande (jj, texte aaaa-mm-jj), heure, id_client, canal, mode_livraison (Domicile, Point relais, Retrait magasin), code_promo (SOLDES, BIENVENUE, FIDELITE, ou vide)
lignes_commande	id_ligne, id_commande, id_produit, quantite, prix_unitaire (€), remise_pct (0, 5, 10 ou 20), montant = quantité × prix unitaire × (1 – remise)
retours	id_retour, id_ligne, date_retour, motif, montant_rembourse

```
commandes du 2023-01-01 au 2025-12-31
  par canal : {'Boutique': 16975, 'Site': 15463, 'Réseaux': 3957}
  par livraison : {'Retrait magasin': 18348, 'Domicile': 10737, 'Point relais': 7310}
  codes promo : {'(aucun)': 30652, 'SOLDES': 3006, 'FIDELITE': 2423, 'BIENVENUE': 314}
lignes par commande : moyenne 2.31 | maximum 8 | quantités : {1: 71218, 2: 9288, 3: 2552, 4: 847}
remises : {0: 70611, 5: 5603, 10: 751, 20: 6940} | part des lignes remisées : 0.158
retours par canal (part des lignes) : {'Boutique': 0.031, 'Réseaux': 0.067, 'Site': 0.09}
  motifs : {'Mauvais choix': 0.314, "Changement d'avis": 0.303, 'Défaut': 0.182, 'Livraison tardive':
↳ 0.121, 'Autre': 0.08}
  remboursé : 221010.01 | chiffre d'affaires : 3653157.28
```

Les commandes s'étalent du 1er janvier 2023 au 31 décembre 2025. Elles sont réparties entre la boutique (16 975), le Site (15 463) et les réseaux (3 957). Un panier compte en moyenne 2,3 lignes (jusqu'à 8), et la quantité vaut presque toujours 1 (85 % des lignes). Les remises existent sur 16 % des lignes. Les **retours** concernent 3,1 % des lignes de la boutique, 6,7 % de celles des réseaux et 9,0 % de celles du Site ; les motifs les plus fréquents sont le « mauvais choix » (31 %) et le « changement d'avis » (30 %), loin devant le défaut du produit (18 %). Le total remboursé (221 010 €) représente un peu plus de 6 % du chiffre d'affaires de trois ans (3 653 157,28 €).

Les jours d'exploitation

La table jours_exploitation résume chaque jour : le nombre de commandes et le chiffre d'affaires **calculés à partir des commandes**, auxquels s'ajoutent des variables extérieures (température et pluie du lieu de la boutique, jour de promotion, dépense publicitaire).

Colonne	Signification
date, jour_semaine	date et jour (1 = lundi ... 7 = dimanche)
nb_commandes, chiffre_affaires	commandes et chiffre d'affaires du jour (€)
temperature_moy, pluie_mm	température moyenne (°C) et pluie (mm)
promo_active	1 les jours de soldes ou de « Vendredi noir » (153 jours en tout)
depense_pub	dépense publicitaire quotidienne moyenne (€)

```
jours : 1096 | jours de promotion : 153 | jours de pluie (> 1 mm) : 290
température moyenne : 13.1 | dépense publicitaire moyenne : 208.1
CA moyen par mois : {1: 2488, 2: 2345, 3: 2747, 4: 3031, 5: 3378, 6: 3355, 7: 3171, 8: 2689, 9:
↳ 3477, 10: 3488, 11: 4418, 12: 5357}
CA moyen par jour de semaine : {1: 3241, 2: 2941, 3: 3102, 4: 3317, 5: 3917, 6: 4634, 7: 2192}
corrélation CA / dépense publicitaire : 0.42 | CA / jour de promotion : -0.01
somme des commandes : 36395 | somme du CA : 3653157.28
```

Le chiffre d'affaires moyen d'un jour passe de 2 488 € en janvier à 5 357 € en décembre : la **saisonnalité** est forte. Le samedi est le meilleur jour (4 634 € en moyenne) et le dimanche le plus faible (2 192 €). Les sommes de cette table redonnent exactement celles des commandes (36 395 commandes, 3 653 157,28 €), ce qui est un bon contrôle. La corrélation de 0,42 entre le chiffre d'affaires et la dépense publicitaire est un piège que le chapitre 1 (section 1.4) démontrera.

L'enquête de satisfaction

L'enquête a été envoyée en fin d'année 2025 aux clients qui avaient commandé dans l'année : 3 875 invitations et 958 réponses (soit 24,7 %). Chaque ligne est une réponse.

Colonne	Signification
id_reponse, date_reponse	identifiant et date de la réponse
id_client	client (vide pour une réponse anonyme)
canal, tranche_age	canal de la dernière commande ; tranche d'âge du répondant
satisfaction_globale, satisfaction_livraison, satisfaction_prix	notes de 1 à 5
satisfaction_conseil	note de 1 à 5 (vide hors boutique : on ne conseille pas en ligne)
recommandation_0_10	« recommanderiez-vous la boutique ? » de 0 à 10
commentaire, duree_reponse_s	commentaire libre (souvent vide) ; durée de réponse en secondes

```
réponses : 958 | invitations (clients ayant commandé en 2025) : 3875
anonymes : 0.204 | doublons exacts : 27 | commentaires vides : 538
satisfaction globale : {1: 27, 2: 136, 3: 266, 4: 256, 5: 273} | moyenne : 3.64 | notes 4 ou 5 : 0.552
réponses 5-5-5 en moins de 20 s : 35 | durée médiane : 70.0 s
taux de réponse selon la carte de fidélité : {0: 0.179, 1: 0.214}
taux de réponse selon que la dernière commande date d'octobre-décembre : {False: 0.167, True: 0.205}
```

Les trois quarts des clients ne répondent pas, et ceux qui répondent **ne ressemblent pas** à ceux qui ne répondent pas : les clients avec carte répondent davantage (21 % contre 18 %), de même que ceux dont la dernière commande est récente (21 % contre 17 %). Le fichier contient aussi des **doublons** (27 réponses identiques en tout), des réponses « 5-5-5 » remplies en quelques secondes, et un cinquième de réponses anonymes. Ces défauts sont **programmés** : ils servent à apprendre à se méfier d'une enquête (sections 1.3 et 5.3).

Les fichiers « de bureau » : un classeur et un export de caisse

Deux fichiers imitent ce que l'on reçoit dans la vraie vie.

- **ventes_2025.xlsx** est un classeur Excel de l'année 2025 avec trois feuilles : Lignes (une ligne de commande par ligne du tableau, avec la date, le client, le canal, le produit, la catégorie, la quantité, le prix, la remise et le montant), Produits (le catalogue) et Clients. Il sert de base au chapitre 2.
- **export_caisse_brut.csv** est un export de la caisse de la boutique pour la semaine du 3 au 9 novembre 2025, **volontairement désordonné** : deux lignes de titre, séparateur point-virgule, virgule décimale, dates jj/mm/aaaa, encodage Windows (cp1252), un en-tête répété à chaque « page », quelques montants absents, des majuscules incohérentes et une ligne de total en bas. Il servira à apprendre à lire un fichier tel qu'il arrive (chapitres 2 et 4, puis le projet).

```
ventes_2025.xlsx : {'Lignes': (29827, 13), 'Produits': (120, 6), 'Clients': (6000, 5)} | somme des
↳ montants : 1324763.72
export brut : 289 lignes | 5 en-têtes | montants vides : 8
```

Le classeur compte 29 827 lignes de commande pour l'année 2025 et un montant total de 1 324 763,72 €, qui est celui que nous avons obtenu en introduction par deux autres chemins. L'export contient 289 lignes dont 5 en-têtes (le premier et quatre répétés) et 8 montants absents.

La base de données boutique.db

Les six premières tables existent aussi dans une **base de données** SQLite, un fichier unique que l'on interroge en SQL (chapitre 3). Les tables sont reliées par des identifiants : c'est le **schéma** de la base.

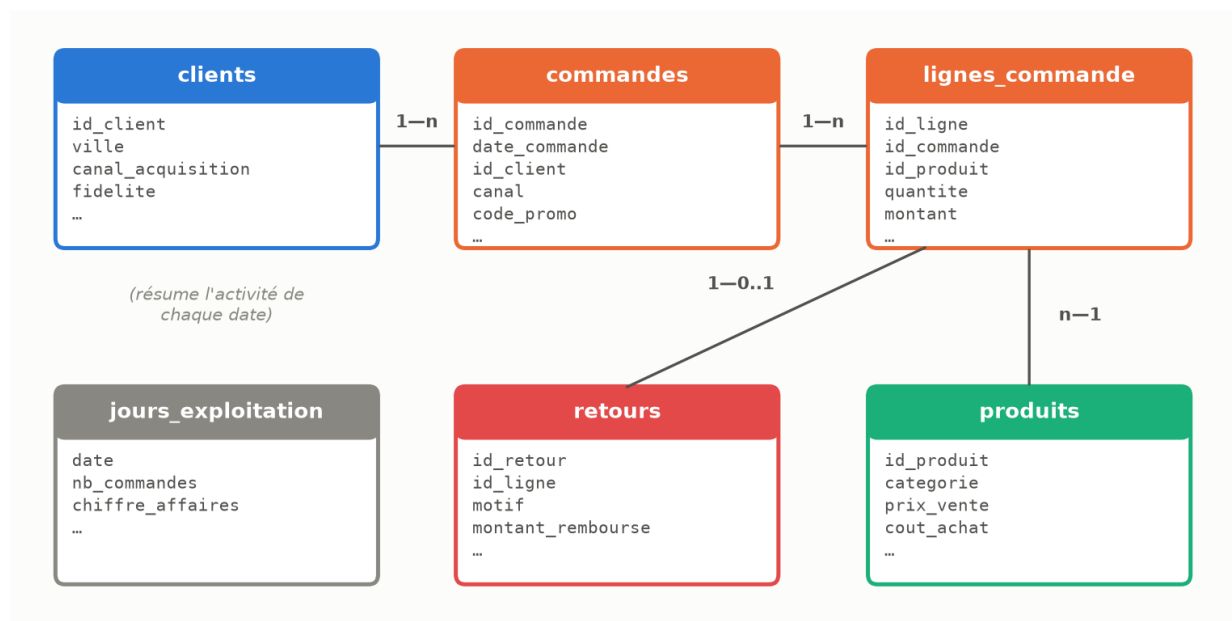


Figure 3: Schéma de la base de la boutique : un client passe **plusieurs** commandes, chaque commande contient **plusieurs** lignes, chaque ligne désigne un produit et peut donner lieu à un retour ; la table des jours d'exploitation résume l'activité par date. Maquette dessinée avec matplotlib.

Les liens se lisent « un client passe **plusieurs** commandes, une commande contient **plusieurs** lignes, une ligne désigne **un** produit et donne lieu à **zéro ou un** retour ». Ce sont ces identifiants communs (id_client, id_commande, id_produit, id_ligne) qui permettent de **joindre** les tables, thème du chapitre 3.

```
tables : ['clients', 'commandes', 'jours_exploitation', 'lignes_commande', 'produits', 'retours']
index : ['idx_cmd_client', 'idx_cmd_date', 'idx_lig_cmd', 'idx_lig_prod']
lignes : {'clients': 6000, 'produits': 120, 'commandes': 36395, 'lignes_commande': 83905, 'retours':
↳ 5002, 'jours_exploitation': 1096}
```

Les quatre **index** accélèrent les recherches par client, par date et par commande (section 3.5).

La vérité programmée

Le script qui fabrique les données y a inscrit des mécanismes connus. Les voici en résumé : nous les retrouverons un par un, et vous verrez que l'analyse les retrouve (ou non, et pourquoi).

Ce qui est programmé	Ce que l'on observe	Où on le retrouve
tendance de +6 % par an	11 418 commandes en 2023, 12 946 en 2025 (+13,4 %)	1.1, 1.5
saison (creux de janvier-février et d'été, pic de novembre-décembre)	CA moyen par jour de 2 488 € en janvier, 5 357 € en décembre	1.1, 2.2
jour de la semaine (samedi +40 %, dimanche -35 %)	4 634 € le samedi, 2 192 € le dimanche	1.1
part du Site : de 35 % à 48 % en trois ans	37,4 % en 2023, 46,9 % en 2025	introduction, 1.5
retours : Site 9 %, réseaux 7 %, boutique 3 %	9,0 %, 6,7 %, 3,1 %	3.1, 4.3
dépense publicitaire liée à la saison, avec un vrai effet faible	corrélation de 0,42 avec le chiffre d'affaires	1.4
promotions : +18 % de commandes les jours de promotion	masqué par la saison et la remise (corrélation brute de -0,01 avec le CA)	1.4, volume III
enquête : biais de réponse, doublons, réponses anonymes, « 5-5-5 »	24,7 % de réponses, 27 doublons, 20 % d'anonymes	1.3, 5.3

Un exemple de piège : la corrélation brute entre jour de promotion et chiffre d'affaires est **quasi nulle** ($-0,01$), alors que la promotion augmente bien les commandes de 18 %. Les promotions tombent en hiver et en été, saisons où les ventes sont plus faibles, et la remise réduit le montant de chaque commande : les effets se masquent. C'est exactement le genre de situation dont l'analyste doit se méfier, et nous la retrouverons au chapitre 1 et au volume III.

L'environnement de travail

Python, R et SQL

Les calculs du livre sont faits en **Python** (bibliothèques pandas, NumPy, SciPy, matplotlib...), en **R** (tidyverse) et en **SQL** (SQLite et DuckDB). Voici les versions utilisées pour écrire ce volume ; les vôtres peuvent différer, tant que tout s'exécute.

```
python 3.13.3 | pandas 3.0.6, numpy 2.5.3, scipy 1.18.1, statsmodels 0.15.0, matplotlib 3.11.2,
↳ openpyxl 3.1.5, XlsxWriter 3.2.9, polars 2.0.0, duckdb 1.5.6
SQLite 3.46.1
R version 4.4.3 (2025-02-28)
```

SQL : les requêtes du chapitre 3 sont exécutées avec **SQLite** (un moteur léger, sans serveur), qui accepte les jointures, les fonctions fenêtres et les CTE. Les différences avec PostgreSQL, MySQL, SQL Server et Oracle sont décrites à la section 3.6, mais **ces moteurs ne sont pas exécutés** : nous le signalons chaque fois.

Le tableur : Excel, LibreOffice et nos maquettes

Une précision d'honnêteté. **Excel n'est pas installé** sur la machine qui a produit ce livre, mais **LibreOffice Calc**, un tableur libre qui lit les fichiers Excel, l'est. Les formules du chapitre 2 ont donc été **écrites dans des classeurs, recalculées par LibreOffice et comparées aux résultats de pandas** : chaque résultat cité est calculé, pas recopié. Trois limites à garder en tête :

- les **copies d'écran** sont des **maquettes dessinées** avec matplotlib, pas des captures d'Excel ;
- les noms de menus et de fonctions varient avec la **version** et la **langue** d'Excel : nous écrivons les formules avec les noms français (SOMME.SI.ENS), les noms anglais figurant dans un tableau de correspondance ; à vérifier dans votre version ;
- ce qui n'est pas exécutable ici (tableaux croisés dynamiques réels, Power Query, Power Pivot, macros, Google Sheets, Looker Studio) est signalé « **non exécuté** », avec un équivalent calculé quand c'est possible.

Régénérer les données et refaire les calculs

Tout le dossier `donnees/` se régénère à l'identique par un script, et les calculs du livre se rejouent par `make check` (qui vérifie que chaque sortie est inchangée).

```
python build/donnees_a1.py      # régénère donnees/ (quelques secondes, graines fixes)
make check                     # rejoue tous les blocs de code et signale toute différence
make pdf                       # reconstruit le livre et le cahier en PDF
```

Conventions du volume

- **Monnaie et villes** : montants en € ; villes « Ville A » à « Ville T » ; canaux Boutique, Site, Réseaux (écrits ainsi, entre accents graves, quand ce sont des valeurs de données).
- **Nombres** : nous écrivons à la française (espace pour les milliers, virgule décimale) dans le texte, et à l'anglaise (point décimal) dans les sorties de programme.
- **Dates** : `aaaa-mm-jj` dans les données et les requêtes (sans ambiguïté) ; `jj/mm/aaaa` dans les exports « à la française ».
- **Aléatoire** : toute simulation fixe sa **graine** (`np.random.default_rng(42)`, par exemple), donc vos résultats sont identiques aux nôtres.
- **Anonymat** : aucun nom réel, ni de boutique, ni de personne, ni de lieu. Les noms d'outils et de bibliothèques, eux, sont réels.
- **Encadrés** : ■ intuition · ■ pour qui veut la formule · ■ expérience · △ piège · ✓ à retenir · ■ repère ou section facultative · ■ pour s'entraîner · ■ données.

À retenir. Les données de la boutique sont **simulées** : 36 395 commandes, 83 905 lignes, 6 000 clients, 120 produits, trois années, plus une enquête, un classeur Excel, un export de caisse désordonné et une base SQLite. La **vérité programmée** permet de juger ce que l'analyse retrouve. Les formules Excel sont vérifiées avec LibreOffice, les copies d'écran sont des maquettes, et tout se régénère par un script.

Chapitre 1 : Les essentiels de la statistique

« Un chiffre sans contexte est une opinion qui se fait passer pour un fait. »

C'est votre deuxième semaine à la boutique. La gérante passe la tête dans la porte de votre bureau, un tableau imprimé à la main :

— Notre panier moyen est de **100 €**. C'est un bon chiffre ?

Vous ouvrez la bouche pour répondre « oui » ou « non », et vous vous arrêtez. Bon **par rapport à quoi** ? À l'an dernier, à ce que fait la concurrence, à ce qu'il faudrait pour couvrir les frais ? Et surtout : ce « 100 € » est-il un montant que **beaucoup de clients dépensent vraiment**, ou le résultat de quelques grosses commandes qui tirent la moyenne vers le haut ? Les deux situations donnent le même chiffre, et pourtant elles appellent des décisions très différentes.

Ce chapitre vous donne le **vocabulaire** pour répondre à ce genre de question. La statistique que vous allez y rencontrer n'est pas de la « grosse mathématique » : c'est un petit nombre d'idées, que l'on retrouve à chaque analyse, et qu'il faut avoir **dans les doigts** avant d'ouvrir Excel, d'écrire une requête SQL ou de lancer un notebook.

Le chemin de ce chapitre

Le **parcours essentiel** compte quatre sections, qui répondent chacune à une question de la gérante :

- **1.1 Statistique descriptive** : *comment résumer des milliers de commandes en quelques nombres honnêtes* ? Le centre (moyenne, médiane), la dispersion (écart-type, quartiles), la forme (asymétrie, valeurs aberrantes), et les moyennes qui trompent.
- **1.2 Distributions et courbe normale** : *quelle forme prennent les phénomènes que nous observons* ? Compter des retours, des commandes, mesurer des âges, des montants : quelques « lois » suffisent pour décrire la plupart des situations.
- **1.3 Échantillonnage et erreur d'échantillonnage** : *si je n'observe qu'une partie des clients, de combien mon chiffre peut-il se tromper* ? L'intervalle de confiance, la taille d'échantillon, et les biais qu'aucune taille d'échantillon ne corrige.
- **1.4 Corrélation et causalité** : *les ventes montent quand la publicité monte : est-ce la publicité qui les fait monter* ? Mesurer une liaison, puis apprendre à ne pas en tirer de conclusion hâtive.

Une section facultative (⊕) complète le tout : **1.5 Mathématiques du quotidien en entreprise**, avec les pourcentages, les taux de croissance, les moyennes pondérées, les marges et les arrondis, c'est-à-dire les calculs que vous ferez **tous les jours**.

Comment lire ce chapitre. Chaque notion suit le même rythme : une **question** concrète, un **petit exemple calculé à la main** (sur quelques lignes, pour que vous voyiez ce qui se passe), la **formule** expliquée, puis l'**application aux données de la boutique**. Le code est réduit au strict nécessaire : l'essentiel est de comprendre ce que l'on calcule. Les exercices et les applications à refaire se trouvent dans le **cahier**, signalé par le symbole ■ à la fin de chaque section.

Les données du chapitre

Les données. Tout le volume s'appuie sur les données **simulées** de la boutique (le générateur est dans `build/donnees_a1.py`) : une petite enseigne de maison et de décoration, avec **6 catégories** et **120 produits**, trois canaux de vente (Boutique, Site, Réseaux) et des clients répartis dans vingt villes fictives (« Ville A » à « Ville T »). Dans ce chapitre, nous utilisons :

- `commandes.csv` et `lignes_commande.csv` : environ 36 395 commandes et 83 905 lignes de commande entre janvier 2023 et décembre 2025 ;
- `clients.csv` : 6 000 clients inscrits ;
- `retours.csv` : les lignes que les clients ont renvoyées ;
- `jours_exploitation.csv` : une ligne par jour (commandes, chiffre d'affaires, météo, promotion, dépense publicitaire).

Comme les données sont fabriquées, nous connaissons la **vérité** qui les a générées, et nous la révélerons au fil du chapitre : c'est le seul moyen de **vérifier** qu'une méthode statistique répond bien à la question posée, ce que la réalité ne permet presque jamais.

Une commande est composée d'une ou de plusieurs **lignes** (un produit, une quantité, un prix). Le **panier** d'une commande est la somme des montants de ses lignes : c'est la grandeur que la gérante appelle « panier moyen » quand elle fait la moyenne sur toutes les commandes. Chargeons les données et calculons ce premier chiffre.

```
import pandas as pd
commandes = pd.read_csv("donnees/commandes.csv")
lignes = pd.read_csv("donnees/lignes_commande.csv")
panier = lignes.groupby("id_commande")["montant"].sum() # une commande = une ou plusieurs lignes
print(len(commandes), "commandes", len(lignes), "lignes")
print("panier moyen :", round(panier.mean(), 2), "€")
```

```
36395 commandes, 83905 lignes
panier moyen : 100.38 €
```

Cette moyenne est exacte, mais elle ne répond pas encore à la question de la gérante. Les quatre sections qui suivent expliquent pourquoi, et ce qu'il faut calculer **en plus**.

1.1 Statistique descriptive : centre, dispersion, forme

Décrire un jeu de données, c'est répondre à trois questions : **autour de quelle valeur** se répartissent les observations (le centre), **à quel point** elles s'écartent les unes des autres (la dispersion), et **quelle allure** a cette répartition (la forme : symétrique ou étirée, avec ou sans valeurs extrêmes). Ces trois familles de nombres sont les **statistiques descriptives**. Elles ne démontrent rien : elles **résumant**. Mais un bon résumé évite la moitié des erreurs d'analyse, et un mauvais résumé (une moyenne seule, par exemple) en fabrique beaucoup.

1.1.1 Résumer un panier : le centre

Pour garder les calculs lisibles, partons de **neuf commandes seulement** : les neuf premières enregistrées en boutique le 15 avril 2025. Leurs paniers, en euros, dans l'ordre où elles sont arrivées :

9,69 ; 207,56 ; 132,37 ; 258,83 ; 66,75 ; 22,56 ; 50,27 ; 48,74 ; 128,25.

La **moyenne** est le total divisé par le nombre d'observations :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{925,02}{9} = 102,78 \text{ €}.$$

C'est le montant que chaque commande aurait si l'on **répartissait le total à parts égales**. C'est aussi le point d'équilibre : si l'on posait les neuf montants sur une règle, la règle tiendrait en équilibre sur 102,78 €.

La **médiane** est la valeur du milieu quand on range les observations par ordre croissant : la moitié des commandes est en dessous, la moitié au-dessus. Rangeons :

9,69 ; 22,56 ; 48,74 ; 50,27 ; **66,75** ; 128,25 ; 132,37 ; 207,56 ; 258,83.

Avec neuf valeurs, c'est la cinquième : **66,75 €**. (Avec un nombre pair de valeurs, on fait la moyenne des deux valeurs centrales.)

Les deux résumés diffèrent de **36 €**, et cet écart est une information : quelques grosses commandes (207,56 € et 258,83 €) **tirent la moyenne vers le haut** sans toucher la médiane. Si l'on retirait la commande de 258,83 €, la moyenne tomberait à 83,3 € et la médiane à 58,5 € ; si on la remplaçait par 2 588,30 € (une erreur de virgule), la moyenne exploserait à 361,6 €, alors que la médiane ne bougerait pas. La médiane est **robuste** aux valeurs extrêmes ; la moyenne ne l'est pas.

Moyenne et médiane, en une image. La moyenne est le **centre de gravité** de la distribution : elle tient compte de la distance de chaque valeur. La médiane est le **centre de position** : elle ne tient compte que de l'ordre. Quand la distribution est étirée d'un côté, la moyenne se laisse entraîner de ce côté ; la médiane reste au milieu des observations.

Deux autres résumés du centre servent régulièrement. Le **mode** est la valeur la plus fréquente : il a un sens pour une variable qui prend peu de valeurs (le nombre d'articles par commande), pas pour un montant en euros où presque toutes les valeurs sont distinctes (on le remplace alors par la **classe modale** d'un histogramme). La **moyenne tronquée** retire un pourcentage de valeurs de chaque extrémité avant de moyenner : c'est un compromis entre la moyenne (sensible aux extrêmes) et la médiane (qui les ignore toutes).

Voyons ces quatre résumés sur les 36 395 commandes de la boutique.

```
from scipy.stats import trim_mean
print(panier.agg(["mean", "median"]).round(2).to_dict())
print("moyenne tronquée à 5 % :", round(trim_mean(panier, 0.05), 2))
print("mode du nombre de lignes par commande :", lignes.groupby("id_commande").size().mode()[0])
```

```
{'mean': 100.38, 'median': 79.8}
moyenne tronquée à 5 % : 92.44
mode du nombre de lignes par commande : 2
```

L'écart entre moyenne et médiane se retrouve à grande échelle : **100,38 €** contre **79,80 €**, soit 21 € d'écart. La moyenne tronquée (92,44 €) tombe entre les deux. Et le fait le plus parlant est que **61 % des commandes sont en dessous de la moyenne** : la gérante voit passer une majorité de paniers inférieurs au « panier moyen » de la boutique. Par ailleurs, le nombre de lignes par commande est le plus souvent de **2** (c'est le mode, atteint par 35 % des commandes).

Quel résumé annoncer ?

Il n'y a pas de bon résumé en soi : il y a un résumé adapté à la **question posée**.

Question	Résumé adapté	Pourquoi
Combien une commande rapporte-t-elle en moyenne , pour prévoir le chiffre d'affaires d'un mois ?	la moyenne	total = moyenne × nombre de commandes ; la moyenne conserve le total
Que dépense un client typique ?	la médiane	elle n'est pas tirée par les grosses commandes
À partir de quel montant une commande est-elle « grosse » ?	un centile (par exemple le 95 ^e : 254 €)	on cherche un seuil, pas un centre
Quel est le nombre d'articles le plus courant ?	le mode	variable à peu de valeurs

Pour la gérante, la réponse honnête est donc : « le panier moyen est de 100 € ; **mais** une commande sur deux est inférieure à 80 €, et les 10 % de commandes les plus importantes représentent à elles seules **28 %** du chiffre d'affaires ». La moyenne sert à prévoir, la médiane à décrire. Les deux ensemble disent quelque chose que ni l'une ni l'autre ne dit seule.

Pour s'entraîner. Cahier, chapitre 1 : application 1.1, exercice 1.1.

1.1.2 Mesurer la dispersion

Deux boutiques peuvent avoir le même panier moyen de 100 € : l'une avec des paniers tous compris entre 90 € et 110 €, l'autre avec des paniers de 5 € à 900 €. Le centre ne les distingue pas. La **dispersion** mesure l'écart entre les observations.

L'étendue est la différence entre la plus grande et la plus petite valeur : $258,83 - 9,69 = 249,14$ € pour nos neuf commandes. Simple, mais fragile : elle ne dépend que de deux observations.

Les quartiles coupent les observations rangées en quatre parts égales. Le premier quartile Q_1 laisse 25 % des valeurs en dessous, le troisième Q_3 en laisse 75 %. Avec n valeurs rangées, la méthode la plus courante (celle d'Excel avec `QUARTILE.INCLUDE`, de R et de pandas par défaut) cherche la valeur à la position $1 + p(n - 1)$, en interpolant entre deux observations voisines si la position n'est pas entière. Pour $n = 9$: Q_1 est à la position $1 + 0,25 \times 8 = 3$, soit **48,74 €**, et Q_3 à la position $1 + 0,75 \times 8 = 7$, soit **132,37 €**. **L'écart interquartile** (EIQ) est $Q_3 - Q_1 = 83,63$ € : c'est l'étendue des 50 % de valeurs centrales, insensible aux extrêmes.

La variance et l'écart-type mesurent la distance **typique** des observations à la moyenne. On part des écarts $x_i - \bar{x}$, on les élève au carré (pour que les écarts positifs et négatifs ne s'annulent pas), on les additionne, et l'on divise par $n - 1$:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s = \sqrt{s^2}.$$

Le calcul sur nos neuf commandes, moyenne 102,78 € :

Commande x_i	Écart $x_i - \bar{x}$	Écart au carré
9,69	-93,09	8 665,7
22,56	-80,22	6 435,2
48,74	-54,04	2 920,3
50,27	-52,51	2 757,3
66,75	-36,03	1 298,2
128,25	25,47	648,7
132,37	29,59	875,6
207,56	104,78	10 978,8
258,83	156,05	24 351,6
Somme	0	58 931,5

La somme des écarts vaut 0 par construction (c'est ce qui définit la moyenne). La variance vaut $58\,931,5/8 \approx 7\,366,4 \text{ €}^2$ et l'écart-type $s = \sqrt{7\,366,4} \approx 85,83 \text{ €}$. L'écart-type s'exprime dans **la même unité** que les données, ce qui le rend interprétable : un panier s'écarte typiquement de 86 € de la moyenne.

Pourquoi diviser par $n - 1$ et pas par n ? Les écarts sont mesurés par rapport à la moyenne de l'échantillon, qui est précisément la valeur la plus proche des observations : ils sont mécaniquement un peu plus petits que les écarts à la vraie moyenne (inconnue). Diviser par $n - 1$ corrige ce biais. On dit aussi que, la moyenne étant connue, **seuls $n - 1$ écarts sont libres** : le dernier se déduit des autres, puisque leur somme vaut zéro. Pour des milliers d'observations, la différence entre n et $n - 1$ disparaît ; pour neuf observations, elle compte.

Enfin, le **coefficient de variation** $CV = s/\bar{x}$ exprime la dispersion **relativement** au centre. Il permet de comparer des grandeurs d'unités différentes (des paniers en euros et des délais en jours) : ici $85,83/102,78 \approx 0,84$.

Sur l'ensemble des commandes :

```
q1, q3 = panier.quantile([0.25, 0.75])
print("quartiles :", round(q1, 2), round(q3, 2), "| écart interquartile :", round(q3 - q1, 2))
print("écart-type :", round(panier.std(), 2), "| coefficient de variation :", round(panier.std() /
↳ panier.mean(), 2))
print("min et max :", panier.min(), panier.max())
```

```
quartiles : 42.9 135.4 | écart interquartile : 92.5
écart-type : 80.73 | coefficient de variation : 0.8
min et max : 2.32 987.9
```

Les paniers de la boutique ont un écart-type de **81 €** pour une moyenne de 100 € : un coefficient de variation de **0,80**, très élevé. Les 50 % de commandes centrales vont de 43 € à 135 € (EIQ de 92 €), alors que la plus petite vaut 2,32 € et la plus grande 988 €. Il y a de tout, et c'est pourquoi **la moyenne seule est un mauvais portrait** de la clientèle.

Pour s'entraîner. Cahier, chapitre 1 : application 1.1, exercice 1.2.

1.1.3 La forme : asymétrie, queues et valeurs aberrantes

Le troisième aspect d'une distribution est sa **forme**. Dessinons les paniers.

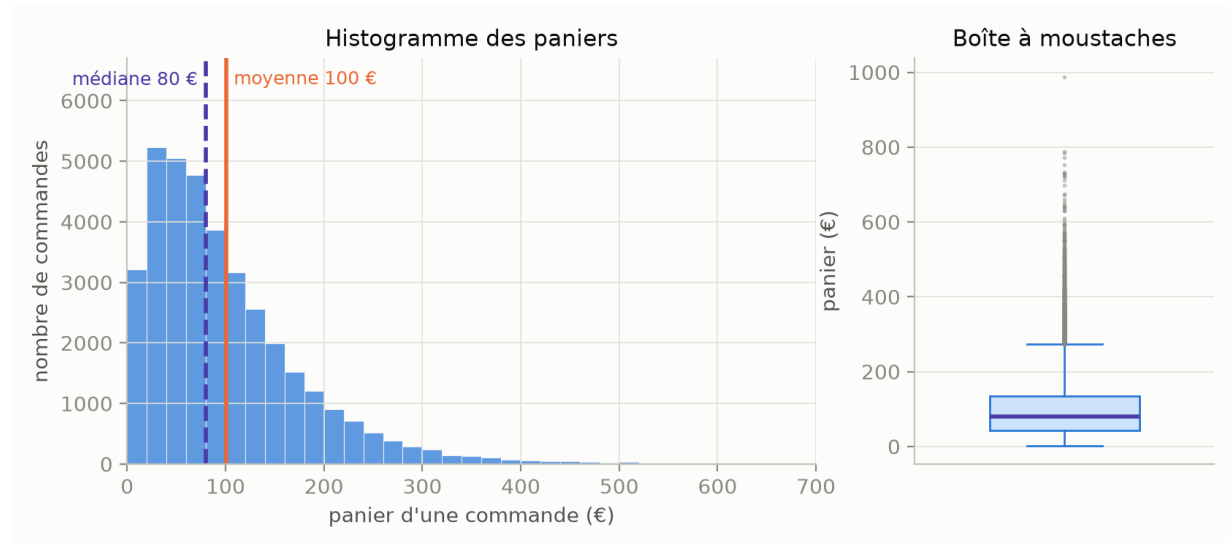


Figure 4: Les paniers des commandes de la boutique. À gauche, l'histogramme (classes de 20 €) : la distribution est étirée vers la droite, la moyenne (orange) est plus grande que la médiane (violet). À droite, la boîte à moustaches : la boîte va du premier au troisième quartile, le trait central est la médiane, les points au-dessus de la moustache sont les valeurs jugées « aberrantes » par la règle de 1,5 écart interquartile.

L'histogramme a une allure typique des montants : un **pic à gauche** (beaucoup de petits paniers) et une **longue traîne à droite** (quelques paniers très gros). On dit que la distribution est **asymétrique à droite** (ou « étalée à droite »). La **boîte à moustaches** résume la même information : la boîte contient la moitié centrale des commandes (Q_1 à Q_3), le trait épais est la médiane, les moustaches s'étendent jusqu'aux valeurs les plus éloignées qui ne sont pas jugées extrêmes, et les points isolés sont les extrêmes.

On mesure l'asymétrie par le **coefficient d'asymétrie** (*skewness*), la moyenne des écarts réduits élevés au cube :

$$g_1 = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3.$$

Il vaut 0 pour une distribution symétrique, est positif quand la traîne est à droite et négatif quand elle est à gauche. Le **coefficient d'aplatissement** (*kurtosis*, ici en « excès » : 0 pour une courbe normale) mesure le poids des **queues** : il est grand quand il y a plus de valeurs extrêmes que ne le prévoirait une courbe normale. Excel (COEFFICIENT.ASYMETRIE, KURTOSIS) et pandas utilisent la même version corrigée pour les petits échantillons.

```
print("asymétrie :", round(panier.skew(), 2), "| aplatissement (excès) :", round(panier.kurt(), 2))
```

```
asymétrie : 1.85 | aplatissement (excès) : 5.8
```

Les valeurs aberrantes : une règle, pas un verdict

Quand une valeur est-elle « aberrante » ? La règle de **Tukey** (celle qui dessine les moustaches) déclare extrême toute valeur située à plus de **1,5 écart interquartile** au-delà des quartiles :

$$x < Q_1 - 1,5 \times \text{EIQ} \quad \text{ou} \quad x > Q_3 + 1,5 \times \text{EIQ}.$$

Pour nos neuf commandes, la borne supérieure est $132,37 + 1,5 \times 83,63 \approx 257,8 \text{ €}$: la commande de 258,83 € la dépasse **d'un euro**. Pour l'ensemble des commandes, la borne inférieure est négative (aucun panier ne peut être « trop petit ») et la borne supérieure vaut :

$135,40 + 1,5 \times 92,50 = 274,15 \text{ €}$. Au-delà, la règle signale **1 400 commandes**, soit 3,8 % du total ; l'asymétrie vaut **1,85** et l'aplatissement **5,8**, confirmant une traîne droite lourde.

« **Aberrant** » ne veut pas dire « faux ». La règle de Tukey signale des valeurs **inhabituelles**, pas des erreurs. Une commande de 700 € passée par un client qui meuble son salon est parfaitement valide ; une commande de 2 588,30 € parce que la virgule a glissé est une erreur de saisie. Devant une valeur extrême, on se pose **trois questions dans cet ordre** : est-ce une *erreur* (à corriger ou à retirer) ? est-ce un cas *légitime mais rare* (à garder, en le sachant) ? ou est-ce un cas d'un **autre type** (une commande de professionnel, une revente) qui mérite une analyse à part ? On **ne supprime jamais** une valeur au seul motif qu'elle est grande, et l'on dit toujours ce que l'on a fait.

Le **résumé des cinq nombres** (minimum, Q_1 , médiane, Q_3 , maximum) est le portrait le plus économique d'une distribution : 2,32 ; 42,90 ; 79,80 ; 135,40 ; 987,90. Il montre l'asymétrie d'un coup d'œil : la médiane est plus proche de Q_1 que de Q_3 , et le maximum est très loin de Q_3 .

Pour s'entraîner. Cahier, chapitre 1 : application 1.1, exercice 1.3.

1.1.4 Quand les moyennes mentent

Une moyenne est un calcul simple, ce qui la rend dangereuse : on la calcule sans réfléchir à **ce qu'elle moyenne**. Deux pièges reviennent sans cesse.

La moyenne des moyennes

La gérante demande : « Quel est le taux de retour de la boutique ? » Vous avez le taux de chacun des trois canaux, calculé sur les lignes vendues en 2023-2025 :

Canal	Lignes vendues	Lignes retournées	Taux de retour
Boutique	39 362	1 211	3,08 %
Réseaux	9 071	605	6,67 %
Site	35 472	3 186	8,98 %

Faire la moyenne des trois taux donne 6,24 %. Mais le taux de **toute la boutique** est le nombre total de retours divisé par le nombre total de lignes : $5\,002 / 83\,905 = 5,96 \text{ %}$. L'écart (0,28 point) vient de ce que la moyenne simple traite les trois canaux **à égalité**, alors que les canaux n'ont pas le même poids : la Boutique vend près de la moitié des lignes, et son taux est le plus bas. **Une moyenne de pourcentages doit être pondérée** par la taille de chaque groupe (c'est la moyenne pondérée, que la section 1.5.3 détaille). La règle pratique : **repartez toujours des totaux** (retours et lignes), jamais des ratios.

Le paradoxe de Simpson

Le second piège est plus troublant : **une tendance observée sur l'ensemble peut s'inverser dans chaque sous-groupe**. Voici un exemple volontairement inventé (les chiffres ne viennent pas des données de la boutique). La boutique a testé deux campagnes d'e-mail, A et B, auprès de 1 500 clients chacune. On compte les clients qui ont acheté dans la semaine, en séparant les clients fidèles (titulaires d'une carte) des nouveaux :

	Campagne A		Campagne B	
	envois	achats	envois	achats
Clients fidèles	1 200	240 (20 %)	300	66 (22 %)
Nouveaux clients	300	15 (5 %)	1 200	78 (6,5 %)
Ensemble	1 500	255 (17,0 %)	1 500	144 (9,6 %)

Dans **chaque** groupe, la campagne B convertit mieux (22 % contre 20 % chez les fidèles, 6,5 % contre 5 % chez les nouveaux). Et pourtant, **sur l'ensemble**, A l'emporte nettement (17,0 % contre 9,6 %). Il n'y a pas d'erreur de calcul : la campagne B a été envoyée surtout à des nouveaux clients, qui achètent beaucoup moins par nature. Le **type de client** est un **facteur de confusion** : il influence à la fois la campagne reçue et le résultat.

La leçon n'est pas « il faut toujours découper » (on peut découper à l'infini et trouver n'importe quoi), mais : **avant de comparer des groupes, demandez-vous s'ils sont comparables**. Les sections 1.3 et 1.4 reviendront sur cette question, qui est au cœur de l'analyse de données.

À retenir. (1) Moyenne et médiane répondent à des questions différentes : la moyenne conserve le total, la médiane décrit le cas typique. (2) Un centre sans dispersion n'est pas un résumé. (3) Une valeur extrême est une question, pas une erreur. (4) Une moyenne de ratios se calcule à partir des totaux ; une comparaison de groupes suppose des groupes comparables.

Pour s'entraîner. Cahier, chapitre 1 : application 1.2, exercices 1.4 et 1.5.

1.1.5 Les mêmes chiffres dans Excel et dans R

Un chiffre qui sort d'un seul outil est un chiffre à vérifier. Voyons comment obtenir les mêmes résumés dans **Excel** et dans **R**, et pourquoi ils concordent.

Dans Excel, on écrit les neuf paniers dans une colonne, puis chaque résumé est une formule. Voici la feuille avec les formules saisies en français ; les formules ont été **vérifiées en les faisant calculer par LibreOffice Calc**, et le dessin est une maquette (ce n'est pas une capture d'Excel).

Les résultats concordent avec le calcul à la main : 102,78 € ; 66,75 € ; 48,74 € ; 132,37 € ; 85,83 €. Trois remarques de lecture :

- **QUARTILE.INCLUDE** utilise l'interpolation « inclusive » vue plus haut (position $1 + p(n - 1)$) ; il existe aussi **QUARTILE.EXCLUDE**, qui donne **d'autres quartiles** sur de petits jeux de données. Précisez toujours la version que vous utilisez.
- **ECARTYPE.STANDARD** divise par $n - 1$ (écart-type d'échantillon) ; **ECARTYPE.PEARSON** divise par n . Sur un échantillon, c'est le premier qu'il faut.
- **MOYENNE.REDUITE(plage ; proportion)** retire la proportion indiquée **au total** (moitié de chaque côté) et arrondit le nombre de valeurs retirées vers le bas, à un nombre pair : avec

D2

fx

=MOYENNE (A2:A10)

	A	B	C	D	E
1	Panier		Statistique	Résultat	Formule (Excel en français)
2	9,69		Moyenne	102,78	=MOYENNE(A2:A10)
3	207,56		Médiane	66,75	=MEDIANE(A2:A10)
4	132,37		Premier quartile	48,74	=QUARTILE.INCLURE(A2:A10;1)
5	258,83		Troisième quartile	132,37	=QUARTILE.INCLURE(A2:A10;3)
6	66,75		Écart-type	85,83	=ECARTYPE.STANDARD(A2:A10)
7	22,56		Asymétrie	0,84	=COEFFICIENT.ASYMETRIE(A2:A10)
8	50,27		Moy. tronquée (7 valeurs)	93,79	=MOYENNE.REDUITE(A2:A10;0,25)
9	48,74				
10	128,25				

Feuil1

Figure 5: Maquette d'une feuille Excel : les neuf paniers en colonne A, et les résumés calculés par des formules (colonnes C à E). La maquette est dessinée à partir des valeurs calculées par LibreOffice Calc : ce n'est pas une capture d'écran d'Excel.

9 valeurs et 0,25, on retire 2,25 → 2 valeurs (la plus petite et la plus grande) ; la moyenne tronquée de la feuille porte donc sur 7 valeurs.

Dans R, les mêmes résumés tiennent en quelques lignes. Calculons-les sur les paniers de l'année 2025 (le même fichier, un autre outil) :

```
cmd <- read.csv("donnees/commandes.csv"); lig <- read.csv("donnees/lignes_commande.csv")
panier <- tapply(lig$montant, lig$id_commande, sum)
p25 <- panier[as.character(cmd$id_commande[substr(cmd$date_commande, 1, 4) == "2025"])]
c(moyenne = mean(p25), mediane = median(p25), ecart_type = sd(p25))
quantile(p25, c(0.25, 0.75))      # type 7 : même méthode que pandas et QUARTILE.INCLUDE
```

```
moyenne  mediane ecart_type
102.32996  81.61500  82.32644
 25%      75%
44.09 137.72
```

En pandas sur les mêmes 12 946 commandes de 2025 : moyenne **102,33**, médiane **81,62**, écart-type **82,33**, quartiles 44,09 et 137,72 : ce sont les nombres que R vient d'afficher (à l'arrondi d'affichage près). Que les trois outils s'accordent n'est pas une formalité : c'est la première **vérification croisée**, que ce livre pratique autant que possible. Quand deux outils donnent deux résultats, la différence est presque toujours une **convention** (quartiles inclusifs ou exclusifs, n ou $n - 1$) ou une **donnée différente** (lignes filtrées, valeurs vides), et il faut la trouver avant de publier.

En pratique. Excel pour explorer et partager, SQL pour interroger de gros volumes, Python ou R pour automatiser et reproduire : le chapitre 2 détaille Excel, le chapitre 3 SQL et le chapitre 4 Python et R. Ce volume apprend à **passer de l'un à l'autre** en retrouvant les mêmes chiffres.

Pour s'entraîner. Cahier, chapitre 1 : application 1.3.

1.2 Distributions et courbe normale

Dire qu'un panier moyen est de 100 € ne dit pas **quelle proportion** de clients dépense plus de 200 €. Pour répondre à ce genre de question, il faut connaître la **forme** de la répartition des valeurs, c'est-à-dire une **distribution**. Cette section présente les quelques distributions qui suffisent à décrire la plupart des phénomènes que l'on rencontre en entreprise : des **comptages** (retours, commandes), des **mesures** (âges) et des **montants**. Elle apprend aussi à reconnaître quand aucune de ces lois ne convient, ce qui arrive plus souvent qu'on ne le croit.

1.2.1 Qu'est-ce qu'une distribution ?

Une **distribution** décrit à **quelle fréquence** chaque valeur (ou chaque intervalle de valeurs) apparaît. On en distingue deux grandes familles :

- une variable **discrète** ne prend que des valeurs isolées (le nombre de lignes d'une commande : 1, 2, 3...) ; sa distribution est la liste des probabilités $P(X = k)$;
- une variable **continue** prend toutes les valeurs d'un intervalle (le panier en euros, une température) ; on ne parle plus de la probabilité d'une valeur exacte (nulle) mais de la probabilité de tomber **dans un intervalle**, qui est l'**aire** sous une courbe appelée **densité**.

L'histogramme est l'image **observée** d'une distribution ; une loi théorique (binomiale, de Poisson, normale...) en est un **modèle**, c'est-à-dire une formule à un ou deux paramètres qui reproduit la forme. Le modèle est pratique : il permet de calculer des probabilités qu'on ne peut pas lire directement sur les données (la probabilité d'un événement rare, par exemple), à condition que **ses hypothèses tiennent**.

La plus simple des lois est la loi **uniforme** : toutes les valeurs ont la même probabilité. La minute à laquelle une commande est passée (de 0 à 59) en est un bon exemple : rien ne distingue 17 heures 03 de 17 heures 48. Les commandes tombent dans les quinze premières minutes de l'heure avec une fréquence de 25,0 %, très près des 25 % que prédit l'uniforme (15/60).

1.2.2 Compter : les lois binomiale et de Poisson

La loi binomiale : combien de retours sur n lignes ?

La gérante veut savoir si un lot de **50 lignes vendues sur le Site** peut contenir 10 retours ou plus, ce qui serait un signal d'alerte. Le taux de retour du Site est de 8,98 %. Chaque ligne est retournée ou non (deux issues), indépendamment des autres, avec la même probabilité p : c'est le schéma de la loi **binomiale** de paramètres $n = 50$ et $p = 0,0898$.

La probabilité d'obtenir exactement k retours est

$$P(K = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad E(K) = np, \quad \text{Var}(K) = np(1 - p).$$

Le coefficient $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ compte les façons de choisir quelles lignes sont retournées. Calculons à la main les deux cas les plus simples : **aucun retour** ($k = 0$), $P(K = 0) = (1 - p)^{50} = 0,9102^{50}$, soit environ 0,0091 ou 0,9 % ; et le nombre de retours **attendu**, $np = 50 \times 0,0898 = 4,49$, avec un écart-type $\sqrt{np(1 - p)}$ de 2,02. Obtenir 10 retours ou plus, c'est donc s'éloigner de la moyenne de plus de deux écarts-types : un événement possible mais rare.

```
from scipy.stats import binom
print("P(0 retour) :", round(binom.pmf(0, 50, 0.0898), 4))
print("P(10 retours ou plus) :", round(binom.sf(9, 50, 0.0898), 4))    # sf(9) = P(K > 9)
```

$P(0 \text{ retour}) : 0.0091$
 $P(10 \text{ retours ou plus}) : 0.0123$

Pour **vérifier** que le modèle décrit bien les données, on tire 20 000 fois 50 lignes du Site au hasard dans les données réelles et l'on compte les retours. La figure de gauche ci-dessous compare les fréquences observées aux probabilités de la loi : elles se superposent presque parfaitement. La fréquence observée de « 10 retours ou plus » est de 1,24 % pour 1,23 % attendus.

La loi de Poisson : combien d'articles de plus dans un panier ?

Quand on **compte des événements dans un intervalle** (une journée, une commande, une heure) et que ces événements surviennent **indépendamment** et à un rythme moyen constant λ , on obtient la loi de **Poisson** :

$$P(N = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad E(N) = \text{Var}(N) = \lambda.$$

Sa signature est que **la variance est égale à la moyenne**. Vérifions sur un cas simple : le nombre d'articles **en plus du premier** dans une commande (nombre de lignes moins un). Sa moyenne vaut 1,31 et sa variance 1,31 : deux nombres voisins, ce qui est bon signe. À la main, avec $\lambda = 1,3$: $P(N = 0) = e^{-1,3} \approx 0,273$, $P(N = 1) = 1,3 e^{-1,3} \approx 0,354$, $P(N = 2) = \frac{1,3^2}{2} e^{-1,3} \approx 0,230$. La figure de droite compare fréquences observées et loi de Poisson : l'accord est excellent.

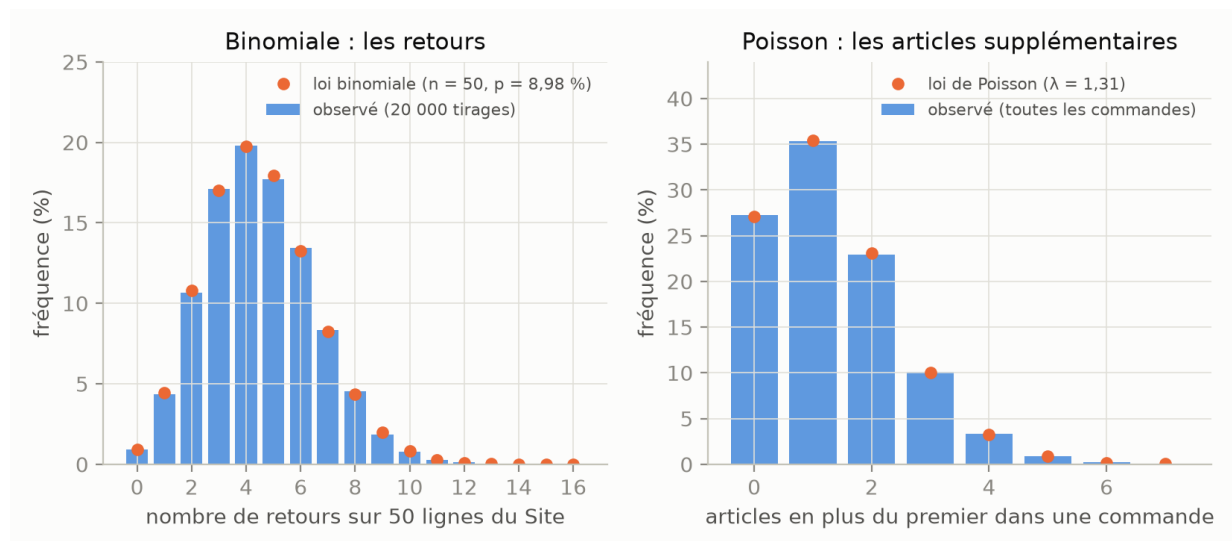


Figure 6: Deux lois de comptage face aux données de la boutique. À gauche : le nombre de retours dans un lot de 50 lignes du Site (barres bleues : 20 000 tirages dans les données ; points orange : loi binomiale). À droite : le nombre d'articles supplémentaires dans une commande (barres : fréquences observées ; points : loi de Poisson de même moyenne).

Quand la loi de Poisson ne convient plus

Appliquons maintenant la même loi à une quantité voisine, le **nombre de commandes par jour**. La moyenne est de 33,2 commandes, mais la variance vaut 169 : **5,1 fois** la moyenne. Ce n'est plus du Poisson. La raison est que les journées ne sont pas comparables : un samedi de décembre en période de soldes n'a rien à voir avec un mardi d'août. La loi de Poisson suppose un **rythme constant** ; ici, le rythme change tous les jours, et mélanger des rythmes différents **gonfle la variance**.

Si l'on se restreint à des journées **comparables** (les mardis hors promotion de septembre et octobre, sur trois ans), le rapport variance/moyenne tombe à 1,3, beaucoup plus près de 1 (il reste un peu au-dessus, car l'activité croît d'une année sur l'autre ; avec 27 journées seulement, l'estimation est elle-même bruitée). La leçon est générale : **un modèle n'est valable que dans les conditions où ses hypothèses tiennent**, et la comparaison variance/moyenne est un test rapide pour la loi de Poisson.

1.2.3 La courbe normale

La loi **normale** (ou de Laplace-Gauss, la « courbe en cloche ») est la plus célèbre. Elle est caractérisée par deux paramètres : la **moyenne** μ (le centre de la cloche) et l'**écart-type** σ (sa largeur). Sa densité est

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Il n'est pas nécessaire de retenir la formule. Il faut retenir **trois propriétés** : la courbe est **symétrique** autour de μ ; elle est **entièrement déterminée** par μ et σ ; et elle obéit à la règle **68-95-99,7** :

- environ **68 %** des valeurs se trouvent à moins de **1 écart-type** de la moyenne ;
- environ **95 %** à moins de **2 écarts-types** ;
- environ **99,7 %** à moins de **3 écarts-types**.

L'âge des clients de la boutique en offre un exemple. Sur les 6 000 clients, l'âge moyen est de 43,3 ans et l'écart-type de 13,7 ans. La règle prédit 68 % des clients entre 30 et 57 ans ; on observe **66,3 %**. À deux écarts-types (de 16 à 71 ans) on observe **97,4 %** (prédit : 95 %), et à trois écarts-types **99,8 %** (prédit : 99,7 %). L'accord est bon : l'âge est une grandeur à peu près normale. (Le pic isolé à 18 ans, visible sur la figure, est un artefact de la simulation : les clients « plus jeunes » ont été ramenés à 18 ans. Dans des données réelles, un tel pic signale une **coupure** ou une valeur par défaut, et mérite une vérification.)

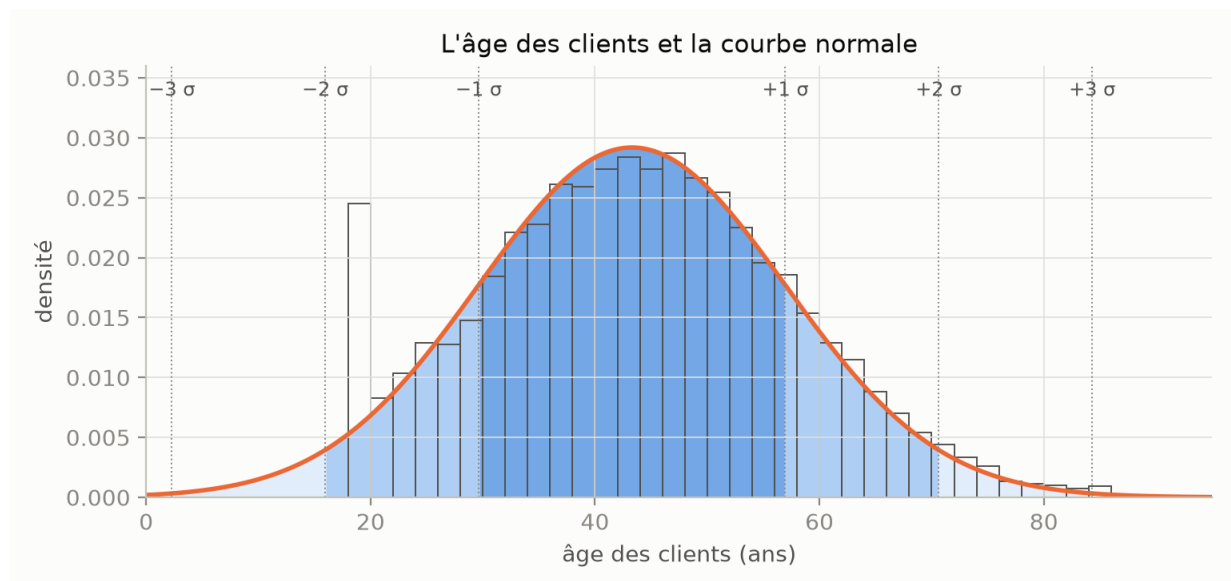


Figure 7: L'âge des clients (histogramme, contour gris) et la courbe normale de même moyenne et de même écart-type (orange). Les trois zones bleues correspondent à ± 1 , ± 2 et ± 3 écarts-types : elles contiennent environ 68 %, 95 % et 99,7 % des valeurs.

Le score z : mesurer en écarts-types

Comment dire qu'une cliente de 70 ans est « très âgée » pour la clientèle ? En la situant **par rapport à la moyenne, en nombre d'écarts-types**. C'est le **score z** :

$$z = \frac{x - \mu}{\sigma}.$$

Pour 70 ans : $z = (70 - 43,3) / 13,7 \approx 1,95$.

Un score de 1,95 signifie que la cliente se situe à presque deux écarts-types **au-dessus** de la moyenne. Les tables de la loi normale (ou la fonction `LOI.NORMALE.STANDARD.N` d'Excel, avec l'option cumulative à `VRAI` : à vérifier dans votre version) donnent la probabilité d'être en dessous d'un score z : pour $z = 1,95$, **97,4 %** (c'est le **centile** de la cliente). Environ **2,5 %** des clients devraient donc avoir 70 ans ou plus si l'âge était exactement normal ; on en observe **3,1 %**. Le score z a un avantage précieux : il permet de **comparer des grandeurs d'unités différentes** (un panier de 250 € et un âge de 70 ans) en les ramenant à la même échelle.

Pourquoi la normale est-elle partout ? Quand on **additionne** (ou moyenne) beaucoup de petites influences indépendantes, le résultat est approximativement normal, même si chaque influence ne l'est pas. C'est ce qu'établit le **théorème central limite**, que la section 1.3 illustre. Voilà pourquoi la taille ou l'âge d'une population suivent souvent une cloche, et pourquoi les **moyennes** d'échantillons sont presque toujours normales, quelle que soit la forme des données d'origine.

1.2.4 Quand la normale ne convient pas

Les montants des paniers, eux, ne sont **pas** normaux : nous l'avons vu, leur histogramme est étiré à droite (asymétrie 1,85). Un modèle normal ajusté à ces paniers prévoirait des paniers **négatifs**. Comment le **vérifier** autrement qu'« à l'œil » ? Avec un **diagramme quantile-quantile** (*Q-Q plot*). On compare chaque quantile des données au quantile correspondant de la loi normale : si les données sont normales, les points s'alignent sur une droite ; une courbure révèle l'écart.

Les paniers forment une courbe très nette (corrélation avec la droite de 0,925) ; leur **logarithme** s'aligne bien mieux (0,986). Quand le logarithme d'une variable est à peu près normal, la variable est dite **log-normale** : c'est le cas typique des montants, des revenus, des tailles d'entreprises, qui résultent de **multiplications** d'effets (un prix multiplié par une quantité, multiplié par une remise...) plutôt que d'additions. Le modèle normal prévoirait 11 % de paniers négatifs, ce qui est absurde ; le modèle log-normal ne le peut pas. Le diagramme montre pourtant que la log-normale n'est elle non plus **qu'une approximation** (asymétrie résiduelle de -0,71) : les paniers très petits sont moins nombreux que le modèle ne le prévoit.

Que faire d'une variable qui n'est pas normale ? Trois attitudes sont courantes. On peut **transformer** (le logarithme) pour se ramener à une cloche. On peut **changer de résumé** : médiane et centiles au lieu de moyenne et écart-type, ce qui ne suppose aucune forme. Ou l'on peut **ne rien supposer** et laisser les données parler (simulation, méthodes dites « non paramétriques »). L'important est de **ne pas appliquer par réflexe** une règle 68-95-99,7 à une variable qui n'est pas en cloche.

Quelle loi pour quel phénomène ?

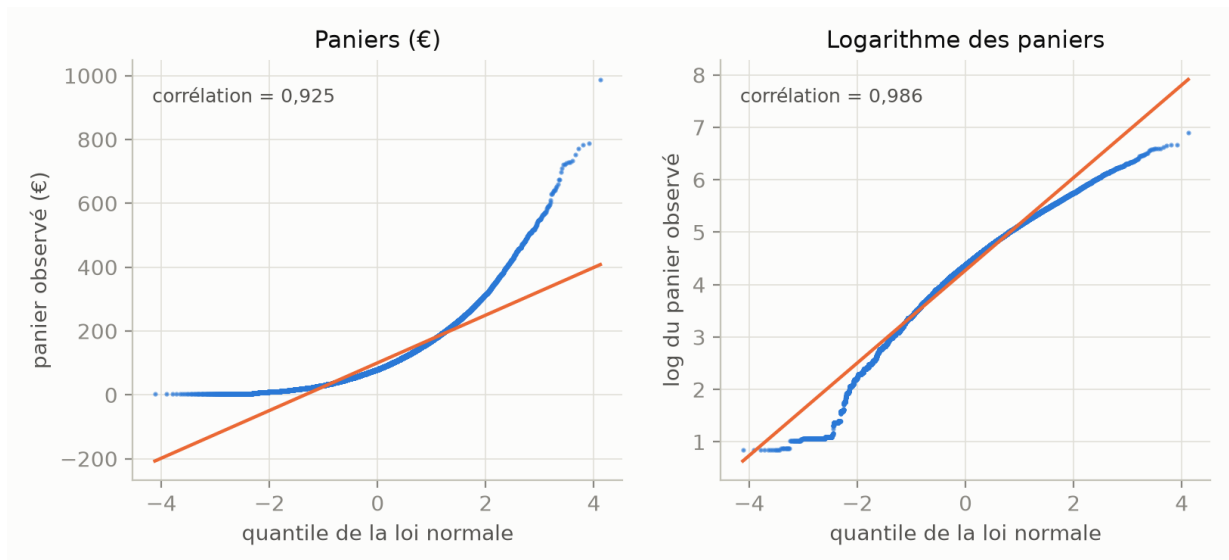


Figure 8: Diagrammes quantile-quantile. À gauche, les paniers : les points s'écartent nettement de la droite (la traîne droite est bien plus longue que celle d'une normale). À droite, leur logarithme : l'alignement est bien meilleur, mais pas parfait (les petits paniers sont plus rares que ne le prévoit une normale).

Phénomène de la boutique	Loi plausible	Paramètres	À vérifier
Minute d'une commande	uniforme	aucun	fréquences égales
Retours parmi n lignes	binomiale	n, p (9,0 % pour le Site)	indépendance, p constant
Articles supplémentaires par commande	Poisson	λ (1,3)	variance $\approx$ moyenne
Commandes par jour	Poisson par tranche homogène	λ qui varie	variance $\approx$ moyenne dans la tranche
Âge des clients	normale	μ (43 ans), σ (14 ans)	Q-Q plot, symétrie
Panier d'une commande	asymétrique ; log-normale en première approximation	moyenne et écart-type du logarithme	Q-Q plot du logarithme

À retenir. (1) Une distribution est un modèle de la forme des données ; elle permet de calculer des probabilités, **si ses hypothèses tiennent**. (2) Binomiale pour compter des succès sur n essais, Poisson pour compter des événements dans un intervalle (variance = moyenne). (3) La normale est décrite par μ et σ ; le score z mesure en écarts-types ; la règle 68-95-99,7 n'a de sens que pour une variable en cloche. (4) Les montants sont asymétriques : prenez le logarithme, ou résumez par la médiane et les centiles ; vérifiez par un Q-Q plot.

Pour s'entraîner. Cahier, chapitre 1 : application 1.4, exercices 1.6 et 1.7.

1.3 Échantillonnage et erreur d'échantillonnage

Presque toute analyse repose sur un **échantillon** : une partie des clients, des commandes, des réponses à une enquête, dont on veut tirer une conclusion sur **l'ensemble**. La gérante demande : « Vous avez regardé 100 commandes et trouvé un panier moyen de 98 € : est-ce que le vrai chiffre peut être de 110 € ? » Pour répondre, il faut savoir de combien un chiffre calculé sur une partie peut s'écarter du chiffre « vrai ». C'est le sujet de cette section : **quantifier l'erreur due au hasard du tirage**, puis reconnaître les erreurs que le hasard n'explique pas.

1.3.1 Population, échantillon, paramètre, statistique

Quatre mots suffisent pour tout ce qui suit.

- La **population** est l'ensemble sur lequel on veut conclure (toutes les commandes de 2025, tous les clients de la boutique).
- L'**échantillon** est la partie effectivement observée, de taille n .
- Un **paramètre** est une caractéristique de la population (la moyenne μ des paniers de 2025), fixe mais généralement inconnue.
- Une **statistique** est une caractéristique de l'échantillon (la moyenne $\bar{x}$ des 100 paniers observés), connue, mais **qui change d'un échantillon à l'autre**.

On utilise la statistique $\bar{x}$ pour **estimer** le paramètre μ . L'**erreur d'échantillonnage** est l'écart $\bar{x} - \mu$ dû uniquement au fait que l'on n'a pas vu toute la population : un autre tirage de 100 commandes aurait donné un autre $\bar{x}$.

Cette boutique a un luxe que vous aurez rarement : les 12 946 commandes de 2025 sont toutes dans le fichier, et leur panier moyen vaut **102,33 €**. Ici, pour une question sur 2025, il n'y a donc pas d'échantillonnage à faire. Nous allons pourtant **faire comme si** nous ne pouvions voir que 100 commandes, parce que c'est le meilleur moyen de voir la méthode fonctionner : on connaît la réponse exacte, on peut juger l'estimation. Deux remarques de fond avant de commencer :

- **Un recensement n'est pas la fin de l'incertitude.** Les commandes de 2025 sont une population complète pour décrire 2025 ; elles sont un **échantillon** de ce que sera 2026. Dès que l'on généralise à l'avenir (« le panier moyen sera de... »), on est de nouveau en situation d'échantillonnage.
- « **Population** » est un **choix de l'analyste**, pas une donnée : si la question porte sur les clients actifs, les clients inscrits depuis dix ans ne font pas partie de la population.

1.3.2 L'erreur d'échantillonnage en action

Simulons le travail de l'analyste qui ne voit que 100 commandes. On tire au hasard 100 commandes de 2025, on calcule leur moyenne, et l'on **recommence 1 000 fois**.

```
import numpy as np
rng = np.random.default_rng(1)
ids25 = commandes.loc[commandes["date_commande"] >= "2025-01-01", "id_commande"]
pop = panier.loc[ids25].values # la population : tous les paniers de 2025
moyennes = [rng.choice(pop, 100, replace=False).mean() for _ in range(1000)]
print("moyenne des moyennes :", round(np.mean(moyennes), 2), "| vraie moyenne :", round(pop.mean(),
    ↳ 2))
print("écart-type des moyennes :", round(np.std(moyennes), 2), "| σ/√n :", round(pop.std() / 10, 2))
```

```
moyenne des moyennes : 102.17 | vraie moyenne : 102.33
écart-type des moyennes : 8.11 | σ/√n : 8.23
```


Trois constats. D'abord, **les 1 000 moyennes sont toutes différentes** : de 79,5 € à 132,1 €, alors que la vraie moyenne est 102,3 €. Ensuite, elles sont **centrées sur la vraie valeur** (leur moyenne vaut 102,17 €) : tirer au hasard ne crée pas de biais. Enfin, leur dispersion, l'**erreur type** (*standard error*), vaut 8,11 € : elle est très proche de $\sigma/\sqrt{n}$ (8,23 €). C'est la formule centrale de cette section :

$$\text{erreur type de } \bar{x} = \frac{\sigma}{\sqrt{n}}.$$

L'erreur type dit **de combien la moyenne d'un échantillon s'écarte typiquement de la vraie moyenne**. Elle décroît avec la **racine** de n : pour diviser l'erreur par 2, il faut **quadrupler** l'échantillon ; pour la diviser par 10, il faut le multiplier par 100. Les rendements décroissent vite. Voici l'erreur observée et l'erreur prévue pour quatre tailles d'échantillon :

Taille n	Erreur type observée (1 000 tirages)	$\sigma/\sqrt{n}$
10	25,4 €	26,0 €
30	14,8 €	15,0 €
100	8,3 €	8,2 €
400	4,1 €	4,1 €

La loi des grands nombres. Quand n augmente, $\bar{x}$ se rapproche de μ . C'est ce que dit l'erreur type : elle tend vers zéro. En pratique : la moyenne de 10 commandes tirées au hasard vaut 68,3 €, celle de 100 commandes 98,1 €, celle de 1 000 commandes 97,2 € et celle des 12 946 commandes 102,3 €. Mais la loi ne dit pas **à quelle vitesse** : c'est le rôle de l'erreur type. Et elle suppose un **tirage au hasard** : les 1 000 *premières* commandes de l'année (en janvier, pendant les soldes) ont un panier moyen de 92,6 €, loin de la vérité. Ce n'est pas un échantillon, c'est un morceau de calendrier.

La forme de la distribution des moyennes : le théorème central limite

Les paniers eux-mêmes ont une distribution très asymétrique. Mais regardons la distribution des **moyennes** de plusieurs paniers.

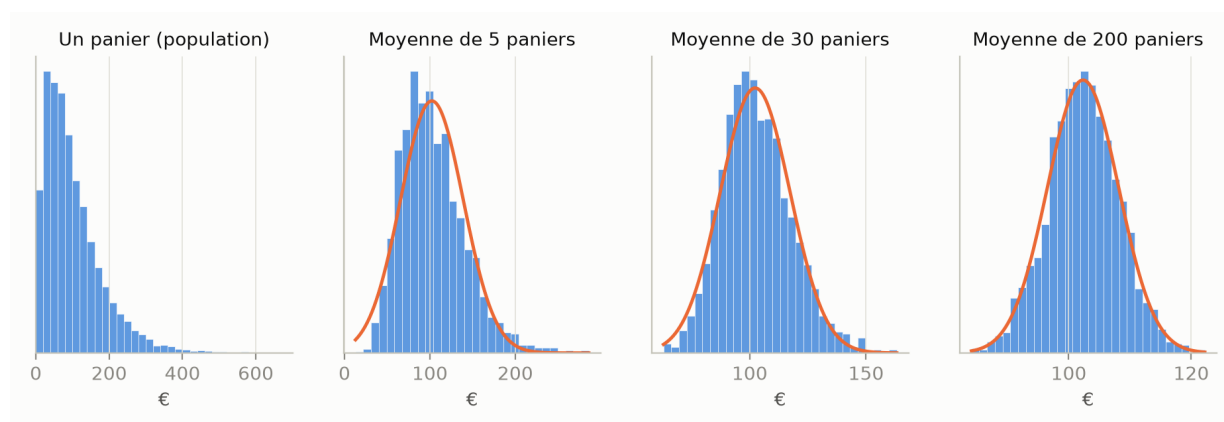


Figure 9: Le théorème central limite sur les paniers de 2025. À gauche, la distribution d'un panier (très asymétrique). Ensuite, la distribution de la moyenne de 5, de 30 puis de 200 paniers, sur 2 000 tirages chacune : elle devient de plus en plus symétrique et se rapproche de la courbe normale (orange) de moyenne μ et d'écart-type $\sigma/\sqrt{n}$.

Avec 5 paniers, la distribution de la moyenne reste étirée à droite (asymétrie 0,88) ; avec 30, elle est déjà presque symétrique (0,43) ; avec 200, elle épouse la courbe normale (0,04). C'est le **théorème central limite** : la moyenne d'un grand nombre d'observations indépendantes suit approximativement une loi normale, quelle que soit la forme des données d'origine, de moyenne μ et d'écart-type $\sigma/\sqrt{n}$. C'est lui qui justifie tous les intervalles de confiance et tous les tests des chapitres suivants. Il demande, en contrepartie, que n soit assez grand : plus la distribution d'origine est asymétrique, plus il faut d'observations (quelques dizaines suffisent ici ; pour des montants très concentrés sur quelques gros clients, il en faudrait bien davantage).

Pour s'entraîner. Cahier, chapitre 1 : application 1.5, exercice 1.8.

1.3.3 L'intervalle de confiance d'une moyenne

Une moyenne seule est une estimation **ponctuelle** : on ne sait pas si elle est précise. Un **intervalle de confiance** (IC) lui ajoute une fourchette. Puisque $\bar{x}$ est à peu près normale, centrée sur μ avec l'écart-type $\sigma/\sqrt{n}$, 95 % des échantillons donnent une moyenne à moins de **deux erreurs types** de la vraie valeur. On en déduit l'intervalle :

$$\bar{x} \pm t \frac{s}{\sqrt{n}},$$

où s est l'écart-type de l'échantillon (on ne connaît pas σ) et t un coefficient (1,96 pour un grand n ; un peu plus pour un petit n , d'après la **loi de Student** : 2,306 pour $n = 9$).

À la main, sur nos neuf commandes (moyenne 102,78 €, écart-type 85,83 €) : l'erreur type vaut $85,83/\sqrt{9} = 28,61$ €, la marge $2,306 \times 28,61 \approx 65,98$ €, d'où l'intervalle [36,8 € ; 168,8 €]. Il est **immense**, et c'est normal : avec neuf commandes très dispersées, on ne sait presque rien du vrai panier moyen.

Avec un échantillon de **100 commandes** de 2025, tiré au hasard :

```
from outils_ch01 import ic_moyenne
echantillon = np.random.default_rng(12).choice(pop, 100, replace=False)
bas, haut = ic_moyenne(echantillon)
print("moyenne :", round(echantillon.mean(), 2), "| IC à 95 % : [", round(bas, 1), " ; ", round(haut, 1), " ]")
```

```
moyenne : 110.94 | IC à 95 % : [ 92.9 ; 129.0 ]
```

On a trouvé 110,9 € avec un écart-type de 91,1 € ; l'intervalle à 95 % est [92,9 € ; 129,0 €], de **demi-largeur 18,1 €**. La vraie moyenne (102,3 €) se trouve bien dans l'intervalle. La gérante peut donc être informée : avec 100 commandes, le panier moyen est connu à environ **plus ou moins 18 €**, soit 16 %. Avec un panier observé de 98 € et la même précision, « 110 € » ne serait donc pas exclu : il se trouverait à l'intérieur de l'intervalle.

Que signifie « 95 % » ?

Le « 95 % » **ne décrit pas** la probabilité que μ soit dans *cet* intervalle (une fois l'intervalle calculé, μ y est ou n'y est pas). Il décrit la **méthode** : si l'on répétait l'échantillonnage un grand nombre de fois, **95 % des intervalles construits ainsi contiendraient** la vraie valeur. Vérifions-le : on construit 1 000 intervalles à partir de 1 000 échantillons de 100 commandes.

Sur les 1 000 intervalles, **94,2 %** contiennent la vraie moyenne : 58 la manquent, soit un peu plus que les 5 % annoncés (le théorème central limite n'est qu'une approximation pour des paniers aussi asymétriques). Sur les 40 premiers de la figure, quelques-uns, en rouge, la manquent : c'est

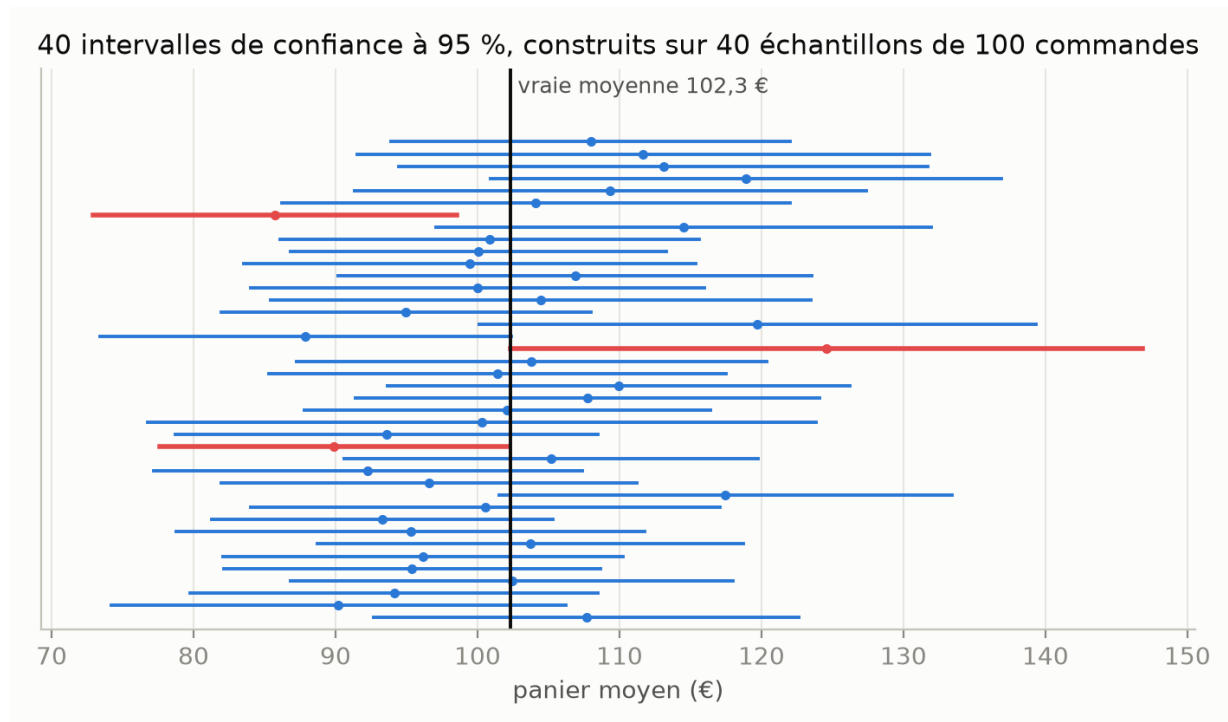


Figure 10: Quarante intervalles de confiance à 95 %, chacun construit sur un échantillon différent de 100 commandes. La ligne verticale est la vraie moyenne ; les intervalles bleus la contiennent, les intervalles rouges la manquent.

le **prix** d'un niveau de confiance de 95 % et non de 100 %. Pour viser 99 %, on élargit l'intervalle (coefficient 2,58 au lieu de 1,96) ; pour un intervalle plus étroit, on paie par un niveau de confiance plus faible.

Trois lectures fausses à éviter. (1) « Il y a 95 % de chances que μ soit dans l'intervalle » : le hasard est dans l'échantillon, pas dans μ . (2) « 95 % des commandes sont dans l'intervalle » : l'intervalle concerne la **moyenne**, pas les commandes individuelles ; les paniers vont de 2 à 988 €. (3) « Deux intervalles qui se chevauchent prouvent qu'il n'y a pas de différence » : c'est plus subtil, et c'est l'objet des tests du volume III.

1.3.4 L'intervalle de confiance d'une proportion

Beaucoup de questions d'entreprise portent sur une **proportion** : le taux de retour, la part de clients satisfaits, le taux d'ouverture d'un e-mail. La gérante demande : « Sur 400 lignes du Site, 36 ont été retournées : est-ce 9 % ? » Le chiffre observé est $\hat{p} = 36/400 = 9,0\%$. Son erreur type est

$$\text{erreur type}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} = \sqrt{\frac{0,09 \times 0,91}{400}} \approx 0,0143,$$

et l'intervalle à 95 % est $\hat{p} \pm 1,96 \times 0,0143$, soit $9,0\% \pm 2,8$ points : **[6,2 % ; 11,8 %]**. Même avec 400 lignes, le taux de retour est connu à près de trois points près, ce qui est large devant un taux de 9 %. Voyons ce que donne un tirage de 400 lignes dans les données.

```
from outils_ch01 import ic_proportion
site = lignes.merge(commandes[["id_commande", "canal"]], on="id_commande").query("canal == 'Site'")
```

```
retourne = site["id_ligne"].isin(pd.read_csv("donnees/retours.csv")["id_ligne"]).values
tirage = np.random.default_rng(8).choice(retourne, 400, replace=False)
print("taux observé :", round(tirage.mean(), 4), "| IC de Wald :",
      ↪ np.round(ic_proportion(tirage.sum(), 400), 4))
```

```
taux observé : 0.0975 | IC de Wald : [0.0684 0.1266]
```

Sur ces 400 lignes, 39 sont retournées : 9,75 %, avec un intervalle de [6,8 % ; 12,7 %]. Le taux de retour de l'ensemble du Site, lui, est de 8,98 %. Quand la proportion est **proche de 0 ou de 1**, ou l'échantillon petit, l'intervalle « de Wald » ci-dessus (symétrique autour de $\hat{p}$) devient peu fiable (il peut même descendre sous 0 %) ; l'intervalle de **Wilson**, légèrement asymétrique, donne [7,2 % ; 13,1 %] ici, et c'est celui qu'on préfère. Retenez surtout l'**ordre de grandeur** : pour une proportion voisine de 50 %, la marge vaut environ $1/\sqrt{n}$ (3 points pour $n = 1\,000$, 5 points pour $n = 400$).

1.3.5 Combien d'observations faut-il ?

Avant une enquête, une question domine : **quelle taille d'échantillon** pour atteindre une précision donnée ? On inverse les formules. Pour une **moyenne** et une marge d'erreur souhaitée e :

$$n = \left(\frac{1,96 \sigma}{e} \right)^2.$$

Pour connaître le panier moyen à plus ou moins 5 € près, avec un écart-type de 82,3 €, il faut $n = (1,96 \times 82,3/5)^2 \approx 1\,041$ **commandes**. À ± 10 €, il en faut le quart : 260. Pour une **proportion** p et une marge e :

$$n = \frac{1,96^2 p (1 - p)}{e^2}.$$

Pour un taux de retour d'environ 9 %, connu à ± 2 points, il faut $n = 1,96^2 \times 0,09 \times 0,91 / 0,02^2 \approx 787$ **lignes** ; à ± 1 point, **3 146**. Si l'on ne sait rien de p , on prend le cas le plus défavorable $p = 0,5$ (le produit $p(1 - p)$ est maximal) : ± 3 points demandent alors 1 067 répondants, le chiffre classique des sondages.

Deux points de méthode. D'abord, **les formules supposent un échantillon tiré au hasard dans une population très grande** devant lui. Quand l'échantillon représente une fraction notable de la population (disons plus de 5 %), l'erreur est plus petite : on la multiplie par le **facteur de correction de population finie** $\sqrt{(N - n)/(N - 1)}$. Pour 400 commandes tirées sur 12 946, il vaut 0,984 (négligeable) ; pour 5 000 commandes, 0,78. Ensuite, la **précision a un coût qui croît très vite** : passer de ± 10 € à ± 5 € demande quatre fois plus de commandes, de ± 5 € à $\pm 2,5$ € encore quatre fois plus. Il est souvent plus rentable de **mieux tirer** un petit échantillon que d'en agrandir un mauvais, ce qui nous amène à la dernière question.

1.3.6 Les biais, que la taille ne corrige pas

Toute la section précédente mesure l'erreur due **au hasard**. Elle disparaît quand n augmente. Il existe une autre sorte d'erreur, le **biais**, qui ne disparaît **jamais** avec la taille : il vient de la **façon** dont l'échantillon a été constitué, et un grand échantillon biaisé donne simplement une mauvaise réponse **avec une grande assurance**.

Un exemple tiré de la boutique. La gérante veut savoir : « **En moyenne, combien de commandes passe un client en trois ans ?** » Deux manières d'interroger les données :

- **Méthode A** : tirer au hasard 300 clients dans la liste des 6 000 clients inscrits et compter leurs commandes.
- **Méthode B** : tirer au hasard 300 *commandes* (« les clients que l'on croise à la caisse ») et compter, pour chacune, les commandes de son client.

La méthode B paraît naturelle (« on prend des gens qui achètent ») ; mais un client qui commande 40 fois a **40 fois plus de chances** d'être croisé qu'un client qui commande une seule fois. On sur-représente les fidèles.

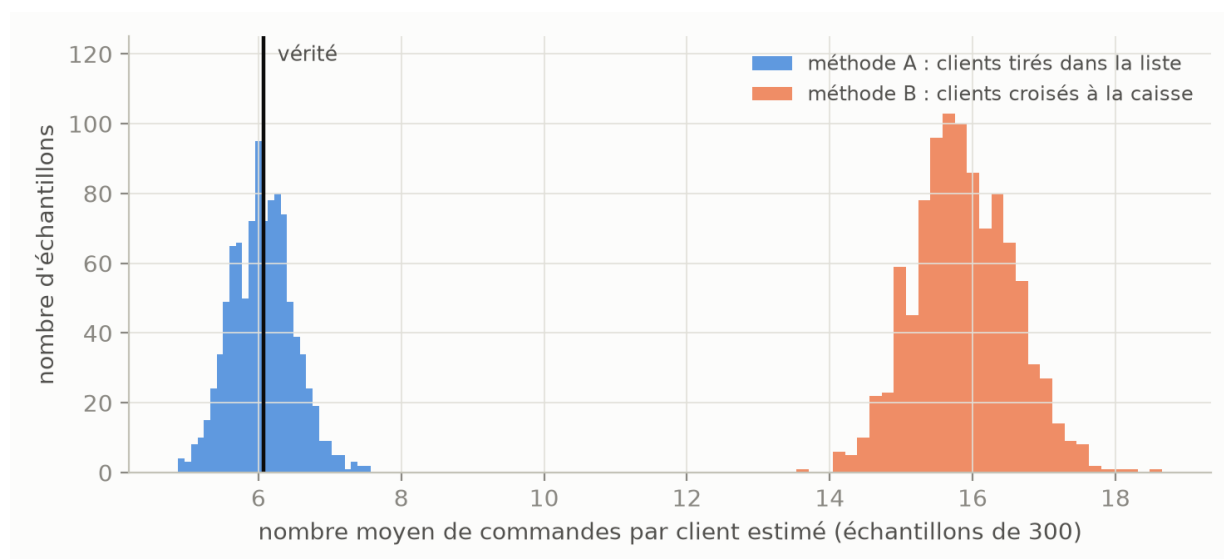


Figure 11: Deux façons d'estimer le nombre moyen de commandes par client, avec 1 000 échantillons de 300 chacune. La méthode A (bleu), qui tire des clients dans la liste, est centrée sur la vérité (trait noir). La méthode B (orange), qui tire des commandes, est centrée très au-dessus : le biais ne dépend pas de la taille de l'échantillon.

La vérité est de **6,07 commandes par client** (en comptant les 1 194 clients inscrits qui n'ont jamais commandé). La méthode A donne en moyenne 6,06, avec une erreur type de 0,44 : sans biais. La méthode B donne en moyenne **15,88**, plus de deux fois trop. Et ce n'est pas une question de taille : avec 3 000 commandes tirées par la méthode B, l'intervalle de confiance à 95 % devient [15,6 ; 16,5], **très étroit et très faux**. C'est l'image de ce que l'on appelle être « précisément à côté de la plaque ».

On retrouve ce mécanisme partout : l'enquête auprès des clients qui ont **accepté** de répondre (les mécontents et les enthousiastes répondent plus que les indifférents), l'analyse des clients **encore actifs** (on a oublié ceux qui sont partis), l'étude des seules **réussites**. La section 5.3 y revient dans le cadre des enquêtes.

Erreur d'échantillonnage, biais, erreur de mesure : trois choses différentes. L'erreur d'échantillonnage vient du hasard du tirage ; elle diminue en $1/\sqrt{n}$ et se quantifie par l'intervalle de confiance. Le **biais de sélection** vient de la façon de constituer l'échantillon ; il ne diminue pas avec n et ne se voit **pas** dans l'intervalle de confiance. L'**erreur de mesure** vient de la façon d'observer (une saisie fautive, une question mal posée, un capteur dérégulé) ; elle aussi persiste quand n augmente. Un intervalle étroit ne garantit que l'absence de hasard, **jamais** l'absence d'erreur.

À retenir. (1) Une statistique d'échantillon varie d'un tirage à l'autre ; son écart-type est l'erreur type $\sigma/\sqrt{n}$ (division par deux : quadrupler n). (2) Le théorème central limite rend la moyenne approximativement normale, d'où l'intervalle $\bar{x} \pm t s/\sqrt{n}$; « 95 % » décrit la méthode, pas une probabilité sur μ . (3) Une proportion a une erreur type $\sqrt{\hat{p}(1-\hat{p})/n}$; n se calcule en inversant la formule. (4) Aucune taille d'échantillon ne corrige un biais de sélection : posez toujours la question « **comment ces observations sont-elles arrivées dans mon fichier ?** ».

Pour s'entraîner. Cahier, chapitre 1 : applications 1.5 et 1.6, exercices 1.9 et 1.10.

1.4 Corrélation et causalité

Voici la question qui revient dans toutes les entreprises : « Les jours où nous dépensons plus en publicité, nous vendons plus. La publicité marche, non ? » La première moitié de la phrase est un constat sur les données (une **corrélacion**) ; la seconde est une **affirmacion sur le monde** (une **causalité**). On passe de l'une à l'autre bien plus vite qu'on ne le devrait. Cette section apprend à mesurer une liaison, à la lire correctement, puis à se demander ce qui la produit.

1.4.1 Mesurer une liaison : covariance et corrélation

Pour deux grandeurs mesurées sur les mêmes jours (la dépense publicitaire x et le nombre de commandes y), la **covariance** moyenne les produits des écarts à la moyenne :

$$\text{cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

Elle est positive quand x et y sont **ensemble** au-dessus ou au-dessous de leur moyenne, négative quand l'un est haut quand l'autre est bas. Mais son échelle dépend des unités (euros $\times$ commandes). On la divise donc par les deux écarts-types pour obtenir le **coefficient de corrélation de Pearson**, sans unité, compris entre -1 et $+1$:

$$r = \frac{\text{cov}(x, y)}{s_x s_y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}.$$

$r = +1$: les points sont exactement alignés sur une droite croissante ; $r = -1$: sur une droite décroissante ; $r = 0$: aucune **liaison linéaire**. Le carré r^2 s'interprète comme la part de la variation de y « partagée » avec celle de x le long de la droite.

À la main sur six jours. Prenons les six premiers jours de novembre 2025 à partir du lundi 3 : dépense publicitaire (en euros arrondis) et nombre de commandes.

Jour	x (pub, €)	y (commandes)	$x - \bar{x}$	$y - \bar{y}$	produit
1	242	32	-153,2	-14,3	2 195,4
2	358	45	-37,2	-1,3	49,6
3	374	32	-21,2	-14,3	303,4
4	583	41	187,8	-5,3	-1 001,8
5	325	58	-70,2	11,7	-818,6
6	489	70	93,8	23,7	2 220,7
Somme			0	0	2 948,7

Avec $\bar{x} = 395,2$ €, $\bar{y} = 46,3$, $\sum (x - \bar{x})^2 = 74\,298,8$ et $\sum (y - \bar{y})^2 = 1\,137,3$:

$$r = \frac{2\,948,7}{\sqrt{74\,298,8 \times 1\,137,3}} = \frac{2\,948,7}{9\,192,4} \approx 0,32.$$

Une corrélation modérée et positive. Mais **six jours, c'est très peu**. En prenant d'autres semaines de six jours, on obtient des coefficients de -0,81, -0,24, 0,32 et 0,45 : de fortement négatif à modérément positif, pour la **même** relation sous-jacente. Un coefficient de corrélation calculé sur peu d'observations est **très instable** ; l'intervalle de confiance de la section 1.3 s'applique aussi à r .

Le coefficient de Pearson mesure une relation **linéaire**. Quand la relation est monotone sans être linéaire, ou en présence de valeurs extrêmes, on préfère le coefficient de **Spearman**, qui n'est autre que le coefficient de Pearson calculé sur les **rangs** (on remplace chaque valeur par sa position dans l'ordre croissant) : il ne dépend que de l'ordre, pas de l'échelle, et résiste aux valeurs aberrantes.

Pour s'entraîner. Cahier, chapitre 1 : exercice 1.11.

1.4.2 Toujours regarder le nuage de points

Un seul nombre ne peut pas résumer une relation. L'exemple classique est celui du statisticien Francis Anscombe (1973) : **quatre jeux de onze points** qui ont exactement les mêmes moyennes, les mêmes variances et le **même coefficient de corrélation** ($r = 0,816$), mais des formes très différentes.

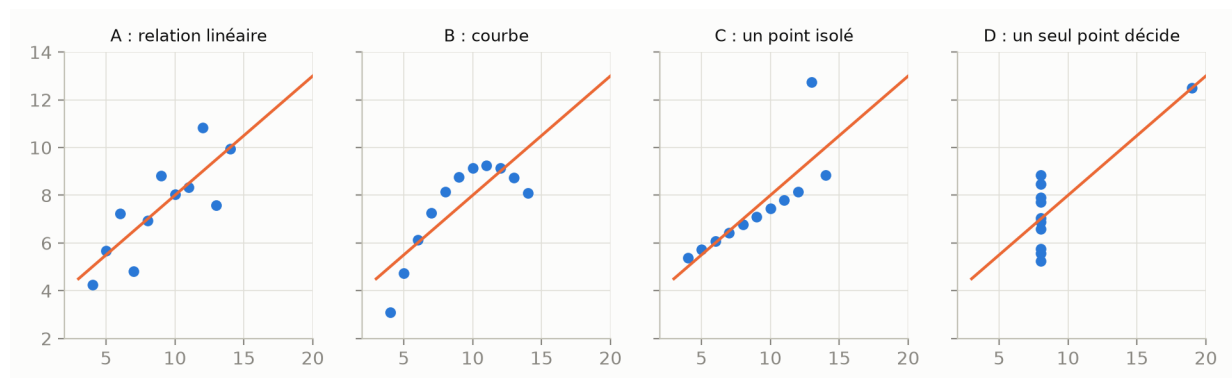


Figure 12: Les quatre jeux d'Anscombe : mêmes moyennes, mêmes écarts-types, même droite de régression et même corrélation (0,816). A : une vraie relation linéaire. B : une relation courbe, que la droite décrit mal. C : une relation parfaitement linéaire, déformée par un point isolé. D : aucune relation, sauf un point extrême qui, à lui seul, crée la corrélation.

Dans le jeu B, la relation est **parfaite mais courbe** ; dans C, un seul point isolé fait baisser un alignement parfait ; dans D, il n'y a **aucune** relation, hormis un point extrême qui fabrique la corrélation à lui seul. Le coefficient r est **aveugle** à ces différences. Les règles qui en découlent :

- **Dessinez toujours** le nuage de points avant de calculer r .
- Un r proche de 0 n'implique pas l'**absence** de relation, seulement l'absence de relation *linéaire* (une relation en U donne $r \approx 0$).
- Un r élevé peut être l'œuvre d'un **seul point** : retirez-le pour voir.
- Le r calculé sur des **moyennes** (par mois, par ville) est généralement plus fort que celui calculé sur les individus, car la moyenne gomme le bruit : c'est l'**erreur écologique** quand on en tire des conclusions sur des individus.

1.4.3 La publicité et les ventes : une corrélation trompeuse

Revenons à la gérante. Sur les 1 096 jours de la boutique, la corrélation de Pearson entre la **dépense publicitaire du jour** et le **chiffre d'affaires du jour** vaut **0,42** (Spearman : 0,29). C'est une liaison positive franche. Regardons le nuage.

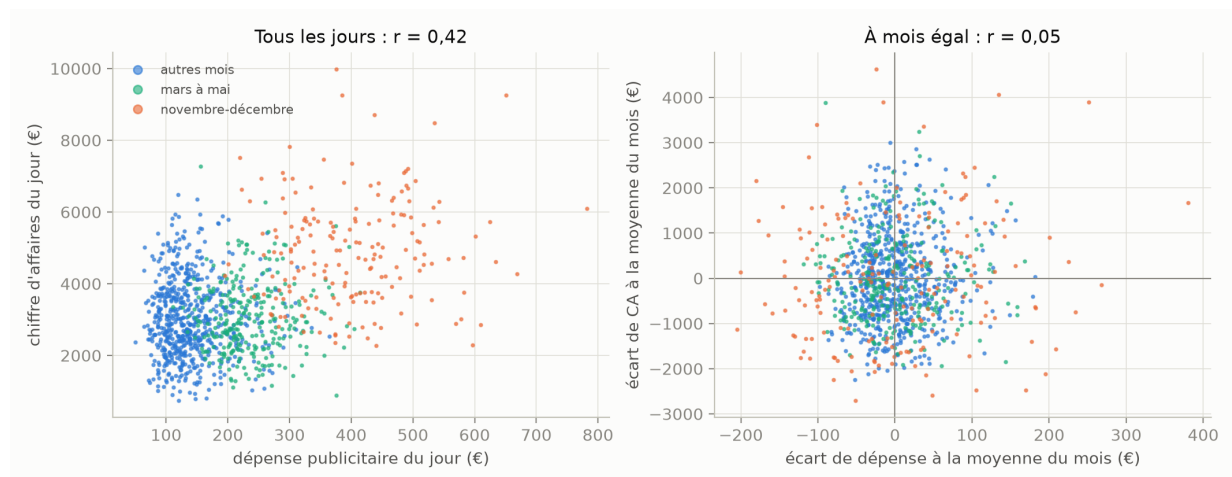


Figure 13: À gauche, chaque point est un jour : la dépense publicitaire (horizontal) et le chiffre d'affaires (vertical), colorés selon la période de l'année. Les points de novembre-décembre (orange) sont en haut à droite : forte dépense et fortes ventes. À droite, les mêmes données après avoir retiré, pour chaque mois, la moyenne du mois : à mois égal, la relation disparaît presque.

Le nuage de gauche a une structure : les jours de **novembre-décembre** (orange) ont à la fois les plus fortes dépenses (en moyenne 400 € par jour, contre 141 € les autres mois) et les plus fortes ventes (4 895 € par jour contre 3 006 €). La **saison** pousse **en même temps** la dépense (la boutique fait plus de publicité avant Noël) et les ventes (les clients achètent plus avant Noël). La publicité et les ventes sont corrélées parce qu'elles **ont une cause commune**, la saison.

Pour tester cette explication, on compare des jours **du même mois** : on retire à chaque jour la moyenne de son mois, pour la dépense et pour le chiffre d'affaires, et l'on regarde la corrélation entre les **écarts**. Elle tombe de 0,42 à **0,05** (graphique de droite) : à mois égal, un jour de forte dépense n'est pas sensiblement meilleur qu'un jour de faible dépense. Le coefficient initial mesurait surtout l'**effet du calendrier**.

Le facteur de confusion. Quand une troisième grandeur Z (ici la saison) influence à la fois X (la dépense) et Y (les ventes), X et Y sont corrélées **même si X n'a aucun effet sur Y** . On dit que Z est un **facteur de confusion**. C'est la raison de principe pour laquelle une corrélation observée ne prouve pas une causalité.

Peut-on aller plus loin, et chiffrer le **vrai** effet de la publicité ? On compare des journées qui ne diffèrent que par la dépense, en neutralisant simultanément le mois, le jour de la semaine, la promotion, la pluie et la tendance (une **régression multiple**, que le volume III de cette série détaille). Voici le résultat pour la dépense cumulée sur les sept derniers jours, en milliers d'euros, et la variation relative du nombre de commandes :

Effet de 1 000 € de dépense hebdomadaire sur le nombre de commandes	Estimation	Intervalle à 95 %
Naïf : sans rien neutraliser	+32 %	
Ajusté : à mois, jour, promotion, pluie et tendance égaux	+0,7 %	de -3,7 % à +5,3 %
Vérité programmée	+1,5 %	

L'estimation naïve annonce que 1 000 € de plus par semaine augmentent les commandes de **32 %** : un résultat énorme, qui est un artefact de la saison. L'estimation ajustée est de l'ordre de **0,7 %**, avec un intervalle qui contient à la fois **zéro** et la vérité (+1,5 %). Honnêtement : avec trois ans de données et un effet aussi petit, on **ne sait pas distinguer** une publicité utile d'une publicité inutile. C'est une conclusion modeste, et c'est la bonne : pour la trancher, il faudrait **faire varier la dépense exprès** (une expérience), pas attendre que le calendrier la fasse varier à notre place.

Pour s'entraîner. Cahier, chapitre 1 : application 1.7.

1.4.4 La température et les ventes de jardin

Un second exemple, plus subtil. La boutique vend, dans sa catégorie « Jardin », des arrosoirs, des parasols, des transats. Sur les 1 096 jours, le nombre de lignes « Jardin » vendues dans la journée est corrélé à la température moyenne du jour (coefficient de corrélation de 0,69). Le froid ou la chaleur du jour pilotent-ils les achats de jardin ?

On refait le test précédent : comparer des jours **du même mois**. À mois égal, la corrélation entre l'écart de température et l'écart de lignes « Jardin » vaut **-0,00** : rien. C'est **la saison**, une fois de plus, qui produit la liaison : en été il fait chaud *et* l'on achète des transats ; mais un jour de juillet plus frais que la moyenne ne fait pas vendre moins. (Dans cette simulation, la température intervient dans le *choix des catégories* selon le mois, jamais selon le temps du jour. Dans la réalité, la météo du jour peut avoir un effet réel : on ne le saurait qu'en le mesurant à saison égale, comme on vient de le faire.)

1.4.5 Les corrélations fortuites

Troisième piège : à force de chercher, on **trouve**. Avec 100 variables, il y a $100 \times 99/2 = 4950$ paires. Si chacune est testée au risque habituel de 5 %, on s'attend à voir environ 5 % de paires « significativement » corrélées **par pur hasard**, soit quelque 250. Vérifions avec 100 séries de 30 nombres tirés au hasard, **indépendantes** par construction.

```
import numpy as np
rng = np.random.default_rng(7)
series = rng.normal(size=(30, 100)) # 100 variables sans aucun lien, 30 observations
r = np.corrcoef(series.T)[np.triu_indices(100, 1)] # les 4 950 corrélations
print("paires :", len(r), "| |r| > 0,36 :", int((abs(r) > 0.36).sum()), "| plus grand |r| :",
      round(abs(r).max(), 2))
```


paires : 4950 | $|r| > 0,36$: 271 | plus grand $|r|$: 0.63

Avec 30 observations, un coefficient dépasse 0,36 en valeur absolue dans environ 5 % des cas **sous l'hypothèse d'indépendance** ; on en trouve ici **271 sur 4 950** (5,5 %), et le plus grand atteint **0,63**, un chiffre qui ferait un joli graphique dans une présentation. Aucune de ces liaisons n'est réelle. La leçon est celle du **dragage de données** (*p-hacking*) : plus on teste de relations, plus on est sûr d'en trouver une « remarquable ». Le remède est de **formuler l'hypothèse avant** de regarder les données, de **corriger** pour le nombre de comparaisons (le volume III y revient), et surtout de **vérifier sur des données nouvelles** : une vraie relation survit, une relation fortuite disparaît.

1.4.6 De la corrélation à la causalité

Quand on observe que X et Y sont corrélées, il y a **quatre** explications possibles :

1. X **cause** Y (la publicité fait vendre).
2. Y **cause** X : la **causalité inverse**. La gérante règle son budget de décembre sur les ventes qu'elle attend : les ventes « causent » la dépense.
3. Un **tiers Z cause les deux** : le **facteur de confusion** (la saison).
4. Le **hasard** : une corrélation fortuite, surtout quand on a cherché ou que n est petit.

Les explications se combinent, et les données seules ne disent pas laquelle est la bonne. Dessiner la situation aide : une flèche par influence supposée.

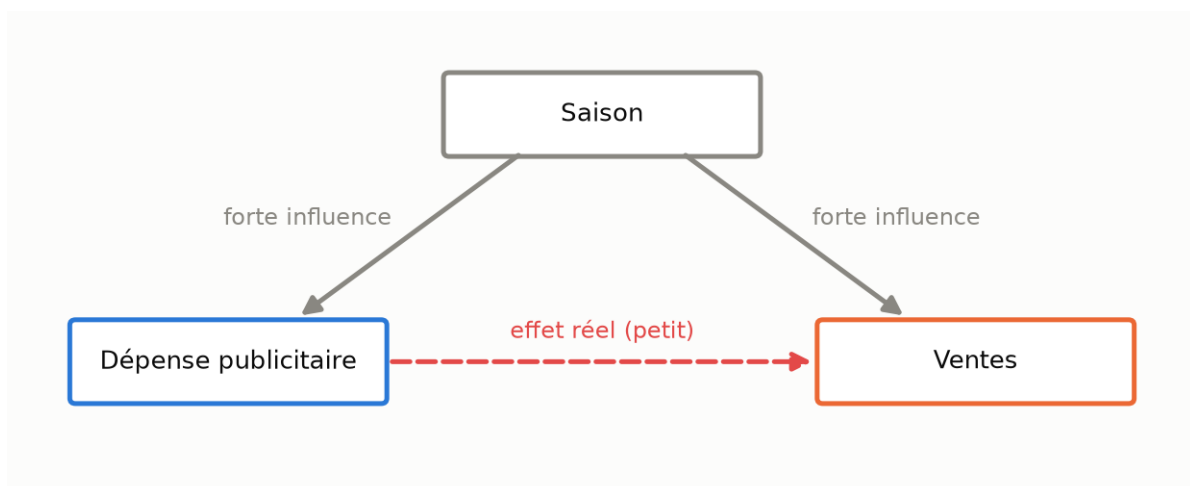


Figure 14: Le schéma du facteur de confusion dans la boutique. La saison influence fortement la dépense publicitaire et les ventes (flèches grises) ; l'effet direct de la dépense sur les ventes (flèche rouge pointillée) est petit. La corrélation observée entre dépense et ventes mélange les deux chemins.

Comment établir qu'une relation est **causale** ? La méthode la plus solide est l'**expérience aléatoire** : on **décide au hasard** quels clients (ou quels jours) reçoivent le « traitement » (la publicité, la promotion, l'e-mail) et l'on compare les groupes. Le tirage au sort garantit que tous les facteurs de confusion, connus ou inconnus, se répartissent également entre les groupes ; la différence qui reste ne peut venir que du traitement. C'est le **test A/B**, que le volume III détaille. Quand l'expérience est impossible (on ne peut pas tirer au sort la saison), on **ajuste** par des facteurs mesurés, comme nous l'avons fait ; mais l'ajustement ne neutralise que ce que l'on a **pensé** à mesurer.

Les données de la boutique, parce qu'elles sont simulées, permettent un contrôle rare : comparer ce que l'on estime à ce qui a **réellement** été programmé.

Effet	Estimation naïve	Estimation ajustée	Vérité programmée
Promotion sur le nombre de commandes du jour	+8 %	+19 % (de +14 à +24 %)	+18 %
Dépense hebdomadaire (+1 000 €) sur les commandes	+32 %	+0,7 % (de -3,7 à +5,3 %)	+1,5 %
Tendance annuelle des commandes		+6,5 %	+6 %

Pour la **promotion**, la comparaison naïve (nombre moyen de commandes les jours de promotion et les autres) donne +8 % : la plupart des jours de promotion tombent **en creux saisonnier** (soldes de janvier et d'été), ce qui réduit l'écart apparent. En comparant à mois, jour de semaine et tendance égaux, on retrouve +19 %, tout près des +18 % programmés. Même méthode que pour la publicité, même facteur de confusion (le calendrier), mais **ici** l'effet est assez grand pour émerger du bruit, **là** il ne l'est pas. Le contrôle par les facteurs mesurés a donc **fonctionné pour la promotion** et **révélé un effet minuscule pour la publicité** : deux conclusions honnêtes à partir des mêmes outils.

Le vocabulaire compte. « La publicité **augmente** les ventes » est une affirmation causale ; « les ventes sont **plus élevées** les jours de forte publicité » est un constat. Dans un rapport, écrivez le constat sauf si votre méthode (expérience, ajustement soigneux, argument de mécanisme) justifie l'affirmation, et **dites laquelle**.

À retenir. (1) r mesure une liaison **linéaire**, entre -1 et $+1$; il est instable sur peu de données et aveugle à la forme (Anscombe) : **dessinez**. (2) Une corrélation a quatre explications : cause, cause inverse, facteur de confusion, hasard. (3) Comparer **à saison égale** fait souvent disparaître une corrélation : la saison, le calendrier, la taille des clients sont les facteurs de confusion habituels. (4) À force de chercher, on trouve : formulez l'hypothèse avant, vérifiez sur des données nouvelles. (5) La preuve causale la plus solide est l'expérience aléatoire ; à défaut, ajuster honnêtement et dire ce que l'on n'a pu mesurer.

Pour s'entraîner. Cahier, chapitre 1 : application 1.7, exercices 1.11 et 1.12.

1.5 ⊕ Pour aller plus loin : mathématiques du quotidien en entreprise

Section complémentaire. Elle ne demande aucune statistique, seulement des calculs que **tout analyste fait chaque semaine** et que l'on rate pourtant régulièrement : pourcentages, taux de croissance, moyennes pondérées, marges, arrondis. Chaque calcul est d'abord fait à la main sur un exemple court, puis vérifié sur les données de la boutique. On peut la lire à tout moment, ou y revenir quand un chiffre « ne tombe pas juste ».

1.5.1 Pourcentages : part, variation, points

Un **pourcentage** est une fraction exprimée sur cent. Il sert à deux choses très différentes qu'il faut savoir distinguer :

- une **part** : « le Site représente 47 % des commandes » (une partie rapportée à un tout) ;
- une **variation** : « le chiffre d'affaires a augmenté de 11 % » (un changement rapporté à la valeur de départ), $\frac{\text{valeur finale} - \text{valeur initiale}}{\text{valeur initiale}}$.

La confusion classique concerne les **points** et les **pour cent**. La part des commandes passées sur le Site est passée de 37,4 % en 2023 à 46,9 % en 2025. On peut dire :

- « la part du Site a augmenté de **9,5 points** » : c'est la différence entre deux pourcentages, exprimée en **points de pourcentage** ;
- « la part du Site a augmenté de **25 %** » : c'est la variation **relative** $((46,9 - 37,4) / 37,4)$.

Les deux sont exacts, et ils ne disent pas la même chose. Dans un rapport, écrivez toujours l'**unité** (« points » ou « % ») : écrire « +9 % » pour 9 points est l'erreur la plus fréquente des tableaux de bord.

Trois pièges de calcul reviennent sans cesse.

- **Les variations ne s'additionnent pas.** Une hausse de 10 % suivie d'une baisse de 10 % ne ramène pas au point de départ : $100 \times 1,10 \times 0,90 = 99$, soit **-1 %**. De même, pour revenir au point de départ après une baisse de 20 %, il faut une hausse de **25 %** ($0,80 \times 1,25 = 1$), pas de 20 %.
- **Une réduction ne se défait pas avec le même pourcentage.** Un prix de 60 € réduit de 20 % vaut 48 € ; pour retrouver 60 €, on ajoute 12 € à 48 €, soit **25 %**.
- **Retrouver la base : on divise, on ne retranche pas.** Un prix TTC de 120 € avec une TVA de 20 % (taux d'exemple) correspond à un prix HT de $120 / 1,20 = 100$ €, **pas** de $120 - 20\% \times 120 = 96$ €. Le pourcentage s'applique à la **base**, c'est-à-dire au prix HT, non au prix TTC.

1.5.2 Taux de croissance

Le chiffre d'affaires de la boutique était de **1 138 932 €** en 2023, **1 189 461 €** en 2024 et **1 324 764 €** en 2025. Comment résumer cette évolution ?

Le **taux de croissance** entre deux périodes est la variation relative : +4,4 % de 2023 à 2024 et +11,4 % de 2024 à 2025. Mais sur deux ans, la hausse totale n'est **pas** la somme des deux taux. Les taux **se composent** (on les multiplie) :

$$(1 + g_{24})(1 + g_{25}) = 1 + g_{23 \rightarrow 25}.$$

Ici, le produit des deux coefficients vaut 1,1632, soit **+16,3 %** sur deux ans.

Pour comparer des périodes de durées différentes, on utilise le **taux de croissance annuel moyen** (TCAM, CAGR en anglais) : le taux constant qui produirait la même croissance totale.

$$\text{TCAM} = \left(\frac{\text{valeur finale}}{\text{valeur initiale}} \right)^{1/\text{nombre d'années}} - 1.$$

À la main. Un chiffre qui passe de 100 à 121 en deux ans a un TCAM de $\sqrt{1,21} - 1 = 10\%$ par an (et non $21/2 = 10,5\%$) : $100 \times 1,10 \times 1,10 = 121$. Pour la boutique : $(1\,324\,764 / 1\,138\,932)^{(1/2)} - 1 = 7,85\%$ par an, alors que la moyenne arithmétique des deux taux annuels vaut 7,91 %. Les deux sont proches ici (les taux annuels sont proches) ; ils divergent quand les taux sont très différents.

Une **échelle en indice** (base 100) facilite la lecture : on fixe la valeur de départ à 100. Avec la base 100 en 2023, le chiffre d'affaires vaut 104,4 en 2024 et 116,3 en 2025 : on lit directement les variations en pourcentage depuis 2023.

Comparer avec le bon mois

Peut-on dire « les ventes de décembre ont augmenté de 28 % : excellent mois » ? Seulement par rapport à novembre, et c'est trompeur : **la saisonnalité** rend tout mois de décembre supérieur à novembre. La figure montre le chiffre d'affaires mensuel de chaque année (à gauche), puis, pour 2025, la variation par rapport au mois précédent (barres bleues) et par rapport au même mois de l'année précédente (barres orange).

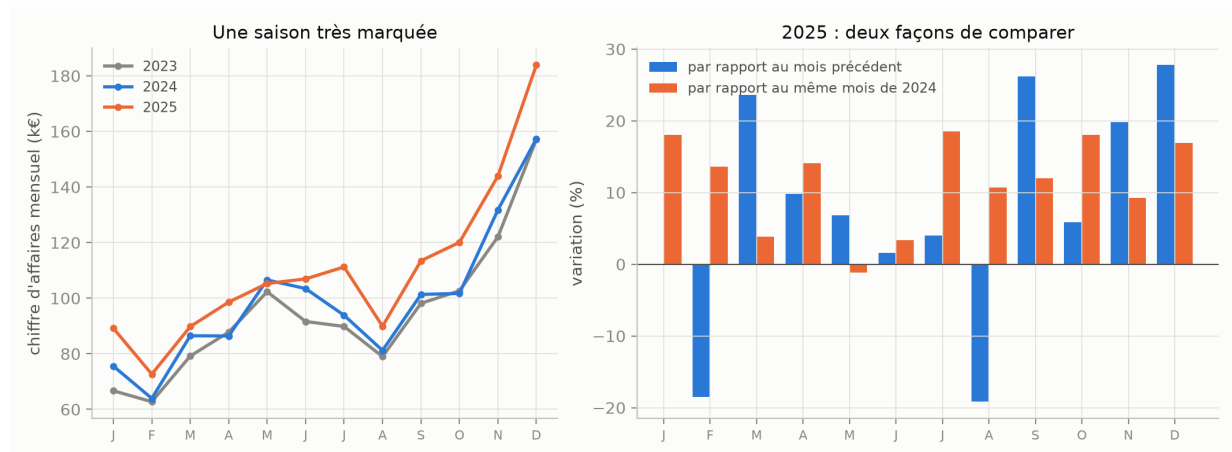


Figure 15: À gauche, le chiffre d'affaires mensuel de chaque année : la saison (creux d'été, pic de novembre-décembre) est la même d'une année à l'autre, avec un niveau qui monte. À droite, pour 2025, la variation par rapport au mois précédent (bleu) oscille fortement à cause de la saison ; la variation par rapport au même mois de l'année précédente (orange) reste dans une fourchette plus étroite, autour de la croissance annuelle.

Les variations « par rapport au mois précédent » vont de -19 % à +28 % : elles décrivent le **calendrier**, pas la santé de l'entreprise. Les variations « par rapport au même mois de 2024 » vont de -1 % à +19 %, une fourchette moitié moins large, centrée sur la croissance annuelle de 11 % : c'est la comparaison qui **neutralise la saison**, et celle qu'il faut privilégier dans un rapport mensuel. Décembre 2025, par exemple, est à +16,9 % de décembre 2024.

1.5.3 Moyennes pondérées et décomposition prix-volume

La moyenne pondérée

Quand on moyenne des groupes de **tailles différentes**, chaque groupe doit compter proportionnellement à sa taille. La **moyenne pondérée** est

$$\bar{x}_w = \frac{\sum_i w_i x_i}{\sum_i w_i},$$

où w_i est le poids du groupe i (le nombre de commandes, le chiffre d'affaires...). **À la main.** Deux canaux : le Site, avec 300 commandes à un panier moyen de 90 €, et la Boutique, avec 100 commandes à 130 € : la moyenne simple des deux paniers moyens est de 110 €, mais la moyenne **pondérée** est $(300 \times 90 + 100 \times 130) / 400 = 100$ €. C'est le panier moyen de l'ensemble.

Sur la boutique en 2025, les trois canaux ont les paniers moyens suivants :

Canal	Commandes 2025	Panier moyen
Boutique	5 442	103,08 €
Réseaux	1 426	102,44 €
Site	6 078	101,63 €

La moyenne simple des trois est de 102,38 € ; la moyenne pondérée par les commandes est de **102,33 €**, qui est exactement le panier moyen de 2025 (102,33 €). Le poids est la **bonne** quantité à utiliser, et la règle générale demeure : **pour recomposer un total, pondérez.**

Prix, volume, mix : pourquoi le chiffre d'affaires a-t-il augmenté ?

La gérante demande : « Le chiffre d'affaires 2025 est en hausse de 11,4 % : est-ce parce que nous avons **vendu plus** ou parce que nous avons **augmenté les prix** ? » Le chiffre d'affaires est un **produit** : $CA = \text{unités vendues} \times CA \text{ moyen par unité}$. Sa variation se décompose donc en deux facteurs qui **se multiplient** :

$$1 + g_{CA} = \underbrace{(1 + g_{\text{volume}})}_{\text{unités vendues}} \times \underbrace{(1 + g_{\text{prix-mix}})}_{CA \text{ moyen par unité}}.$$

En 2025, le nombre d'unités vendues a augmenté de **7,4 %** (de 33 323 à 35 801 unités), et le chiffre d'affaires moyen par unité de **3,7 %** (de 35,69 € à 37,00 €). Vérification : $1,0744 \times 1,0367 = 1,1138$, soit bien $1 + 11,4 \%$. Le catalogue a en effet été augmenté de **3 %** le 1^{er} janvier 2025 ; l'écart avec les 3,7 % constatés vient du **mix** (la part de chaque produit dans les ventes) et des **remises** (la part des ventes sous promotion). Moralité : en proportion (logarithmique) de la croissance de 2025, environ **67 %** vient du **volume** et le reste du prix et du mix : une décomposition qu'aucun des deux chiffres, pris seuls, ne révèle.

1.5.4 Marge, marque et TVA

Le prix affiché en rayon est **TTC** (toutes taxes comprises) ; le chiffre d'affaires comptable est **HT** (hors taxes), car la TVA est reversée à l'État. Pour parler de rentabilité, il faut donc d'abord passer en HT : $\text{prix HT} = \text{prix TTC} / (1 + \text{taux de TVA})$. Dans ce livre, la TVA est de **20 %** : c'est un taux d'exemple, il varie selon les pays et les produits.

Trois notions se ressemblent et ne se confondent pas :

- la **marge brute** (en euros) : $\text{prix de vente HT} - \text{coût d'achat HT}$;
- le **taux de marque** : $\text{marge brute} / \text{prix de vente HT}$ (c'est la part du prix qui reste) ;
- le **taux de marge** : $\text{marge brute} / \text{coût d'achat}$ (c'est le « coefficient » ajouté au coût).

À la main. Un produit vendu 30,00 € TTC, acheté 15,00 € HT. Prix HT : $30/1,2 = 25,00$ €. Marge brute : $25 - 15 = 10$ €. Taux de marque : $10/25 = 40 \%$. Taux de marge : $10/15 \approx 66,7 \%$. Les deux sont reliés par :

$$\text{taux de marque} = \frac{\text{taux de marge}}{1 + \text{taux de marge}} \quad \Leftrightarrow \quad \text{taux de marge} = \frac{\text{taux de marque}}{1 - \text{taux de marque}}.$$

L'erreur de coefficient. On veut un taux de marque de 40 % sur un produit acheté 15 € HT. Appliquer « +40 % » au coût donne un prix HT de $15 \times 1,4 = 21$ €, soit une marque de $6/21 \approx 28,6 \%$ seulement. Le bon prix HT est $15/(1 - 0,40) = 25$ €. **Précisez toujours** quelle

marge vous annoncez : selon les entreprises et les pays, « taux de marge » et « taux de marque » sont employés l'un pour l'autre.

Sur la boutique, calculons le taux de marque de l'année 2025, par catégorie (chiffre d'affaires HT = montant / 1,2 ; coût = quantité × coût d'achat HT).

```
lg25 = lignes.merge(commandes[["id_commande", "date_commande"]],
    ↪ on="id_commande").query("date_commande >= '2025-01-01'")
produits = pd.read_csv("donnees/produits.csv")
lg25 = lg25.merge(produits[["id_produit", "categorie", "cout_achat"]], on="id_produit")
lg25["ca_ht"] = lg25["montant"] / 1.2
lg25["cout"] = lg25["quantite"] * lg25["cout_achat"]
cat = lg25.groupby("categorie")[["ca_ht", "cout"]].sum()
cat["taux_marque_%"] = ((cat["ca_ht"] - cat["cout"]) / cat["ca_ht"] * 100).round(1)
print(cat["taux_marque_%"].to_dict())
print("ensemble :", round((cat["ca_ht"].sum() - cat["cout"].sum()) / cat["ca_ht"].sum() * 100, 1),
    ↪ "%")

{'Bien-être': 35.1, 'Cuisine': 37.3, 'Décoration': 39.7, 'Jardin': 38.3, 'Maison': 37.9, 'Papeterie':
    ↪ 36.9}
ensemble : 38.0 %
```

Les taux de marque vont de **35,1 %** (Bien-être) à **39,7 %** (Décoration) ; celui de l'**ensemble** est de **38,0 %** (soit un taux de marge de 61 %), sur un chiffre d'affaires HT de 1 103 970 €. La moyenne **simple** des taux par catégorie (37,5 %) ne coïncide pas avec le taux de l'ensemble : c'est, une fois de plus, la moyenne de ratios qu'il faut pondérer par le chiffre d'affaires. (Cette marge est « brute » : elle ignore les retours remboursés, les frais de livraison, de personnel et de loyer.)

1.5.5 Arrondis : où et quand

Les arrondis semblent anodins, jusqu'à ce qu'un total ne tombe pas juste. Deux difficultés.

La somme des arrondis n'est pas l'arrondi de la somme. Une commande de trois articles à 1,04 € HT chacun. Si l'on calcule la TVA ligne par ligne, chaque ligne donne $1,04 \times 0,20 = 0,208$ €, arrondi à **0,21 €**, soit **0,63 €** au total ; si on la calcule sur le total, $3,12 \times 0,20 = 0,624$, arrondi à **0,62 €**. Un centime d'écart, qui devient **des milliers d'euros** à l'échelle de millions de lignes. La règle : **arrondissez le plus tard possible** (à l'affichage), conservez les décimales dans les calculs, et quand la règle comptable impose un arrondi par ligne, **écrivez-le dans la documentation**.

Les outils n'arrondissent pas tous de la même manière. Excel arrondit les cas « à égalité » (le chiffre 5 exactement) à l'**écart de zéro** ($2,5 \rightarrow 3$; $-2,5 \rightarrow -3$). Python (la fonction `round`) et pandas arrondissent **au pair le plus proche** ($2,5 \rightarrow 2$; $3,5 \rightarrow 4$), et à cela s'ajoute la représentation binaire des décimaux ($2,675$ n'est pas exactement représentable). Comparons, avec les formules vérifiées par LibreOffice Calc :

Cas	Excel (ARRONDI)	Python (round)
2,5 à 0 décimale	3	2
3,5 à 0 décimale	4	4
-2,5 à 0 décimale	-3	-2
0,125 à 2 décimales	0,13	0,12
2,675 à 2 décimales	2,68	2,67

Excel, sur la ligne « 0,125 à 2 décimales », renvoie 0,13 (arrondi « commercial » à l'écart de zéro) ; Python renvoie 0,12 parce que 0,125 est exactement représentable en binaire et que l'égalité

est départagée **au pair**. Pour 2,675, Excel renvoie 2,68 (il raisonne sur le nombre décimal tel qu'on l'a écrit) et Python 2,67 (la valeur binaire réellement stockée est légèrement inférieure à 2,675). Le calcul de TVA ligne par ligne contre sur le total se vérifie aussi par formule : =SOMME-PROD (ARRONDI (A9:A11*0,2;2)) donne **0,63** et =ARRONDI (SOMME (A9:A11)*0,2;2) donne **0,62**.

Deux outils, deux arrondis. Quand un total Excel et un total Python diffèrent d'un centime, cherchez d'abord la **règle d'arrondi** avant de chercher une erreur de données. Dans un rapport financier, la règle (à l'écart de zéro, au pair, par ligne, sur le total) fait partie de la méthode.

À retenir. (1) Distinguez **points** et **pour cent**, **part** et **variation** ; retrouvez une base en **divisant**. (2) Les variations se **multiplient** ; le TCAM est la racine n -ième du rapport final sur initial ; comparez au **même mois** de l'an passé. (3) Pour recomposer un total, **pondérez** ; CA = unités $\times$ CA moyen par unité, et la croissance se décompose en volume et prix-mix. (4) Marque (sur le prix) et marge (sur le coût) ne sont pas interchangeables : précisez laquelle vous annoncez ; passez en HT avant de parler de rentabilité. (5) Arrondissez le plus tard possible et notez la règle.

Pour s'entraîner. Cahier, chapitre 1 : application 1.8, exercices 1.13 et 1.14.

Bilan du chapitre 1

Vous savez maintenant :

- **résumer** une variable par son **centre** (moyenne, médiane, mode, moyenne tronquée), sa **dispersion** (étendue, quartiles, écart interquartile, écart-type, coefficient de variation) et sa **forme** (asymétrie, aplatissement, valeurs aberrantes par la règle de 1,5 écart interquartile), et **choisir** le résumé qui répond à la question posée ;
- **éviter** deux pièges de moyenne : la moyenne de ratios non pondérée (on repart des totaux) et le **paradoxe de Simpson** (comparer des groupes comparables) ;
- **retrouver les mêmes résumés** dans Excel, dans R et dans pandas, en connaissant les conventions qui les distinguent (quartiles inclusifs ou exclusifs, n ou $n - 1$) ;
- **reconnaître** la loi qui décrit un phénomène : **binomiale** pour des succès parmi n essais, **de Poisson** pour des événements dans un intervalle (variance = moyenne, valable par tranche homogène), **normale** pour une grandeur en cloche (règle 68-95-99,7, score z), **log-normale** en première approximation pour des montants ; et le **vérifier** par un diagramme quantile-quantile ;
- **mesurer l'erreur d'échantillonnage** : erreur type $\sigma / \sqrt{n}$, théorème central limite, **intervalle de confiance** d'une moyenne et d'une proportion, **taille d'échantillon** nécessaire, et distinguer l'erreur du hasard, le **biais de sélection** (qu'aucune taille ne corrige) et l'erreur de mesure ;
- **mesurer une liaison** (corrélation de Pearson et de Spearman), **dessiner** avant de calculer (jeux d'Anscombe), repérer un **facteur de confusion** en comparant « à saison égale », se méfier des corrélations fortuites, et énoncer les quatre explications d'une corrélation (cause, cause inverse, confusion, hasard) ;
- (en option) faire les **calculs du quotidien** : points et pour cent, variations qui se composent, taux de croissance annuel moyen, comparaison au même mois, moyennes pondérées, décomposition prix-volume, marque, marge et TVA, arrondis.

Le tableau suivant résume **ce que nous avons mesuré** sur les données de la boutique, avec, quand elle est connue, la **vérité programmée** :

Question	Résultat mesuré	Ce qu'il faut en retenir
Panier moyen et panier médian	100,4 € et 79,8 €	61 % des commandes sont sous la moyenne : annoncer les deux
Dispersion des paniers	écart-type 81 €, CV 0,80 ; asymétrie 1,85	la moyenne seule est un mauvais portrait
Taux de retour : moyenne des canaux ou taux global	6,24 % contre 5,96 %	repartir des totaux
L'âge des clients et la règle 68-95-99,7	66 %, 97 %, 99,8 %	l'âge est à peu près normal ; les paniers ne le sont pas
Erreur type de la moyenne de 100 paniers	8,1 € (formule : 8,2 €)	quadrupler n divise l'erreur par 2
Intervalle à 95 % : couverture réelle	94,2 % des 1 000 intervalles contiennent la vérité	« 95 % » décrit la méthode
Commandes par client : tirer des clients ou des commandes	6,1 contre 15,9 (vérité : 6,1)	un biais de sélection ne se corrige pas par la taille
Corrélation dépense publicitaire – chiffre d'affaires	0,42 au total, 0,05 à mois égal	la saison est un facteur de confusion
Effet de la dépense publicitaire (par 1 000 € par semaine)	+32 % naïf, +0,7 % ajusté (vérité +1,5 %)	l'effet est trop petit pour être mesuré avec ces données
Effet de la promotion sur les commandes	+8 % naïf, +19 % ajusté (vérité +18 %)	ajuster sur le calendrier retrouve la vérité
Croissance du chiffre d'affaires 2023-2025	TCAM 7,85 % par an ; en 2025, 67 % portée par le volume	décomposer prix et volume

Le fil conducteur du chapitre tient en une phrase : **un chiffre n'a de valeur que si l'on sait ce qu'il résume, ce qu'il ignore, et de combien il peut se tromper**. Une moyenne sans dispersion, un pourcentage sans base, une corrélation sans explication ni intervalle sont des demi-vérités ; les corriger est le travail le plus ordinaire, et le plus utile, de l'analyste.

En pratique : cinq questions avant de publier un chiffre.

1. **Quel résumé ?** (moyenne, médiane, centile : lequel répond à la question ?)
2. **Quelle dispersion ?** (écart-type, quartiles : le chiffre est-il représentatif ?)
3. **Quelle base ?** (population, échantillon : comment les observations sont-elles arrivées dans mon fichier ?)
4. **Quelle précision ?** (intervalle de confiance, taille d'échantillon)
5. **Quelle explication ?** (corrélation ou causalité : quels facteurs de confusion ai-je écartés ?)

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 à 1.8 (résumer un panier, moyennes qui trompent, Excel contre pandas, quelle loi pour quoi, simuler un échantillonnage, intervalles de confiance et biais, corrélation et confusion, mathématiques du quotidien) et exercices 1.1 à 1.14.

Le chapitre 2 aborde le premier outil du quotidien de l'analyste : **Excel**. Vous y retrouverez les résumés de ce chapitre (moyennes, quartiles, pourcentages) sous forme de formules, puis les tableaux croisés dynamiques et Power Query pour importer et transformer des données.

Chapitre 2 : Excel

« Le tableur est le premier outil de la plupart des analystes, et le dernier dont ils se méfient. »

La gérante de la boutique ouvre un classeur Excel tous les lundis matin. Elle y colle l'export des ventes de la semaine, recalcule à la main les totaux par catégorie, les compare à ceux du mois précédent, puis recopie les chiffres dans un tableau qu'elle envoie à son associé. Elle vous pose aujourd'hui la question que se posent tous les gens qui travaillent ainsi :

« Peux-tu me dire, **par catégorie et par mois, ce que nous avons vendu en 2025**, sans que je refasse le calcul à la main chaque lundi ? Et peux-tu faire en sorte que je puisse **me fier** aux chiffres ? »

La seconde phrase compte autant que la première. Un tableur est un outil extraordinaire de rapidité : en quelques minutes, on obtient un tableau, un graphique, un chiffre. C'est aussi un outil sans garde-fou : une formule recopiée sur une plage trop courte, un nombre stocké comme du texte, une cellule fusionnée, un total qui ignore les dernières lignes, et personne ne le voit. Ce chapitre vous apprend à la fois à **aller vite** (formules, tableaux croisés dynamiques, Power Query) et à **vérifier** (contrôles, recoupements, bonnes pratiques de structure).

Honnêteté sur les outils. Le livre a été produit sur une machine **sans Excel**. Voici ce que cela change pour vous :

- **Les formules sont vérifiées**, mais avec **LibreOffice Calc**, un tableur libre qui lit les fichiers Excel (.xlsx) et calcule les mêmes fonctions. Chaque résultat chiffré du chapitre a été calculé ainsi, puis **recoupé par un second outil** (pandas ou SQL). LibreOffice et Excel peuvent différer sur des fonctions récentes ou des détails d'affichage : les cas connus sont signalés.
- **Les formules sont écrites comme dans l'Excel en français** (SOMME.SI.ENS, séparateur ;, virgule décimale). Un tableau de correspondance avec les noms anglais figure en section 2.1.10.
- **Les « copies d'écran » sont des maquettes dessinées** avec un programme de dessin : elles montrent ce que vous verrez à l'écran, avec des valeurs réellement calculées, mais ce **ne sont pas des captures d'Excel**. Les menus, les libellés et les icônes varient selon la **version** d'Excel et la **langue** : traitez nos descriptions de menus comme des repères, à **vérifier** sur votre poste.
- **Ce qui n'a pas pu être exécuté** : les tableaux croisés dynamiques réels, Power Query, Power Pivot et le langage DAX, VBA, Office Scripts, Google Sheets et Looker Studio. Ces blocs sont signalés « non exécuté » ; quand c'est possible, le même résultat est **recalculé en pandas ou en SQL** pour que vous puissiez vérifier ce que l'outil devrait donner.

Le chemin de ce chapitre

Section	Question	Idée centrale
2.1	Comment calculer, chercher et nettoyer avec des formules ?	Références, fonctions conditionnelles, recherches, dates, texte, erreurs
2.2	Comment résumer 30 000 lignes en un tableau ?	Le tableau croisé dynamique, et pourquoi il faut le recouper
2.3	Comment importer et nettoyer un export désordonné ?	Power Query : des étapes enregistrées et rejouables
2.4	Comment ne pas se tromper ?	Structure, données propres, contrôles, erreurs célèbres
⊕ 2.5	Que fait Excel au-delà du tableur ?	Modèle de données, DAX, macros, et quand passer à SQL ou Python
⊕ 2.6	Et dans le nuage ?	Google Sheets, Looker Studio, équivalences

Les données du chapitre

Données simulées. Tous les fichiers sont **simulés** (graines fixes, générateur `build/donnees_a1.py`) : la boutique, ses clients et ses ventes n'existent pas. La « vérité » du simulateur est documentée en tête du générateur ; nous n'en avons pas besoin ici, mais elle nous permet de savoir que les totaux sont exacts.

- **ventes_2025.xlsx**, le classeur de la gérante, contient trois feuilles : Lignes (une ligne par article vendu en 2025), Produits (le catalogue) et Clients.
- **export_caisse_brut.csv** est un export de caisse de la boutique physique, tel que la caisse le fournit : désordonné, avec un titre, un total et des formats « à la française ». Il sert à la section 2.3.
- Les autres fichiers du volume (`commandes.csv`, `boutique.db...`) servent dans les chapitres suivants ; nous les utilisons ici pour **recouper** les résultats.

Le classeur de la gérante compte **29 827 lignes** de ventes, réparties en 12 946 commandes passées par 3 875 clients, du 1^{er} janvier au 31 décembre 2025, pour un montant total de **1 324 763,72 €**. La figure suivante montre ce que la gérante voit en ouvrant la feuille Lignes.

Remarquez déjà trois choses, que tout analyste vérifie en ouvrant un classeur inconnu. **Une ligne d'en-tête** unique, sans cellule fusionnée. **Une ligne = une observation** (ici, un article vendu). **Des colonnes homogènes** : une colonne de dates ne contient que des dates, une colonne de montants que des montants. C'est la forme que les outils d'analyse attendent, et nous y reviendrons en section 2.4.2. Les cellules vides de la colonne `code_promo` ne sont pas une erreur : elles signifient « pas de code promo » (25 221 lignes sur 29 827).

Pour s'entraîner. Cahier, chapitre 2 : application 2.1 (explorer et contrôler le classeur).

M2	fx 24,62								
	A	B	C	E	F	H	I	J	M
1	id_ligne	id_commande	date_commande	canal	code_promo	nom_produit	categorie	quantite	montant
2	54079	23450	01/01/2025	Site		Cadre design	Décoration	1	24,62
3	54080	23450	01/01/2025	Site		Poêle mat	Cuisine	1	39,04
4	54081	23451	01/01/2025	Boutique		Hamac rustique	Jardin	1	74,06
5	54082	23451	01/01/2025	Boutique		Panier mat	Maison	1	38,01
6	54083	23451	01/01/2025	Boutique		Étagère compact	Maison	1	34,92
7	54084	23451	01/01/2025	Boutique		Vase mat	Décoration	1	29,77
8	54085	23451	01/01/2025	Boutique		Poêle mat	Cuisine	1	40,07
9	54086	23451	01/01/2025	Boutique		Boîte rustique	Maison	1	52,43
Lignes Produits Clients									

Figure 16: La feuille Lignes de ventes_2025.xlsx : une ligne par article vendu, une colonne par caractéristique (les colonnes D, G, K et L sont masquées pour que la figure reste lisible). Maquette dessinée avec matplotlib, pas une capture d'Excel.

2.1 Formules et fonctions

Une formule Excel est un **petit programme** que l'on écrit dans une cellule : elle commence par =, elle lit d'autres cellules et elle affiche un résultat qui se **met à jour tout seul** quand ces cellules changent. Cette section présente les familles de fonctions dont un analyste se sert tous les jours : agréger sous conditions, décider, chercher, nettoyer du texte, manipuler des dates, et comprendre les erreurs. Tous les résultats cités ont été calculés sur le classeur ventes_2025.xlsx et **recoupés par pandas** : le bloc suivant (caché dans le livre) le fait une fois pour toutes, et nous n'y reviendrons que pour citer les chiffres.

2.1.1 Anatomie d'un classeur et d'une formule

Un **classeur** (le fichier .xlsx) contient des **feuilles** ; une feuille est une grille de **cellules** repérées par une lettre (la colonne) et un numéro (la ligne) : M2 est la cellule de la colonne M, ligne 2. Une **plage** désigne un rectangle de cellules : M2:M29828 est la colonne des montants, de la ligne 2 à la ligne 29828. Pour parler d'une cellule d'une autre feuille, on préfixe : Lignes!M2.

Une cellule contient soit une **valeur** (un nombre, un texte, une date), soit une **formule**. Pour calculer le montant d'une ligne de vente, la formule est =J2*K2*(1-L2/100) : le prix unitaire, multiplié par la quantité, multiplié par un moins la remise exprimée en pourcentage. Excel applique les règles de priorité de l'arithmétique : d'abord les parenthèses, puis les pourcentages et les puissances, puis les multiplications et divisions, puis les additions et soustractions, puis la concaténation de texte (&), enfin les comparaisons (=, <>, <, >=...). **Dans le doute, mettez des parenthèses** : elles ne coûtent rien et se lisent mieux.

Quand vous modifiez une cellule, Excel **recalcule** toutes les formules qui en dépendent (le mode par défaut est le calcul automatique ; la touche F9 force un recalcul en mode manuel). C'est ce qui fait la force du tableur, et c'est aussi ce qui le rend dangereux : un changement oublié se propage sans bruit.

Faisons le calcul sur toute l'année. Si l'on recalcule chaque ligne par prix × quantité × (1 – remise) puis que l'on additionne, on trouve un total **légèrement différent** de la somme de la colonne montant :

```
=SOMME(Lignes!M2:M29828) → 1 324 763,7200
=SOMMEPROD(Lignes!J2:J29828;Lignes!K2:K29828;1-Lignes!L2:L29828/100) → 1 324 763,6395
```

écart : 0,0805 € sur 29 827 lignes

L'écart est de **8 centimes** pour 29 827 lignes : la colonne montant est arrondie au centime **ligne par ligne** (c'est ce que fait une caisse), alors que le recalcul additionne des produits non arrondis. Ce n'est pas une erreur, mais c'est la première leçon de rigueur : **une somme de valeurs arrondies n'est pas l'arrondi de la somme**. Nous retrouverons ce phénomène en section 2.4.5.

Intuition. Une formule ne « contient » pas son résultat : elle contient une **recette**. Si vous copiez la recette sur une autre ligne, Excel adapte les ingrédients à la nouvelle ligne. C'est le sujet de la sous-section suivante.

2.1.2 Références relatives, absolues et mixtes

Quand on copie la formule =J2*K2 de la ligne 2 vers la ligne 3, Excel écrit =J3*K3 : la référence est **relative**, elle se décale avec la formule. Pour qu'une référence **ne bouge pas**, on la fige avec des dollars : \$A\$1 est **absolue** (ni la colonne ni la ligne ne changent), \$A1 est **mixte** (la colonne est figée, la ligne se décale) et A\$1 est mixte dans l'autre sens. La touche F4 fait défiler ces quatre variantes.

Écriture	Copiée une ligne plus bas	Copiée une colonne plus à droite
B2 (relative)	B3	C2
\$B\$2 (absolue)	\$B\$2	\$B\$2
\$B2 (colonne figée)	\$B3	\$B2
B\$2 (ligne figée)	B\$2	C\$2

Prenons un exemple où les références mixtes sont indispensables : une table de multiplication, avec les facteurs 1, 2, 3 en ligne 1 et en colonne A. La formule de la cellule B2 est =B\$1*\$A2. Elle fige la **ligne** du facteur en haut (B\$1) et la **colonne** du facteur à gauche (\$A2) : copiée partout, elle fait toujours le produit de l'en-tête de colonne et de l'en-tête de ligne.

C3		fx		=C1 * A3	
	A	B	C	D	
1		1	2	3	
2	1	1	2	3	
3	2	2	4	6	
4	3	3	6	9	
Feuil1					

Figure 17: Une table de multiplication avec une seule formule recopiée : =B\$1*\$A2. En colonne et en ligne, les en-têtes (surlignés) restent figés. Maquette dessinée avec matplotlib, valeurs calculées par LibreOffice.

△ **Piège.** La cause la plus fréquente de résultats faux dans un tableur est une **référence qui devait être figée et ne l'était pas** : par exemple une cellule de taux de remise B1 utilisée dans =J2*B1, qui devient =J3*B2 à la ligne suivante, et multiplie par une cellule vide. Règle : toute cellule de paramètre (un taux, un seuil) est référencée avec des **dollars** ou, mieux, par un **nom** (sous-section suivante).

Pour s'entraîner. Cahier, chapitre 2 : exercice 2.1.

2.1.3 Plages nommées et tableaux structurés

Écrire Lignes!\$M\$2:\$M\$29828 est exact mais illisible, et fragile : si l'on ajoute une ligne en bas, la plage ne s'étend pas. Deux mécanismes règlent le problème.

La **plage nommée** donne un nom à une plage (menu *Formules* → *Gestionnaire de noms*, ou la zone de nom à gauche de la barre de formule) : après avoir nommé Montant la colonne M, la formule devient =SOMME(Montant). Le nom se lit comme une phrase, et il est stable quand on copie la formule.

Le **tableau structuré** (*Insertion* → *Tableau*, ou Ctrl+T) transforme la plage de données en objet : il reçoit un nom (ici Ventes), ses colonnes se citent par leur en-tête (Ventes[montant]), et surtout **il grandit tout seul** quand on ajoute des lignes en dessous. Toutes les formules, les tableaux croisés et les requêtes qui s'y réfèrent suivent.

```
=SOMME(Lignes!M2:M29828) → 1 324 763,72
=SOMME(Montant) → 1 324 763,72
=SOMME(Ventes[montant]) → 1 324 763,72
trois écritures, un même total : True
```

Les trois écritures donnent le même total, **1 324 763,72 €**. La différence est ailleurs : elle apparaît le jour où le classeur change. Les tableaux structurés sont la première des bonnes pratiques de la section 2.4.

2.1.4 Agréger sous conditions : SOMME.SI.ENS, NB.SI.ENS, MOYENNE.SI.ENS

La plupart des questions d'un analyste commencent par « combien ... **pour** ... ? ». Trois fonctions y répondent, dans leur version à conditions multiples (le suffixe .ENS signifie « ensemble de critères ») :

Question	Fonction	Syntaxe
Combien d'euros ?	SOMME.SI.ENS	(plage_à_sommer ; plage_critère1 ; critère1 ; ...)
Combien de lignes ?	NB.SI.ENS	(plage_critère1 ; critère1 ; ...)
Quelle moyenne ?	MOYENNE.SI.ENS	(plage_à_moyenner ; plage_critère1 ; critère1 ; ...)

△ La **plage à sommer vient en premier** dans SOMME.SI.ENS, alors qu'elle vient **en dernier** dans l'ancienne SOMME.SI(plage_critère ; critère ; plage_à_sommer). Cette inversion est une source d'erreurs classique.

Les critères sont des textes ou des nombres, éventuellement précédés d'un opérateur écrit **entre guillemets** : "Site", ">0", "<>" (non vide). Pour comparer à une date ou à une cellule, on **concatène** : ">="&DATE(2025;3;1). Les jokers * (une suite de caractères) et ? (un caractère) fonctionnent dans les critères de texte.

Voici les réponses à quelques questions de la gérante, calculées par LibreOffice :

```

=SOMME.SI.ENS(Lignes!M2:M29828;Lignes!E2:E29828;"Site") → 617 715,45
=SOMME.SI.ENS(Lignes!M2:M29828;Lignes!E2:E29828;"Boutique") → 560 973,91
=SOMME.SI.ENS(Lignes!M2:M29828;Lignes!E2:E29828;"Réseaux") → 146 074,36
=NB.SI.ENS(Lignes!L2:L29828;">0") → 4 606
=MOYENNE.SI.ENS(Lignes!M2:M29828;Lignes!I2:I29828;"Jardin") → 65,63
J
↪ =SOMME.SI.ENS(Lignes!M2:M29828;Lignes!I2:I29828;"Jardin";Lignes!C2:C29828;">="&DATE(2025;3;1);Lignes!C2:C29828;"
↪ → 14 788,58
=SOMME.SI(Lignes!H2:H29828;"Bougie*";Lignes!M2:M29828) → 36 833,11
J
↪ =NB.SI.ENS(Lignes!E2:E29828;"Site";Lignes!I2:I29828;"Décoration";Lignes!C2:C29828;">="&DATE(2025;11;1))
↪ → 971
=NB.SI.ENS(Lignes!M2:M29828;">=100") → 2 482

```

Lisons-les : le canal **Site** a rapporté 617 715,45 €, la **Boutique** 560 973,91 € et les **Réseaux** 146 074,36 € (ces trois montants s'additionnent bien en 1 324 763,72 € : c'est le premier **contrôle de cohérence** à toujours faire). Sur 29 827 lignes, 4 606 portent une remise ; une ligne de la catégorie Jardin pèse en moyenne 65,63 € ; les ventes de Jardin de mars 2025 valent 14 788,58 € ; les produits dont le nom commence par « Bougie » 36 833,11 € ; le Site a vendu 971 articles de décoration depuis le 1^{er} novembre ; et 2 482 lignes valent 100 € ou plus.

Chacun de ces chiffres a été recoupé par pandas (groupby, sum, mean) : les écarts maximaux sont nuls au centime près. C'est l'habitude à prendre : **un chiffre obtenu par deux chemins indépendants est un chiffre auquel on peut se fier.**

Les pièges de l'agrégation conditionnelle

- **Les plages doivent avoir exactement la même taille.** Si l'on écrit =SOMME.SI.ENS(M2:M29828 ; E2:E29818 ; "Site") (dix lignes de moins pour le critère), Excel renvoie #VALEUR! : ici, LibreOffice l'a confirmé (#VALEUR!). C'est un bon signe : l'erreur est **visible**.
- **Une plage trop courte, au contraire, ne dit rien.** =SOMME(M2:M29028) oublie les 800 dernières lignes et renvoie 1 291 124,89 € au lieu de 1 324 763,72 € : **33 638,83 € disparus** sans le moindre message. C'est l'erreur la plus courante des classeurs réels, et la raison pour laquelle on préfère les tableaux structurés.
- **Les critères sont sensibles aux détails :** "Site" ne trouve pas "site " (avec une espace à la fin). Nous verrons comment nettoyer en 2.1.7.
- **Les dates doivent être de vraies dates :** une date stockée comme du texte ("03/11/2025") n'est pas comparée comme une date (2.1.8).

Pour s'entraîner. Cahier, chapitre 2 : application 2.2, exercice 2.2.

2.1.5 Décider : SI, SI.CONDITIONS, ET, OU, SIERREUR

La fonction SI(test ; valeur_si_vrai ; valeur_si_faux) renvoie une valeur ou une autre selon un test. Pour ranger un panier dans une taille — « petit » en dessous de 40 €, « moyen » de 40 à 100 €, « gros » à partir de 100 € — on imbrique deux SI :

```
=SI(A2>=100 ; "gros" ; SI(A2>=40 ; "moyen" ; "petit"))
```

La fonction SI.CONDITIONS (Excel 2019 et suivants, à **vérifier** selon votre version) évite l'imbrication : =SI.CONDITIONS(A2>=100 ; "gros" ; A2>=40 ; "moyen" ; VRAI ; "petit"). Le dernier couple VRAI ; "petit" joue le rôle de la valeur par défaut : sans lui, un montant qui ne remplit aucune condition produit #N/A.

```

panier de 12 € → petit
panier de 45 € → moyen

```

```
panier de 99,90 € → moyen
panier de 150 € → gros
SI.CONDITIONS sur 99,90 € → moyen | ET(99,90 ≥ 40 ; 99,90 < 100) → VRAI | OU(12 ≥ 100 ; 150 ≥ 100) →
↪ VRAI
```

Les fonctions ET(...), OU(...) et NON(...) combinent des tests : elles renvoient VRAI ou FAUX, et se placent à l'intérieur d'un SI. L'erreur classique est d'écrire =SI(40<=A2<100 ; ...), qui n'est **pas** un test d'intervalle en Excel : il faut =SI(ET(A2>=40 ; A2<100) ; ...).

SIERREUR(formule ; valeur_de_replacement) remplace n'importe quelle erreur par une valeur : =SIERREUR(A6/B6 ; 0) renvoie 0 quand le dénominateur est nul. Utilisez-la **avec parcimonie** : elle **masque toutes les erreurs**, y compris celles qui révèlent un vrai problème (une référence cassée, un nom mal écrit). Si seule l'erreur « non trouvé » vous intéresse, préférez SI.NON.DISP(...), qui ne traite que #N/A.

Piège. Un SIERREUR qui renvoie 0 transforme une donnée **manquante** en donnée **égale à zéro**. Sur une moyenne ou une somme, cela fausse silencieusement le résultat. Préférez un message explicite ("à vérifier") ou une cellule vide, et comptez ensuite les erreurs.

2.1.6 Chercher : RECHERCHEV, INDEX+EQUIV, RECHERCHEX

Presque toutes les analyses croisent deux tables : ici, les lignes de vente ne contiennent que l'identifiant du produit, et le prix d'achat est dans la feuille Produits. Retrouver une information d'une table à partir d'une clé s'appelle une **recherche** (en base de données, une **jointure**, chapitre 3).

RECHERCHEV(clé ; table ; n° de colonne ; FAUX) cherche la clé dans la **première colonne** de la table et renvoie la valeur de la colonne numéro n. Son quatrième argument est le piège principal : FAUX (ou 0) demande la correspondance **exacte** ; VRAI (ou omis !) demande la correspondance **approchée**, qui suppose la première colonne **triée** et renvoie la plus grande valeur inférieure ou égale à la clé.

```
=RECHERCHEV(7;Produits!A2:F121;2;FAUX) → Moule mat
=RECHERCHEV(5;Ref!A2:B5;2;FAUX) → Cadre design
=RECHERCHEV(5;Ref!A2:B5;2;VRAI) → #N/A
=RECHERCHEV(Ref!G2;Ref!E2:F5;2;FAUX) → #N/A
=RECHERCHEV(SUPPRESPE(Ref!G2);Ref!E2:F5;2;FAUX) → 3
=RECHERCHEV(4;Bareme!A2:B4;2;VRAI) → 0,05
```

Lisons ces résultats. La clé 7 est trouvée exactement (Moule mat). Avec une table **non triée** (identifiants 7, 3, 12, 5), chercher la clé 5 en correspondance exacte donne bien Cadre design, mais **la même recherche en mode approché échoue** (#N/A) ; dans Excel, selon l'ordre des données, le résultat peut même être **faux sans erreur** : c'est pourquoi il ne faut **jamais oublier le FAUX**. Une clé tapée avec une espace de trop ("Bo1 design ") ne se trouve pas (#N/A) tant qu'on ne la nettoie pas avec SUPPRESPE. Enfin, la correspondance approchée a un **bon usage** : lire un **barème** trié, comme une remise par palier de quantité (1 → 0 %, 3 → 5 %, 10 → 10 %) ; ici, une quantité de 4 donne 5 %.

Trois autres limites de RECHERCHEV : elle **ne regarde qu'à droite** de la colonne clé ; le **numéro de colonne** est écrit en dur, donc **insérer une colonne casse la formule sans prévenir** ; et une clé numérique (7) ne correspond pas à la même clé écrite comme du texte ("7") dans Excel (l'erreur est #N/A ; LibreOffice, plus indulgent, convertit silencieusement : le **comportement diffère**, nous ne pouvons donc pas le montrer ici et il est à tester dans votre Excel).

INDEX et **EQUIV** se combinent pour faire mieux : EQUIV(clé ; colonne_clés ; 0) renvoie la

position de la clé, et INDEX(colonne_résultat ; position) renvoie la valeur à cette position. La colonne résultat peut être **n'importe où** (à gauche aussi), et rien ne casse si l'on insère une colonne.

RECHERCHEX(clé ; colonne_clés ; colonne_résultat ; si_non_trouvé) (Excel 2021 et Microsoft 365, à vérifier pour les versions antérieures) cumule les avantages : correspondance exacte par défaut, valeur de remplacement intégrée, recherche dans les deux sens.

```
=INDEX(Produits!B2:B121;EQUIV(7;Produits!A2:A121;0)) → Moule mat
=RECHERCHEX(7;Produits!A2:A121;Produits!B2:B121;"absent") → Moule mat
=RECHERCHEX(999;Produits!A2:A121;Produits!B2:B121;"absent") → absent
=RECHERCHEX("Bol design";Produits!B2:B121;Produits!A2:A121;"absent") → 3
```

La dernière ligne illustre la recherche « à l'envers » : retrouver l'identifiant (3) d'un produit à partir de son nom, ce que RECHERCHEV ne sait pas faire.

⚠ **Mais attention : un nom n'est pas une clé.** La recherche précédente renvoie l'identifiant 3, la **première** correspondance, sans prévenir qu'il y en a d'autres. Vérifions l'unicité des noms dans le catalogue :

catalogue : 120 produits, 60 noms distincts, 120 produits dont le nom est partagé

Le catalogue compte **120 produits mais seulement 60 noms distincts** : chaque nom est porté par deux produits (de prix différents). Une recherche par nom renvoie donc l'un des deux, silencieusement. Une **clé de recherche doit être unique** : ici l'identifiant du produit. Nous retrouverons cette anomalie, avec ses conséquences sur une jointure, en section 2.3.5.

Enrichir les lignes : le coût d'achat et la marge

Ajoutons à chaque ligne de vente le **coût d'achat unitaire** du produit, par =RECHERCHEX(G2 ; Produits!A:A ; Produits!E:E) (l'identifiant du produit est en colonne G) recopiée sur les 29 827 lignes dans une nouvelle colonne N. On obtient alors la **marge** de l'année : le chiffre d'affaires moins la somme des quantités multipliées par le coût unitaire.

N2	fx	=RECHERCHEX(G2;Produits!A:A;Produits!E:E)					
	A	G	H	J	K	M	N
1	id_ligne	id_produit	nom_produit	quantite	prix_unitaire	montant	cout_u
2	54079	43	Cadre design	1	24,62	24,62	15,47
3	54080	12	Poêle mat	1	39,04	39,04	20,55
4	54081	90	Hamac rustique	1	74,06	74,06	31,91
5	54082	37	Panier mat	1	38,01	38,01	19,96
6	54083	24	Étagère compact	1	34,92	34,92	15,21
7	54084	42	Vase mat	1	29,77	29,77	13,97
8	54085	2	Poêle mat	1	40,07	40,07	21,47

Lignes Produits Clients

Figure 18: Une colonne ajoutée par recherche : le coût d'achat unitaire de chaque ligne, retrouvé dans la feuille Produits à partir de l'identifiant du produit. Extrait : seules quelques colonnes de la feuille sont affichées. Maquette dessinée avec matplotlib, valeurs issues du classeur.

marge de l'année : 639 810,88 € (CA 1 324 763,72 €)
dont catégorie Jardin : 171 875,43 €

La **marge de 2025** s'élève à **639 810,88 €**, dont **171 875,43 €** pour la seule catégorie Jardin ; les deux

chiffres sont recoupés par pandas (fusion des tables, puis somme). La marge est ici calculée sur le prix de vente tel quel : nous ignorons la TVA, simplification que nous reprendrons en section 2.2.4.

Pour s'entraîner. Cahier, chapitre 2 : application 2.3, exercices 2.3 et 2.4.

2.1.7 Nettoyer du texte

Les données réelles sont sales : espaces en trop, majuscules incohérentes, nombres écrits comme du texte. Quelques fonctions de texte suffisent à corriger l'essentiel.

Besoin	Fonction	Exemple (cellule B2 = " bÔÎTE rustique ")
Enlever les espaces superflus	SUPPRESPE	SUPPRESPE(B2)
Normaliser la casse	MAJUSCULE, MINUSCULE, NOMPROPRE	NOMPROPRE(SUPPRESPE(B2))
Extraire un morceau	GAUCHE, DROITE, STXT	GAUCHE(A2 ; 1)
Mesurer, repérer	NBCAR, TROUVE, CHERCHE	TROUVE("-" ; F2)
Remplacer	SUBSTITUE	SUBSTITUE(E2 ; ", " ; ".")
Assembler	CONCAT, JOINDRE.TEXTE, opérateur &	CONCAT(A2 ; "-" ; D2)
Convertir en nombre	CNUM	CNUM(E2)

```
=NOMPROPRE(SUPPRESPE(Texte!B2)) → Boîte Rustique
=GAUCHE(Texte!A2;1) → T
=CNUM(DROITE(Texte!A2;NBCAR(Texte!A2)-1)) → 33 133
=NBCAR(Texte!B2) → 17
=STXT(Texte!C2;1;3) → abc
=CONCAT(Texte!A2;"-";Texte!D2) → T33133-77
=JOINDRE.TEXTE(" | ";VRAI;SUPPRESPE(Texte!B2);SUPPRESPE(Texte!B3);SUPPRESPE(Texte!B4)) →
↳ bÔÎTE rustique | moule MAT | Plaid NORDIQUE
=TROUVE("-";Texte!F2) → 2
=CNUM(Texte!E2) → 12,50
```

L'exemple le plus courant est le **numéro de ticket** de l'export de caisse : T33133, un code qui commence par une lettre. Pour obtenir le numéro seul, on enlève le premier caractère et on convertit : =CNUM(DROITE(A2 ; NBCAR(A2) - 1)) donne 33 133. De même, le nom d'article bÔÎTE rustique devient Boîte Rustique avec NOMPROPRE(SUPPRESPE(...)) : l'espace en tête et les doubles espaces disparaissent, et la casse est homogène. TROUVE est sensible à la casse et renvoie #VALEUR! si le texte n'existe pas ; CHERCHE ne l'est pas et accepte des jokers.

⚠ **Piège.** SUPPRESPE ne supprime pas l'**espace insécable** (code 160) que l'on obtient en copiant des données depuis une page web ou un PDF. Il faut alors SUBSTITUE(A2 ; CAR(160) ; " ") avant. Les décimales posent un autre piège : "12,5" est un nombre en Excel français, mais un texte qui **ne se convertit pas** en Excel anglais, où le séparateur décimal est le point. Quand vous échangez des fichiers entre langues, **testez**.

2.1.8 Manipuler des dates

Pour Excel, une date est un **numéro de série** : le nombre de jours écoulés depuis le 1^{er} janvier 1900 (système par défaut de Windows, dans lequel le 1^{er} janvier 1900 vaut 1). Le 1^{er} janvier 2025 vaut **45 658** ; la cellule n'affiche une date que grâce à son **format**. Ce choix rend les calculs triviaux (la différence de deux dates est un nombre de jours) mais cache deux pièges : on peut afficher un nombre à la place d'une date (si le format est « Standard »), et inversement un nombre peut s'afficher comme une date.

Un héritage historique. Excel traite par erreur 1900 comme une année bissextile (pour rester compatible avec un ancien tableur) : les numéros de série d'avant le 1^{er} mars 1900 sont décalés d'un jour. Sans conséquence pour des données récentes ; sachez-le si vous travaillez sur des dates anciennes. Les Mac utilisaient autrefois un système différent (point de départ en 1904) : une option du classeur le permet encore.

Besoin	Fonction	Résultat
Extraire l'année, le mois, le jour	ANNEE, MOIS, JOUR	ANNEE(3/11/2025) = 2025
Fabriquer une date	DATE(année ; mois ; jour)	DATE(2025 ; 1 ; 1) = 45 658
Dernier jour du mois	FIN.MOIS(date ; 0)	FIN.MOIS(10/02/2025 ; 0) = 28/02/2025
Décaler de n mois	MOIS.DECALER(date ; n)	MOIS.DECALER(31/01/2025 ; 1) = 28/02/2025
Jour de la semaine	JOURSEM(date ; 2)	lundi = 1
Écart	DATEDIF(début ; fin ; "M")	mois entiers écoulés
Jours ouvrés	NB.JOURS.OUVRES(début ; fin)	exclut samedis et dimanches
Afficher	TEXTE(date ; "mmm")	« mars »

```
=DATE(2025;1;1)*1 → 45 658
=FIN.MOIS(DATE(2025;2;10);0) → 45 716
=MOIS.DECALER(DATE(2025;1;31);1) → 45 716
=JOURSEM(DATE(2025;11;3);2) → 1
=TEXTE(DATE(2025;11;3);"dddd") → lundi
=DATEDIF(DATE(2025;1;15);DATE(2025;11;3);"M") → 9
=DATEDIF(DATE(2025;1;15);DATE(2025;11;3);"D") → 292
=NB.JOURS.OUVRES(DATE(2025;11;3);DATE(2025;11;9)) → 5
=TEXTE(DATE(2025;3;15);"mmm") → mars
```

Les résultats sont des numéros de série, que l'on met en forme de date pour les lire (FIN.MOIS donne 45 716, c'est-à-dire le 28 février 2025) ; le 3 novembre 2025 est un **lundi** (JOURSEM(...;2) renvoie 1), il y a **9 mois entiers** et **292 jours** entre le 15 janvier et le 3 novembre, et la semaine du 3 au 9 novembre compte **5 jours ouvrés**. Les numéros de série sont recoupés par pandas (écart de dates).

DATEDIF est une fonction **non documentée dans l'aide** d'Excel mais qui fonctionne ; évitez son argument "MD", dont les résultats sont réputés incohérents.

Dates en texte : le piège du format régional

Quand un export écrit les dates comme 03/11/2025, Excel les reconnaît comme des dates **si** le format correspond à la langue du poste : en français, jour/mois/année ; en anglais américain, mois/jour/année. La chaîne 11/03/2025 est donc le **11 mars** dans un Excel français et le **3 novembre** dans un Excel américain. Si Excel ne la reconnaît pas, elle reste du **texte** : une SOMME de ces cellules vaut 0, mais DATEVALUE les convertit.

somme de deux dates stockées en texte : 0
 somme après DATEVALUE (03/11/2025 + 11/03/2025) : 91 691 = 45 964 + 45 727
 11/03/2025 lu par un Excel français : 45 727 (le 11 mars 2025)

Notre classeur est en français : 11/03/2025 donne 45 727, le 11 mars. **Vérifiez toujours** quelques dates connues (un 25, un 31 : ceux-là ne peuvent pas être des mois) après un import : c'est le test le plus rapide pour détecter une inversion jour/mois.

△ **Piège.** Les codes de format dans TEXTE dépendent de la **langue** : en français, l'année s'écrit aaaa et le jour jjjj ; en anglais yyyy et dddd. Un classeur partagé entre un poste français et un poste anglais peut donc afficher #VALEUR! ou un format inattendu. Dans ce livre, nous utilisons les codes valables dans les deux langues (mmm pour le mois).

Pour s'entraîner. Cahier, chapitre 2 : application 2.4, exercice 2.5.

2.1.9 Les formules à résultats multiples : FILTRE, TRIER, UNIQUE, LET

Dans Excel 2021 et Microsoft 365, certaines fonctions renvoient **plusieurs valeurs** qui « débordent » (*spill*) dans les cellules voisines : UNIQUE(plage) liste les valeurs distinctes, TRIER(plage) les range, FILTRE(plage ; condition) extrait les lignes qui vérifient une condition, SEQUENCE(n) génère une suite de nombres. Avant ces fonctions, il fallait des astuces (formules matricielles, Ctrl+Maj+Entrée).

La fonction LET donne un **nom** à un résultat intermédiaire à l'intérieur d'une formule, ce qui la rend lisible et évite de recalculer deux fois la même chose : =LET(somme ; SOMME(Montant) ; nb ; NBVAL(A:A)-1 ; somme/nb) calcule le montant moyen par ligne.

```
=NBVAL(UNIQUE(Lignes!I2:I29828)) → 6
=SOMME(FILTRE(Lignes!M2:M29828;Lignes!E2:E29828="Site")) → 617 715,45
=LET(a;SOMME(Lignes!M2:M29828);b;NBVAL(Lignes!A2:A29828);ARRONDI(a/b;2)) → 44,41
```

Les résultats scalaires ont été vérifiés : il y a **6 catégories** distinctes, le total des lignes du **Site** est de 617 715,45 € (le même que SOMME.SI.ENS, trouvé par un autre chemin), et le **montant moyen d'une ligne** est de **44,41 €**. En revanche, le **débordement lui-même** (la liste affichée sur plusieurs cellules) n'est pas vérifiable avec notre outil : nous l'avons recoupé par pandas (unique, sort_values, filtre booléen), qui donne les mêmes lignes. Ces fonctions sont absentes des versions anciennes d'Excel : un classeur qui les utilise **ne se recalcule pas** chez un collègue qui en possède une (△ à vérifier avant de partager).

2.1.10 Les erreurs, et la correspondance anglais-français

Excel signale une formule impossible par un code d'erreur commençant par #. Les reconnaître vous fait gagner un temps considérable.

```
=1/0 → #DIV/0!
=RECHERCHEV("zz";Produits!A2:B121;2;FAUX) → #N/A
="a"+1 → #VALUE!
=INDEX(Produits!A:A;2000000) → #REF!
=NOMINCONNU(1) → #NAME?
=SIERREUR(1/0;"n/a") → n/a
```

Code (Excel français)	Signification	Cause typique
#DIV/0!	division par zéro	dénominateur vide ou nul
#N/A	valeur non disponible	recherche sans résultat

Code (Excel français)	Signification	Cause typique
#VALEUR!	mauvais type d'argument	texte à la place d'un nombre, plages de tailles différentes
#REF!	référence invalide	ligne ou colonne supprimée, INDEX hors plage
#NOM?	nom inconnu	faute de frappe dans une fonction ou un nom
#NOMBRE!	valeur numérique impossible	racine carrée d'un négatif
#NUL!	intersection vide	espace utilisé à la place de ; ou :
#DÉBOREMENT!	résultat qui déborde sur des cellules occupées	formule à résultats multiples bloquée
#####	colonne trop étroite	pas une erreur de calcul : élargir la colonne

Le bloc précédent a produit cinq de ces erreurs avec LibreOffice (division par zéro, recherche sans résultat, texte plus un nombre, référence hors plage, nom inconnu) : #DIV/0!, #N/A, #VALEUR!, #REF! et #NAME? (le nom anglais de #NOM?). Une **différence** connue : la racine carrée d'un nombre négatif renvoie #NOMBRE! dans Excel et #VALEUR! dans LibreOffice ; la liste ci-dessus suit Excel et n'a pas pu être entièrement vérifiée.

Pour **chercher** les erreurs dans un classeur : ESTERREUR(cellule) renvoie VRAI ou FAUX ; le compte =SOMMEPROD(--ESTERREUR(plage)) dénombre les erreurs d'une plage ; l'outil *Audit de formules* (onglet *Formules*) trace les antécédents d'une cellule. Un classeur propre n'affiche **aucune erreur** : un #N/A laissé en place est une alarme qu'on a éteinte.

Correspondance anglais–français

Un fichier .xlsx stocke les **noms anglais** des fonctions et le séparateur , ; Excel les affiche dans la langue de l'utilisateur. Quand vous lisez un tutoriel en anglais, voici les équivalents des fonctions de ce chapitre.

Anglais	Français	Anglais	Français
SUM, SUMIFS, SUMIF	SOMME, SOMME.SI.ENS, SOMME.SI	LEFT, RIGHT, MID	GAUCHE, DROITE, STXT
COUNTIFS, COUNTBLANK	NB.SI.ENS, NB.VIDE	TRIM, PROPER, LEN	SUPPRESSESPACE, NOMPROPRE, NBCAR
AVERAGEIFS	MOYENNE.SI.ENS	FIND, SUBSTITUTE, VALUE	TROUVE, SUBSTITUE, CNUM
IF, IFS, IFERROR	SI, SI.CONDITIONS, SIERREUR	CONCAT, TEXTJOIN, TEXT	CONCAT, JOINDRE.TEXTE, TEXTE
AND, OR, NOT	ET, OU, NON	YEAR, MONTH, EOMONTH	ANNEE, MOIS, FIN.MOIS
VLOOKUP, XLOOKUP	RECHERCHEV, RECHERCHEX	EDATE, WEEKDAY, NETWORKDAYS	MOIS.DECALER, JOURSEM, NB.JOURS.OUVRES
INDEX, MATCH	INDEX, EQUIV	FILTER, SORT, UNIQUE	FILTRE, TRIER, UNIQUE
SUMPRODUCT, LET	SOMMEPROD, LET	TRUE, FALSE	VRAI, FAUX

À retenir.

- Une formule est une **recette** : les références relatives se décalent, les références absolues (\$) et les **noms** ne bougent pas ; un tableau structuré **grandit** tout seul.
- SOMME.SI.ENS : la plage à sommer **vient d'abord** ; les plages doivent avoir **la même taille** ; une plage trop courte ne signale **rien**.
- RECHERCHEV : toujours FAUX pour l'exacte ; elle regarde à droite et casse si l'on insère

une colonne ; préférez INDEX+EQUIV ou RECHERCHEX.

- SIERREUR masque **toutes** les erreurs : utilisez-la avec parcimonie.
- Une date est un **numéro de série** ; vérifiez le format jour/mois après un import.
- **Chaque chiffre important se recoupe par un second chemin** (ici pandas) : sur le classeur, 29 827 lignes, 1 324 763,72 €, un écart nul sur toutes les formules vérifiées.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.2 à 2.4, exercices 2.1 à 2.5.

2.2 Tableaux croisés dynamiques et graphiques croisés dynamiques

Le **tableau croisé dynamique** (TCD, *PivotTable* en anglais) est l'outil le plus puissant d'Excel pour un analyste : il résume des milliers de lignes en un tableau de quelques lignes et colonnes, **sans écrire une seule formule**, et se remanie en deux clics de souris. C'est lui qui répond à la question de la gérante : « par catégorie et par mois, qu'avons-nous vendu ? ». Cette section explique comment il raisonne, comment le construire, et surtout comment **se méfier de ce qu'il affiche**.

Ce qui est vérifié, et ce qui ne l'est pas. Le tableau croisé dynamique est un objet propre à Excel, que notre outil de vérification ne sait pas construire. Pour chaque TCD de cette section, nous donnons donc **trois choses** : la description des gestes à faire, le **résultat que le TCD doit afficher** (calculé en pandas avec `pivot_table`), et un **recoupement par des formules SOMME.SI.ENS** calculées par LibreOffice. Si votre TCD affiche autre chose, c'est le TCD (ou sa source) qui est en cause.

2.2.1 Le principe : lignes, colonnes, valeurs, filtres

Un tableau croisé dynamique prend une **table de données** (une ligne par observation, une colonne par variable) et la regroupe selon **quatre zones** que l'on remplit en faisant glisser des champs :

- **Lignes** : les champs dont chaque valeur distincte devient une ligne du tableau (par exemple catégorie) ;
- **Colonnes** : les champs dont chaque valeur distincte devient une colonne (par exemple canal) ;
- **Valeurs** : le champ que l'on agrège (par exemple montant) et la **manière** de l'agréger : somme, nombre, moyenne, minimum, maximum... ;
- **Filtres** : des champs qui restreignent les lignes prises en compte (par exemple année), sans apparaître dans le tableau.

Pour chaque combinaison (ligne, colonne), le TCD prend les lignes de la table qui ont ces valeurs et leur applique l'agrégation. C'est exactement l'opération que SOMME.SI.ENS fait cellule par cellule ; le TCD la fait **pour toutes les cellules à la fois**.

Un TCD à la main sur huit lignes

Prenons huit lignes de vente (catégorie, canal, montant) :

#	Catégorie	Canal	Montant
1	Cuisine	Boutique	40
2	Cuisine	Site	25

#	Catégorie	Canal	Montant
3	Jardin	Boutique	80
4	Jardin	Site	60
5	Cuisine	Boutique	10
6	Jardin	Site	30
7	Cuisine	Site	15
8	Jardin	Boutique	20

Avec *categorie* en lignes, *canal* en colonnes et la **somme** du montant en valeurs, chaque case regroupe les lignes concernées : Cuisine × Boutique = 40 + 10 = 50 ; Cuisine × Site = 25 + 15 = 40 ; Jardin × Boutique = 80 + 20 = 100 ; Jardin × Site = 60 + 30 = 90. Les totaux de lignes et de colonnes s'obtiennent en additionnant (Cuisine = 90, Jardin = 190, Boutique = 150, Site = 130), et le total général est **280**, qui doit être égal à la somme des huit montants : 40 + 25 + 80 + 60 + 10 + 30 + 15 + 20 = 280. Ce **contrôle** (total du TCD = total de la source) est le plus important de toute la section.

2.2.2 Construire un tableau croisé pas à pas

Sur le classeur de la gérante, voici la marche à suivre (les noms de menus varient selon la version : à **vérifier** sur votre poste) :

1. Cliquez dans une cellule de la table de données ; si ce n'est pas déjà un tableau structuré, faites Ctrl+T pour en faire un (c'est ce qui permettra au TCD de suivre les nouvelles lignes).
2. Onglet *Insertion*, bouton *Tableau croisé dynamique* ; choisissez une **nouvelle feuille** comme destination.
3. Dans le volet des champs, glissez *categorie* dans *Lignes*, *canal* dans *Colonnes* et *montant* dans *Valeurs*.
4. Excel affiche par défaut **Somme de montant** (si la colonne ne contient que des nombres) ; un clic sur le champ permet de choisir une autre agrégation et le **format** des nombres (séparateur de milliers, deux décimales).

Le résultat attendu, calculé par pandas :

```
pt = L.pivot_table(index="categorie", columns="canal", values="montant", aggfunc="sum", margins=True,
↳ margins_name="Total")
print(pt.round(2).to_string())
```

canal	Boutique	Réseaux	Site	Total
categorie				
Bien-être	48682.72	13670.69	55157.43	117510.84
Cuisine	98565.17	25137.50	109609.91	233312.58
Décoration	111898.40	28008.19	118835.74	258742.33
Jardin	149067.30	38686.18	166201.16	353954.64
Maison	129080.56	34744.95	140788.49	304614.00
Papeterie	23679.76	5826.85	27122.72	56629.33
Total	560973.91	146074.36	617715.45	1324763.72

La question qui compte : **peut-on se fier à ce tableau ?** Recoupons-le avec 18 formules SOMME.SI.ENS (une par case), calculées par LibreOffice :

18 cases SOMME.SI.ENS contre le tableau croisé : écart maximal 0.0 €
somme des 18 cases : 1 324 763,72 | total de la source : 1 324 763,72

Les 18 cases sont identiques au centime près, et leur somme retombe sur le total de la source, **1 324 763,72 €**. La lecture est immédiate : le Jardin est la première catégorie (353 954,64 €), devant la

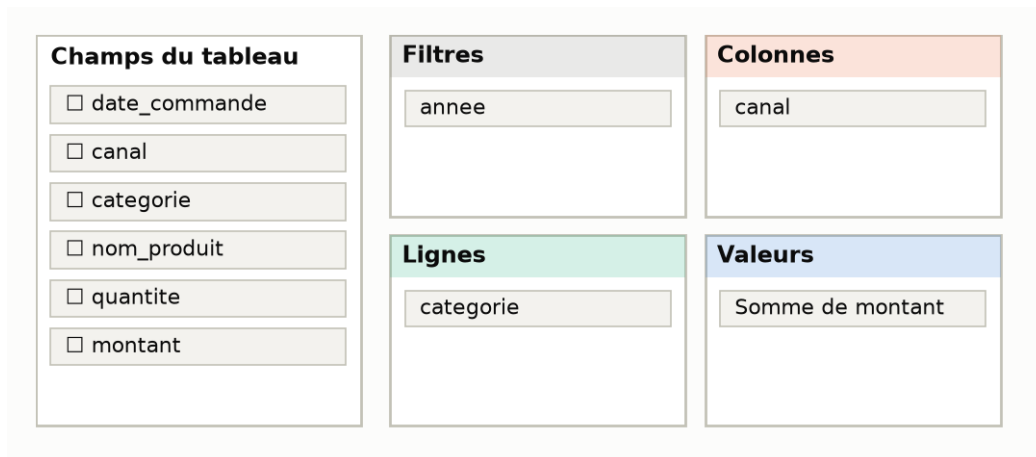


Figure 19: Le volet des champs d'un tableau croisé dynamique : on fait glisser les champs de la table dans les quatre zones. Schéma dessiné avec matplotlib, pas une capture d'Excel (les libellés varient avec la version et la langue).

Maison (304 614,00 €) et la Décoration (258 742,33 €) ; le Site pèse 617 715,45 €, devant la Boutique (560 973,91 €).

A3	fx				
	A	B	C	D	E
1	Somme de montant	Étiquettes de colonnes			
2	Étiquettes de lignes	Boutique	Réseaux	Site	Total général
3	Bien-être	48 682,72	13 670,69	55 157,43	117 510,84
4	Cuisine	98 565,17	25 137,50	109 609,91	233 312,58
5	Décoration	111 898,40	28 008,19	118 835,74	258 742,33
6	Jardin	149 067,30	38 686,18	166 201,16	353 954,64
7	Maison	129 080,56	34 744,95	140 788,49	304 614,00
8	Papeterie	23 679,76	5 826,85	27 122,72	56 629,33
9	Total général	560 973,91	146 074,36	617 715,45	1 324 763,72
	TCD	Lignes			

Figure 20: Le tableau croisé dynamique attendu : catégories en lignes, canaux en colonnes, somme du montant, totaux généraux. Maquette dessinée avec matplotlib à partir de valeurs calculées (pas une capture d'Excel).

Observons enfin ce que le tableau ne montre pas : **les proportions**. La section 2.2.4 y revient.

2.2.3 Regrouper les dates : par mois, par trimestre, par année

La gérante voulait un tableau **par mois**. Mettre date_commande en lignes produit 365 lignes, une par jour. Excel (dans ses versions récentes) regroupe souvent les dates automatiquement en mois, trimestres et années ; sinon, un clic droit sur une date puis *Grouper* permet de choisir les niveaux (secondes, minutes, heures, jours, **mois**, **trimestres**, **années**). Les dates doivent être de **vraies dates** (2.1.8) : une colonne contenant du texte ne se regroupe pas.


```
mp = L.assign(mois=L["date_commande"].dt.to_period("M")).pivot_table(index="mois",
↪ columns="categorie", values="montant", aggfunc="sum", margins=True, margins_name="Total")
print(mp.round(0).astype(int).iloc[[0, 1, 5, 6, 11, 12]].to_string())
```

categorie	Bien-être	Cuisine	Décoration	Jardin	Maison	Papeterie	Total
mois							
2025-01	10816	19475	19653	7455	27648	4132	89179
2025-02	7990	15519	16774	7649	20964	3746	72642
2025-06	6972	14775	13858	48204	19339	3782	106930
2025-07	6597	13837	13555	56711	17619	2902	111221
2025-12	17997	33689	60891	18763	44688	7817	183845
Total	117511	233313	258742	353955	304614	56629	1324764

On lit trois histoires dans ce tableau. La catégorie **Jardin** est saisonnière : 7 455 € en janvier, 56 711 € en juillet, puis 18 763 € en décembre. La **Décoration** a un pic en décembre (60 891 €, plus du double de novembre, 27 247 €). Et le **total** culmine en décembre à 183 845 €, soit plus du double de février (72 642 €).

72 cases (6 catégories × 12 mois) SOMME.SI.ENS contre le tableau croisé : écart maximal 0.0 €
somme des 72 cases : 1 324 763,72

Les 72 formules (une par case, avec des bornes de dates ">=&DATE(...)" et "<&DATE(...)" donnent les mêmes valeurs que le regroupement. Si l'on souhaite un résumé par trimestre : 251 609 € au premier trimestre, 310 783 € au deuxième, 314 572 € au troisième et 447 800 € au quatrième.

par trimestre : {1: 251609, 2: 310783, 3: 314572, 4: 447800}

part de la Boutique selon la catégorie : de 41.4 à 43.2 % | du Site : de 45.9 à 47.9 %

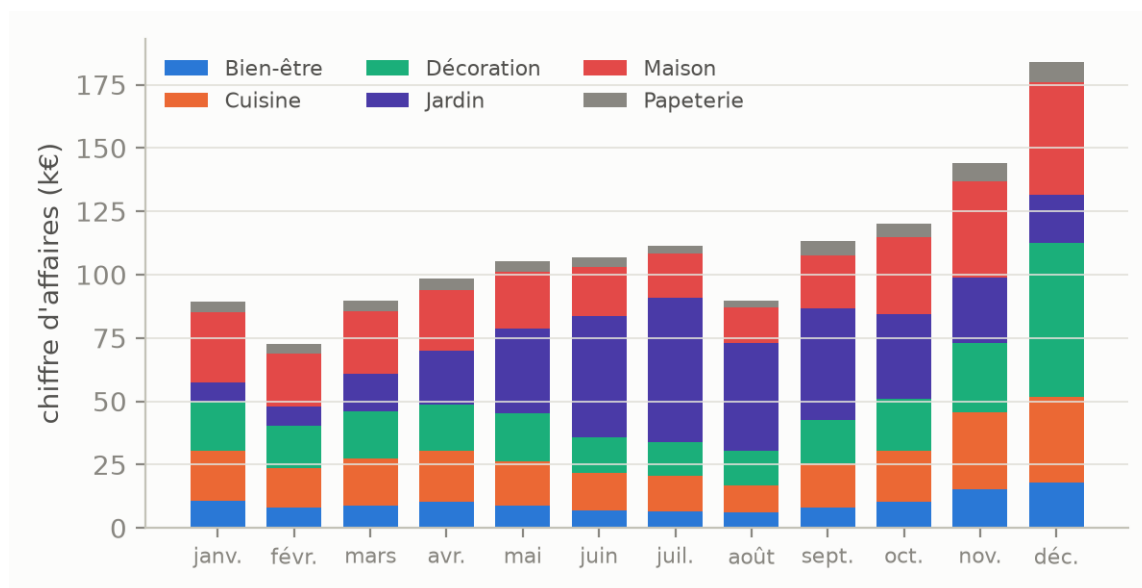


Figure 21: Un graphique croisé dynamique : le chiffre d'affaires de 2025 par mois et par catégorie (empilé). Le même tableau que ci-dessus, dessiné avec matplotlib.

Le **graphique croisé dynamique** est simplement le graphique d'un TCD : il en suit les lignes, les colonnes et les filtres, et se met à jour avec lui. C'est un bon outil d'exploration, moins bon pour la présentation finale (volume IV de la série (visualisation et communication)).

Pour s'entraîner. Cahier, chapitre 2 : application 2.5, exercice 2.6.

2.2.4 Afficher des proportions, calculer dans un TCD

Dans la zone *Valeurs*, le menu *Afficher les valeurs* remplace les montants par des **proportions** : % du total général, % du total de la colonne, % du total de la ligne, écart par rapport à une période précédente, cumul... Elles se calculent à partir des mêmes cases.

```
% du total général (Jardin × Site) : 12.5 % | Site (colonne) : 46.6 %
% du total de la ligne, Jardin : {'Boutique': 42.1, 'Réseaux': 10.9, 'Site': 47.0}
% du total de la ligne, toutes catégories : {'Boutique': 42.3, 'Réseaux': 11.0, 'Site': 46.6}
```

La lecture change tout. En **% du total général**, la case Jardin × Site vaut 12,5 % : c'est la plus grosse case, mais ce chiffre mélange la taille de la catégorie et celle du canal. En **% du total de la ligne**, on voit que **toutes les catégories ont à peu près le même partage entre canaux** (de 41 % à 43 % en Boutique, de 46 % à 48 % sur le Site) : les canaux ne se spécialisent pas par catégorie. Le Site pèse 46,6 % du total, la Boutique 42,3 %, les Réseaux 11,0 %. Quelle proportion choisir dépend de la question posée : **la bonne proportion est celle dont le dénominateur correspond à la question**.

Les champs calculés : attention au ratio de sommes

Un **champ calculé** ajoute au TCD une colonne définie par une formule sur les autres champs (par exemple = marge / montant). Mais le TCD **somme d'abord, puis calcule** : un champ calculé marge / montant donne le **rapport des sommes**, et non la **moyenne des rapports** ligne par ligne. Les deux ne coïncident pas :

	moyenne des taux de ligne	rapport des sommes
categorie		
Bien-être	46.3	45.9
Cuisine	47.6	47.8
Décoration	48.6	49.8
Jardin	49.0	48.6
Maison	48.8	48.2
Papeterie	48.2	47.4

Pour la **Décoration**, la moyenne des taux de marge par ligne est de 48,6 % ; le rapport des sommes (ce que calcule un champ calculé) est de 49,8 %. La différence vient de ce que les lignes chères ont une marge relative plus forte et pèsent plus dans le rapport des sommes que dans la moyenne simple. **Aucun des deux n'est « faux »** : ils répondent à des questions différentes (« le taux de marge d'une ligne type » ou « la marge de la catégorie »). Il faut savoir lequel on affiche.

Autre limite : un champ calculé ne peut pas compter des **valeurs distinctes** (le nombre de clients différents, par exemple). Pour cela il faut le modèle de données de Power Pivot (section 2.5) ou un autre outil.

2.2.5 Segments, chronologies et pièges

Les **segments** (*slicers*) sont des boutons de filtre posés à côté du TCD (un bouton par canal, par exemple) ; la **chronologie** (*timeline*) est un curseur de dates. Ils rendent un TCD utilisable par quelqu'un qui ne le connaît pas : c'est le moyen le plus simple de fabriquer un petit tableau de bord interactif.

Les erreurs les plus fréquentes avec un TCD ne viennent pas du TCD, mais de sa **source** :

- **Le TCD ne s'actualise pas tout seul**. Si les données changent, il faut cliquer sur *Actualiser* (Alt+F5) ; un TCD daté de la semaine dernière sur des données de cette semaine est un grand classique. On peut demander l'actualisation à l'ouverture du fichier.
- **Une source en plage fixe** n'inclut pas les nouvelles lignes ; un **tableau structuré** (2.1.3) règle le problème.

- **Les doublons gonflent les totaux sans bruit.** Si 500 lignes sont copiées deux fois dans la source, le total passe de 1 324 763,72 € à 1 345 630,05 € (+ 20 866,33 €) et personne ne reçoit de message. Le contrôle du **nombre de lignes** (29 827 attendu) et de l'**unicité** de l'identifiant de ligne les détecte.
- **Les cellules vides** de la source comptent comme « (vide) » dans les lignes et colonnes, et faussent les moyennes si on les confond avec des zéros.
- **Les nombres stockés comme du texte** ne se somment pas : un TCD affichera alors **Nombre de** (un compte) au lieu de **Somme de**, ce qui doit vous alerter.

doublons : total de la source 1 324 763,72 → avec 500 lignes en double 1 345 630,05 | lignes en double
↪ détectées : 500
nombres stockés comme du texte (12,5 ; 30 ; 7) : SOMME = 0 | NBVAL = 3 | SOMME après CNUM = 49,50

La dernière ligne reproduit ce piège : trois valeurs écrites comme du texte (12,5, 30, 7) donnent une somme de 0 et un compte de 3 ; après conversion par CNUM, la somme retombe sur 49,5. Toute somme à zéro sur une colonne que l'on croyait remplie mérite un coup d'œil au **type** des cellules.

À retenir.

- Un TCD **regroupe** une table selon des champs en lignes, colonnes et filtres, et **agrège** un champ de valeurs ; chaque case est un SOMME.SI.ENS.
- Vérifiez toujours : **total du TCD = total de la source**, et **nombre de lignes** attendu ; sur nos données, 18 cases et 72 cases recoupées, écart nul, total 1 324 763,72 €.
- Une **proportion** n'a de sens qu'avec le bon dénominateur ; un **champ calculé** donne un rapport de sommes, pas une moyenne de rapports.
- Les pièges viennent de la **source** : actualisation oubliée, plage fixe, doublons, vides, nombres en texte.

Pour s'entraîner. Cahier, chapitre 2 : application 2.5, exercices 2.6 et 2.7.

2.3 Power Query pour l'import et la transformation

Chaque lundi, la gérante reçoit l'export de la caisse, l'ouvre, supprime à la main les lignes de titre, retire la ligne de total, corrige les majuscules, convertit les virgules en points... puis recommence la semaine suivante. **Power Query** est l'outil d'Excel qui remplace ces gestes par une **recette enregistrée** : on la construit une fois, on la rejoue en un clic sur chaque nouvel export. Cette section en explique le principe, montre la recette sur l'export de caisse, et vérifie chaque étape avec pandas.

Ce qui est vérifié, et ce qui ne l'est pas. Power Query est un composant d'Excel (et de Power BI) que nous n'avons pas pu exécuter. Les **scripts en langage M** de cette section sont donc **non exécutés** et leur syntaxe est à **vérifier** dans votre version. En revanche, **chaque étape est reproduite en pandas** sur le vrai fichier export_caisse_brut.csv, avec le nombre de lignes à chaque étape et un contrôle final : ce sont les résultats que votre requête doit donner.

2.3.1 L'idée : une recette d'étapes enregistrées

Power Query (*Données → Obtenir des données*) ouvre un éditeur dans lequel chaque transformation (supprimer des lignes, changer un type, scinder une colonne, fusionner deux tables...) devient une **étape appliquée**, listée dans un volet à droite. L'ensemble des étapes est la **requête**. Trois propriétés la distinguent d'un nettoyage à la main :

- **elle est rejouable** : sur le prochain fichier, un clic sur *Actualiser* refait toutes les étapes dans le même ordre ;
- **elle est lisible** : les étapes portent un nom, on voit où une donnée a changé ; la requête est aussi un **document** du traitement (section 2.4) ;
- **elle ne touche pas à la source** : le fichier d'origine reste intact, le résultat est chargé dans une feuille ou dans le modèle de données.

Derrière l'interface, chaque étape est une ligne de code dans un langage fonctionnel appelé **M**. On peut l'ignorer au début (on clique), et le lire ensuite pour comprendre ou corriger une requête.

2.3.2 Importer : formats, encodage, paramètres régionaux

Power Query sait lire des classeurs Excel, des fichiers texte et CSV, des **dossiers entiers** (tous les fichiers d'un répertoire, empilés), des bases de données, des pages web. Pour un fichier texte, **trois réglages** décident de tout, et c'est là que naissent la plupart des erreurs d'import :

- le **délimiteur** (point-virgule, virgule, tabulation) : l'export français utilise le point-virgule parce que la virgule sert de séparateur décimal ;
- l'**encodage** : les anciens exports de Windows sont en cp1252 (ANSI), les fichiers modernes en UTF-8 ;
- les **paramètres régionaux** (*culture*) : ils définissent la virgule décimale (52,43) et l'ordre des dates (03/11/2025 = jour/mois/année).

Regardons le fichier tel que la caisse le fournit. Nous le lisons **sans rien interpréter** : tout en texte, sans en-tête, en conservant les lignes vides.

```
brut = pd.read_csv("donnees/export_caisse_brut.csv", sep=";", encoding="cp1252", header=None,
↳ dtype=str, skip_blank_lines=False)
print(brut.shape)
print(brut.iloc[:6, :4].fillna("").to_string(header=False))
```

```
(289, 8)
0      Export caisse - Boutique
1 Période du 03/11/2025 au 09/11/2025
2
3      N° ticket      Date  Heure      Article
4      T33133  03/11/2025  09:00  BOÎTE RUSTIQUE
5      T33137  03/11/2025  10:35  Tapis nordique
```

Le fichier compte **289 lignes** et 8 colonnes. On y voit un **titre**, une ligne de **période**, une ligne **vide**, puis l'en-tête réel (N° ticket, Date, ...) à la quatrième ligne. Un import automatique, qui supposerait un en-tête en première ligne, rangerait ce titre dans les noms de colonnes et ferait de toutes les colonnes du **texte**.

```
caractères illisibles si l'on décode en UTF-8 au lieu de cp1252 : 169
dates 03/11/2025 lues « à l'américaine » : ['11/03/2025', '11/03/2025', '11/03/2025']
```

Deux erreurs d'import illustrent les réglages. Avec le **mauvais encodage**, 169 caractères accentués sont illisibles (é devient  ) : un Boîte ne correspond plus à rien. Avec les **mauvais paramètres régionaux**, le 3 novembre (03/11/2025) est lu comme le **11 mars** : les trois premières dates se retrouvent au 11/03/2025. Comme toutes les dates de cette semaine ont un jour inférieur à 10 et le mois 11, **toutes** les lignes seraient décalées de plusieurs mois, sans le moindre message d'erreur.

2.3.3 Les étapes de nettoyage, une à une

Voici la recette pour l'export de caisse. Pour chaque étape, nous donnons l'opération de Power Query (son nom français dans l'interface, à **vérifier** selon la version) et l'équivalent pandas, avec le **nombre de lignes** après l'étape.

#	Étape Power Query	Équivalent pandas	Lignes après
1	Supprimer les premières lignes (3)	<code>.iloc[3:]</code>	286
2	Utiliser la première ligne comme en-têtes	<code>.columns = ...</code>	285
3	Supprimer les lignes d'en-tête répétées (filtrer N° ticket ≠ N° ticket)	<code>[t["N° ticket"] != "N° ticket"]</code>	281
4	Supprimer la ligne de total (filtrer les Qté vides)	<code>[t["Qté"].notna()]</code>	280
5	Changer les types (avec les paramètres régionaux fr-FR)	<code>to_datetime(format=...), astype</code>	280
6	Nettoyer le texte (espaces, majuscules)	<code>.str.strip().str.capitalize()</code>	280
7	Ajouter une colonne : montant corrigé	<code>fillna(Qté × Prix)</code>	280

```
t = brut.iloc[3:].reset_index(drop=True)
t.columns = t.iloc[0]; t = t.iloc[1:].reset_index(drop=True)
n_apres_entete = len(t)
t = t[t["N° ticket"] != "N° ticket"]; n_sans_repetes = len(t)
t = t[t["Qté"].notna()].copy(); n_sans_total = len(t)
print(len(brut), "→", n_apres_entete, "→", n_sans_repetes, "→", n_sans_total, "lignes")
```

289 → 285 → 281 → 280 lignes

On passe de 289 lignes à **280 lignes de vente** : trois lignes de titre, l'en-tête réel, quatre en-têtes répétées (l'export les a reproduits à chaque « page ») et une ligne de total. La ligne de total est précieuse : nous la conservons de côté, elle servira de **total de contrôle**.

Viennent les types et le texte :

```
for c in ["Prix unitaire", "Montant"]:
    t[c] = t[c].str.replace(",", ".").astype(float)      # virgule décimale -> point
t["Qté"] = t["Qté"].astype(int)
t["Date"] = pd.to_datetime(t["Date"], format="%d/%m/%Y")
t["Article"] = t["Article"].str.strip().str.capitalize() # « BOÎTE RUSTIQUE » -> « Boîte rustique
↳ »
t["Catégorie"] = t["Catégorie"].str.strip().str.capitalize() # « maison » -> « Maison »
print(t["Catégorie"].value_counts().to_dict())
print("montants manquants :", int(t["Montant"].isna().sum()))
```

```
{'Décoration': 68, 'Maison': 52, 'Cuisine': 50, 'Papeterie': 40, 'Bien-être': 38, 'Jardin': 32}
montants manquants : 8
```

Deux choix méritent l'attention. D'abord la **casse** : la fonction « Première lettre de chaque mot en majuscule » de Power Query (Text.Proper) donnerait Bien-Être et Bol Design, qui ne correspondent plus au catalogue (Bien-être, Bol design) ; la bonne transformation est **une majuscule**

initiale, le reste en minuscules (Text.Upper du premier caractère, Text.Lower du reste). Ensuite les **8 montants manquants** : que mettre ?

Comblant les montants manquants, et se contrôler

Une solution naturelle est de recalculer Qté × Prix unitaire. Mais la ligne de total du fichier permet de **vérifier** cette réparation : elle annonce 11 561,47 €.

total annoncé par l'export : 11 561,47 | somme après réparation : 11 564,09 | écart : 2,62

La somme réparée vaut **11 564,09 €** contre **11 561,47 €** : un écart de **2,62 €**. La réparation est donc **un peu trop généreuse** : les huit lignes concernées appartiennent à des tickets qui avaient une **remise** (code promo), et Qté × Prix l'ignore. Le contrôle ne dit pas *quelle* ligne est fausse, mais il **prouve** qu'il y a un écart, ce qu'une réparation silencieuse n'aurait jamais révélé. Deux attitudes sont défendables : signaler l'écart et laisser un indicateur « montant estimé » dans une colonne, ou aller chercher la remise dans la source (ici la table des commandes). **Ne jamais réparer sans contrôler.**

Voici la requête complète en langage M (non exécutée) ; les étapes portent les noms de l'éditeur.

```
let
    Source = Csv.Document(File.Contents("export_caisse_brut.csv"), [Delimiter=";", Columns=8,
    ↳ Encoding=1252]),
    SansTitre = Table.Skip(Source, 3),
    EnTetes = Table.PromoteHeaders(SansTitre, [PromoteAllScalars=true]),
    SansRepetes = Table.SelectRows(EnTetes, each [#"N° ticket"] <> "N° ticket"),
    SansTotal = Table.SelectRows(SansRepetes, each [Qté] <> null and [Qté] <> ""),
    Types = Table.TransformColumnTypes(SansTotal, {"Date", type date}, {"Qté", Int64.Type},
    ↳ {"Prix unitaire", type number}, {"Montant", type number}}, "fr-FR"),
    Texte = Table.TransformColumns(Types, {"Article", each Text.Upper(Text.Start(Text.Trim(_), 1)) &
    ↳ Text.Lower(Text.Middle(Text.Trim(_), 1))},
    ↳ {"Catégorie", each Text.Upper(Text.Start(Text.Trim(_), 1)) &
    ↳ Text.Lower(Text.Middle(Text.Trim(_), 1))}},
    Corrige = Table.AddColumn(Texte, "Montant corrigé", each if [Montant] = null then [Qté] * [Prix
    ↳ unitaire] else [Montant], type number)
in
    Corrige
```

Syntaxe à vérifier dans votre version de Power Query, en particulier les noms de colonnes entre #"..." et le traitement de la colonne Qté vide.

Pour s'entraîner. Cahier, chapitre 2 : application 2.6, exercice 2.8.

2.3.4 Scinder, fusionner des colonnes, dépivoter

Trois transformations de forme reviennent sans cesse.

Scinder une colonne. Le numéro de ticket T33133 mélange une lettre et un nombre. *Fractionner la colonne* → *par nombre de caractères* donne T et 33133, ce que fait en pandas `t["N° ticket"].str[0]` et `.str[1:]`. On peut aussi scinder Nom Prénom par délimiteur (l'espace), une adresse par virgule.

Fusionner des colonnes. L'inverse : Date et Heure (deux colonnes de texte) deviennent un seul horodatage utilisable pour trier ou regrouper.

Dépivoter. Beaucoup de tableaux de reporting sont **larges** : une ligne par catégorie, une colonne par mois. Pour un TCD, un graphique ou une jointure, il faut le format **long** : une ligne par couple (catégorie, mois). L'opération *Dépivoter les colonnes* (*unpivot*) fait ce passage ; l'inverse est *Pivoter*. Prenons le tableau large du chiffre d'affaires par catégorie et par mois de 2025 : 6 lignes et 12 colonnes de mois.

Étapes appliquées	Aperçu du résultat (280 lignes)
✓ Source	N° ticket Date Article Catégorie Qté Montant corrigé
✓ Lignes du haut supprimées	T33133 03/11/2025 Boîte rustique Maison 2 104,86
✓ En-têtes promus	T33137 03/11/2025 Tapis nordique Maison 1 64,79
✓ Doublons d'en-tête filtrés	T33137 03/11/2025 Statuette rustique Décoration 1 66,85
✓ Total supprimé	T33137 03/11/2025 Étagère compact Maison 1 109,08
✓ Types modifiés	T33157 03/11/2025 Bol design Cuisine 3 117,12
✓ Texte nettoyé	T33153 03/11/2025 Bol design Cuisine 2 78,08
► Montant corrigé	T33138 03/11/2025 Moule mat Cuisine 1 17,41

Figure 22: L'éditeur de Power Query : à gauche, les étapes appliquées (chacune rejouable) ; à droite, l'aperçu du résultat. Schéma dessiné avec matplotlib, avec les vraies valeurs de l'export, pas une capture d'écran.

```
large =
↳ L.assign(mois=L["date_commande"].dt.to_period("M").astype(str)).pivot_table(index="categorie",
↳ columns="mois", values="montant", aggfunc="sum")
long = large.reset_index().melt(id_vars="categorie", var_name="mois", value_name="montant")
print(large.shape, "→", long.shape, "| total conservé :", round(long["montant"].sum(), 2))
```

(6, 12) → (72, 3) | total conservé : 1324763.72

Les 6×12 cases deviennent **72 lignes**, et le total (1 324 763,72 €) est **conservé** : c'est le contrôle de toute restructuration. Le format long est celui que les outils d'analyse attendent (chapitre 4 du volume) : une colonne par variable, une ligne par observation.

2.3.5 Fusionner et ajouter des requêtes

Ajouter des requêtes (*Append*) empile des tables de même structure : les exports de chaque semaine de l'année, par exemple. Avec l'option **Dossier**, Power Query lit tous les fichiers d'un répertoire et les empile : déposer le fichier de la semaine suivante dans le dossier suffit, un clic sur *Actualiser* met tout à jour. Notez que les fichiers doivent avoir **la même structure** (mêmes colonnes, même ordre) : un export dont une colonne a changé de nom fait échouer toute la requête, ce qui est **un bon signe** (l'erreur est visible).

Fusionner des requêtes (*Merge*) est une **jointure** : on rattache à chaque ligne d'une table des colonnes d'une autre, à partir d'une clé commune. Les types de jointure de Power Query ont leur équivalent pandas et SQL (chapitre 3) :

Power Query	pandas	Ce que l'on garde
Externe gauche	how="left"	toutes les lignes de la table de gauche
Interne	how="inner"	seulement les lignes qui ont une correspondance
Anti gauche	how="left" puis filtre sur le manque	les lignes sans correspondance

Rattachons à l'export de caisse le **coût d'achat** du catalogue, pour calculer la marge de la semaine. L'export ne contient pas l'identifiant du produit, seulement le **nom de l'article** : la clé de jointure sera le nom.

```
cle = lambda s: s.str.lower()
prod = P.assign(cle=cle(P["nom_produit"]))
t["cle"] = cle(t["Article"])
m1 = t.merge(prod[["cle", "cout_achat"]], on="cle", how="left")
print(len(t), "lignes avant,", len(m1), "après la jointure sur le seul nom | total", 0.fr(m1["Montant"]
↳ corrigé"].sum()))
```

280 lignes avant, 560 après la jointure sur le seul nom | total 23 128,18

Le nombre de lignes a doublé (280 → 560) et le total aussi (23 128,18 € au lieu de 11 564,09 €) : le nom d'article **n'est pas une clé**. Le catalogue compte 120 produits mais seulement **60 noms distincts** : chaque nom désigne deux produits, à des prix différents. La jointure rattache chaque ligne de vente à **chacun** des deux produits, et personne ne reçoit de message d'erreur. C'est l'erreur de fusion la plus coûteuse : **toujours vérifier le nombre de lignes avant et après une jointure**.

Pour lever l'ambiguïté, ajoutons le **prix** à la clé : le prix de caisse de novembre 2025 est le prix du catalogue majoré de 3 % (hausse du 1^{er} janvier 2025, arrondie au centime).

```
prod["prix_caisse"] = (prod["prix_vente"] * 1.03).round(2)
m2 = t.merge(prod[["cle", "prix_caisse", "cout_achat"]], left_on=["cle", "Prix unitaire"],
↳ right_on=["cle", "prix_caisse"], how="left")
print(len(t), "→", len(m2), "lignes | sans correspondance :", int(m2["prix_caisse"].isna().sum()))
doublons_cat = prod[prod.duplicated(["cle", "prix_caisse"], keep=False)][["id_produit",
↳ "nom_produit", "prix_vente", "cout_achat"]]
print(doublons_cat.to_string(index=False))
```

```
280 → 285 lignes | sans correspondance : 0
id_produit nom_produit prix_vente cout_achat
62 Carnet mat 2.9 1.53
72 Carnet mat 2.9 1.53
```

Le résultat compte 285 lignes pour 280 attendues : **cinq lignes sont encore doublées**, parce que le catalogue lui-même contient **deux produits identiques** (Carnet mat, identifiants 62 et 72, même prix, même coût). C'est une anomalie du **catalogue** à signaler à son propriétaire ; en attendant, on **supprime les doublons du catalogue** avant de fusionner (l'étape *Supprimer les doublons* de Power Query).

```
après dédoublonnage du catalogue : 280 lignes | sans correspondance : 0
marge de la semaine : 5 715,57 € sur 11 564,09 € de ventes
recoupement avec la base : lignes 280 | montant 11 561,47 | total de contrôle du fichier 11 561,47
```

Après dédoublonnage, la jointure rend bien **280 lignes**, toutes appariées. La marge de la semaine est de **5 715,57 €** sur 11 564,09 € de ventes (avec le montant réparé) ; et le recoupement avec la base des ventes confirme le **total de contrôle** : 280 lignes, 11 561,47 €, exactement la valeur de la ligne de total de l'export. Les 2,62 € de l'écart de réparation sont donc bien des remises que Qté × Prix ignorait.

Voici la fusion en langage M (non exécutée) :

```
Fusion = Table.NestedJoin(Corrige, {"Article", "Prix unitaire"}, Catalogue, {"Nom", "PrixCaisse"},
↳ "Cat", JoinKind.LeftOuter),
Colonnes = Table.ExpandTableColumn(Fusion, "Cat", {"cout_achat"})
```

2.3.6 Power Query, formules ou Python ?

	Formules Excel	Power Query	Python / SQL
Point fort	immédiat, visible	rejouable sans code	très gros volumes, reproductible, versionnable
Point faible	difficile à rejouer proprement	outil propre à l'écosystème Microsoft	demande d'apprendre un langage
À choisir pour	calculs ponctuels, petits tableaux	import et nettoyage récurrents de fichiers	traitements lourds ou à partager en équipe

En pratique. Si vous faites deux fois la même manipulation de fichier, **faites-en une requête**. Si le fichier dépasse le million de lignes, ou si le traitement doit tourner sans personne, passez à SQL ou à Python (chapitres 3 et 4).

À retenir.

- Une requête Power Query est une **liste d'étapes rejouables** ; elle ne modifie pas la source.
- Trois réglages d'import décident de tout : **délimiteur, encodage, paramètres régionaux** (sur notre export : 169 caractères illisibles avec le mauvais encodage ; toutes les dates décalées avec les mauvais paramètres).
- **Compter les lignes à chaque étape** (289 → 280 sur l'export) et **contrôler un total** (11 561,47 €) : l'écart de 2,62 € a révélé que la réparation ignorait des remises.
- Une jointure sur une clé **non unique** multiplie les lignes (280 → 560) sans erreur : vérifiez l'unicité des clés et le nombre de lignes avant/après.
- Dépivoter conserve le total (72 lignes, 1 324 763,72 €) : la restructuration se contrôle comme le reste.

Pour s'entraîner. Cahier, chapitre 2 : application 2.6, exercices 2.8 et 2.9.

2.4 Bonnes pratiques de tableur

Un classeur n'est pas seulement un calcul : c'est un **document que quelqu'un d'autre ouvrira**, souvent des mois plus tard, souvent la personne qui l'a écrit et qui ne se souvient plus de rien. Les erreurs de tableur ne viennent presque jamais d'une formule mal comprise ; elles viennent d'une **structure** qui les cache. Cette section rassemble les pratiques qui rendent un classeur **lisible, vérifiable et durable**.

2.4.1 Une structure en trois étages

La règle d'or est de **séparer ce qui entre, ce qui se calcule et ce qui se montre**. Un bon classeur comporte au moins quatre types de feuilles :

Feuille	Contenu	Règle
README	à quoi sert le classeur, qui l'a fait, quand, d'où viennent les données, comment l'actualiser	écrite en premier , lue en premier
Données (ou plusieurs)	une table par feuille, brute ou nettoyée	aucune mise en forme, aucune formule de synthèse
Calculs	les formules et tableaux croisés qui lisent les données	rien de saisi à la main, sauf les hypothèses
Présentation	les chiffres et graphiques destinés aux lecteurs	ne contient que des renvois aux calculs

Cette séparation a trois avantages. Quand les données changent (nouvelle semaine, nouvel export), on **remplace la feuille de données** et tout se recalcule. Quand on se demande d'où vient un chiffre, on remonte de la présentation aux calculs puis aux données. Et quand on donne le classeur à quelqu'un d'autre, il sait **où il peut toucher** et où il ne le doit pas.

Le contraire est le classeur « tout-en-un » : un tableau de bord où se mélangent des données collées, des calculs intermédiaires dans des cellules dispersées, et des chiffres retapés. Il fonctionne le jour où on le fabrique, puis plus jamais.

2.4.2 Des données propres : une ligne, une observation

Les outils d'analyse (tableaux croisés, Power Query, pandas, SQL) attendent des données **ordonnées** (*tidy*) :

1. **une ligne = une observation** (ici, un article vendu) ;
2. **une colonne = une variable**, avec **un seul type** (que des dates, que des montants) ;
3. **une seule ligne d'en-tête**, avec des noms courts et **sans cellules fusionnées** ;
4. **pas de ligne ni de colonne de totaux, de titres ou de blancs** au milieu des données ;
5. **une information par cellule** (pas de « Boutique / Ville A » dans la même cellule) ;
6. **des catégories écrites toujours de la même façon** (une liste de valeurs autorisées).

Comparons deux manières de présenter les mêmes ventes. À gauche, la présentation « pour être lue », qu'un humain aime ; à droite, la présentation « pour être analysée ».

Dans la première forme, un tableau croisé dynamique prendrait « Sous-total T1 » pour un mois, la ligne de titre pour un en-tête, et la colonne Total pour une catégorie. Dans la seconde, tout fonctionne : on regroupe par ce que l'on veut. Quand on **reçoit** des données sous la première forme (c'est fréquent), c'est précisément ce que Power Query (section 2.3) remet en forme.

△ **Deux détails qui comptent.** D'abord les **identifiants** : un code comme 007 ou 00512 perd ses zéros si Excel le prend pour un nombre ; stockez-le comme **texte** (format texte *avant* la saisie). Ensuite les **cellules fusionnées** : elles cassent le tri, le filtre et les tableaux croisés ; pour centrer un titre au-dessus de plusieurs colonnes, utilisez l'alignement *Centré sur plusieurs colonnes*, qui ne fusionne rien.

A1	fx	Ventes 2025 - Cuisine et Maison (en €)		
	A	B	C	D
1	Ventes 2025 - Cuisine et Maison (en €)			
2				
3	Mois	Cuisine	Maison	Total
4	Janvier	19475	27648	47123
5	Février	15519	20964	36483
6	Mars	18695	24579	43274
7	Sous-total T1	53689	73191	126880
8	Avril	20196	23997	44192
Synthèse				

Figure 23: À éviter : un titre sur la première ligne, une ligne blanche, une colonne de total et, surtout, une ligne de sous-total au milieu des données. Agréable à lire, impossible à analyser. Maquette dessinée avec matplotlib.

A2	fx			
	A	B	C	D
1	date	categorie	canal	montant
2	01/01/2025	Décoration	Site	24,62
3	01/01/2025	Cuisine	Site	39,04
4	01/01/2025	Jardin	Boutique	74,06
5	01/01/2025	Maison	Boutique	38,01
6	01/01/2025	Maison	Boutique	34,92
7	01/01/2025	Décoration	Boutique	29,77
8	01/01/2025	Cuisine	Boutique	40,07
Données				

Figure 24: À faire : une ligne d'en-tête, une ligne par article vendu, une colonne par variable, aucun total. Maquette dessinée avec matplotlib.

2.4.3 Séparer les hypothèses des formules

Une formule qui contient un **nombre écrit en dur** est une bombe à retardement : `=B2*1,03` ne dit pas ce que représente 1,03, et personne ne pense à le changer quand le taux change. La règle est simple : **tout paramètre vit dans une cellule étiquetée**, et les formules y renvoient (par une référence absolue ou un nom, 2.1.2 et 2.1.3).

Illustrons-le avec une prévision simple pour 2026. Les hypothèses sont posées dans une feuille Hyp : volume +5 %, prix +3 %, inflation du coût d'achat +2 %. Le chiffre d'affaires 2026 est le chiffre de 2025 multiplié par $(1 + \text{volume}) \times (1 + \text{prix})$; le coût d'achat est celui de 2025 multiplié par $(1 + \text{volume}) \times (1 + \text{inflation})$; la marge est la différence.

CA 2025 : 1 324 763,72 | coûts d'achat 2025 : 684 952,84 | marge 2025 : 639 810,88
 marge 2026 (volume +5 %) : 699 147,47 | CA 2026 : 1 432 731,96 | recoupement en Python : 699 147,47
 marge 2026 si le volume reste stable : 665 854,73
 marge 2026 par rapport à 2025 : 9.3 % (volume +5 %) et 4.1 % (volume stable)

Avec ces hypothèses, la marge passe de **639 810,88 €** à **699 147,47 €** (+ 9,3 %) ; si le volume reste stable, elle s'élève à **665 854,73 €** (+ 4,1 %). Le **changement d'une seule cellule** (le volume) suffit à passer de l'un à l'autre : c'est ce qu'on appelle une **analyse de sensibilité**, et elle n'est possible que parce que l'hypothèse n'est écrite **qu'à un seul endroit**.

En pratique. Donnez une **couleur** aux cellules de saisie (par exemple un fond jaune pâle) et **une seule** couleur : tout le reste est calculé et ne se modifie pas. Ajoutez une colonne « source » ou « commentaire » à côté de chaque hypothèse : d'où vient ce 3 % ?

2.4.4 Se contrôler : validation, mise en forme conditionnelle, feuille de contrôles

On ne corrige bien que ce que l'on voit. Trois outils d'Excel aident à **rendre les erreurs visibles** :

- la **validation de données** (*Données* → *Validation*) restreint ce qu'une cellule accepte : une liste de valeurs (les trois canaux), un nombre entre deux bornes, une date dans la période. Une saisie Sit au lieu de Site est refusée à l'entrée plutôt que détectée dans un total faux ;
- la **mise en forme conditionnelle** colore les cellules qui remplissent une condition (valeurs négatives, doublons, écarts supérieurs à un seuil) ;
- une **feuille de contrôles** réunit des formules qui répondent à la question « ce classeur est-il intact ? ».

Voici notre feuille de contrôles, calculée par LibreOffice sur les données de la gérante, puis sur une copie où **500 lignes sont comptées deux fois** (un accident de copier-coller, l'erreur de la section 2.2.5). Les contrôles sont : nombre de lignes, nombre de doublons (lignes moins valeurs distinctes de l'identifiant), cellules vides dans une colonne clé, plage de dates, montants négatifs, total, et un verdict qui compare le nombre de lignes et le total à leurs **valeurs attendues**.

contrôle	données saines	avec 500 doublons
nombre de lignes	29 827	30 327
doublons d'identifiant	0	500
canaux vides	0	0
date minimale	01/01/2025	01/01/2025
date maximale	31/12/2025	31/12/2025
montants ≤ 0	0	0
total des montants	1 324 763,72	1 345 630,05
verdict	OK	ÉCART

Le classeur sain passe tous les contrôles (29 827 lignes, aucun doublon, aucune valeur manquante, dates du 01/01/2025 au 31/12/2025, aucun montant négatif, verdict **OK**). La copie abîmée est détectée immédiatement : 30 327 lignes, **500 doublons**, un total de 1 345 630,05 € au lieu de 1 324

763,72 €, et un verdict **ÉCART**. Les **valeurs attendues** (29 827 lignes, 1 324 763,72 €) viennent d'une **autre source** que le classeur (par exemple la compta, ou la base de données) : c'est ce qui rend le contrôle utile, car un contrôle qui compare le classeur à lui-même ne prouve rien.

2.4.5 Les erreurs de tableur que tout le monde commet

Les erreurs spectaculaires de tableur ont des **mécanismes** simples et récurrents. Les reconnaître vaut mieux que mémoriser des anecdotes.

1. La conversion automatique. Excel interprète ce que l'on saisit ou importe : 1-2 devient une date, 007 devient 7, un nom qui ressemble à une date devient une date. Les chercheurs en génétique en ont fait l'amère expérience : des noms de gènes comme SEPT2 ou MARCH1 étaient convertis en dates dans des listes publiées, au point que les organismes de nomenclature ont fini par **renommer** des gènes (à vérifier, mais le cas est documenté). Pour éviter la conversion, importez en **texte** (Power Query, section 2.3) ou mettez la colonne au format texte **avant** de coller.

2. L'arrondi et la précision. Un tableur affiche un nombre arrondi mais calcule avec le nombre complet, et inversement. Deux illustrations sur nos données :

```
somme des montants arrondis au dixième, 29 premières lignes : 1 281,90 | arrondi de la somme : 1 281,70
ARRONDI(2,675 ; 2) dans le tableur : 2,68 | round(2.675, 2) en Python : 2.67
(0,1 + 0,2) - 0,3 dans le tableur : 0 | en Python : 5.551115123125783e-17
12345678901234567 dans le tableur : 12 345 678 901 234 600
```

- **La somme des arrondis n'est pas l'arrondi de la somme** : 1 281,90 € contre 1 281,70 € sur seulement 29 lignes.
- **Outils différents, arrondis différents** : le tableur arrondit 2,675 à 2,68 (convention « 5 vers le haut » appliquée au nombre décimal), alors que Python donne 2,67, parce que 2,675 n'est **pas représentable exactement** en binaire (il est stocké un peu en dessous) ; de même, $(0,1 + 0,2) - 0,3$ vaut 0 dans le tableur et $5,55 \times 10^{-17}$ en Python : le tableur « nettoie » certains petits résidus, ce qui peut masquer un problème de précision.
- **La précision est limitée à 15 chiffres significatifs** : un identifiant de 17 chiffres (12345678901234567) perd ses derniers chiffres (LibreOffice affiche ici 12 345 678 901 234 600). Les **identifiants longs (numéros de carte, de compte) doivent être stockés comme du texte**.

Pour des montants, la règle est de **choisir où l'on arrondit** (à la ligne ? au total ?) et de s'y tenir : un écart de quelques centimes entre deux documents est presque toujours une affaire d'arrondi, pas d'erreur de calcul.

3. La plage oubliée. =SOMME(M2:M29028) oublie les 800 dernières lignes (33 638,83 € de moins, section 2.1.4). Un grand article d'économie très cité a, dit-on, subi une erreur de ce type (une plage qui omettait quelques pays) ; le détail est à vérifier, le mécanisme est certain. Contre cela : tableaux structurés, contrôle de total.

4. Les limites du format. Un fichier .xlsx est limité à **1 048 576 lignes** et 16 384 colonnes ; l'ancien format .xls s'arrêtait à **65 536 lignes**. Un service de santé publique a, en 2020, perdu plusieurs milliers de cas déclarés parce qu'un fichier de résultats était enregistré dans l'ancien format (à vérifier : l'incident est largement rapporté). Des données au-delà du million de lignes **n'ont rien à faire dans un tableur** : base de données, SQL, Python.

5. Les liens et les valeurs figées. Un classeur qui lit un autre classeur par un lien externe affiche l'ancienne valeur si le fichier source a bougé ; un « copier / collage spécial valeurs » fige un chiffre qui ne se mettra plus jamais à jour. Les deux sont légitimes, **à condition de les documenter**.

2.4.6 Documenter, protéger, versionner, partager

Documenter. La feuille README répond en dix lignes à : *à quoi sert ce classeur ? qui l'a produit, quand, pour qui ? quelles sont les sources des données et leurs dates ? comment l'actualiser ? quelles sont les hypothèses ? quelles sont les limites connues ?* Les commentaires de cellules servent aux exceptions (« valeur corrigée à la main le 12/03, voir le mail du fournisseur »).

Nommer et versionner. Un fichier s'appelle ventes_2025_v03_2025-12-31.xlsx plutôt que ventes_final_vraiment_final.xlsx : un numéro de version, une date ISO (année-mois-jour, qui se range correctement par ordre alphabétique). Si le classeur est partagé dans un espace en ligne, l'**historique des versions** permet de revenir en arrière ; si le travail est important, un **journal des modifications** (une feuille de plus) note qui a changé quoi.

Protéger. La protection d'une feuille ou de cellules verrouille les formules et laisse libres les cellules de saisie : c'est un **garde-fou contre les fausses manœuvres**, pas une mesure de sécurité (les mots de passe de protection d'un classeur sont faciles à contourner). Pour des données confidentielles, le contrôle d'accès se fait au niveau du **dossier ou du fichier**, pas de la feuille. Les classeurs qui contiennent des **données personnelles** (noms, adresses, téléphones de clients) sont soumis à des règles de confidentialité, et circulent mal par courrier électronique : le volume II y consacre un chapitre complémentaire.

Partager. Pour diffuser un chiffre, mieux vaut un **PDF** ou une feuille de présentation en lecture seule que le classeur de travail ; pour échanger des données, un **CSV** (simple, universel) plutôt qu'un classeur plein de formules et de liens.

Liste de contrôle d'un classeur fiable.

1. Une feuille README : objet, auteur, date, sources, hypothèses.
2. Données et calculs **séparés** ; données en **tableau structuré**, une ligne par observation.
3. **Aucun nombre en dur** dans une formule : des cellules d'hypothèses étiquetées.
4. Une **feuille de contrôles** comparant à une source indépendante.
5. **Aucune erreur** affichée (#N/A, #REF!...), aucune cellule fusionnée.
6. Un nom de fichier **versionné** et daté.

À retenir.

- Séparez ce qui **entre** (données, hypothèses), ce qui se **calcule** et ce qui se **montre**.
- Des données **ordonnées** (une ligne, une observation) font fonctionner tous les outils ; un tableau « pour l'œil » les met en échec.
- Un paramètre écrit en dur est une erreur future : une cellule, un nom, une analyse de sensibilité (ici : marge 2026 de 699 147,47 € ou 665 854,73 € selon le volume).
- **Contrôlez** : lignes, doublons, vides, dates, total comparé à une **source indépendante** (500 doublons détectés, verdict ÉCART).
- Les erreurs célèbres ont des mécanismes simples : conversion automatique, arrondis, plage oubliée, limites du format, liens et valeurs figées.

Pour s'entraîner. Cahier, chapitre 2 : application 2.7, exercices 2.10 et 2.11.

2.5 ⊕ Pour aller plus loin : Excel avancé

Section complémentaire. Elle présente ce qu'Excel sait faire au-delà de la feuille de calcul : un **modèle de données** relié (Power Pivot), un langage de **mesures** (DAX), l'**automatisation** (macros VBA, scripts Office), puis elle répond à la question qui conclut toute formation sur Excel : **quand faut-il le quitter ?** Aucun des composants de cette section n'a pu être exécuté sur la machine qui a produit le livre : les scripts DAX, VBA et Office Scripts sont **non exécutés** et leur syntaxe est à **vérifier** ; nous donnons à chaque fois le **résultat attendu**, calculé en pandas ou en SQL.

2.5.1 Power Pivot et le modèle de données

Un tableau croisé classique travaille sur **une seule table**. Or les données d'une entreprise sont réparties en plusieurs tables : les lignes de vente (les **faits**, des événements nombreux), le catalogue des produits et la liste des clients (les **dimensions**, des objets décrits une fois). Plutôt que d'ajouter des colonnes à la table des ventes par RECHERCHEX (2.1.6), Excel permet de **déclarer des relations** entre tables dans un **modèle de données** (le composant s'appelle *Power Pivot*), puis de construire un tableau croisé qui s'appuie sur plusieurs tables à la fois.

Une relation relie une colonne de la table des faits à la **clé** d'une dimension : Lignes[id_produit] vers Produits[id_produit]. Elle est de type **plusieurs à un** (plusieurs lignes de vente pour un produit). Pour qu'elle fonctionne, la clé du côté « un » doit être **unique** : c'est ce que la section 2.3.5 nous a appris à vérifier (le nom d'un produit ne l'est pas, son identifiant l'est).

```
relation Lignes → Produits : identifiants du catalogue uniques : True | lignes sans produit connu : 0
relation Lignes → Clients : identifiants clients uniques : True | lignes sans client connu : 0
lignes de ventes : 29 827 | produits : 120 | clients : 6 000
```

Les deux relations sont **saines** : les identifiants du côté « un » sont uniques, et aucune ligne de vente ne renvoie à un produit ou à un client inconnu (pas de ligne « orpheline »). C'est le **contrôle d'intégrité** que l'on fait avant de déclarer une relation : une clé en double ou une ligne orpheline faussent tous les résultats sans message.

Ce dessin s'appelle un **schéma en étoile**. Il a quatre avantages décisifs sur la table unique avec colonnes collées : le **catalogue n'est stocké qu'une fois** (on change un prix à un seul endroit) ; le modèle gère **bien plus de lignes** que la grille d'Excel, car il les stocke en mémoire de façon compressée ; il permet de **compter des valeurs distinctes** (le nombre de clients différents), ce qu'un champ calculé de tableau croisé ne sait pas faire (2.2.4) ; et il prépare le terrain aux **mesures** de la sous-section suivante. La table Calendrier est une dimension particulière : une ligne par jour, avec le mois, le trimestre et l'année, indispensable pour comparer des périodes.

2.5.2 Les mesures DAX

DAX (*Data Analysis Expressions*) est le langage des formules du modèle de données. Il ressemble aux formules d'Excel, mais fonctionne différemment : au lieu de calculer une cellule, on définit une **mesure**, un calcul qui se **réévalue dans chaque case du tableau croisé**, selon les filtres de cette case. La même mesure donne le chiffre d'affaires de toute l'année dans le total général et celui d'une catégorie dans la ligne de cette catégorie.

C'est la différence avec la **colonne calculée**, évaluée **ligne par ligne** à la création (comme une colonne de formules) et stockée. Règle simple : une colonne calculée sert à **décrire** une ligne (le coût unitaire d'une vente) ; une mesure sert à **agréger** (une somme, un ratio, une comparaison).

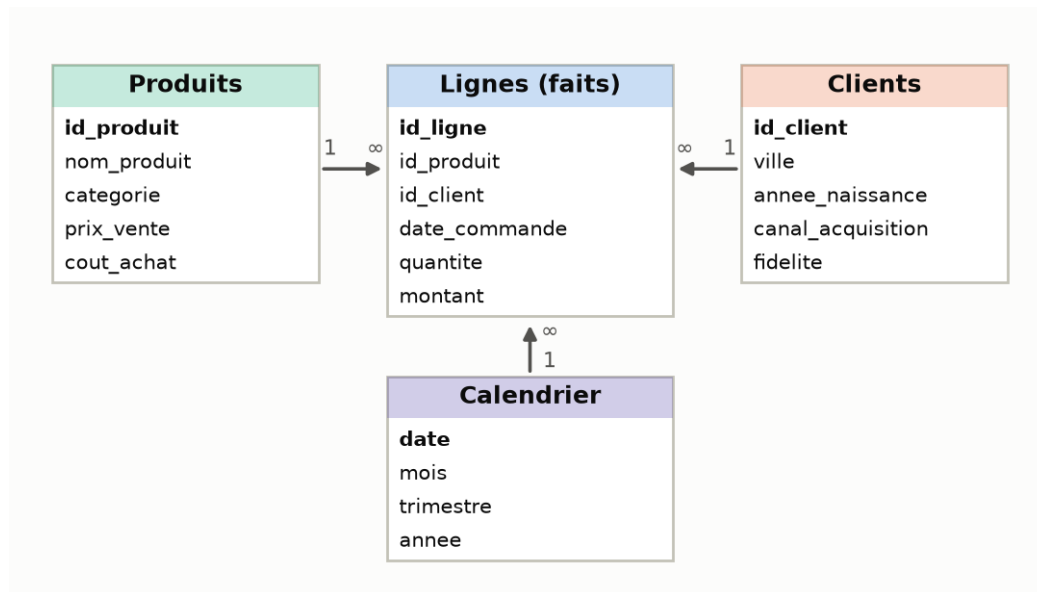


Figure 25: Un modèle de données en étoile : au centre la table des faits (Lignes), autour d'elle les dimensions (Produits, Clients, Calendrier). Chaque flèche est une relation « plusieurs à un » (∞ côté faits, 1 côté dimension). Schéma dessiné avec matplotlib.

Voici les mesures dont la gérante a besoin, en DAX (non exécuté, syntaxe à vérifier selon votre version) :

```

CA := SUM ( Lignes[montant] )
CA Site := CALCULATE ( [CA], Lignes[canal] = "Site" )
Marge := SUMX ( Lignes, Lignes[montant] - Lignes[quantite] * RELATED ( Produits[cout_achat] ) )
Marge % := DIVIDE ( [Marge], [CA] )
Clients distincts := DISTINCTCOUNT ( Lignes[id_client] )
Panier moyen := DIVIDE ( [CA], DISTINCTCOUNT ( Lignes[id_commande] ) )
CA N-1 := CALCULATE ( [CA], SAMEPERIODLASTYEAR ( Calendrier[date] ) )
Croissance := DIVIDE ( [CA] - [CA N-1], [CA N-1] )

```

Quelques lignes méritent une explication. **CALCULATE modifie le contexte de filtre** avant d'évaluer sa première expression : **CALCULATE**([CA] ; canal = "Site") calcule le chiffre d'affaires **comme si** le tableau était filtré sur le Site, quelle que soit la case où on le place. **SUMX itère** sur chaque ligne de la table, calcule une expression (**RELATED** va chercher le coût dans la table Produits grâce à la relation), puis additionne : c'est le même calcul que le **SOMMEPROD** de la section 2.1.6. **DIVIDE** renvoie un résultat vide, au lieu d'une erreur, quand le dénominateur est nul. **SAMEPERIODLASTYEAR** décale le contexte de dates d'un an : c'est l'une des fonctions d'**intelligence temporelle**, qui exigent une table Calendrier complète, sans trou.

Les résultats attendus, calculés en pandas sur le même classeur (et sur toutes les années pour la comparaison avec 2024) :

CA	: 1 324 763,72
CA Site	: 617 715,45
Marge	: 639 810,88
Marge %	: 48,30 %
Clients distincts	: 3 875
Panier moyen	: 102,33 € (12 946 commandes)
CA N-1 (2024)	: 1 189 461,17
Croissance	: 11,38 %

On lit : un chiffre d'affaires de 1 324 763,72 €, dont 617 715,45 € sur le Site ; une marge de 639 810,88 €, soit **48,30 %** du chiffre d'affaires ; **3 875** clients distincts ; un **panier moyen de 102,33 €** sur 12 946

commandes ; et une **croissance de 11,4 %** par rapport à 2024 (1 189 461,17 €). Remarquez que Marge % est un **ratio de sommes**, par construction (2.2.4) : c'est le comportement voulu d'une mesure, et c'est ce qui la distingue d'une moyenne de ratios. Chaque mesure s'adapte ensuite **au contexte du tableau** : placée dans un tableau croisé par catégorie, Marge % donne le taux de chaque catégorie, avec une seule définition.

Piège. Le contexte de filtre est la notion la plus difficile de DAX, et la source des erreurs : une mesure qui donne un bon résultat dans le total peut en donner un faux dans une case, parce qu'un filtre d'une autre table s'y propage (ou ne s'y propage pas). **Testez chaque mesure sur un cas que vous savez calculer à la main**, comme nous l'avons fait ici avec pandas.

2.5.3 Macros VBA et scripts Office

Une **macro** enregistre une suite de gestes (mise en forme, copie, tri...) et la rejoue sur demande. Excel peut l'enregistrer pendant que vous travaillez, et la traduit en code **VBA** (*Visual Basic for Applications*). Un classeur qui contient des macros doit être enregistré au format **.xlsm**, que les messageries et les politiques de sécurité accueillent avec méfiance, à juste titre : les macros peuvent faire n'importe quoi sur votre poste.

Voici une macro minimale (non exécutée) qui actualise toutes les connexions de données puis exporte la feuille de présentation en PDF :

```
Sub PublierRapport()
    ThisWorkbook.RefreshAll           ' actualise requêtes et tableaux croisés
    Application.CalculateUntilAsyncQueriesDone ' attend la fin des requêtes
    Sheets("Présentation").ExportAsFixedFormat Type:=xlTypePDF, _
        Filename:=ThisWorkbook.Path & "\rapport_" & Format(Date, "yyyy-mm-dd") & ".pdf"
End Sub
```

Les **Office Scripts** (Excel pour le web, scénarios automatisés par Power Automate) jouent le même rôle avec un langage différent, TypeScript :

```
function main(workbook: ExcelScript.Workbook) {
    const feuille = workbook.getWorksheet("Données");
    const plage = feuille.getUsedRange();
    console.log("lignes :", plage.getRowCount());
}
```

Le premier script (VBA) tourne sur le poste, le second dans le nuage. Retenez trois principes. **Une macro ne remplace pas une formule** : tout ce qu'une formule fait, elle le fait de façon visible et recalculable, une macro non. **Toute macro doit être documentée**, car personne ne peut la relire dans l'interface d'Excel. Et **ne faites pas confiance aux macros reçues** : désactivez-les par défaut.

2.5.4 Quand quitter Excel : SQL ou Python ?

Excel est le bon outil dans beaucoup de situations, et le mauvais dans d'autres. Les signaux qui disent « il faut changer d'outil » sont assez nets :

Signal	Pourquoi Excel peine	Alternative
Plus d'un million de lignes (ou plus de quelques centaines de milliers avec des formules)	limite de la grille, lenteur, fichier énorme	base de données et SQL (chapitre 3)
La même opération chaque semaine, avec plusieurs étapes	gestes à la main, erreurs	Power Query, ou un script Python

Signal	Pourquoi Excel peine	Alternative
Plusieurs personnes modifient en même temps	conflits de versions	base partagée, dépôt de code
Il faut pouvoir prouver et rejouer l'analyse	formules dispersées, pas d'historique	script versionné, notebook (chapitre 4)
Calculs statistiques évolués (modèles, simulations)	fonctions limitées	Python ou R

Une dernière illustration, pour **triangler** : le chiffre d'affaires du Site en 2025, calculé trois fois, avec trois outils indépendants (la formule du tableur, une requête SQL sur la base, pandas).

```
con = sqlite3.connect("donnees/boutique.db")
sql = """SELECT ROUND(SUM(l.montant), 2) FROM lignes_commande l JOIN commandes c ON c.id_commande =
↳ l.id_commande
WHERE c.canal = 'Site' AND c.date_commande >= '2025-01-01'"""
print("SQL :", con.execute(sql).fetchone()[0])
print("pandas :", round(L.loc[L["canal"] == "Site", "montant"].sum(), 2))
```

```
SQL : 617715.45
pandas : 617715.45
```

Les deux donnent **617 715,45 €**, exactement le résultat de SOMME.SI.ENS de la section 2.1.4. Trois outils, un seul chiffre : c'est ce qui permet de s'y fier. Le chapitre suivant apprend à écrire cette requête SQL de zéro.

À retenir.

- Un **modèle de données** relie des tables par des clés **uniques** côté « un » ; contrôlez l'intégrité (clés uniques, pas de lignes orphelines) avant de déclarer une relation.
- Une **mesure DAX** s'évalue dans chaque case selon son contexte de filtre ; une colonne calculée est évaluée ligne par ligne. CALCULATE modifie le contexte ; un ratio de mesures est un **ratio de sommes**.
- Une **macro** automatise mais ne se lit pas comme une formule : documentez-la, méfiez-vous de celles qu'on vous envoie.
- **Changez d'outil** quand les données dépassent la grille, quand le traitement doit être rejoué sans personne, ou quand il faut le prouver : SQL, Python, Power Query.
- **Triangulez** : un même chiffre obtenu par trois outils (ici 617 715,45 €) est un chiffre fiable.

Pour s'entraîner. Cahier, chapitre 2 : application 2.8, exercice 2.12.

2.6 ☉ Pour aller plus loin : Google Sheets et Looker Studio

Section complémentaire. Beaucoup d'équipes travaillent dans le nuage plutôt que dans un fichier : **Google Sheets** pour le tableur, **Looker Studio** pour les tableaux de bord. Cette section en présente l'esprit, les fonctions qui n'existent pas dans Excel, et les équivalences avec ce que vous avez appris. **Rien ici n'a pu être exécuté** (nous n'avons pas de compte ni d'accès à ces services) : tous les exemples de formules sont **non exécutés** et à vérifier dans votre environnement ; nous recalculons en SQL ou en pandas les résultats qu'ils doivent donner.

2.6.1 Google Sheets : ce qui ressemble, ce qui diffère

Google Sheets reprend l'essentiel du vocabulaire d'Excel : cellules, plages, références relatives et absolues, tableaux croisés dynamiques, la plupart des fonctions de la section 2.1 (SUM, SUMIFS, IF, VLOOKUP, XLOOKUP, FILTER, SORT, UNIQUE...). Un classeur Excel s'importe, un classeur Sheets s'exporte au format .xlsx, avec des différences possibles sur les fonctions propres à chacun et la mise en forme. Dans l'interface en français, les noms de fonctions sont traduits comme dans Excel (avec quelques écarts : à vérifier).

Les différences qui comptent pour un analyste :

- **La collaboration en temps réel** est native : plusieurs personnes éditent le même document, avec commentaires, suggestions, historique des versions. C'est le grand avantage, et le risque (modifications non contrôlées : section 2.4).
- **Les formules sur toute une colonne** s'écrivent avec ARRAYFORMULA : =ARRAYFORMULA(J2:J * K2:K * (1 - L2:L / 100)) applique le calcul à chaque ligne sans recopier la formule.
- **QUERY** interroge une plage avec un langage proche de SQL (voir ci-dessous) : c'est la fonction la plus puissante de Sheets, et elle n'a pas d'équivalent direct dans Excel.
- **IMPORTRANGE** lit une plage d'un autre document Sheets : =IMPORTRANGE("adresse_du_document" ; "Lignes!A1:M"), après autorisation d'accès. Pratique, mais fragile (le document source peut être déplacé, supprimé, ou son accès retiré).
- Des fonctions utiles propres à Sheets : SPLIT (découper un texte), IMAGE, GOOGLEFINANCE (cours de bourse), REGEXEXTRACT (expressions régulières).
- **La puissance de calcul est limitée** : un document Sheets accepte un nombre maximal de cellules (de l'ordre de dix millions, à vérifier), et devient lent bien avant. Pour des volumes importants, la même conclusion qu'avec Excel s'impose (2.5.4).
- L'automatisation se fait avec **Google Apps Script**, un langage dérivé de JavaScript, comparable aux macros VBA (non exécuté ici).

QUERY : du SQL dans une cellule

La fonction QUERY(données ; requête ; en-têtes) prend une plage et une requête écrite dans un petit langage inspiré de SQL. Les colonnes sont désignées par leur **lettre** (Col1, Col2... si la plage est une formule). Pour lister le chiffre d'affaires du Site par catégorie, du plus grand au plus petit :

```
=QUERY(Lignes!A1:M ; "select I, sum(M) where E = 'Site' group by I order by sum(M) desc label sum(M)
↪ 'CA'" ; 1)
```

(Non exécuté ; I est la colonne categorie, M la colonne montant, E la colonne canal.) Cette requête est exactement une requête SQL de regroupement : SELECT, WHERE, GROUP BY, ORDER BY. Nous pouvons donc **vérifier le résultat attendu** en SQL, avec SQLite, sur les mêmes données.

```
con = sqlite3.connect(":memory:")
L.to_sql("lignes", con, index=False)
req = "SELECT categorie, ROUND(SUM(montant), 2) AS CA FROM lignes WHERE canal = 'Site' GROUP BY
↪ categorie ORDER BY CA DESC"
res_sql = pd.read_sql(req, con)
print(res_sql.head(4).to_string(index=False))
```

```
categorie      CA
Jardin 166201.16
Maison 140788.49
Décoration 118835.74
Cuisine 109609.91
```

écart maximal entre la requête SQL et la colonne « Site » du tableau croisé de la section 2.2 : 0.0
total des six catégories : 617 715,45

La requête SQL, et donc la fonction QUERY qui en est le miroir, donne le **Jardin en tête** (166 201,16 €), devant la Maison (140 788,49 €), la Décoration (118 835,74 €) et la Cuisine (109 609,91 €). Elle retombe exactement sur la colonne « Site » du tableau croisé de la section 2.2, et le total des six catégories (617 715,45 €) est le chiffre d'affaires du Site. Apprendre SQL (chapitre 3) est donc aussi la meilleure façon d'apprendre QUERY.

2.6.2 Looker Studio : des tableaux de bord reliés à des sources

Looker Studio est un outil de tableaux de bord en ligne (anciennement « Data Studio »). Il ne stocke pas les données : il se **connecte** à des sources (feuilles Google Sheets, fichiers CSV, bases de données, services d'analyse du web...) et affiche des graphiques, des tableaux et des indicateurs qui se mettent à jour quand la source change. Trois notions suffisent pour démarrer :

- une **dimension** est un champ qui décrit (catégorie, canal, mois) ; une **métrique** est un champ que l'on agrège (montant, quantité). C'est la distinction « lignes et colonnes » contre « valeurs » d'un tableau croisé ;
- un **champ calculé** se définit par une formule sur les champs de la source : `SUM(montant) / COUNT_DISTINCT(id_commande)` donne le **panier moyen**, un ratio de sommes (2.2.4), dont nous avons calculé la valeur attendue en 2.5.2 : 102,33 € ;
- les **filtres** et **contrôles** (liste déroulante, sélecteur de dates) laissent le lecteur explorer le tableau de bord seul.

Les points de vigilance sont ceux de tout tableau de bord relié à des données vivantes. **La fraîcheur** : le tableau de bord peut afficher des données mises en cache (une actualisation manuelle ou programmée peut être nécessaire). **L'accès** : celui qui voit le tableau de bord voit-il les données de la source ? Les identifiants de connexion et les droits sont à régler. **La responsabilité** : si la source change (une colonne renommée), le tableau de bord casse, et le lecteur ne le sait pas toujours. **La confidentialité** : relier des données personnelles de clients à un service en ligne demande de savoir où elles sont hébergées et qui y a accès (volume II, chapitre complémentaire sur la confidentialité). Le volume IV de la série (visualisation et communication) reviendra sur la conception de tableaux de bord.

2.6.3 Tableau d'équivalences

Ce tableau rassemble les opérations du chapitre dans les cinq outils que vous rencontrerez. Il sert de **pense-bête** quand vous passez de l'un à l'autre.

Opération	Excel	Google Sheets	SQL	pandas
Somme conditionnelle	SOMME.SI.ENS	SUMIFS	SELECT SUM(...) ... WHERE ...	df.loc[cond, "col"].sum()
Compter sous condition	NB.SI.ENS	COUNTIFS	SELECT COUNT(*) ... WHERE ...	(cond).sum()
Recherche	RECHERCHEX	XLOOKUP, VLOOKUP	JOIN	merge
Valeurs distinctes	UNIQUE	UNIQUE	SELECT DISTINCT	drop_duplicates, unique
Filtrer	FILTRE	FILTER	WHERE	masque booléen
Trier	TRIER	SORT	ORDER BY	sort_values
Regrouper et agréger	tableau croisé dynamique	tableau croisé, QUERY	GROUP BY	groupby, pivot_table
Requête dans une cellule	—	QUERY	—	—
Découper un texte	FRACTIONNER.TEXTE, SPLIT assistant	SPLIT	SUBSTR, INSTR	str.split

Opération	Excel	Google Sheets	SQL	pandas
Dépivoter	Power Query	formules, QUERY	UNION ALL	melt
Calcul sur toute une colonne	formule recopiée, tableaux dynamiques	ARRAYFORMULA	expression dans SELECT	vectorisation

À retenir.

- **Google Sheets** ressemble à Excel pour les fonctions et les tableaux croisés ; il se distingue par la **collaboration**, QUERY, IMPORTRANGE et ARRAYFORMULA. Ses limites de volume sont les mêmes qu'Excel, en plus strictes.
- **QUERY est du SQL** : le résultat attendu se vérifie par une requête SQL (ici, 617 715,45 € répartis sur 6 catégories, Jardin en tête).
- **Looker Studio** relie des tableaux de bord à des sources vivantes : pensez **fraîcheur, droits d'accès, confidentialité**.
- Un **tableau d'équivalences** entre Excel, Sheets, SQL et pandas évite de réapprendre chaque fois.

Pour s'entraîner. Cahier, chapitre 2 : application 2.8, exercice 2.12.

Bilan du chapitre 2

Vous savez maintenant :

- **écrire et lire des formules** : références relatives, absolues et mixtes, plages nommées, tableaux structurés qui grandissent tout seuls ;
- **agréger sous conditions** (SOMME.SI.ENS, NB.SI.ENS, MOYENNE.SI.ENS) et **décider** (SI, SI.CONDITIONS, ET, OU, SIERREUR, en sachant que ce dernier masque toutes les erreurs) ;
- **chercher** une valeur dans une autre table (RECHERCHEV et ses pièges, INDEX+EQUIV, RECHERCHEX), en vérifiant que la clé est **unique** ;
- **nettoyer du texte** et **manipuler des dates** (numéros de série, FIN.MOIS, DATEDIF, jours ouvrés), et repérer une **inversion jour/mois** ;
- **reconnaître les erreurs** d'Excel (#N/A, #REF!, #DIV/0!, #VALEUR!...) et traduire les fonctions entre l'anglais et le français ;
- **construire un tableau croisé dynamique**, regrouper des dates, afficher des proportions, comprendre qu'un champ calculé donne un **ratio de sommes**, et **recouper** le tableau par un autre chemin ;
- **importer et nettoyer un export désordonné** avec Power Query (étapes rejouables), en réglant le délimiteur, l'**encodage** et les **paramètres régionaux**, en comptant les lignes à chaque étape et en **contrôlant un total** ;
- **structurer un classeur** (README, données, calculs, présentation), tenir des **données ordonnées**, séparer les **hypothèses** des formules, tenir une **feuille de contrôles** et reconnaître les erreurs classiques de tableur ;
- (en option) décrire un **modèle de données** en étoile, écrire des **mesures DAX**, situer les **macros** et les **scripts Office**, décider **quand quitter Excel** ; situer **Google Sheets** (QUERY, IMPORTRANGE, ARRAYFORMULA) et **Looker Studio**.

Le chapitre a mis des chiffres sur des habitudes qui restent souvent des slogans. Tous ces chiffres ont été **recoupés par un second outil** (pandas ou SQL).

Ce que nous avons mesuré	Résultat
Le classeur de la gérante	29 827 lignes, 12 946 commandes, 3 875 clients, 1 324 763,72 €
25 formules vérifiées (LibreOffice) contre pandas	écart maximal nul au centime
Recalcul ligne par ligne contre colonne montant	8 centimes d'écart (arrondi à la ligne)
SOMME sur une plage trop courte de 800 lignes	33 638,83 € disparus, aucun message
Marge brute 2025	639 810,88 € (48,30 % du chiffre d'affaires)
Tableau croisé : 18 cases (catégorie × canal), 72 cases (catégorie × mois)	écart nul contre SOMME.SI.ENS
500 lignes copiées deux fois	total gonflé de 20 866,33 € , doublons détectés par les contrôles
Export de caisse : 289 lignes brutes → lignes de vente	280 (3 de titre, 1 en-tête, 4 en-têtes répétés, 1 total)
Réparation des montants manquants contre le total de l'export	2,62 € d'écart : des remises ignorées
Jointure sur le nom d'article (non unique)	280 lignes → 560 , total doublé
Marge 2026 selon l'hypothèse de volume	699 147,47 € (+ 5 %) ou 665 854,73 € (stable)
Chiffre d'affaires du Site, trois outils (formule, SQL, pandas)	617 715,45 € partout

Le fil conducteur du chapitre tient en une phrase : **un chiffre de tableur n'est fiable que s'il peut être recoupé**. Le recoupement prend plusieurs formes : un total comparé à une source indépendante, le nombre de lignes avant et après chaque transformation, la même requête dans deux outils, une feuille de contrôles. Quelles que soient les fonctions que vous maîtrisez, la discipline reste la même.

Ce que ce chapitre n'a pas pu vérifier. Excel n'était pas installé : les formules ont été calculées avec LibreOffice, qui peut différer d'Excel (dans un cas, une racine carrée d'un nombre négatif donne #VALEUR! au lieu de #NOMBRE!, et une clé numérique est comparée à du texte avec plus d'indulgence). Les codes de format de TEXTE dépendent de la langue d'Excel. Les **tableaux croisés dynamiques réels, Power Query, Power Pivot et DAX, VBA, Office Scripts, Google Sheets et Looker Studio** n'ont pas été exécutés : les scripts correspondants sont signalés « non exécutés », avec le résultat attendu recalculé en pandas ou en SQL. Les menus et les libellés des « copies d'écran » (des maquettes dessinées) sont à **vérifier** dans votre version.

Le chapitre 3 apprend le **SQL**, le langage des bases de données. Vous y retrouverez la plupart des questions de ce chapitre (« combien, pour quelle catégorie, quel mois ? »), écrites cette fois dans un langage qui traite sans difficulté des millions de lignes, qui se **rejoue** exactement et qui ne dépend d'aucun menu.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 à 2.8 (explorer et contrôler le classeur, agréger sous conditions, enrichir par recherche, nettoyer texte et dates, recouper un tableau croisé, rejouer une chaîne Power Query, auditer un classeur, trianguler Excel, SQL et pandas) et exercices 2.1 à 2.12.

Chapitre 3 : SQL

« Une question bien posée à une base de données vaut mieux que dix tableaux copiés à la main. »

La gérante de la boutique vous arrête dans le couloir : « Je prépare mon bilan de l'année. **Quels sont nos dix meilleurs clients de 2025, et que représentent-ils dans notre chiffre d'affaires ?** » La question tient en une phrase. La réponse, elle, se cache dans deux tables de plusieurs dizaines de milliers de lignes : la liste des commandes, et le détail de chacune. Vous pourriez ouvrir un classeur, copier, trier, additionner à la main ; vous y passeriez la matinée, et personne ne saurait demain comment vous avez obtenu votre chiffre.

Ce chapitre vous apprend à poser cette question **à la base de données elle-même**, dans le langage prévu pour cela : le **SQL** (*Structured Query Language*, que l'on prononce « esse-ku-elle » ou « sicouèle »). À la fin du chapitre, vous répondrez à la question de la gérante en une requête de quelques lignes, **vous la vérifierez par un autre outil**, et vous pourrez la relancer l'an prochain en changeant une date.

Pourquoi un analyste a besoin du SQL

Une grande partie des données d'une entreprise ne vit pas dans des fichiers, mais dans des **bases de données relationnelles** : la caisse, le site, la comptabilité, la logistique y écrivent en continu. Pour les lire, on ne copie pas les tables dans un tableur : on **interroge** la base. Trois raisons rendent cette compétence centrale pour une analyste.

- **L'échelle.** Un tableur se traîne à partir de quelques centaines de milliers de lignes ; une base répond en quelques instants à des questions sur des millions de lignes, parce qu'elle sait **filtrer, joindre et agréger près des données**, sans les déplacer.
- **La source unique.** Quand tout le monde interroge la même base, tout le monde parle du même chiffre d'affaires. Un classeur recopié, lui, devient vite un chiffre de plus.
- **La reproductibilité.** Une requête est un texte : on la relit, on la fait relire, on la rejoue le mois suivant, on la met sous contrôle de version. Un clic dans un tableur ne laisse aucune trace.

Intuition. Une requête SQL ne dit pas **comment** calculer (quelles boucles, dans quel ordre), mais **ce que l'on veut** : quelles colonnes, de quelles tables, avec quelles conditions, regroupées comment. C'est le moteur de la base qui choisit la méthode. On parle d'un langage **déclaratif**. C'est ce qui rend le SQL à la fois court et exigeant : il faut savoir **décrire précisément le résultat attendu**, y compris la ligne qui, sur le tableau final, représente « une commande », « un client » ou « un mois ».

Le chemin de ce chapitre

Le parcours essentiel suit les quatre idées qui font 90 % du travail quotidien d'une analyste.

- **3.1 SELECT, filtrage, tri, agrégation** : lire une table, garder les lignes qui comptent, les trier, les regrouper et les résumer ; comprendre l'ordre dans lequel le moteur exécute une requête, et la valeur absente (NULL).
- **3.2 Jointures** : recoller les tables entre elles (clients, commandes, produits), garder ou non les lignes sans correspondance, et éviter le piège le plus coûteux du métier : **la multiplication silencieuse des lignes**.
- **3.3 Fonctions fenêtres** : calculer un classement, un cumul, une moyenne mobile ou une évolution d'un mois sur l'autre **sans perdre le détail** des lignes.
- **3.4 CTE et structuration de requêtes complexes** : découper une longue requête en étapes nommées, la construire avec méthode, et la **vérifier** par un second outil.

Deux sections facultatives prolongent ce parcours : **3.5 SQL avancé** (sous-requêtes, index, vues, transactions, déclencheurs, sécurité) et **3.6 Différences entre PostgreSQL, MySQL, SQL Server et Oracle** (les dialectes que vous rencontrerez en entreprise).

Les données du chapitre

La base boutique.db. Tout le chapitre travaille sur une petite base **SQLite** (un fichier unique, sans serveur à installer) qui décrit **trois ans d'activité de la boutique** (2023 à 2025) : les clients, les produits, les commandes et leurs lignes, les retours, et un tableau de bord journalier. **Les données sont simulées**, avec des graines fixes : aucun client, aucun produit réel. La section 3.1.1 en dessine le schéma.

Les requêtes de ce chapitre sont **réellement exécutées** : les tableaux que vous verrez sous chaque bloc sont les résultats produits par SQLite (version 3.46) au moment où le livre a été fabriqué, pas des résultats imaginés. Deux précautions de lecture :

- le chapitre travaille sur une **copie jetable** de la base, ce qui nous autorise à créer des index ou des vues sans rien abîmer ;
- le SQL est un langage normalisé, mais **chaque produit a son dialecte**. Ce que vous lirez ici est du SQL courant, qui fonctionne tel quel sur la plupart des bases ; ce qui est propre à SQLite est signalé, et la section 3.6 dresse la liste des différences.

En pratique : lire les résultats avec un œil d'analyste. Après chaque requête, posez-vous toujours trois questions : *combien de lignes attendais-je ? quelle est la signification d'une ligne du résultat ? ce chiffre est-il plausible ?* Le chapitre vous y entraîne en comparant systématiquement les résultats à des ordres de grandeur connus (36 395 commandes, un chiffre d'affaires annuel de l'ordre du million d'euros...) et, quand c'est possible, à un second outil.

Les applications guidées et les exercices de ce chapitre sont dans le cahier : vous y trouverez une base prête à l'emploi et une trentaine de requêtes à écrire, corrigées.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 à 3.9 et exercices 3.1 à 3.14 (chacun renvoie à la section du livre qu'il met en pratique).

3.1 SELECT, filtrage, tri, agrégation

Cette première section pose les fondations : lire une table, ne garder que les lignes utiles, les ordonner, calculer de nouvelles colonnes, puis **résumer** (compter, additionner, moyenner) par groupes. Presque toute requête d'analyse, même la plus longue, est une variation sur ces gestes. Nous les appliquons d'emblée aux données de la boutique, en gardant à l'esprit la question de la gérante : *qui sont nos meilleurs clients, et pèsent-ils lourd dans le chiffre d'affaires ?*

3.1.1 Une base, des tables, des clés

Une base de données relationnelle est un ensemble de **tables**. Une table ressemble à une feuille de tableur, avec deux différences essentielles : **chaque colonne a un type** (entier, décimal, texte, date...), et **chaque ligne décrit une chose du même genre**. Cette dernière idée est la plus importante du chapitre : le **grain** d'une table, c'est ce que représente **une ligne**.

Table	Une ligne =	Clé de la table
clients	un client	id_client
produits	un article du catalogue	id_produit
commandes	une commande (un passage en caisse, un panier sur le site)	id_commande
lignes_commande	un produit dans une commande	id_ligne
retours	une ligne de commande retournée	id_retour
jours_exploitation	un jour d'activité de la boutique	date

Une **clé primaire** identifie chaque ligne de façon unique. Une **clé étrangère** est une colonne qui contient la clé primaire d'une autre table : `commandes.id_client` désigne un client, `lignes_commande.id_commande` désigne la commande à laquelle appartient la ligne. Ces liens sont ce qui rend la base « relationnelle » ; la figure suivante les dessine.

Demandons à la base ce qu'elle contient. La requête ci-dessous compte les lignes de chaque table ; nous en profitons pour découvrir `UNION ALL`, qui empile des résultats de même forme.

```
SELECT 'clients' AS table_, COUNT(*) AS nb_lignes FROM clients
UNION ALL SELECT 'produits', COUNT(*) FROM produits
UNION ALL SELECT 'commandes', COUNT(*) FROM commandes
UNION ALL SELECT 'lignes_commande', COUNT(*) FROM lignes_commande
UNION ALL SELECT 'retours', COUNT(*) FROM retours
UNION ALL SELECT 'jours_exploitation', COUNT(*) FROM jours_exploitation
```

table_	nb_lignes
clients	6000
produits	120
commandes	36395
lignes_commande	83905
retours	5002
jours_exploitation	1096

La boutique compte donc **6 000 clients**, **36 395 commandes** et **83 905 lignes de commande** sur trois ans, soit un peu plus de deux lignes par commande, et **1 096 jours** d'exploitation (trois ans, dont une année bissextile). Deux lectures s'imposent. D'abord, le rapport entre les tables : il y a plus de lignes que de commandes, **parce qu'une commande contient plusieurs lignes** ; nous y reviendrons en 3.2.6, car c'est la source de la plupart des erreurs de chiffres d'affaires. Ensuite, le type des colonnes. Interrogeons le catalogue de la base pour la table `commandes` :

Les six tables de la base boutique.db : un trait relie une clé à la colonne qui la référence (1 = un, n = plusieurs)

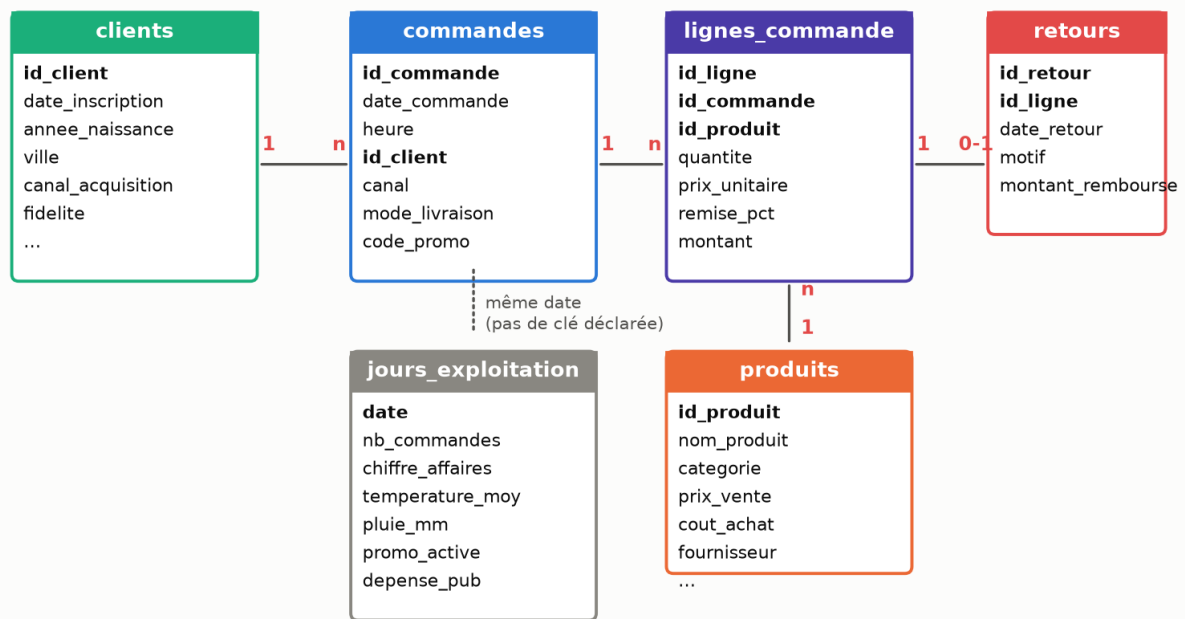


Figure 26: Les six tables de la base boutique.db et leurs liens : un client passe plusieurs commandes, une commande compte plusieurs lignes, une ligne désigne un produit et peut donner lieu à un retour.

```
SELECT name AS colonne, type AS type FROM pragma_table_info('commandes')
```

colonne	type
id_commande	INTEGER
date_commande	TEXT
heure	TEXT
id_client	INTEGER
canal	TEXT
mode_livraison	TEXT
code_promo	TEXT

Les dates sont stockées en **texte**, au format ISO AAAA-MM-JJ. Ce n'est pas un détail : ce format a la propriété agréable que **l'ordre alphabétique est l'ordre chronologique**, ce qui permet de comparer et de trier des dates comme du texte. Dans une autre base (PostgreSQL, SQL Server...) les dates ont un vrai type DATE, mais les requêtes de ce chapitre s'écrivent presque de la même façon.

Piège : la base de la boutique ne déclare pas ses clés. Les six tables ont été créées à partir de fichiers, sans contrainte : rien n'empêche, dans cette base, d'insérer une commande pour un client qui n'existe pas. Dans une base bien conçue, on **déclare** les clés pour que la base elle-même refuse les incohérences. Voici un exemple minimal, deux tables de démonstration :

```
CREATE TABLE demo_clients (id_client INTEGER PRIMARY KEY, ville TEXT NOT NULL);
CREATE TABLE demo_commandes (
  id_commande INTEGER PRIMARY KEY,
  id_client INTEGER NOT NULL REFERENCES demo_clients(id_client),
  montant REAL CHECK (montant >= 0));
PRAGMA foreign_keys = ON;
INSERT INTO demo_clients VALUES (1, 'Ville A'), (2, 'Ville B');
```

```
INSERT INTO demo_commandes VALUES (10, 1, 25.0);
```

La clause REFERENCES déclare la clé étrangère, CHECK impose une règle sur la valeur, NOT NULL interdit l'absence. Essayons maintenant d'insérer deux lignes incohérentes :

```
for requete in ("INSERT INTO demo_commandes VALUES (11, 99, 10.0)", "INSERT INTO demo_commandes VALUES
↳ (12, 1, -5.0)"):
    try:
        con.execute(requete)
    except sqlite3.IntegrityError as erreur:
        print("refusé :", erreur)
```

```
refusé : FOREIGN KEY constraint failed
refusé : CHECK constraint failed: montant >= 0
```

La base refuse les deux, en nommant la règle violée. C'est l'intérêt d'un modèle déclaré : **les erreurs de saisie sont arrêtées à l'entrée**, et non découvertes six mois plus tard dans un rapport. Nous supprimons nos tables de démonstration avant de continuer.

3.1.2 SELECT et FROM : choisir des colonnes

Toute lecture commence par SELECT (quelles colonnes ?) et FROM (dans quelle table ?). La forme la plus simple affiche les premières lignes d'une table :

```
SELECT id_commande, date_commande, canal
FROM commandes
LIMIT 5
```

id_commande	date_commande	canal
1	2023-01-01	Site
2	2023-01-01	Boutique
3	2023-01-01	Boutique
4	2023-01-01	Boutique
5	2023-01-01	Boutique

LIMIT 5 borne le nombre de lignes renvoyées : c'est le bon réflexe pour **regarder** une table sans en afficher soixante mille lignes. On peut écrire SELECT * pour obtenir toutes les colonnes ; c'est pratique en exploration, mais à éviter dans une requête que l'on garde : si la table gagne une colonne demain, le résultat change à votre insu, et la base transporte inutilement des données.

Une requête ne se contente pas de **recopier** des colonnes : elle peut en **calculer**. On donne alors un nom au résultat avec AS (un **alias**). La marge d'un produit est la différence entre son prix de vente et son coût d'achat ; le taux de marge la rapporte au prix :

```
SELECT nom_produit, prix_vente, cout_achat,
       ROUND(prix_vente - cout_achat, 2) AS marge,
       ROUND(100 * (prix_vente - cout_achat) / prix_vente, 1) AS taux_marge
FROM produits
ORDER BY taux_marge DESC
LIMIT 5
```

nom_produit	prix_vente	cout_achat	marge	taux_marge
Pochette compact	7.9	3.16	4.74	60.0
Transat nordique	11.9	4.76	7.14	60.0
Tablier compact	29.9	12.01	17.89	59.8
Rideau design	31.9	12.84	19.06	59.7
Guirlande compact	54.9	22.17	32.73	59.6

Nous avons ajouté ORDER BY (section suivante) pour afficher les cinq produits au taux de marge le plus élevé : ils se situent autour de 60 %, ce qui est cohérent avec la façon dont les coûts d'achat

de la boutique ont été fixés (entre 40 et 65 % du prix). **Tout calcul dans un SELECT se fait ligne par ligne** : la marge de chaque produit est calculée indépendamment des autres. Nous verrons en 3.1.6 comment calculer à l'échelle de **groupes** de lignes.

3.1.3 WHERE : ne garder que les lignes utiles

WHERE filtre les lignes **avant** tout calcul de groupe. Sa condition combine des comparaisons avec AND, OR et NOT. Voici les opérateurs que l'analyste utilise tous les jours :

Besoin	Écriture	Exemple
égalité, différence	=, <> (ou !=)	canal = 'Site'
comparaison	<, <=, >, >=	prix_vente >= 50
intervalle (bornes incluses)	BETWEEN a AND b	date_commande BETWEEN '2025-11-24' AND '2025-11-30'
appartenance à une liste	IN (...)	categorie IN ('Jardin', 'Cuisine')
motif de texte	LIKE (% = n'importe quelle suite, _ = un caractère)	nom_produit LIKE 'Bougie%'
valeur absente	IS NULL, IS NOT NULL	date_retour IS NULL

Les textes s'écrivent entre **apostrophes simples** ('Site') ; les guillemets servent aux noms de colonnes ou de tables. Pour isoler les commandes passées sur le site avec un code promo pendant la semaine du « Vendredi noir » de 2025 :

```
SELECT id_commande, date_commande, heure, canal, code_promo
FROM commandes
WHERE canal = 'Site'
      AND code_promo <> ''
      AND date_commande BETWEEN '2025-11-24' AND '2025-11-30'
ORDER BY date_commande, heure
LIMIT 6
```

id_commande	date_commande	heure	canal	code_promo
34177	2025-11-24	12:02	Site	SOLDES
34176	2025-11-24	12:28	Site	SOLDES
34138	2025-11-24	12:42	Site	SOLDES
34141	2025-11-24	12:57	Site	SOLDES
34167	2025-11-24	13:32	Site	SOLDES
34147	2025-11-24	13:52	Site	SOLDES

Nous voyons les six premières de ces commandes, toutes avec le code SOLDES. Remarquez code_promo <> '' : dans cette base, l'absence de code promo est enregistrée par un **texte vide** '', et non par une valeur absente (NULL) ; nous reviendrons sur cette différence en 3.1.8.

Pour les motifs de texte et les listes, voici les bougies des catégories « Décoration » et « Bien-être » :

```
SELECT id_produit, nom_produit, categorie, prix_vente
FROM produits
WHERE nom_produit LIKE 'Bougie%' AND categorie IN ('Décoration', 'Bien-être')
ORDER BY prix_vente
```

id_produit	nom_produit	categorie	prix_vente
51	Bougie nordique	Décoration	16.9
109	Bougie parfumée compact	Bien-être	27.9
41	Bougie nordique	Décoration	28.9
119	Bougie parfumée compact	Bien-être	56.9

Le résultat compte quatre lignes, et un détail doit vous alerter : « **Bougie nordique** » apparaît **deux fois**, avec deux identifiants et deux prix différents. Dans cette base, **le nom d'un produit n'est pas une clé** ; c'est `id_produit` qui l'est. Regrouper par nom dans une analyse fusionnerait à tort deux articles distincts. Un analyste se méfie toujours des colonnes « qui ressemblent » à une clé.

Piège : AND passe avant OR. Comme en arithmétique où la multiplication précède l'addition, AND est évalué **avant** OR. La condition `canal = 'Site' OR canal = 'Réseaux' AND code_promo = 'SOLDES'` signifie « toutes les commandes du site, **plus** celles des réseaux avec un code SOLDES ». Écrivez toujours des **parenthèses** pour dire ce que vous pensez : `(canal = 'Site' OR canal = 'Réseaux') AND code_promo = 'SOLDES'`. Sur les données de la boutique, la première lecture renvoie 15 810 commandes, la seconde 1 593 : le même texte, écrit sans parenthèses, change le résultat d'un facteur dix.

Les deux chiffres de l'encadré proviennent des deux requêtes suivantes, que nous exécutons pour que vous puissiez les vérifier :

```
SELECT COUNT(*) AS sans_parentheses FROM commandes
WHERE canal = 'Site' OR canal = 'Réseaux' AND code_promo = 'SOLDES';

SELECT COUNT(*) AS avec_parentheses FROM commandes
WHERE (canal = 'Site' OR canal = 'Réseaux') AND code_promo = 'SOLDES'
```

```
sans_parentheses
      15810

avec_parentheses
      1593
```

3.1.4 ORDER BY, LIMIT, DISTINCT

ORDER BY trie le résultat, par ordre croissant (ASC, par défaut) ou décroissant (DESC), sur une ou plusieurs colonnes ; la seconde colonne départage les ex æquo de la première. LIMIT *n* ne garde que les *n* premières lignes **du résultat trié** : c'est la combinaison ORDER BY ... DESC LIMIT *n* qui répond aux questions « les *n* meilleurs... ». Les cinq articles les plus chers du catalogue :

```
SELECT nom_produit, categorie, prix_vente
FROM produits
ORDER BY prix_vente DESC
LIMIT 5
```

nom_produit	categorie	prix_vente
Transat nordique	Jardin	152.9
Lampe design	Maison	126.9
Lanterne mat	Jardin	117.9
Étagère compact	Maison	105.9
Panier mat	Maison	104.9

Piège : LIMIT sans ORDER BY. Sans tri, « les cinq premières lignes » n'ont aucune signification : l'ordre dans lequel une base renvoie ses lignes **n'est pas garanti**, et il peut changer d'un jour à l'autre (après la création d'un index, par exemple). Tout LIMIT qui compte doit être précédé d'un ORDER BY, et d'un critère de départage si des ex æquo sont possibles.

DISTINCT élimine les doublons du résultat. Quels canaux de vente apparaissent dans les commandes ?

```
SELECT DISTINCT canal FROM commandes
```

```
canal
Site
Boutique
Réseaux
```

Le résultat liste les **trois canaux** Boutique, Site et Réseaux. DISTINCT est une excellente façon d'**explorer** une colonne qualitative ; nous verrons qu'on le retrouve aussi à l'intérieur d'un comptage, COUNT(DISTINCT ...), pour compter des éléments **différents**.

3.1.5 Calculer des colonnes : expressions, CASE et dates

Une colonne calculée peut combiner des colonnes, des constantes, des fonctions (ROUND, LENGTH, UPPER, SUBSTR...) et des conditions. L'expression CASE est le « si... alors... sinon » du SQL : elle range chaque ligne dans une catégorie. Classons les produits en trois gammes de prix et comptons-les :

```
SELECT CASE WHEN prix_vente < 20 THEN 'petit prix'
           WHEN prix_vente < 60 THEN 'moyen'
           ELSE 'haut de gamme' END AS gamme,
       COUNT(*) AS nb_produits
FROM produits
GROUP BY gamme
ORDER BY MIN(prix_vente)
```

```
gamme  nb_produits
petit prix      43
moyen          56
haut de gamme   21
```

Le catalogue compte 43 produits « petit prix », 56 « moyen » et 21 « haut de gamme » : les conditions du CASE sont évaluées **dans l'ordre**, la première vraie l'emporte, et le ELSE ramasse tout le reste. (Nous avons utilisé GROUP BY avant de l'avoir présenté : le principe est expliqué en 3.1.6.)

Les **dates** méritent une attention particulière, car une analyse de ventes est presque toujours une analyse **dans le temps**. En SQLite, la fonction strftime(format, date) extrait une partie de la date : '%Y' l'année, '%m' le mois, '%Y-%m' le mois complet, '%w' le jour de la semaine (0 pour le dimanche). Comptons les commandes par mois sur la fin de l'année 2025 :

```
SELECT strftime('%Y-%m', date_commande) AS mois, COUNT(*) AS nb_commandes
FROM commandes
WHERE date_commande >= '2025-07-01'
GROUP BY mois
ORDER BY mois
```

```
mois  nb_commandes
2025-07  963
2025-08  788
2025-09  1097
2025-10  1150
2025-11  1509
2025-12  1853
```

Le volume passe de 963 commandes en juillet à 788 en août (creux estival), puis grimpe jusqu'à **1 853 en décembre**, soit près du double de juillet : la saisonnalité de fin d'année est forte. Le même mécanisme donne le profil de la semaine :


```
SELECT strftime('%w', date_commande) AS jour_semaine, COUNT(*) AS nb_commandes
FROM commandes
GROUP BY jour_semaine
ORDER BY jour_semaine
```

jour_semaine	nb_commandes
0	3496
1	4976
2	4706
3	4893
4	5122
5	5979
6	7223

Avec 0 pour le dimanche et 6 pour le samedi, on lit que **le samedi est le jour le plus chargé** (7 223 commandes sur trois ans) et le dimanche le plus calme (3 496). Une réponse comme celle-ci se traduit directement en décision : la gérante saura quand renforcer l'équipe.

En pratique : stocker des dates comme des dates. Dans les bases qui ont un vrai type DATE, on écrit `date_commande >= DATE '2025-07-01'` et l'on dispose de fonctions d'extraction (`EXTRACT(YEAR FROM ...)`, `DATE_TRUNC('month', ...)`). Le principe est identique ; seuls les noms de fonctions changent (voir 3.6).

3.1.6 Agréger : COUNT, SUM, AVG, MIN, MAX et GROUP BY

Jusqu'ici, chaque ligne du résultat correspondait à une ligne de la table. L'**agrégation** change le grain : elle **résume** plusieurs lignes en une seule. Les cinq fonctions d'agrégation de base sont :

Fonction	Rôle
COUNT(*)	nombre de lignes
COUNT(colonne)	nombre de lignes où la colonne n'est pas absente
COUNT(DISTINCT colonne)	nombre de valeurs différentes
SUM(colonne), AVG(colonne)	somme, moyenne
MIN(colonne), MAX(colonne)	plus petite, plus grande valeur

Sans GROUP BY, l'agrégat porte sur **toute la table** et la requête renvoie une seule ligne. Voici les chiffres globaux des lignes de commande :

```
SELECT COUNT(*) AS nb_lignes,
       ROUND(SUM(montant), 2) AS ca,
       ROUND(AVG(montant), 2) AS montant_moyen_par_ligne,
       COUNT(DISTINCT id_commande) AS nb_commandes,
       ROUND(SUM(montant) / COUNT(DISTINCT id_commande), 2) AS panier_moyen
FROM lignes_commande
```

nb_lignes	ca	montant_moyen_par_ligne	nb_commandes	panier_moyen
83905	3653157.28	43.54	36395	100.38

Le chiffre d'affaires cumulé des trois ans est de **3 653 157,28 €**. Remarquez la différence entre deux moyennes que l'on confond souvent : le **montant moyen par ligne** (43,54 €) et le **panier moyen** (100,38 €), c'est-à-dire le montant moyen **par commande**. Le panier moyen se calcule en divisant la somme par le nombre de **commandes distinctes** ; AVG(montant) moyenne, lui, des **lignes**. Les deux sont corrects, ils répondent à des questions différentes, et c'est à vous de choisir celle que la gérante pose.

Avec `GROUP BY`, la table est d'abord découpée en **paquets** (un par valeur de la colonne de regroupement), puis chaque agrégat est calculé **paquet par paquet**. Le résultat compte une ligne par paquet. Les statistiques du catalogue par catégorie :

```
SELECT categorie, COUNT(*) AS nb_produits,
       ROUND(AVG(prix_vente), 2) AS prix_moyen,
       MIN(prix_vente) AS prix_min, MAX(prix_vente) AS prix_max
FROM produits
GROUP BY categorie
```

categorie	nb_produits	prix_moyen	prix_min	prix_max
Bien-être	20	27.50	9.9	68.9
Cuisine	20	34.75	9.9	57.9
Décoration	20	38.05	11.9	96.9
Jardin	20	51.75	7.9	152.9
Maison	20	51.85	12.9	126.9
Papeterie	20	11.35	2.9	21.9

Chaque catégorie compte 20 produits ; la papeterie a le prix moyen le plus bas (11,35 €), la maison (51,85 €) et le jardin (51,75 €) les plus hauts. On peut regrouper par **plusieurs** colonnes : un paquet par combinaison. Les commandes par année et par canal :

```
SELECT strftime('%Y', date_commande) AS annee, canal, COUNT(*) AS nb_commandes
FROM commandes
GROUP BY annee, canal
ORDER BY annee, canal
```

annee	canal	nb_commandes
2023	Boutique	5916
2023	Réseaux	1230
2023	Site	4272
2024	Boutique	5617
2024	Réseaux	1301
2024	Site	5113
2025	Boutique	5442
2025	Réseaux	1426
2025	Site	6078

On lit deux tendances en même temps : le **Site** progresse chaque année (4 272, 5 113 puis 6 078 commandes) tandis que la **Boutique** recule (5 916, 5 617 puis 5 442). Voilà de quoi alimenter la question que la gérante se pose depuis longtemps : *le site fait-il perdre des clients à la boutique ?* Ce tableau ne prouve rien (les clients du site sont-ils les mêmes ?), mais il oriente l'analyse, et c'est ce qu'on attend d'un premier regard.

Règle d'or du `GROUP BY`. Dans un `SELECT` qui regroupe, chaque colonne est soit **dans le `GROUP BY`**, soit **à l'intérieur d'une fonction d'agrégation**. Il n'y a pas de troisième possibilité raisonnable : si vous affichez une autre colonne, la base ne saurait pas laquelle des lignes du paquet choisir. (SQLite tolère l'écart et choisit une ligne au hasard ; la plupart des autres bases refusent la requête. Ne vous appuyez jamais dessus.)

3.1.7 HAVING et l'ordre logique d'exécution

`WHERE` filtre les **lignes** avant le regroupement. Pour filtrer sur le **résultat d'un agrégat** (« les clients qui ont passé au moins quinze commandes »), il faut une condition **après** le regroupement : c'est le rôle de `HAVING`.

```
SELECT id_client, COUNT(*) AS nb_commandes
FROM commandes
```

```
WHERE date_commande >= '2025-01-01'
GROUP BY id_client
HAVING COUNT(*) >= 15
ORDER BY nb_commandes DESC
LIMIT 5
```

id_client	nb_commandes
1360	25
2987	23
2997	21
4782	20
90	19

Le client numéro 1360 a passé 25 commandes en 2025, le 2987 en a passé 23. Ces deux conditions ne sont pas interchangeables : `WHERE date_commande >= ...` agit sur chaque commande **avant** de les compter ; `HAVING COUNT(*) >= 15` agit sur le compte **une fois les paquets formés**. Pour ne pas se tromper, il faut connaître l'ordre dans lequel le moteur **exécute** une requête, qui n'est **pas** l'ordre dans lequel on l'écrit :

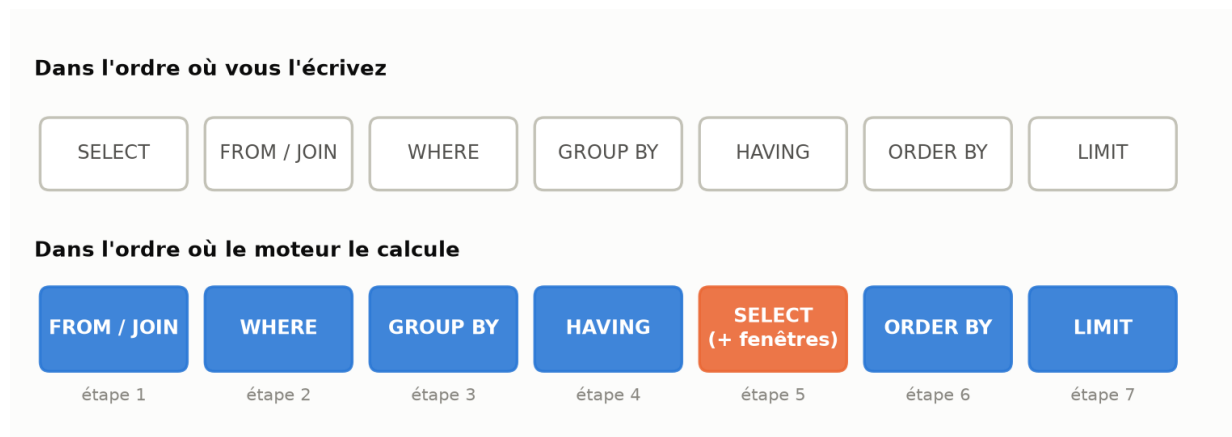


Figure 27: L'ordre d'écriture d'une requête SQL (en haut) n'est pas l'ordre de son exécution (en bas) : le moteur commence par FROM et les jointures, filtre, regroupe, filtre les groupes, et seulement alors calcule les colonnes du SELECT, trie et limite.

Cet ordre explique beaucoup de messages d'erreur et de comportements surprenants :

- On ne peut pas utiliser dans le `WHERE` le résultat d'un agrégat, puisque les groupes n'existent pas encore à l'étape 2 : c'est le travail de `HAVING`.
- Les alias définis dans le `SELECT` (étape 5) n'existent pas encore au moment du `WHERE` (étape 2). Beaucoup de bases refusent donc `WHERE marge > 10` si `marge` est un alias ; **SQLite l'accepte par tolérance**, PostgreSQL le refuse. Pour rester portable, on répète l'expression ou l'on passe par une étape intermédiaire (3.4).
- `ORDER BY`, lui, s'exécute après le `SELECT` : on peut trier sur un alias.

Un dernier exemple combine agrégat, sous-requête et pourcentage. La part de chaque canal dans l'ensemble des commandes :

```
SELECT canal, COUNT(*) AS nb_commandes,
       ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM commandes), 1) AS part_pct
FROM commandes
GROUP BY canal
ORDER BY nb_commandes DESC
```

canal	nb_commandes	part_pct
Boutique	16975	46.6
Site	15463	42.5
Réseaux	3957	10.9

La boutique physique réalise 46,6 % des commandes, le site 42,5 %, les réseaux 10,9 %. La sous-requête (`SELECT COUNT(*) FROM commandes`) renvoie un nombre unique, utilisé comme dénominateur ; nous reverrons ces sous-requêtes en 3.5.

3.1.8 NULL : la valeur absente

En SQL, une cellule peut contenir une valeur **absente** : NULL. Ce n'est ni zéro, ni un texte vide : c'est « on ne sait pas » (une date de retour pour un article jamais retourné, un âge non renseigné). Cette idée a des conséquences, car **toute opération avec une valeur inconnue donne un résultat inconnu**. Quelques expériences pures :

```
SELECT NULL = NULL AS egal, NULL IS NULL AS est_nul,
       1 + NULL AS somme, COALESCE(NULL, 0) AS avec_default
```

egal	est_nul	somme	avec_default
None	1	None	0

Le résultat est parlant : `NULL = NULL` n'est **pas vrai** (on ne peut pas affirmer que deux inconnues sont égales : le résultat est lui-même inconnu), il faut écrire `IS NULL` ; `1 + NULL` donne NULL ; et `COALESCE(x, y)` renvoie la première valeur qui n'est pas absente, ce qui permet de **remplacer** une absence par une valeur par défaut. On dit que le SQL a une **logique à trois valeurs** : vrai, faux, inconnu. Une condition `WHERE` ne garde que les lignes **vraies** : les lignes dont la condition est « inconnue » sont éliminées comme les fausses.

Les fonctions d'agrégation **ignorent** les valeurs absentes, ce qui provoque des erreurs silencieuses si l'on n'y prend garde. Prenons une petite table à trois lignes, 1, NULL et 3 :

```
WITH t(x) AS (VALUES (1), (NULL), (3))
SELECT COUNT(*) AS nb_lignes, COUNT(x) AS nb_valeurs, SUM(x) AS somme, AVG(x) AS moyenne
FROM t
```

nb_lignes	nb_valeurs	somme	moyenne
3	2	4	2.0

`COUNT(*)` compte les **3 lignes**, `COUNT(x)` seulement les **2 valeurs présentes**, et la moyenne vaut 2 (= 4/2) et non 1,33 (= 4/3) : l'absence n'est pas comptée comme un zéro. (`WITH t(x) AS (VALUES ...)` est une façon de poser une mini-table sans la créer ; nous la détaillons en 3.4.) De même, `WHERE x <> 1` **élimine** la ligne NULL : elle n'est ni égale à 1, ni différente de 1, elle est inconnue.

Reste le cas de la boutique, où l'absence de code promo est un **texte vide**. Comparons trois façons de compter :

```
SELECT COUNT(*) AS toutes,
       COUNT(code_promo) AS non_nulles,
       COUNT(NULLIF(code_promo, '')) AS avec_code
FROM commandes
```

toutes	non_nulles	avec_code
36395	36395	5743

`COUNT(code_promo)` renvoie 36 395, soit **toutes** les commandes : aucune valeur n'est NULL, puisque l'absence est codée par ''. La fonction `NULLIF(x, '')` transforme le texte vide en NULL, et le comptage tombe à **5 743 commandes avec un code promo réel**. Retenez-le : **deux données qui « ont**

l'air vides » (`NULL`, ' ', 0, 'N/A') ne sont pas la même absence, et chaque base a ses conventions. C'est le travail de l'analyste de les découvrir avant de calculer.

3.1.9 Quatre pièges classiques

Nous avons croisé plusieurs pièges ; en voici quatre autres, que tout analyste rencontre un jour.

1. La division entière. Dans la plupart des bases (dont SQLite), diviser un entier par un entier renvoie un **entier** : `7 / 2` donne 3. Pour calculer une part, on force une division décimale en multipliant par `100.0` ou en divisant par `2.0`. Voici la part de lignes vendues avec une remise, calculée de façon naïve puis correcte :

```
SELECT SUM(CASE WHEN remise_pct > 0 THEN 1 ELSE 0 END) / COUNT(*) AS part_naive,
       ROUND(100.0 * SUM(CASE WHEN remise_pct > 0 THEN 1 ELSE 0 END) / COUNT(*), 1) AS part_pct
FROM lignes_commande
```

part_naive	part_pct
0	15.8

La version naïve donne **0** ; la bonne, 15,8 %. Ce piège est vicieux parce qu'**aucune erreur ne s'affiche** : on lirait « 0 % de remises » sans sourciller.

2. La moyenne des moyennes. La moyenne de deux moyennes n'est la moyenne de l'ensemble que si les groupes ont **la même taille**. Prenons deux canaux fictifs : A, avec 2 commandes de panier moyen 100 €, et B, avec 8 commandes de panier moyen 50 € :

```
WITH paniers(canal, nb_commandes, panier_moyen) AS (VALUES ('A', 2, 100.0), ('B', 8, 50.0))
SELECT ROUND(AVG(panier_moyen), 1) AS moyenne_des_moyennes,
       ROUND(SUM(nb_commandes * panier_moyen) / SUM(nb_commandes), 1) AS moyenne_ponderee
FROM paniers
```

moyenne_des_moyennes	moyenne_ponderee
75.0	60.0

La moyenne naïve donne 75 €, la bonne (pondérée par le nombre de commandes) **60 €**. Sur les trois vrais canaux de la boutique l'écart est minuscule (nous le verrons en 3.4) parce que les paniers y sont très semblables ; la règle, elle, ne change pas : **on agrège toujours à partir du détail**, jamais à partir de résumés.

3. BETWEEN et les heures. `BETWEEN '2025-11-01' AND '2025-11-30'` inclut le 30 novembre si la colonne ne contient que des dates. Si elle contient aussi l'heure, '2025-11-30 14:20' est plus grand que '2025-11-30' et la dernière journée disparaît. Reconstituons un horodatage en collant la date et l'heure :

```
SELECT COUNT(*) AS avec_between_sur_horodatage
FROM commandes
WHERE date_commande || ' ' || heure BETWEEN '2025-11-01' AND '2025-11-30'
```

avec_between_sur_horodatage
1461

On obtient **1 461** commandes au lieu des 1 509 de novembre vues plus haut : **48 commandes ont disparu**, toutes celles du 30. La parade est de borner **par le haut avec une inégalité stricte** : `>= '2025-11-01' AND < '2025-12-01'`, qui fonctionne qu'il y ait une heure ou non.

4. Le NULL dans une comparaison. Nous l'avons vu en 3.1.8 : `WHERE canal <> 'Site'` n'inclut pas les lignes où `canal` est `NULL`. Quand une colonne peut être absente, **demandez-vous toujours ce que devient la ligne absente**.

À retenir.

- Une table a un **grain** : sachez toujours ce que représente une ligne.
- WHERE filtre les **lignes**, HAVING filtre les **groupes** ; l'ordre d'exécution est FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- Dans un SELECT qui regroupe, chaque colonne est dans le GROUP BY ou dans un agrégat.
- NULL n'est pas zéro : = NULL ne marche pas, les agrégats ignorent les absences, COALESCE et NULLIF sont vos outils.
- Méfiez-vous des divisions entières, des moyennes de moyennes, des bornes de dates et de LIMIT sans ORDER BY.

Pour s'entraîner. Cahier, chapitre 3 : application 3.1 (découvrir la base), application 3.2 (les ventes par canal et par année), exercices 3.1 à 3.4.

3.2 Jointures

Les données d'une entreprise sont **réparties** entre plusieurs tables : les clients d'un côté, les commandes de l'autre, les produits ailleurs. Cette organisation évite de répéter cent fois l'adresse d'un client dans chacune de ses commandes ; en contrepartie, presque toute question utile (« combien chaque catégorie rapporte-t-elle ? », « quels clients n'ont jamais commandé ? ») oblige à **recoller** les morceaux. C'est le rôle des **jointures**. Elles sont aussi la principale source d'erreurs de chiffres dans le métier : cette section vous apprend à les écrire, et surtout à les **vérifier**.

3.2.1 Pourquoi joindre

Reprenons la commande 34177, passée sur le site pendant la semaine du « Vendredi noir ». Dans la table `lignes_commande`, elle apparaît sous forme de numéros : `id_produit`, `quantite`, `montant`. Pour savoir **de quel produit** il s'agit, il faut aller chercher le nom dans `produits`. On écrit la correspondance dans une clause ON :

```
SELECT c.id_commande, c.canal, p.nom_produit, l.quantite,
       l.prix_unitaire, l.remise_pct, l.montant
FROM commandes c
JOIN lignes_commande l ON l.id_commande = c.id_commande
JOIN produits p ON p.id_produit = l.id_produit
WHERE c.id_commande = 34177
```

id_commande	canal	nom_produit	quantite	prix_unitaire	remise_pct	montant
34177	Site	Lanterne mat	2	121.44	20	194.30
34177	Site	Lanterne mat	1	121.44	20	97.15

La commande contient deux lignes du même article, une lanterne vendue 121,44 € l'unité avec 20 % de remise : 194,30 € pour deux, 97,15 € pour une seule. Trois tables, deux jointures, un seul SELECT. Quelques conventions de lecture :

- `commandes c` donne à la table un **alias court** (`c`) pour ne pas avoir à écrire son nom en entier ; `c.id_commande` se lit « la colonne `id_commande` de `c` ».
- **Préfixer chaque colonne par son alias** évite l'ambiguïté (deux tables ont une colonne `id_commande`) et rend la requête lisible : on voit d'où vient chaque information.
- `JOIN ... ON a = b` dit comment recoller : les lignes dont les deux valeurs sont égales. Quand les deux colonnes portent le même nom, `USING (id_commande)` est un raccourci.

3.2.2 INNER JOIN : ne garder que les correspondances

JOIN seul est un raccourci de **INNER JOIN** : on ne garde que les lignes qui ont une correspondance **des deux côtés**. Pour voir ce que cela implique, utilisons de minuscules tables inventées, deux clients (Ana, Bao, Chloé) et quatre commandes :

```
WITH clients_mini(id, nom) AS (VALUES (1, 'Ana'), (2, 'Bao'), (3, 'Chloé')),
     cmd_mini(id_cmd, id_client, montant) AS
     (VALUES (10, 1, 30.0), (11, 1, 20.0), (12, 3, 50.0), (13, 4, 10.0))
SELECT c.nom, m.id_cmd, m.montant
FROM clients_mini c
JOIN cmd_mini m ON m.id_client = c.id
```

nom	id_cmd	montant
Ana	10	30.0
Ana	11	20.0
Chloé	12	50.0

Le résultat compte trois lignes : Ana avec ses deux commandes, Chloé avec la sienne. **Bao a disparu** (elle n'a passé aucune commande) et **la commande 13 aussi** (son client, le numéro 4, n'existe pas dans la table des clients). C'est l'effet d'une jointure interne : toute ligne sans partenaire est écartée, silencieusement. La figure de la page suivante résume les quatre grandes jointures sur deux tables de trois lignes.

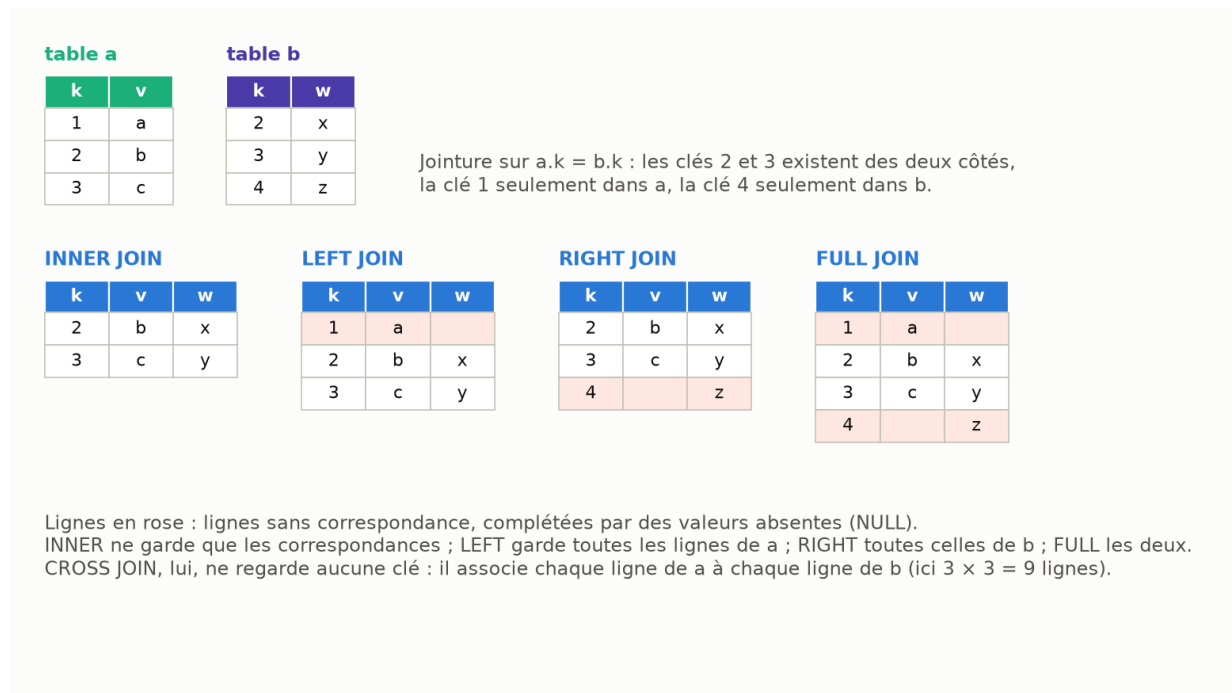


Figure 28: Deux tables de trois lignes jointes sur leur clé : la jointure interne garde les correspondances, la jointure à gauche garde toutes les lignes de la première table, la jointure à droite toutes celles de la seconde, la jointure complète les deux ; les lignes sans correspondance sont complétées par des valeurs absentes.

Passons aux vraies données. Quel chiffre d'affaires chaque catégorie de produits a-t-elle réalisé en 2025 ? Il faut recoller **trois tables** : les lignes (qui portent le montant), les commandes (qui portent la date) et les produits (qui portent la catégorie).

```
SELECT p.categorie, COUNT(*) AS nb_lignes, ROUND(SUM(l.montant), 0) AS ca
FROM lignes_commande l
JOIN commandes c ON c.id_commande = l.id_commande
```



```
JOIN produits p ON p.id_produit = l.id_produit
WHERE c.date_commande >= '2025-01-01'
GROUP BY p.categorie
ORDER BY ca DESC
```

categorie	nb_lignes	ca
Jardin	5393	353955.0
Maison	5300	304614.0
Décoration	5897	258742.0
Cuisine	5141	233313.0
Bien-être	3675	117511.0
Papeterie	4421	56629.0

Le **jardin** arrive en tête (353 955 €), devant la maison (304 614 €) et la décoration (258 742 €) ; la papeterie ferme la marche (56 629 €). Pour vérifier le résultat, additionnez les six chiffres d'affaires : on retrouve **1 324 764 €**, le chiffre d'affaires total de 2025 que nous obtiendrons de façon indépendante en 3.4. Quand une jointure ne perd ni ne duplique de lignes, **les sous-totaux se recourent avec le total**. Faire ce contrôle est un réflexe d'analyste.

3.2.3 LEFT JOIN : garder tout le monde

La jointure à gauche (LEFT JOIN) conserve **toutes les lignes de la table de gauche**, qu'elles aient ou non une correspondance à droite ; les colonnes de droite sont alors remplies de NULL. C'est l'outil des questions de type « quels sont ceux qui n'ont pas... ». Combien de clients inscrits n'ont **jamais** commandé, et d'où viennent-ils ?

```
SELECT c.canal_acquisition, COUNT(*) AS clients_sans_commande
FROM clients c
LEFT JOIN commandes o ON o.id_client = c.id_client
WHERE o.id_commande IS NULL
GROUP BY c.canal_acquisition
```

canal_acquisition	clients_sans_commande
Boutique	580
Réseaux	157
Site	457

Le motif est toujours le même : on joint à gauche, puis on garde les lignes où une colonne **de la table de droite** (idéalement sa clé) est NULL. On appelle cela une **anti-jointure**. Le résultat montre 580 clients de la boutique, 457 du site et 157 des réseaux, soit **1 194 clients** (un sur cinq) qui n'ont encore rien acheté : un public naturel pour une campagne de première commande. Il existe une autre écriture, souvent plus lisible, avec NOT EXISTS :

```
SELECT COUNT(*) AS clients_sans_commande
FROM clients c
WHERE NOT EXISTS (SELECT 1 FROM commandes o WHERE o.id_client = c.id_client)
```

clients_sans_commande
1194

Elle renvoie bien les mêmes 1 194 clients. Les deux formulations sont équivalentes ; choisissez celle qui se lit le mieux (nous détaillons EXISTS en 3.5).

Piège : le filtre sur la table de droite, dans le WHERE. Combien de clients n'ont **pas** commandé en 2025 ? L'écriture naturelle semble être : LEFT JOIN commandes o ON ... WHERE o.date_commande >= '2025-01-01' AND o.id_commande IS NULL. Exécutons-la :

```
SELECT COUNT(*) AS clients_sans_commande_2025_faux
FROM clients c
LEFT JOIN commandes o ON o.id_client = c.id_client
WHERE o.date_commande >= '2025-01-01' AND o.id_commande IS NULL
```

```
clients_sans_commande_2025_faux
0
```

Elle renvoie **0 ligne**, ce qui est absurde : la condition `o.date_commande >= ...` est fausse quand `o` est absente (NULL), donc le **WHERE élimine** précisément les lignes que le **LEFT JOIN** voulait garder, et la jointure à gauche se comporte comme une jointure interne. La bonne écriture met le filtre **dans le ON** :

```
SELECT COUNT(*) AS clients_sans_commande_2025
FROM clients c
LEFT JOIN commandes o ON o.id_client = c.id_client AND o.date_commande >= '2025-01-01'
WHERE o.id_commande IS NULL
```

```
clients_sans_commande_2025
2125
```

On obtient **2 125 clients** sans commande en 2025 (6 000 clients au total, 3 875 actifs en 2025 : $6\,000 - 3\,875 = 2\,125$). La règle : **une condition qui décrit quelles lignes de droite peuvent correspondre va dans le ON ; une condition qui décrit quelles lignes finales on garde va dans le WHERE.**

Les anti-jointures servent aussi à **contrôler l'intégrité** d'une base dont les clés ne sont pas déclarées, comme la nôtre. Combien de lignes de commande désignent un produit inexistant, de commandes sans aucune ligne, de commandes dont le client est inconnu ?

```
SELECT 'lignes sans produit' AS controle, COUNT(*) AS nb
FROM lignes_commande l LEFT JOIN produits p ON p.id_produit = l.id_produit WHERE p.id_produit IS NULL
UNION ALL
SELECT 'commandes sans ligne', COUNT(*)
FROM commandes c LEFT JOIN lignes_commande l ON l.id_commande = c.id_commande WHERE l.id_ligne IS NULL
UNION ALL
SELECT 'commandes de client inconnu', COUNT(*)
FROM commandes c LEFT JOIN clients cl ON cl.id_client = c.id_client WHERE cl.id_client IS NULL
```

```
          controle  nb
lignes sans produit  0
commandes sans ligne 0
commandes de client inconnu 0
```

Les trois contrôles renvoient **0** : la base est cohérente. Sur un vrai système, ce genre de requête est lancé à chaque chargement de données ; un résultat non nul est un message à destination de l'équipe qui fournit les données.

3.2.4 RIGHT JOIN et FULL JOIN

La jointure à droite (**RIGHT JOIN**) est le symétrique de la jointure à gauche : elle garde toutes les lignes de la table de droite. En pratique, **on l'évite** : inverser l'ordre des tables et utiliser **LEFT JOIN** dit la même chose et se lit mieux. La jointure complète (**FULL JOIN**) garde **les deux côtés** : elle sert à **comparer deux sources** et à repérer à la fois ce qui manque d'un côté et de l'autre.

```
WITH a(k, v) AS (VALUES ('A', 'a1'), ('B', 'b1'), ('C', 'c1')),
      b(k, w) AS (VALUES ('B', 'b2'), ('C', 'c2'), ('D', 'd2'))
SELECT a.k AS k_a, b.k AS k_b, v, w
FROM a FULL JOIN b ON a.k = b.k
```

k_a	k_b	v	w
A	NaN	a1	NaN
B	B	b1	b2
C	C	c1	c2
NaN	D	NaN	d2

Les clés B et C sont communes ; A n'existe que dans la première table (les colonnes de b sont absentes), D que dans la seconde. Les versions récentes de SQLite (à partir de la 3.39), PostgreSQL, SQL Server et Oracle savent faire un `FULL JOIN`. **MySQL ne le connaît pas** : on l'émule alors en empilant deux jointures à gauche, la seconde limitée aux lignes qui n'ont pas de partenaire :

```
SELECT a.k AS k_a, b.k AS k_b, v, w FROM a LEFT JOIN b ON a.k = b.k
UNION ALL
SELECT a.k, b.k, v, w FROM b LEFT JOIN a ON a.k = b.k WHERE a.k IS NULL
```

Nous ne l'exécutons pas ici (les tables a et b n'existent que le temps de la requête précédente, et SQLite sait faire le `FULL JOIN`) ; c'est un schéma à retenir pour les moteurs qui n'en disposent pas.

3.2.5 CROSS JOIN et auto-jointure

Le **CROSS JOIN** associe **chaque ligne** de la première table à **chaque ligne** de la seconde, sans regarder aucune clé : trois lignes par trois lignes donnent neuf lignes. C'est un outil précieux pour fabriquer une **grille complète** de combinaisons, y compris celles qui n'existent pas dans les données. Quelles combinaisons canal × mode de livraison la boutique observe-t-elle ?

```
WITH canaux(canale) AS (VALUES ('Boutique'), ('Site'), ('Réseaux')),
     modes(mode_livraison) AS (VALUES ('Domicile'), ('Point relais'), ('Retrait magasin'))
SELECT k.canale, m.mode_livraison, COUNT(c.id_commande) AS nb_commandes
FROM canaux k CROSS JOIN modes m
LEFT JOIN commandes c ON c.canale = k.canale AND c.mode_livraison = m.mode_livraison
GROUP BY k.canale, m.mode_livraison
ORDER BY k.canale, m.mode_livraison
```

canale	mode_livraison	nb_commandes
Boutique	Domicile	0
Boutique	Point relais	0
Boutique	Retrait magasin	16975
Réseaux	Domicile	2164
Réseaux	Point relais	1519
Réseaux	Retrait magasin	274
Site	Domicile	8573
Site	Point relais	5791
Site	Retrait magasin	1099

Sans la grille, un simple `GROUP BY canale, mode_livraison` aurait renvoyé sept lignes seulement et **n'aurait jamais montré** que la boutique n'offre ni livraison à domicile ni point relais (deux lignes à 0). Avec elle, les combinaisons impossibles apparaissent explicitement. Notez le `COUNT(c.id_commande)`, et non `COUNT(*)` : il compte les commandes **réellement présentes** (zéro pour les lignes complétées), alors que `COUNT(*)` compterait la ligne de la grille elle-même et donnerait 1.

Une **auto-jointure** joint une table **avec elle-même**, sous deux alias différents. Elle sert à comparer des lignes entre elles. Un client a-t-il passé deux commandes le même jour (doublon de saisie, ou commande oubliée complétée après coup) ?

```
SELECT a.id_client, a.date_commande, a.id_commande AS cmd_1, b.id_commande AS cmd_2
FROM commandes a
JOIN commandes b ON a.id_client = b.id_client
```

```

        AND a.date_commande = b.date_commande
        AND a.id_commande < b.id_commande
WHERE a.date_commande >= '2025-12-01'
LIMIT 5

```

id_client	date_commande	cmd_1	cmd_2
3330	2025-12-03	34699	34701
487	2025-12-08	34988	35000
3737	2025-12-09	35071	35085
3937	2025-12-11	35192	35209
2723	2025-12-16	35464	35483

La condition `a.id_commande < b.id_commande` est l'astuce standard : elle garde chaque **paire** une seule fois (sans elle, la paire (1, 2) apparaîtrait aussi sous la forme (2, 1), et chaque commande serait comparée à elle-même). La même requête, sans le filtre de décembre ni la limite, donne le total sur les trois ans :

```

nb_paires_sur_trois_ans
321

```

On compte **321 paires** de ce type : à étudier avant de conclure à une erreur, car un client peut très bien passer deux commandes dans la journée.

3.2.6 Le piège de la multiplication des lignes

C'est la source d'erreurs de chiffres du métier. Rappelons la règle : une jointure « un à plusieurs » **recopie** chaque ligne du côté « un » autant de fois qu'elle a de correspondances du côté « plusieurs ». Les colonnes du côté « un » se retrouvent alors **multipliées**, et toute somme sur ces colonnes est fausse.

Un exemple concret avec le budget publicitaire. La table `jours_exploitation` contient la dépense publicitaire de chaque **jour** ; la table `commandes` contient plusieurs commandes par jour. Si l'on joint les deux sur la date, la dépense d'un jour est recopiée **une fois par commande** de ce jour-là :

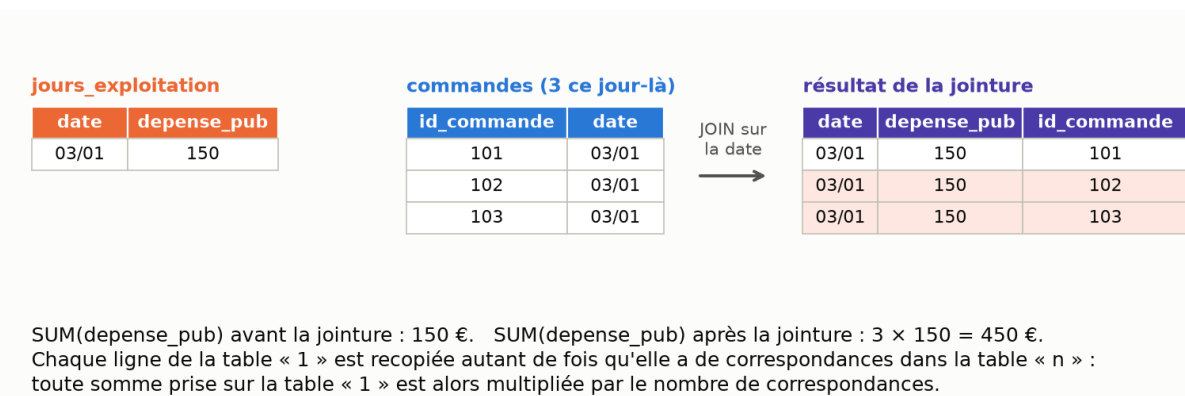


Figure 29: Un jour de dépense publicitaire de 150 € joint aux trois commandes de ce jour : la dépense apparaît trois fois dans le résultat, et la somme passe de 150 € à 450 €.

Voyons l'effet sur les vraies données. La dépense publicitaire de 2025, sommée directement, puis après la jointure :

```
SELECT ROUND(SUM(depense_pub), 0) AS pub_2025
FROM jours_exploitation WHERE date >= '2025-01-01';

SELECT ROUND(SUM(j.depense_pub), 0) AS pub_apres_jointure
FROM jours_exploitation j
JOIN commandes c ON c.date_commande = j.date
WHERE j.date >= '2025-01-01'
```

```
pub_2025
75995.0

pub_apres_jointure
2991731.0
```

Le budget réel est de **75 995 €**. La jointure en affiche **2 991 731 €**, soit près de **quarante fois trop** : chaque jour a été compté autant de fois qu'il comptait de commandes (une trentaine en moyenne). Ce genre d'erreur se produit **sans message d'erreur**, et le résultat n'a rien d'absurde au premier regard : un directeur marketing qui lirait trois millions d'euros de publicité aurait d'abord un doute sur les données, pas sur la requête.

Même phénomène, plus discret, pour un simple comptage. Combien de commandes y a-t-il après jointure avec leurs lignes ?

```
SELECT COUNT(*) AS nb_lignes_jointes, COUNT(DISTINCT c.id_commande) AS nb_commandes
FROM commandes c
JOIN lignes_commande l ON l.id_commande = c.id_commande
```

```
nb_lignes_jointes  nb_commandes
83905              36395
```

COUNT(*) renvoie **83 905** (le nombre de lignes de commande), alors que la base compte **36 395 commandes** ; seul COUNT(DISTINCT id_commande) rend la bonne réponse. La règle est donc de **toujours savoir le grain du résultat d'une jointure** : après avoir joint commandes et lignes, une ligne est une **ligne de commande**, pas une commande.

Trois parades, par ordre de préférence :

1. **Agréger avant de joindre.** Ramenez le côté « plusieurs » au grain du côté « un » (par exemple, un nombre de commandes **par jour**), puis joignez. La jointure devient « un à un » et plus rien n'est recopié :

```
SELECT ROUND(SUM(j.depense_pub), 0) AS pub_2025
FROM jours_exploitation j
LEFT JOIN (SELECT date_commande, COUNT(*) AS nb FROM commandes GROUP BY date_commande) c
ON c.date_commande = j.date
WHERE j.date >= '2025-01-01'
```

```
pub_2025
75995.0
```

On retrouve les **75 995 €** : la jointure à un seul partenaire par jour ne multiplie rien.

2. **Compter des éléments distincts** (COUNT(DISTINCT ...)) quand on ne peut pas éviter la jointure.
3. **Contrôler**, avant et après chaque jointure, **le nombre de lignes** et **une somme connue**. Si SUM(montant) change, ou si le nombre de lignes dépasse celui de la table « un », c'est qu'une multiplication a eu lieu.

En pratique : le test des trois comptes. Avant de livrer un chiffre issu d'une jointure, comparez : (1) le nombre de lignes de la table de départ, (2) le nombre de lignes après jointure, (3) un total de référence calculé sans jointure. Vous attrapez ainsi 90 % des erreurs de jointure en moins d'une minute.

3.2.7 Empiler et comparer des ensembles

Une jointure **élargit** une table en largeur ; les opérations ensemblistes **empilent** ou **comparent** des résultats de même forme (même nombre de colonnes, types compatibles).

Opérateur	Résultat
UNION ALL	toutes les lignes des deux résultats, doublons compris
UNION	toutes les lignes, sans doublons (plus coûteux : la base doit chercher les doublons)
INTERSECT	les lignes présentes dans les deux
EXCEPT	les lignes du premier résultat absentes du second

INTERSECT et EXCEPT répondent élégamment à des questions de **fidélisation**. Parmi les clients actifs, combien sont fidèles (déjà clients avant 2025), combien sont nouveaux, et combien a-t-on perdus ?

```
SELECT 'fidèles (avant et en 2025)' AS groupe, COUNT(*) AS nb_clients FROM (
  SELECT id_client FROM commandes WHERE date_commande >= '2025-01-01'
  INTERSECT SELECT id_client FROM commandes WHERE date_commande < '2025-01-01')
UNION ALL SELECT 'nouveaux en 2025', COUNT(*) FROM (
  SELECT id_client FROM commandes WHERE date_commande >= '2025-01-01'
  EXCEPT SELECT id_client FROM commandes WHERE date_commande < '2025-01-01')
UNION ALL SELECT 'perdus (avant, pas en 2025)', COUNT(*) FROM (
  SELECT id_client FROM commandes WHERE date_commande < '2025-01-01'
  EXCEPT SELECT id_client FROM commandes WHERE date_commande >= '2025-01-01')
```

	groupe	nb_clients
fidèles (avant et en 2025)		3133
nouveaux en 2025		742
perdus (avant, pas en 2025)		931

On compte **3 133 clients fidèles**, **742 nouveaux** et **931 perdus**. Les deux premiers font **3 875 clients** actifs en 2025, le chiffre de la section 3.1 : les groupes se recoupent, la requête est cohérente. Notez enfin que INTERSECT et EXCEPT **éliminent les doublons** d'eux-mêmes, ce qui évite d'avoir à écrire DISTINCT : un client est compté une fois, qu'il ait passé une ou vingt commandes.

Piège : UNION ou UNION ALL ? UNION supprime les doublons, ce qui peut **effacer des lignes légitimes** (deux ventes identiques le même jour). Si vous empilez deux périodes sans chevauchement, utilisez UNION ALL : c'est plus rapide et ça ne perd rien.

3.2.8 Joindre sur plusieurs colonnes

Il arrive que la clé d'une table soit **composée** : une ligne d'objectifs est identifiée par un couple (année, canal). On joint alors sur **toutes** les colonnes de la clé, reliées par AND. Comparons les objectifs de chiffre d'affaires de la direction (valeurs inventées) à la réalité :

```
WITH objectifs(annee, canal, objectif) AS (VALUES
    ('2024', 'Boutique', 600000), ('2024', 'Site', 500000),
    ('2025', 'Boutique', 580000), ('2025', 'Site', 650000)),
reel AS (SELECT strftime('%Y', c.date_commande) AS annee, c.canal, SUM(l.montant) AS ca
FROM commandes c JOIN lignes_commande l USING (id_commande) GROUP BY 1, 2)
SELECT o.annee, o.canal, o.objectif, ROUND(r.ca, 0) AS reel,
    ROUND(100.0 * r.ca / o.objectif, 1) AS atteinte_pct
FROM objectifs o JOIN reel r ON r.annee = o.annee AND r.canal = o.canal
ORDER BY 1, 2
```

annee	canal	objectif	reel	atteinte_pct
2024	Boutique	600000	558143.0	93.0
2024	Site	500000	502531.0	100.5
2025	Boutique	580000	560974.0	96.7
2025	Site	650000	617715.0	95.0

Le site dépasse de peu son objectif de 2024 (100,5 %), mais manque celui de 2025 de cinq points (95,0 %) ; la boutique reste sous les siens (93,0 % en 2024, 96,7 % en 2025). Oublier **une** des deux colonnes de la clé (joindre seulement sur canal) aurait associé chaque objectif aux chiffres de **toutes** les années, et multiplié les lignes (retour au piège précédent). Une jointure sur clé composite se vérifie donc, elle aussi, par le **nombre de lignes** : ici, quatre lignes en entrée, quatre lignes en sortie.

À retenir.

- Une jointure recolle des tables sur une **clé** ; INNER garde les correspondances, LEFT garde tout le côté gauche, FULL garde les deux, CROSS fait toutes les combinaisons.
- Une anti-jointure (LEFT JOIN ... WHERE droite.clé IS NULL, ou NOT EXISTS) répond aux questions « qui n'a pas... ». Le filtre qui définit les correspondances possibles va dans le ON.
- **Le grain du résultat est celui de la table « plusieurs »** : après une jointure un-à-plusieurs, toute somme sur une colonne du côté « un » est multipliée. Agrégez avant de joindre, ou comptez des éléments distincts.
- Contrôlez toujours le **nombre de lignes** et un **total connu** avant et après une jointure.
- UNION ALL empile, INTERSECT et EXCEPT comparent des ensembles.

Pour s'entraîner. Cahier, chapitre 3 : application 3.3 (les meilleurs clients), application 3.4 (panier moyen et double comptage), exercices 3.5 à 3.8.

3.3 Fonctions fenêtres

Le GROUP BY écrase les lignes : une ligne par groupe, et le détail est perdu. Or beaucoup de questions d'analyse demandent **les deux à la fois** : garder chaque ligne **et** la situer par rapport à son groupe (sa part du total du client, son rang dans sa catégorie, l'écart avec le mois précédent, le cumul depuis janvier). Les **fonctions fenêtres** (*window functions*) répondent exactement à ce besoin. Elles sont l'outil le plus puissant que vous apprendrez dans ce chapitre : une fois maîtrisées, elles remplacent des dizaines de lignes de tableur ou de code.

3.3.1 Agréger sans perdre le détail

Voici un exemple simple. Pour un client donné (le numéro 2), on veut afficher chacune de ses lignes d'achat de 2025 et la part qu'elle représente dans ses achats de l'année :

```
SELECT c.id_commande, c.id_client, ROUND(l.montant, 2) AS montant,
       ROUND(SUM(l.montant) OVER (PARTITION BY c.id_client), 2) AS total_client,
       ROUND(100 * l.montant / SUM(l.montant) OVER (PARTITION BY c.id_client), 1) AS part_pct
FROM commandes c
JOIN lignes_commande l USING (id_commande)
WHERE c.id_client = 2 AND c.date_commande >= '2025-01-01'
ORDER BY c.id_commande
LIMIT 5
```

id_commande	id_client	montant	total_client	part_pct
29212	2	24.62	211.18	11.7
30003	2	24.52	211.18	11.6
30003	2	28.74	211.18	13.6
30003	2	22.56	211.18	10.7
35090	2	21.43	211.18	10.1

Chaque ligne d'achat est conservée, et deux colonnes nouvelles apparaissent : le **total du client** (211,18 €, répété sur chaque ligne) et la **part** de la ligne dans ce total (de 10,1 % à 13,6 % ici). L'expression `SUM(l.montant) OVER (PARTITION BY c.id_client)` se lit : « la somme de montant, calculée sur la fenêtre formée par toutes les lignes du même client ». La somme n'est **pas** un `GROUP BY` : aucune ligne n'a été réduite. Avec un `GROUP BY id_client`, le résultat aurait compté une seule ligne par client et la part de chaque ligne aurait été impossible à calculer.

Intuition. Une fonction fenêtre fait deux choses. Pour **chaque ligne**, elle regarde un ensemble de lignes voisines (la **fenêtre**) et calcule un résultat sur cet ensemble, puis elle **attache** ce résultat à la ligne. La fenêtre est définie par la clause `OVER (...)`.

3.3.2 Anatomie d'une fonction fenêtre

Une fonction fenêtre a toujours la même forme : `fonction(...) OVER (PARTITION BY ... ORDER BY ... cadre)`. Chaque élément de la parenthèse est facultatif.

Élément	Rôle	Exemple
<code>PARTITION BY</code>	découpe les lignes en paquets indépendants, comme <code>GROUP BY</code> mais sans les réduire	<code>PARTITION BY id_client</code>
<code>ORDER BY</code>	ordonne les lignes à l'intérieur du paquet (indispensable pour un classement ou un cumul)	<code>ORDER BY date_commande</code>
cadre (<i>frame</i>)	précise quelles lignes autour de la ligne courante entrent dans le calcul	<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>

Trois familles de fonctions s'utilisent après `OVER` : les fonctions d'**agrégation** vues en 3.1 (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`), les fonctions de **classement** (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`) et les fonctions de **décalage** (`LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`).

Un point technique décide de la correction de vos requêtes : **les fonctions fenêtres sont calculées après `WHERE`, `GROUP BY` et `HAVING`**, à l'étape du `SELECT` (la figure de la section 3.1.7). Conséquence : un `WHERE` **supprime des lignes avant** que la fenêtre ne les voie. Prenons l'évolution mensuelle

du chiffre d'affaires, calculée avec LAG, qui renvoie la valeur de la ligne précédente. Si l'on filtre sur 2025 dans le même niveau de requête que le LAG, le mois de décembre 2024 n'existe plus et janvier n'a pas de « mois précédent » :

```
WITH mensuel AS (
  SELECT strftime('%Y-%m', c.date_commande) AS mois, ROUND(SUM(l.montant), 0) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  GROUP BY mois)
SELECT mois, ca, LAG(ca) OVER (ORDER BY mois) AS ca_precedent
FROM mensuel
WHERE mois >= '2025-01'
ORDER BY mois
LIMIT 3
```

mois	ca	ca_precedent
2025-01	89179.0	NaN
2025-02	72642.0	89179.0
2025-03	89787.0	72642.0

Le chiffre d'affaires de janvier 2025 (89 179 €) n'a pas de mois précédent (NaN, une valeur absente), alors que décembre 2024 existe bel et bien. **Le filtre a été appliqué avant la fenêtre.** La parade consiste à calculer la fenêtre dans une étape, puis à filtrer dans l'étape suivante (ici, deux niveaux avec WITH) :

```
WITH mensuel AS (
  SELECT strftime('%Y-%m', c.date_commande) AS mois, ROUND(SUM(l.montant), 0) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  GROUP BY mois),
evol AS (SELECT mois, ca, LAG(ca) OVER (ORDER BY mois) AS ca_precedent FROM mensuel)
SELECT mois, ca, ca_precedent,
  ROUND(100.0 * (ca - ca_precedent) / ca_precedent, 1) AS evolution_pct
FROM evol
WHERE mois >= '2025-07'
ORDER BY mois
```

mois	ca	ca_precedent	evolution_pct
2025-07	111221.0	106930.0	4.0
2025-08	89898.0	111221.0	-19.2
2025-09	113453.0	89898.0	26.2
2025-10	120064.0	113453.0	5.8
2025-11	143892.0	120064.0	19.8
2025-12	183845.0	143892.0	27.8

(WITH nom AS (requête) met une requête de côté sous un nom ; nous le détaillons en 3.4.) Le tableau est précieux pour une analyste : août perd **19,2 %** (creux estival), septembre regagne 26,2 %, et la fin d'année accélère (+19,8 % en novembre, **+27,8 % en décembre**). La question « le chiffre d'affaires baisse-t-il en août ? » se règle par un décalage, sans copier une seule cellule.

3.3.3 Classer : ROW_NUMBER, RANK, DENSE_RANK

Les trois fonctions de classement numérotent les lignes selon un ordre. Elles diffèrent **sur les ex æquo** :

Fonction	Ex æquo	Exemple de numérotation (valeurs 506, 506, 493, 467)
ROW_NUMBER()	numéros tous différents (l'un des ex æquo passe avant l'autre, arbitrairement)	1, 2, 3, 4
RANK()	même rang , puis un « trou »	1, 1, 3, 4

Fonction	Ex æquo	Exemple de numérotation (valeurs 506, 506, 493, 467)
DENSE_RANK()	même rang, sans trou	1, 1, 2, 3

Regardons-le sur les cinq produits de la papeterie les plus vendus en 2025 (en quantités), où deux articles sont à égalité :

```
WITH ventes AS (
  SELECT p.nom_produit, SUM(l.quantite) AS qte
  FROM lignes_commande l JOIN commandes c USING (id_commande) JOIN produits p USING (id_produit)
  WHERE c.date_commande >= '2025-01-01' AND p.categorie = 'Papeterie'
  GROUP BY p.id_produit)
SELECT nom_produit, qte,
       ROW_NUMBER() OVER (ORDER BY qte DESC) AS rn,
       RANK() OVER (ORDER BY qte DESC) AS rang,
       DENSE_RANK() OVER (ORDER BY qte DESC) AS rang_dense
FROM ventes ORDER BY qte DESC LIMIT 5
```

nom_produit	qte	rn	rang	rang_dense
Cahier nordique	506	1	1	1
Stylo design	506	2	1	1
Carnet mat	493	3	3	2
Agenda compact	467	4	4	3
Classeur rustique	402	5	5	4

« Cahier nordique » et « Stylo design » ont vendu 506 unités chacun : ROW_NUMBER les départage arbitrairement (1 puis 2), RANK leur donne le rang 1 à tous les deux puis saute au rang 3, DENSE_RANK passe au rang 2 sans trou. Le choix dépend de la question : pour **garder exactement les n premiers**, utilisez ROW_NUMBER (et un critère de départage explicite) ; pour **garder tous les gagnants**, RANK.

L'usage le plus fréquent est le **top-N par groupe** : « *le produit le plus vendu de chaque catégorie* ». PARTITION BY recommence le classement à chaque catégorie. Comme on ne peut pas filtrer directement sur le résultat d'une fonction fenêtre (elle est calculée après le WHERE), on filtre dans une requête extérieure :

```
SELECT categorie, nom_produit, qte FROM (
  SELECT p.categorie, p.nom_produit, SUM(l.quantite) AS qte,
         RANK() OVER (PARTITION BY p.categorie ORDER BY SUM(l.quantite) DESC) AS rang
  FROM lignes_commande l JOIN commandes c USING (id_commande) JOIN produits p USING (id_produit)
  WHERE c.date_commande >= '2025-01-01'
  GROUP BY p.id_produit)
WHERE rang = 1
ORDER BY categorie
```

categorie	nom_produit	qte
Bien-être	Savon nordique	447
Cuisine	Casserole nordique	596
Décoration	Vase mat	698
Jardin	Arrosoir nordique	675
Maison	Plaid nordique	659
Papeterie	Cahier nordique	506
Papeterie	Stylo design	506

La « Casserole nordique » domine la cuisine (596 unités), le « Vase mat » la décoration (698), l'« Arrosoir nordique » le jardin (675). Dans la papeterie, **deux** produits sortent, puisque RANK conserve les ex æquo : le résultat compte sept lignes pour six catégories. Notez aussi qu'une fonction fenêtre peut s'appliquer à un **agrégat** (SUM(l.quantite) à l'intérieur de RANK() OVER) : le GROUP BY s'exécute d'abord, la fenêtre classe ensuite les groupes.

3.3.4 Comparer à la ligne précédente : LAG et LEAD

LAG(colonne) renvoie la valeur de la ligne **précédente** dans l'ordre de la fenêtre, LEAD(colonne) celle de la ligne **suivante** ; un second argument choisit le nombre de lignes (LAG(x, 2)). Elles servent à calculer des **écarts**, des **variations** (comme plus haut) et des **durées entre événements**. Combien de temps s'écoule-t-il entre deux commandes successives d'un même client ?

```
SELECT id_client, date_commande,
       LAG(date_commande) OVER (PARTITION BY id_client ORDER BY date_commande, id_commande) AS
       ↳ commande_precedente,
       CAST(julianday(date_commande) - julianday(LAG(date_commande)
       OVER (PARTITION BY id_client ORDER BY date_commande, id_commande)) AS INTEGER) AS
       ↳ ecart_jours
FROM commandes
WHERE id_client = 2
ORDER BY date_commande
```

id_client	date_commande	commande_precedente	ecart_jours
2	2023-01-18	NaN	NaN
2	2023-04-11	2023-01-18	83.0
2	2023-09-27	2023-04-11	169.0
2	2024-08-05	2023-09-27	313.0
2	2024-10-05	2024-08-05	61.0
2	2025-07-05	2024-10-05	273.0
2	2025-08-01	2025-07-05	27.0
2	2025-12-09	2025-08-01	130.0
2	2025-12-16	2025-12-09	7.0

Pour le client 2, les écarts successifs sont de 83, 169, 313, 61, 273, 27, 130 et 7 jours : un rythme très irrégulier. La première commande n'a pas de précédente (valeur absente). Remarquez PARTITION BY id_client : sans lui, LAG comparerait la commande d'un client à celle du **client précédent** dans la table. Étendons maintenant le calcul à **tous** les clients, pour résumer ces écarts :

```
WITH ecarts AS (
  SELECT julianday(date_commande) - julianday(LAG(date_commande)
         OVER (PARTITION BY id_client ORDER BY date_commande, id_commande)) AS jours
  FROM commandes)
SELECT COUNT(jours) AS nb_ecarts, ROUND(AVG(jours), 1) AS moyenne, MIN(jours) AS minimum, MAX(jours)
↳ AS maximum
FROM ecarts
```

nb_ecarts	moyenne	minimum	maximum
31589	86.4	0.0	1014.0

On dispose de **31 589 écarts**, d'une moyenne de **86,4 jours** (un client recommande en moyenne tous les trois mois), avec un minimum de 0 (deux commandes le même jour, que nous avons repérées en 3.2.5) et un maximum de plus de mille jours. COUNT(jours) ignore les valeurs absentes (la première commande de chaque client), ce qui est exactement ce qu'on veut.

3.3.5 Cumuls, moyennes mobiles et cadres

Quand on ajoute un ORDER BY à une fonction d'agrégation fenêtre, elle devient **cumulative** : SUM(ca) OVER (ORDER BY mois) additionne le mois courant et tous les précédents, c'est-à-dire le **cumul depuis le début**. Pour une **moyenne mobile**, on précise le **cadre** : AVG(ca) OVER (ORDER BY mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) moyenne la ligne courante et les deux précédentes, donc trois mois glissants. Enfin SUM(ca) OVER (), sans rien dans la parenthèse, est le total général, utile pour calculer des parts.

Voici ces quatre indicateurs pour le second semestre de 2025. Les fenêtres sont calculées sur **toute l'année**, le filtre sur le semestre n'intervient qu'à l'extérieur (le piège de la section 3.3.2) :

```

WITH mensuel AS (
  SELECT strftime('%Y-%m', c.date_commande) AS mois, SUM(l.montant) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2025-01-01' GROUP BY mois),
calc AS (
  SELECT mois, ca, SUM(ca) OVER (ORDER BY mois) AS cumul,
    AVG(ca) OVER (ORDER BY mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moy3,
    100.0 * ca / SUM(ca) OVER () AS part
  FROM mensuel)
SELECT mois, ROUND(ca) AS ca, ROUND(cumul) AS cumul,
  ROUND(moy3) AS moyenne_mobile_3_mois, ROUND(part, 1) AS part_annee_pct
FROM calc WHERE mois >= '2025-07'

```

mois	ca	cumul	moyenne_mobile_3_mois	part_annee_pct
2025-07	111221.0	673612.0	107808.0	8.4
2025-08	89898.0	763511.0	102683.0	6.8
2025-09	113453.0	876963.0	104857.0	8.6
2025-10	120064.0	997027.0	107805.0	9.1
2025-11	143892.0	1140919.0	125803.0	10.9
2025-12	183845.0	1324764.0	149267.0	13.9

Le cumul de décembre (1 324 764 €) est le chiffre d'affaires de l'année 2025 : on retrouve exactement le total de la section 3.2.2. La moyenne mobile lisse les accidents : en août, le chiffre d'affaires chute à 89 898 €, mais la moyenne des trois derniers mois ne baisse que de 107 808 € à 102 683 €. Enfin, décembre représente à lui seul 13,9 % de l'année. La figure présente ces résultats sur l'année entière.

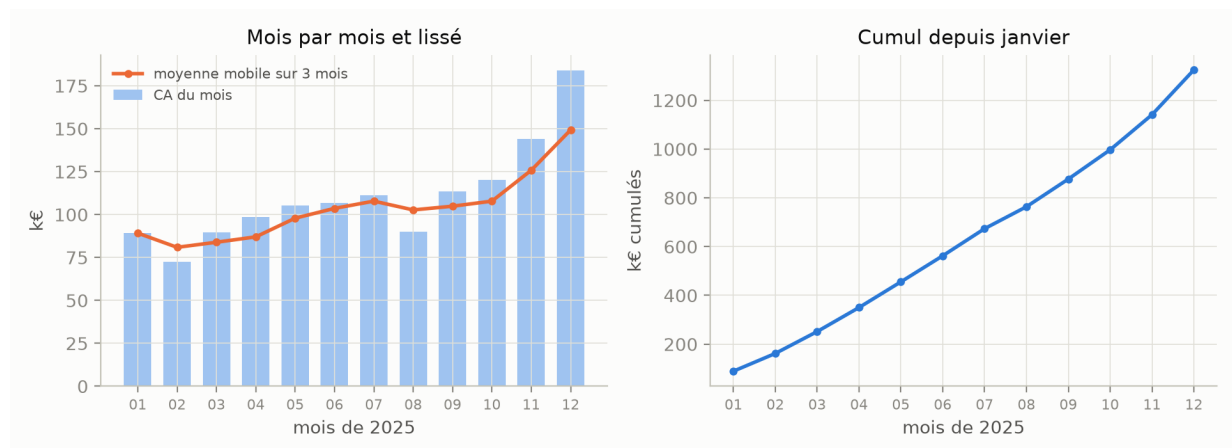


Figure 30: Chiffre d'affaires mensuel de 2025 avec sa moyenne mobile sur trois mois (à gauche) et cumul depuis janvier (à droite) : la moyenne mobile atténue le creux d'août ; le cumul finit à 1,32 million d'euros.

Piège : les premières lignes d'une moyenne mobile. Pour janvier, il n'y a pas deux mois précédents : la moyenne mobile sur trois mois s'appuie sur **une** seule valeur (puis deux en février). Les premiers points ne sont pas comparables aux suivants ; en pratique, on supprime ou l'on signale les mois incomplets.

Remarque : ROWS ou RANGE ? Le cadre ROWS compte des **lignes** ; le cadre RANGE raisonne sur des **valeurs** (toutes les lignes dont la valeur de tri est égale à celle de la ligne courante forment un seul « pair »). Quand ORDER BY est présent et qu'aucun cadre n'est précisé, le cadre par défaut est RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW : les ex æquo sont cumulés **ensemble**, ce qui surprend. Pour un cumul, précisez ROWS dès que votre ordre peut contenir

des ex æquo.

3.3.6 Découper en tranches et mesurer la concentration : NTILE

NTILE(n) répartit les lignes ordonnées en **n tranches de taille presque égale** : NTILE(10) donne les déciles, NTILE(4) les quartiles. Combiné à un GROUP BY, il mesure la **concentration** du chiffre d'affaires, un classique de l'analyse client : *quelle part du chiffre d'affaires les meilleurs clients représentent-ils ?* Nous y répondons ici par déciles de clients (du plus dépensier au moins dépensier) :

```
WITH ca_client AS (
  SELECT c.id_client, SUM(l.montant) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2025-01-01' GROUP BY c.id_client),
deciles AS (SELECT id_client, ca, NTILE(10) OVER (ORDER BY ca DESC) AS decile FROM ca_client)
SELECT decile, COUNT(*) AS nb_clients, ROUND(SUM(ca)) AS ca,
       ROUND(100.0 * SUM(ca) / SUM(SUM(ca)) OVER (), 1) AS part_ca
FROM deciles GROUP BY decile ORDER BY decile
```

decile	nb_clients	ca	part_ca
1	388	435870.0	32.9
2	388	251185.0	19.0
3	388	181434.0	13.7
4	388	134516.0	10.2
5	388	103459.0	7.8
6	387	79489.0	6.0
7	387	60268.0	4.5
8	387	42040.0	3.2
9	387	26058.0	2.0
10	387	10446.0	0.8

Les 3 875 clients actifs en 2025 sont répartis en dix tranches de 387 ou 388 clients (3 875 n'est pas divisible par dix : les premières tranches reçoivent un client de plus). Les 10 % de clients qui dépensent le plus réalisent **32,9 %** du chiffre d'affaires, les 20 % premiers **51,9 %** (32,9 + 19,0) ; la moitié la moins dépensière (les déciles 6 à 10) n'en apporte que **16,5 %**. La clientèle est donc **concentrée mais pas dramatiquement** : on est loin de la règle des « 80/20 » que l'on cite souvent. Remarquez l'expression SUM(SUM(ca)) OVER () : le SUM(ca) interne est l'agrégat du groupe, le SUM(...) OVER () externe additionne ces agrégats pour obtenir le total.

3.3.7 Les clients qui ne reviennent plus

Un dernier exemple rassemble plusieurs idées. La gérante veut la liste des clients **dont la dernière commande date d'avant 2025** (donc inactifs depuis plus d'un an à la fin de 2025). La **dernière** commande de chaque client est celle de rang 1 quand on classe ses commandes de la plus récente à la plus ancienne :

```
WITH derniere AS (
  SELECT id_client, date_commande,
         ROW_NUMBER() OVER (PARTITION BY id_client ORDER BY date_commande DESC, id_commande DESC) AS rn
  FROM commandes)
SELECT COUNT(*) AS clients_inactifs_depuis_2025
FROM derniere
WHERE rn = 1 AND date_commande < '2025-01-01'
```

```
clients_inactifs_depuis_2025
931
```

931 clients n'ont plus commandé depuis la fin de 2024. Ce chiffre n'est pas nouveau : en 3.2.7,

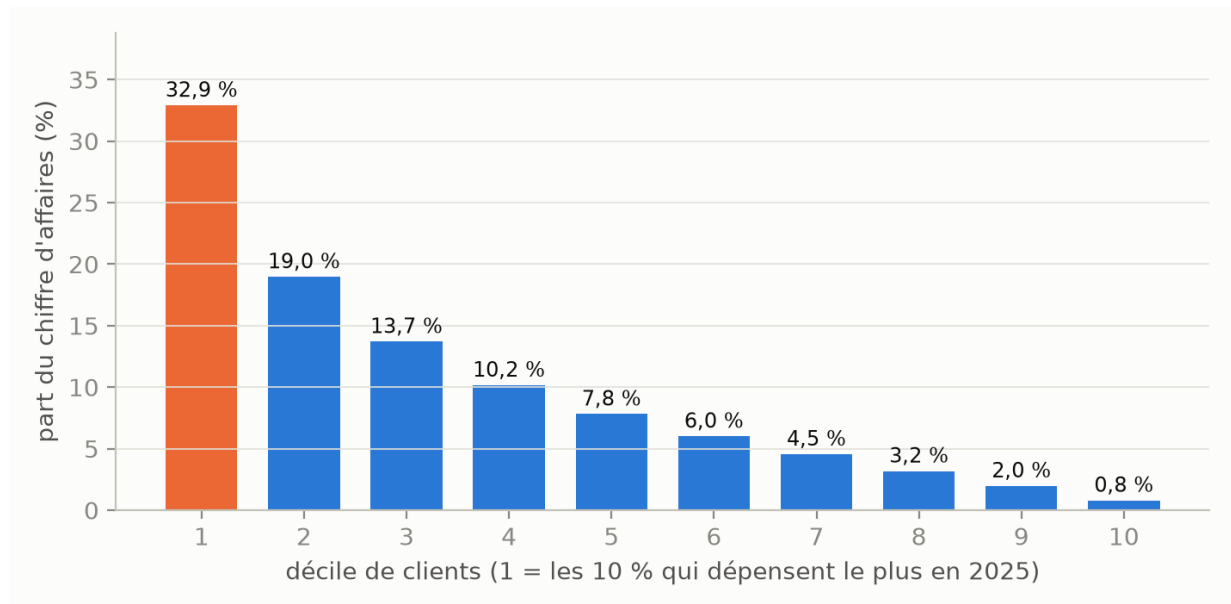


Figure 31: Part du chiffre d'affaires 2025 réalisée par chaque décile de clients : les 10 % de clients qui dépensent le plus réalisent un tiers du chiffre d'affaires, la moitié la moins dépensière moins d'un cinquième.

la requête EXCEPT des « clients perdus » (clients d'avant 2025 absents en 2025) renvoyait **931** aussi. **Deux méthodes très différentes, le même résultat** : une jointure ensembliste d'un côté, une fonction fenêtre de l'autre. C'est le meilleur des contrôles : quand deux écritures indépendantes s'accordent, la probabilité que les deux soient fausses de la même façon est faible. Prenez l'habitude de le faire pour les chiffres que vous livrez.

À retenir.

- Une fonction fenêtre calcule un résultat **sur un groupe de lignes** mais **conserve chaque ligne** ; PARTITION BY définit les groupes, ORDER BY l'ordre, le **cadre** les lignes voisines retenues.
- Elles sont calculées **après** WHERE, GROUP BY et HAVING : pour filtrer sur leur résultat, ou pour qu'un WHERE ne prive pas la fenêtre de ses voisines, on passe par deux niveaux (WITH ou sous-requête).
- ROW_NUMBER numérote sans ex æquo, RANK laisse des trous, DENSE_RANK n'en laisse pas ; LAG et LEAD comparent à la ligne précédente ou suivante ; SUM ... OVER (ORDER BY ...) cumule ; NTILE découpe en tranches.
- Contrôlez toujours un résultat de fenêtre par un total connu ou par une seconde méthode.

Pour s'entraîner. Cahier, chapitre 3 : application 3.6 (évolution mensuelle, cumul et comparaison à l'an dernier), application 3.8 (une segmentation RFM en SQL), exercices 3.9 à 3.11.

3.4 CTE et structuration de requêtes complexes

Une requête d'analyse réelle ne tient presque jamais en un seul SELECT : il faut d'abord calculer le chiffre d'affaires de chaque client, puis classer les clients, puis rapporter les dix premiers au total. On peut empiler ces étapes en sous-requêtes imbriquées, mais le résultat se lit comme une poupée russe, de l'intérieur vers l'extérieur. Les **expressions de table commune** (*Common Table Expressions*, CTE) offrent une alternative bien plus lisible : on **nomme** chaque étape, et on les enchaîne de haut en bas comme une recette. Cette section est aussi celle où nous répondons enfin à la question de la gérante.

3.4.1 Nommer ses étapes avec WITH

Une CTE se déclare avec WITH nom AS (requête), **avant** la requête principale. Elle se comporte ensuite comme une table temporaire que l'on peut interroger autant de fois qu'on veut, **le temps de la requête seulement** (rien n'est stocké). On peut en déclarer plusieurs, séparées par des virgules ; chacune peut utiliser celles qui la précèdent.

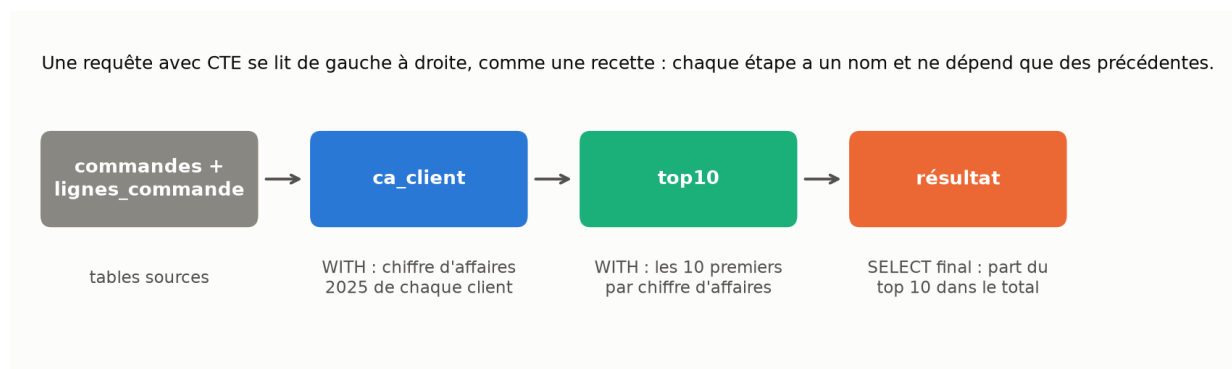


Figure 32: La requête qui répond à la gérante, vue comme une chaîne d'étapes : les tables sources, le chiffre d'affaires de chaque client en 2025, les dix premiers, puis le calcul final de leur part dans le total.

Voici la requête qui répond à la question posée en introduction : *quels sont nos dix meilleurs clients de 2025, et que représentent-ils dans le chiffre d'affaires ?* Elle suit exactement la figure : une CTE `ca_client` (une ligne par client, avec son chiffre d'affaires de 2025), une CTE `top10` (les dix premiers), puis le calcul final.

```

WITH ca_client AS (
  SELECT c.id_client, SUM(l.montant) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2025-01-01'
  GROUP BY c.id_client),
top10 AS (SELECT id_client, ca FROM ca_client ORDER BY ca DESC LIMIT 10)
SELECT (SELECT COUNT(*) FROM ca_client) AS nb_clients,
  ROUND((SELECT SUM(ca) FROM top10), 0) AS ca_top10,
  ROUND((SELECT SUM(ca) FROM ca_client), 0) AS ca_total,
  ROUND(100.0 * (SELECT SUM(ca) FROM top10) / (SELECT SUM(ca) FROM ca_client), 2) AS part_pct
  
```

nb_clients	ca_top10	ca_total	part_pct
3875	24886.0	1324764.0	1.88

La réponse : parmi 3 875 clients actifs en 2025, les dix meilleurs ont dépensé 24 886 € sur un chiffre d'affaires total de 1 324 764 €, soit 1,88 %. Les dix premiers clients de la boutique ne pèsent donc même pas 2 % de l'activité. C'est une information utile pour la gérante : la boutique **ne dépend pas de quelques gros clients** (une perte de dix clients serait indolore), mais d'une large

clientèle. Ce résultat est cohérent avec ce que nous avons vu en 3.3.6 : dix clients, c'est 0,26 % de la clientèle, loin du premier décile qui, lui, pèse 32,9 %. Regardons maintenant les dix clients eux-mêmes, en réutilisant la même CTE :

```
WITH ca_client AS (
  SELECT c.id_client, SUM(l.montant) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2025-01-01'
  GROUP BY c.id_client)
SELECT id_client, ROUND(ca, 2) AS ca_2025,
       ROUND(100.0 * ca / (SELECT SUM(ca) FROM ca_client), 2) AS part_pct
FROM ca_client
ORDER BY ca DESC
LIMIT 10
```

id_client	ca_2025	part_pct
1360	3382.33	0.26
2987	2898.79	0.22
1395	2675.71	0.20
3126	2531.71	0.19
3802	2402.11	0.18
3737	2332.84	0.18
68	2230.69	0.17
4409	2226.18	0.17
2953	2140.43	0.16
5095	2065.63	0.16

Le meilleur client (le numéro 1360) a dépensé 3 382 € en 2025, soit 0,26 % du chiffre d'affaires ; le dixième, 2 066 €. L'écart entre le premier et le dixième est modeste : il n'y a **pas de « super-client »** dans cette clientèle. On notera une propriété de la CTE : dans la première requête, `ca_client` est utilisée **quatre fois** (dans `top10`, puis dans trois sous-requêtes du `SELECT` final), alors qu'on ne l'a écrite qu'une fois. Avec des sous-requêtes imbriquées, il aurait fallu copier quatre fois le même texte.

Intuition. Une CTE n'est ni plus rapide ni plus lente qu'une sous-requête : c'est **la même requête, mieux écrite**. Son seul rôle est de rendre la lecture linéaire, étape après étape, et de **donner un nom à chaque idée** (`ca_client`, `top10`). Une requête que l'on comprend est une requête que l'on peut vérifier.

3.4.2 Vérifier par un autre outil

Un chiffre n'est vraiment acquis que lorsqu'un **second outil, indépendant, le confirme**. Recalculons la réponse avec pandas, la bibliothèque d'analyse de Python (que vous découvrirez au chapitre 4) : on charge les deux tables brutes, on les joint, on additionne par client, et l'on compare.

```
cmd = pd.read_sql_query("SELECT id_commande, id_client FROM commandes WHERE date_commande >=
↳ '2025-01-01'", con)
lig = pd.read_sql_query("SELECT id_commande, montant FROM lignes_commande", con)
ca = cmd.merge(lig,
↳ on="id_commande").groupby("id_client")["montant"].sum().sort_values(ascending=False)
print(len(ca), round(ca.head(10).sum()), round(ca.sum()), round(100 * ca.head(10).sum() / ca.sum(),
↳ 2))
```

```
3875 24886 1324764 1.88
```

pandas renvoie **3 875 clients, 24 886 €, 1 324 764 € et 1,88 %** : exactement les mêmes chiffres. Le résultat vient donc de deux implémentations indépendantes (un moteur SQL, une bibliothèque

Python), et la jointure n'a ni perdu ni dupliqué de lignes. Dans le chapitre 2, vous retrouverez le même chiffre d'affaires par un tableau croisé dynamique ; l'idée est la même : **un chiffre important se calcule au moins deux fois, par deux chemins.**

3.4.3 Enchaîner plusieurs étapes : le taux de retour

Prenons une question plus riche : *les retours coûtent-ils cher, et d'où viennent-ils ?* Le taux de retour (le nombre de lignes retournées rapporté au nombre de lignes vendues) et le montant remboursé se calculent par canal. Il faut trois étapes : isoler les lignes vendues en 2025, résumer les retours **par ligne**, puis recoller les deux avec une jointure à gauche (une ligne vendue n'a pas forcément de retour).

```
WITH lignes25 AS (
  SELECT l.id_ligne, c.canal, l.montant
  FROM lignes_commande l JOIN commandes c USING (id_commande)
  WHERE c.date_commande >= '2025-01-01'),
retournees AS (
  SELECT id_ligne, SUM(montant_rembourse) AS rembourse FROM retours GROUP BY id_ligne)
SELECT l.canal, COUNT(*) AS nb_lignes, COUNT(r.id_ligne) AS nb_retours,
  ROUND(100.0 * COUNT(r.id_ligne) / COUNT(*), 1) AS taux_retour_pct,
  ROUND(COALESCE(SUM(r.rembourse), 0), 0) AS rembourse
FROM lignes25 l LEFT JOIN retournees r ON r.id_ligne = l.id_ligne
GROUP BY l.canal
ORDER BY taux_retour_pct DESC
```

canal	nb_lignes	nb_retours	taux_retour_pct	rembourse
Site	13928	1229	8.8	56583.0
Réseaux	3288	228	6.9	9429.0
Boutique	12611	419	3.3	18456.0

Les retours se concentrent sur le **site** : 8,8 % des lignes y sont retournées, contre 6,9 % pour les réseaux et seulement **3,3 % en boutique**, et le site rembourse près de **56 600 €** contre 18 500 € en boutique. Plusieurs choix de construction méritent d'être soulignés, car ils illustrent des idées du chapitre :

- La CTE retournees **agrège avant de joindre** : on ramène la table des retours au grain « une ligne par ligne vendue » (GROUP BY id_ligne). Si une ligne avait fait l'objet de deux retours, la jointure aurait sinon dupliqué la ligne vendue, et les montants avec elle (le piège de la section 3.2.6).
- La jointure est une jointure **à gauche** : sans elle, les lignes sans retour disparaîtraient et le taux serait de 100 %.
- COUNT(r.id_ligne) compte les retours **réels** (les absences ne comptent pas), alors que COUNT(*) compte toutes les lignes vendues : le rapport des deux est le taux de retour.
- COALESCE(SUM(r.rembourse), 0) remplace par zéro la somme d'un groupe sans aucun retour.

Ce tableau déclenche une vraie discussion de gestion : le site vend plus, mais **il rend près de trois fois plus que la boutique**. Faut-il revoir les fiches produits, les tailles, les délais de livraison ? Les motifs de retour (table retours) permettront d'aller plus loin ; c'est l'objet d'une application du cahier.

3.4.4 Une méthode pour construire une requête complexe

Devant une question compliquée, la tentation est d'écrire d'un coup une requête de cinquante lignes, puis de chercher pourquoi elle ne marche pas. Une méthode simple évite la plupart des erreurs. Elle tient en six étapes, que l'on applique **dans l'ordre**.

1. **Reformuler la question** en une phrase précise, avec la période et la mesure (« chiffre d'affaires TTC, remises déduites, 2025 »).
2. **Fixer le grain du résultat** : que représentera une ligne du tableau final ? Un client ? Un mois ? Un couple canal × catégorie ? C'est la décision la plus importante : elle détermine le GROUP BY.
3. **Repérer les tables sources** et la colonne qui porte la mesure (le montant est dans lignes_commande, la date dans commandes, la catégorie dans produits).
4. **Écrire les jointures une par une**, en vérifiant à chaque fois le nombre de lignes (le test des trois comptes de la section 3.2.6).
5. **Ajouter les filtres, puis les agrégats**, en les nommant dans des CTE s'ils sont nombreux.
6. **Vérifier** : un total connu, un ordre de grandeur, un second outil. Si le résultat est trop beau ou trop laid, doutez-en d'abord.

Appliquons-la à une question dont la réponse pourrait paraître évidente : *quel est le panier moyen, canal par canal, et que vaut la moyenne de ces trois paniers moyens ?* On l'a vu en 3.1.9 sur un exemple inventé : la moyenne des moyennes diffère de la vraie moyenne quand les groupes n'ont pas la même taille. Voyons ce que donne la réalité :

```
WITH par_commande AS (
  SELECT c.id_commande, c.canal, SUM(l.montant) AS panier
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  GROUP BY c.id_commande, c.canal),
par_canal AS (
  SELECT canal, COUNT(*) AS n, AVG(panier) AS panier_moyen FROM par_commande GROUP BY canal)
SELECT ROUND((SELECT AVG(panier) FROM par_commande), 2) AS panier_global,
       ROUND(AVG(panier_moyen), 2) AS moyenne_des_moyennes
FROM par_canal
```

panier_global	moyenne_des_moyennes
100.38	100.39

Le **panier moyen global** vaut 100,38 € ; la **moyenne des trois paniers moyens de canal** vaut 100,39 €. L'écart est minuscule parce que les trois canaux ont des paniers presque identiques (autour de 100 €) ; il serait considérable si un petit groupe avait un panier très différent. La règle ne change pas pour autant : **on agrège toujours à partir du détail**, jamais à partir de résumés. Remarquez aussi le grain de par_commande : **une ligne par commande**, ce qui fait que le panier moyen est bien une moyenne **par commande** et non par ligne (la confusion de 3.1.6).

3.4.5 CTE récursives : fabriquer un calendrier

Une CTE peut s'**appeler elle-même** : on la déclare avec WITH RECURSIVE, on donne une ligne de départ, puis une règle qui produit la ligne suivante à partir de la précédente, jusqu'à une condition d'arrêt. L'usage le plus courant pour un analyste est de **fabriquer un calendrier**, c'est-à-dire une liste de tous les jours d'une période, y compris ceux où rien ne s'est passé :

```
WITH RECURSIVE jours(j) AS (
  SELECT '2025-01-01'
  UNION ALL
  SELECT date(j, '+1 day') FROM jours WHERE j < '2025-01-10'),
reseaux AS (
  SELECT date_commande AS j, COUNT(*) AS nb FROM commandes WHERE canal = 'Réseaux' GROUP BY
  ↪ date_commande)
```

```
SELECT jours.j, COALESCE(reseaux.nb, 0) AS commandes_reseaux
FROM jours LEFT JOIN reseaux ON reseaux.j = jours.j
```

j	commandes_reseaux
2025-01-01	3
2025-01-02	1
2025-01-03	3
2025-01-04	8
2025-01-05	3
2025-01-06	3
2025-01-07	1
2025-01-08	3
2025-01-09	4
2025-01-10	5

La première ligne de jours est le 1er janvier ; à chaque tour, la partie récursive ajoute le jour suivant, tant que la date reste inférieure au 10 janvier. En joignant ce calendrier à **gauche** avec les commandes des réseaux sociaux, on obtient une ligne par jour, y compris les jours sans commande (0). Étendu à l'année entière, le même calendrier révèle :

```
WITH RECURSIVE jours(j) AS (
  SELECT '2025-01-01' UNION ALL SELECT date(j, '+1 day') FROM jours WHERE j < '2025-12-31'),
  reseaux AS (SELECT date_commande AS j, COUNT(*) AS nb FROM commandes WHERE canal = 'Réseaux' GROUP BY
    ↪ date_commande)
SELECT COUNT(*) AS jours_total, SUM(reseaux.nb IS NULL) AS jours_sans_commande
FROM jours LEFT JOIN reseaux ON reseaux.j = jours.j
```

jours_total	jours_sans_commande
365	13

13 jours sur 365 n'ont enregistré aucune commande par les réseaux sociaux en 2025. Sans calendrier, ces jours n'existeraient tout simplement pas dans le résultat d'un GROUP BY : une moyenne par jour calculée sur les seuls jours « présents » serait **trop haute**. C'est l'une des erreurs les plus fréquentes dans les analyses de séries temporelles : les zéros qui manquent.

Les CTE récursives servent aussi à parcourir des **hiérarchies** (un organigramme où chaque ligne désigne son chef, des catégories à plusieurs niveaux). La ligne de départ est la **racine** (la personne sans chef) ; à chaque tour, la partie récursive ajoute les personnes dont le chef vient d'être trouvé, en incrémentant un compteur de niveau. Le cahier propose de l'écrire sur une équipe de six personnes (exercice 3.12). Une récursion s'arrête quand la partie récursive ne produit plus de ligne nouvelle ; **si l'on oublie la condition d'arrêt, la requête tourne indéfiniment**, ce qui fait de la CTE récursive l'un des rares endroits où le SQL peut « geler » votre session.

3.4.6 Écrire des requêtes que l'on relit

Une requête est lue bien plus souvent qu'elle n'est écrite : par vous dans six mois, par un collègue, par la personne qui reprendra l'analyse. Quelques habitudes les rendent relisibles :

- **Une clause par ligne**, mots-clés en majuscules, indentation qui montre la structure ; des alias **parlants** (ca_client, pas t1).
- **Des commentaires** qui disent **pourquoi** (-- on exclut les retours internes), pas ce que fait le code. En SQL, un commentaire de ligne commence par --.
- **Tester chaque CTE séparément** : sélectionnez-la seule, vérifiez son nombre de lignes et un total, puis passez à la suivante.
- **Nommer les colonnes** du résultat (AS ca_2025) et arrondir **à la fin**, pas dans les étapes intermédiaires.

- **Garder les requêtes sous contrôle de version** (Git) à côté des résultats : c'est ce qui rend l'analyse reproductible.
- **Éviter SELECT *** et les nombres « magiques » (un taux de TVA, une date) répétés dans la requête : mettez-les dans une CTE paramètres ou dans un commentaire.

Piège : une CTE n'est pas un cache. Selon la base, une CTE référencée plusieurs fois peut être recalculée à chaque référence (c'est le cas de `ca_client` dans la première requête de cette section). Si une étape est coûteuse et utilisée plusieurs fois, on peut la matérialiser dans une table temporaire. Pour l'analyste, la règle pratique est de **mesurer** avant de s'inquiéter : sur les volumes de la boutique, rien de tout cela ne se voit ; sur des centaines de millions de lignes, cela se voit tout de suite (la section 3.5 donne les outils).

À retenir.

- Une CTE (`WITH nom AS (...)`) donne un nom à une étape ; on enchaîne les étapes de haut en bas, chacune pouvant utiliser les précédentes.
- Pour une requête complexe : **question** → **grain du résultat** → **tables sources** → **jointures une à une** → **filtres et agrégats** → **vérification**.
- **Un chiffre important se vérifie par un second outil** : ici, SQL et pandas donnent 24 886 € pour les dix meilleurs clients, soit 1,88 % de 1 324 764 €.
- Une CTE **réursive** fabrique un calendrier (pour ne pas oublier les jours vides) ou parcourt une hiérarchie.
- Agrégez toujours **depuis le détail** ; commentez, nommez, testez étape par étape.

Pour s'entraîner. Cahier, chapitre 3 : application 3.5 (les retours), application 3.7 (fidélité et cohortes), exercice 3.12 (un organigramme par une CTE réursive).

3.5 ⊕ Pour aller plus loin : SQL avancé

Section complémentaire. Elle prolonge le parcours essentiel par les notions que l'on rencontre dès que l'on travaille sur une vraie base d'entreprise : les sous-requêtes, les index et l'optimisation, les vues, les transactions, les déclencheurs et la sécurité. Rien ici n'est nécessaire pour la suite du volume.

3.5.1 Sous-requêtes : une requête dans une requête

Une **sous-requête** est un `SELECT` placé entre parenthèses à l'intérieur d'un autre. On en a déjà croisé (le dénominateur d'un pourcentage en 3.1.7). Il en existe trois usages principaux, selon ce que renvoie la sous-requête.

Une valeur unique (sous-requête *scalaire*), utilisée comme une constante dans une comparaison ou un calcul. Quels produits sont plus chers que le prix moyen du catalogue ?

```
SELECT COUNT(*) AS nb_produits_au-dessus_de_la_moyenne,
       (SELECT ROUND(AVG(prix_vente), 2) FROM produits) AS prix_moyen
FROM produits
WHERE prix_vente > (SELECT AVG(prix_vente) FROM produits)
```

nb_produits_au-dessus_de_la_moyenne	prix_moyen
47	35.88

La sous-requête (`SELECT AVG(prix_vente) FROM produits`) est évaluée une fois et remplacée par son résultat : **47 produits sur 120** dépassent le prix moyen de 35,88 €. On ne pourrait pas écrire `WHERE prix_vente > AVG(prix_vente)` : un agrégat n'est pas autorisé dans le `WHERE` (l'ordre d'exécution de la section 3.1.7).

Une liste de valeurs (sous-requête de liste), avec `IN`. Combien de produits ont donné lieu à au moins un retour de plus de 100 € ?

```
SELECT COUNT(*) AS produits_concernes
FROM produits
WHERE id_produit IN (SELECT l.id_produit
                     FROM lignes_commande l JOIN retours r USING (id_ligne)
                     WHERE r.montant_rembourse > 100)
```

produits_concernes
55

La sous-requête renvoie la liste des identifiants de produits concernés, et la requête extérieure ne garde que ces produits : **55 produits** (près de la moitié du catalogue) ont déjà fait l'objet d'un remboursement de plus de 100 €, ce qui invite à regarder de près les articles chers.

Une sous-requête corrélée est une sous-requête qui **dépend de la ligne courante** de la requête extérieure : elle est « réévaluée » pour chaque ligne. Elle sert aux comparaisons avec le groupe de la ligne. Quel est, dans chaque catégorie, le produit le plus cher ?

```
SELECT p.categorie, p.nom_produit, p.prix_vente
FROM produits p
WHERE p.prix_vente = (SELECT MAX(q.prix_vente) FROM produits q WHERE q.categorie = p.categorie)
ORDER BY p.categorie
```

categorie	nom_produit	prix_vente
Bien-être	Tisane nordique	68.9
Cuisine	Casserole nordique	57.9
Décoration	Horloge nordique	96.9
Jardin	Transat nordique	152.9
Maison	Lampe design	126.9
Papeterie	Trousse mat	21.9

Pour chaque produit `p`, la sous-requête cherche le prix maximal des produits `q` **de la même catégorie** (`q.categorie = p.categorie`), et la condition ne garde que les produits qui atteignent ce maximum. Le résultat compte six lignes, une par catégorie, du transat à 152,90 € pour le jardin à la trousse à 21,90 € pour la papeterie. Cette écriture est correcte, mais un **classement avec fonction fenêtre** (section 3.3.3) fait la même chose plus lisiblement et, sur de gros volumes, bien plus vite : les sous-requêtes corrélées sont des **candidates à la réécriture**.

Enfin, `EXISTS` (sous-requête) ne demande pas **quoi**, seulement **s'il existe** au moins une ligne. C'est la forme préférée pour les tests de présence : elle s'arrête à la première correspondance trouvée. Combien de clients ont déjà fait au moins un retour ?

```
SELECT COUNT(*) AS clients_avec_retour
FROM clients cl
WHERE EXISTS (SELECT 1
              FROM commandes c
              JOIN lignes_commande l USING (id_commande)
              JOIN retours r USING (id_ligne)
              WHERE c.id_client = cl.id_client)
```



```
clients_avec_retour
2443
```

2 443 clients (sur 6 000) ont déjà renvoyé un article : un client sur quatre environ, une proportion qui pèsera sur la politique de reprise. Le `SELECT 1` de la sous-requête n'a pas d'importance : seule compte l'existence d'une ligne. `NOT EXISTS` en est le contraire, et c'est une anti-jointure (section 3.2.3).

Piège : `NOT IN` et les valeurs absentes. `x NOT IN (liste)` renvoie un résultat **inconnu** dès que la liste contient une valeur absente (`NULL`), et donc **aucune ligne**. La même requête écrite avec `NOT EXISTS` ne souffre pas de ce défaut. Dans le doute, préférez `NOT EXISTS`.

3.5.2 Index et plans d'exécution

Quand on demande à la base les commandes d'un client, elle peut lire la table **ligne par ligne** jusqu'à la fin (un **balayage complet**, *full scan*), ou consulter un **index**, comme le fait un lecteur qui cherche un mot dans l'index d'un livre au lieu de relire tout l'ouvrage. Un index est une structure triée, maintenue par la base, qui permet de **retrouver des lignes sans tout lire**. En contrepartie, il occupe de la place et ralentit un peu les écritures, puisqu'il faut le mettre à jour.

La base propose d'**expliquer** le plan qu'elle a choisi : `EXPLAIN QUERY PLAN` en SQLite, `EXPLAIN` ailleurs. Le mot `SCAN` signale une lecture complète, `SEARCH` une recherche par index. Écrivons une petite fonction qui renvoie le plan d'une requête, et interrogeons trois requêtes :

```
def plan(requete):
    return [ligne[3] for ligne in con.execute("EXPLAIN QUERY PLAN " + requete)]

print(plan("SELECT COUNT(*) FROM commandes WHERE canal = 'Site'"))
print(plan("SELECT COUNT(*) FROM commandes WHERE id_client = 2"))
print(plan("SELECT COUNT(*) FROM commandes WHERE strftime('%Y', date_commande) = '2025'"))
print(plan("SELECT COUNT(*) FROM commandes WHERE date_commande >= '2025-01-01' AND date_commande <
↳ '2026-01-01'"))
```

```
['SCAN commandes']
['SEARCH commandes USING COVERING INDEX idx_cmd_client (id_client=?)']
['SCAN commandes USING COVERING INDEX idx_cmd_date']
['SEARCH commandes USING COVERING INDEX idx_cmd_date (date_commande>? AND date_commande<?)']
```

Les quatre plans sont instructifs. Aucun index n'existe sur `canal` : la première requête fait un **balayage complet** de commandes. La deuxième utilise l'index de `id_client` (`SEARCH ... USING INDEX`). La troisième et la quatrième portent sur la date, qui est indexée, **mais pas de la même façon** : quand on applique une **fonction** à la colonne (`strftime('%Y', date_commande)`), la base ne peut plus se servir de l'ordre de l'index et doit le **parcourir en entier** (`SCAN`) ; quand on exprime la même condition par un **intervalle** (`>= ... AND < ...`), elle fait une **recherche** directe (`SEARCH`). Les deux écritures donnent le même résultat, mais **pas le même effort**. Créons maintenant un index sur `canal` pour voir le plan changer :

```
con.execute("CREATE INDEX idx_cmd_canal ON commandes(canal)")
print(plan("SELECT COUNT(*) FROM commandes WHERE canal = 'Site'"))
```

```
['SEARCH commandes USING COVERING INDEX idx_cmd_canal (canal=?)']
```

Le plan passe de `SCAN` à `SEARCH ... USING COVERING INDEX idx_cmd_canal (canal=?)`. Un index n'est pourtant **pas toujours utile** : ici, la colonne ne contient que trois valeurs, et un index ne rend de grands services que si la condition **isole peu de lignes** (on dit qu'elle est *sélective*). Chercher les commandes d'un seul client sur 6 000 (sélectif) en profite largement ; chercher « toutes celles

du site » (42 % de la table) beaucoup moins.

3.5.3 Quelques règles d'optimisation

Sur les volumes de la boutique, toutes les requêtes de ce chapitre répondent instantanément. Sur des millions de lignes, quelques règles font la différence :

- **Filtrer tôt** : placer les conditions WHERE le plus près possible des tables, pour réduire le nombre de lignes avant les jointures et les agrégats.
- **Ne sélectionner que les colonnes utiles** : éviter SELECT *, qui transporte des colonnes inutiles et empêche certains accès rapides par index.
- **Ne pas appliquer de fonction à une colonne indexée** dans un WHERE (le plan de 3.5.2) : exprimer la condition sur la colonne brute.
- **Agréger avant de joindre** quand on le peut : c'est plus rapide et cela évite la multiplication des lignes (section 3.2.6).
- **Éviter LIKE '%mot'** (joker en tête) : aucun index ne sait chercher « se termine par ». LIKE 'mot%' est, lui, efficace.
- **Lire le plan** avant d'accuser la base : un SCAN sur une grosse table dans une requête lente indique souvent l'index manquant.

En pratique. Un analyste ne crée pas, en général, les index d'une base de production : c'est le rôle de l'équipe qui la gère. Savoir **lire un plan** vous permet en revanche de lui faire une demande précise (« la requête X lit toute la table commandes ; un index sur canal, date_commande l'accélérerait »), ce que les administrateurs de bases apprécient.

3.5.4 Vues : donner un nom à une requête

Une **vue** est une requête enregistrée sous un nom : elle se comporte comme une table, mais ne contient aucune donnée propre, elle est recalculée à chaque utilisation. Les vues servent à **partager** une logique (« le chiffre d'affaires, c'est ceci ») pour que tous les rapports calculent la même chose, et à **simplifier** les requêtes. Créons une vue qui recolle les lignes, leurs commandes et leurs produits, c'est-à-dire la jointure que nous avons écrite en 3.2.2 :

```
CREATE VIEW v_ventes AS
SELECT l.id_ligne, c.id_commande, c.date_commande, c.canal, c.id_client,
       p.categorie, p.nom_produit, l.quantite, l.montant
FROM lignes_commande l
JOIN commandes c ON c.id_commande = l.id_commande
JOIN produits p ON p.id_produit = l.id_produit
```

Elle s'interroge ensuite comme n'importe quelle table, sans réécrire les jointures :

```
SELECT canal, ROUND(SUM(montant), 0) AS ca_2025, COUNT(DISTINCT id_client) AS clients
FROM v_ventes
WHERE date_commande >= '2025-01-01'
GROUP BY canal
ORDER BY ca_2025 DESC
```

canal	ca_2025	clients
Site	617715.0	2844
Boutique	560974.0	2731
Réseaux	146074.0	1139

Le **site** réalise 617 715 € de chiffre d'affaires en 2025, devant la **boutique** (560 974 €) et les **réseaux** (146 074 €) ; les trois montants totalisent 1 324 763 €, soit les 1 324 764 € de la section 3.3 à l'arrondi près. Les nombres de clients (2 844, 2 731 et 1 139) **ne s'additionnent pas** : un même client peut

acheter par plusieurs canaux, et `COUNT(DISTINCT id_client)` ne compte chacun qu'une fois **par canal**. L'intérêt de la vue est **organisationnel** : si l'on décide demain d'exclure les commandes de test, on corrige **la vue**, et tous les rapports sont corrigés d'un coup. Dans certaines bases (PostgreSQL, Oracle), une **vue matérialisée** stocke le résultat et le rafraîchit à la demande : on gagne en vitesse, on perd en fraîcheur.

Piège : une vue fige un grain. `v_ventes` a le grain « une ligne de commande » : calculer un nombre de commandes dessus exige `COUNT(DISTINCT id_commande)`, comme dans l'exemple (`COUNT(DISTINCT id_client)` pour les clients). Documentez le grain de chaque vue en commentaire.

3.5.5 Transactions : tout ou rien

Une **transaction** regroupe plusieurs opérations en un bloc indivisible : soit **toutes** réussissent (`COMMIT`), soit **aucune** n'est appliquée (`ROLLBACK`). C'est la garantie qui évite, par exemple, de débiter un stock sans enregistrer la commande correspondante si la machine s'arrête entre les deux. On résume les propriétés d'une transaction par le sigle **ACID** : atomicité (tout ou rien), cohérence (les règles de la base sont respectées), isolation (deux transactions simultanées ne se voient pas à moitié faites), durabilité (ce qui est validé survit à une panne). Une démonstration sur une petite table de stock :

```
con.execute("CREATE TABLE demo_stock (id_produit INTEGER PRIMARY KEY, quantite INTEGER)")
con.execute("INSERT INTO demo_stock VALUES (1, 10), (2, 5)")
con.commit()
con.execute("UPDATE demo_stock SET quantite = quantite - 3 WHERE id_produit = 1")
print("pendant la transaction :", con.execute("SELECT quantite FROM demo_stock WHERE id_produit =
↳ 1").fetchone()[0])
con.rollback()
print("après annulation      :", con.execute("SELECT quantite FROM demo_stock WHERE id_produit =
↳ 1").fetchone()[0])
```

```
pendant la transaction : 7
après annulation      : 10
```

Pendant la transaction, la quantité est de 7 ; après le `ROLLBACK`, elle est revenue à 10 : l'opération n'a jamais eu lieu. Un analyste, qui **lit** surtout, rencontre rarement les transactions, mais il doit savoir qu'elles existent : un chiffre lu **au milieu** d'un chargement de données peut être incohérent si la base n'isole pas bien les lectures, d'où l'intérêt de lire les données **après** la fin des traitements de nuit.

3.5.6 Déclencheurs et procédures stockées

Un **déclencheur** (*trigger*) est un morceau de SQL que la base exécute **automatiquement** quand un événement survient (insertion, modification, suppression). Il sert à tenir un **journal d'audit** : qui a changé quel prix, quand. Exemple minimal : à chaque modification d'un prix, une ligne est ajoutée à un journal.

```
con.executescript("""
CREATE TABLE demo_prix (id_produit INTEGER PRIMARY KEY, prix REAL);
CREATE TABLE demo_journal (id_produit INTEGER, ancien REAL, nouveau REAL);
CREATE TRIGGER trg_prix AFTER UPDATE OF prix ON demo_prix
BEGIN INSERT INTO demo_journal VALUES (OLD.id_produit, OLD.prix, NEW.prix); END;
INSERT INTO demo_prix VALUES (1, 20.0);
UPDATE demo_prix SET prix = 22.0 WHERE id_produit = 1;""")
print(con.execute("SELECT * FROM demo_journal").fetchall())
```

```
[(1, 20.0, 22.0)]
```

Le journal contient une ligne : le produit 1, l'ancien prix 20, le nouveau 22. OLD et NEW désignent les valeurs avant et après la modification. Les déclencheurs ont un coût : ils rendent le comportement de la base **moins visible** (une modification déclenche des effets que l'on ne voit pas dans la requête), et il vaut mieux les réserver à l'audit et aux règles d'intégrité.

Une **procédure stockée** est un programme enregistré **dans** la base, appelé par son nom avec des paramètres. SQLite n'en possède pas ; PostgreSQL, SQL Server, MySQL et Oracle, si, chacun avec son propre langage. Voici, à titre d'illustration, la forme d'une fonction PostgreSQL qui renvoie le chiffre d'affaires d'une année :

```
-- PostgreSQL – non exécuté (SQLite n'a pas de procédures stockées)
CREATE FUNCTION ca_annee(annee integer) RETURNS numeric AS $$
    SELECT SUM(l.montant)
    FROM lignes_commande l JOIN commandes c USING (id_commande)
    WHERE EXTRACT(YEAR FROM c.date_commande) = annee;
$$ LANGUAGE sql;
-- appel : SELECT ca_annee(2025);
```

Pour un analyste, l'intérêt est de **réutiliser** une logique validée sans la recopier ; l'inconvénient est qu'elle vit dans la base et non dans votre dépôt de code : veillez à ce qu'elle soit **documentée** et **versionnée**.

3.5.7 Sécurité : l'injection SQL

Quand une application ou un script construit une requête **en collant du texte saisi par un utilisateur**, un malin peut y glisser du SQL. C'est l'**injection SQL**, l'une des failles les plus répandues et les plus coûteuses. Supposons qu'un formulaire demande « quel canal ? » et que le script colle la réponse dans la requête :

```
saisie = "Site' OR '1'='1"
dangereux = f"SELECT COUNT(*) FROM commandes WHERE canal = '{saisie}'"
prudent = "SELECT COUNT(*) FROM commandes WHERE canal = ?"
print("collé dans le texte :", con.execute(dangereux).fetchone()[0])
print("paramètre          :", con.execute(prudent, (saisie,)).fetchone()[0])
```

```
collé dans le texte : 36395
paramètre           : 0
```

La saisie piégée transforme la condition en `canal = 'Site' OR '1'='1'`, **toujours vraie** : la requête renvoie **toutes** les commandes (36 395), alors que le canal demandé n'existe pas. Avec un **paramètre** (?), la base traite la saisie comme une **valeur** et non comme du SQL, et renvoie 0 ligne, ce qui est la bonne réponse. La règle est absolue : **ne jamais assembler une requête par concaténation avec une donnée extérieure** ; utiliser des requêtes paramétrées. Dans le même esprit, on donne à un analyste un compte en **lecture seule** et limité aux tables dont il a besoin (principe du moindre privilège) : une erreur de frappe ne doit pas pouvoir effacer une table.

À retenir.

- Une **sous-requête** renvoie une valeur, une liste, ou sert à tester l'existence (EXISTS) ; les sous-requêtes corrélées se réécrivent souvent avec une fonction fenêtre. Préférez NOT EXISTS à NOT IN.
- Un **index** accélère les recherches **sélectives** ; une fonction appliquée à une colonne indexée l'empêche de servir. EXPLAIN montre le plan : SCAN (tout lire) ou SEARCH (par index).

- Une **vue** nomme une requête pour que tous calculent la même chose ; une **transaction** est un bloc tout-ou-rien ; un **déclencheur** réagit automatiquement à un événement.
- **Ne jamais coller** une saisie dans une requête : utilisez des **paramètres**.

Pour s'entraîner. Cahier, chapitre 3 : application 3.9 (index et plan d'exécution), exercice 3.14.

3.6 ⊕ Pour aller plus loin : PostgreSQL, MySQL, SQL Server, Oracle

Section complémentaire. Le SQL que vous avez appris est un langage **normalisé**, mais chaque éditeur y a ajouté ses habitudes. Un analyste change souvent d'entreprise, de projet ou d'entrepôt de données : cette section vous donne la carte des principales différences, pour que vous sachiez quoi vérifier avant de recopier une requête d'un système à l'autre.

Honnêteté. Seul SQLite (et DuckDB, un moteur voisin de PostgreSQL) est installé sur la machine qui a produit ce livre. **Les exemples PostgreSQL, MySQL, SQL Server et Oracle sont donc écrits de mémoire et non exécutés**, et signalés comme tels. Les détails varient avec la version : **vérifiez toujours dans la documentation de votre produit**.

3.6.1 Un standard, plusieurs dialectes

Le langage SQL est normalisé par l'ISO depuis 1986, et la norme s'est enrichie par étapes (jointures explicites, requêtes récursives, fonctions fenêtres, types temporels...). En pratique, chaque système suit la norme **à sa façon** : il ne l'implémente jamais en entier, et il ajoute ses propres fonctions. Ce que vous avez appris dans ce chapitre (SELECT ... FROM ... WHERE ... GROUP BY, les jointures, CASE, COALESCE, les fonctions fenêtres, les CTE) fonctionne, **à quelques détails près**, dans les quatre systèmes ; c'est tout ce qui touche aux **dates**, au **texte**, à la **limitation des lignes** et aux **types** qui diffère.

Système	Où on le rencontre	Particularités à connaître
PostgreSQL	base libre, très répandue pour l'analyse et les entrepôts	proche de la norme ; types riches (dates, tableaux, JSON) ; ILIKE ; DISTINCT ON ; GENERATE_SERIES
MySQL (et MariaDB)	sites web, applications	fonctions fenêtres et CTE seulement depuis la version 8 ; pas de FULL JOIN ; comportements historiquement permissifs
SQL Server	entreprises utilisant l'écosystème Microsoft	dialecte T-SQL : TOP n, crochets pour les noms, + pour concaténer, GETDATE(), DATEADD
Oracle	grandes entreprises, banques, assurances	FETCH FIRST, table fictive DUAL, NVL, SYSDATE ; une chaîne vide y est traitée comme NULL
SQLite	applications embarquées, fichiers de travail (ce chapitre)	typage souple ; pas de procédures stockées ; une seule écriture à la fois

3.6.2 La table de correspondance

Voici les différences que l'analyste rencontre le plus souvent, une ligne par besoin. Elles sont données **de mémoire, à vérifier** pour votre version.

Besoin	SQLite	PostgreSQL	MySQL	SQL Server	Oracle
Limiter à n lignes	LIMIT n	LIMIT n	LIMIT n	SELECT TOP n ...	FETCH FIRST n ROWS ONLY
Concaténer du texte	a b	a b	CONCAT(a, b)	a + b ou CONCAT	a b
Date du jour	date('now')	CURRENT_DATE	CURDATE()	CAST(GETDATE() AS date)	TRUNC(SYSDATE)
Année d'une date	strftime('%Y', d)	EXTRACT(YEAR FROM d)	YEAR(d)	YEAR(d)	EXTRACT(YEAR FROM d)
Premier jour du mois	strftime('%Y- %m-01', d)	DATE_TRUNC('month', d)	DATE_FORMAT(d, '%Y-%m-01')	DATEFROMPARTS(YEAR(d), MONTH(d), 1)	TRUNC(d, 'MM')
Ajouter 7 jours	date(d, '+7 day')	d + INTERVAL '7 days'	DATE_ADD(d, INTERVAL 7 DAY)	DATEADD(day, 7, d)	d + 7
Texte sans casse	LIKE (insensible pour l'ASCII)	ILIKE	LIKE (selon le jeu de caractères)	LIKE (selon la collation)	LIKE (sensible)
Nom entre guillemets	"nom"	"nom"	`nom`	[nom] ou "nom"	"nom"
7 / 2	3	3	3,5	3	3,5
FULL JOIN	oui (≥ 3.39)	oui	non	oui	oui
Série de nombres ou de dates	CTE récursive	GENERATE_SERIES	CTE récursive	CTE récursive (ou GENE- RATE_SERIES récent)	CONNECT BY LEVEL
Colonne non agrégée dans GROUP BY	tolérée	refusée	selon la configuration	refusée	refusée

Deux lignes méritent un commentaire. La **division entière** d'abord : le même 7 / 2 vaut 3 ou 3,5 selon le système, ce qui est exactement le piège de la section 3.1.9, avec une raison de plus de **forcer le type décimal** (7 / 2.0) pour être portable. Ensuite la **chaîne vide** d'Oracle, qui la traite comme NULL : code_promo = '' n'y a pas de sens, et la différence entre « vide » et « absent » que nous avons vue en 3.1.8 y disparaît.

3.6.3 Les mêmes requêtes, ailleurs

Prenons deux requêtes de ce chapitre et écrivons-les dans d'autres dialectes. Les **cinq articles les plus chers** (section 3.1.4) :

```
-- SQL Server – non exécuté
SELECT TOP 5 nom_produit, categorie, prix_vente FROM produits ORDER BY prix_vente DESC;

-- Oracle – non exécuté
SELECT nom_produit, categorie, prix_vente FROM produits ORDER BY prix_vente DESC FETCH FIRST 5 ROWS
↵ ONLY;

-- MySQL et PostgreSQL : identique à SQLite (LIMIT 5)
```

Le **nombre de commandes par mois** (section 3.1.5), où se voit la différence de traitement des dates :

```
-- PostgreSQL – non exécuté
SELECT DATE_TRUNC('month', date_commande) AS mois, COUNT(*) AS nb_commandes
FROM commandes WHERE date_commande >= DATE '2025-07-01' GROUP BY 1 ORDER BY 1;
```

Dans SQL Server, il faudrait remplacer DATE_TRUNC par DATEFROMPARTS(YEAR(date_commande), MONTH(date_commande), 1) et **répéter cette expression** dans le GROUP BY : ce système n'accepte pas l'alias à cet endroit, conséquence de l'ordre d'exécution étudié en 3.1.7 (l'alias n'existe pas encore à l'étape du regroupement) ; PostgreSQL et MySQL, eux, l'acceptent par tolérance. Enfin, une même logique de **jointure à gauche avec recherche d'absence** s'écrit identiquement dans les quatre systèmes : c'est le cœur portable du langage.

3.6.4 DuckDB : un dialecte voisin de PostgreSQL, que l'on peut exécuter

Puisque nous ne disposons pas d'un serveur PostgreSQL, nous utilisons **DuckDB** pour montrer quelques constructions de ce dialecte. DuckDB est un moteur analytique qui s'exécute dans votre session, comme SQLite, mais dont la syntaxe est très proche de celle de PostgreSQL ; il sait lire **directement des fichiers CSV**. Branchons-le sur nos fichiers :

```
import duckdb
DONN = os.environ["DONNEES"]
dk = duckdb.connect()
dk.sql(f"CREATE VIEW commandes AS SELECT * FROM read_csv('{DONN}/commandes.csv', header=true)")
dk.sql(f"CREATE VIEW produits AS SELECT * FROM read_csv('{DONN}/produits.csv')")
print(dk.sql("SELECT canal, COUNT(*) AS n FROM commandes GROUP BY canal ORDER BY n
↳ DESC").df().to_string(index=False))
```

```
canal      n
Boutique 16975
Site      15463
Réseaux   3957
```

Le comptage par canal est identique à celui obtenu avec SQLite (16 975, 15 463 et 3 957 commandes) : le **même résultat**, par un autre moteur, depuis les fichiers d'origine et non plus depuis la base. C'est un contrôle de plus. Voyons trois fonctions du dialecte PostgreSQL : DATE_TRUNC (début de période), ILIKE (recherche sans casse) et GENERATE_SERIES (série de dates).

```
print(dk.sql("SELECT date_trunc('month', date_commande) AS mois, COUNT(*) AS n FROM commandes WHERE
↳ date_commande >= DATE '2025-11-01' GROUP BY 1 ORDER BY 1").df().to_string(index=False))
print(dk.sql("SELECT nom_produit, prix_vente FROM produits WHERE nom_produit ILIKE 'bougie%' ORDER BY
↳ prix_vente LIMIT 3").df().to_string(index=False))
print(dk.sql("SELECT * FROM generate_series(DATE '2025-01-01', DATE '2025-01-03', INTERVAL 1 DAY) AS
↳ t(jour)").df().to_string(index=False))
```

```
mois      n
2025-11-01 1509
2025-12-01 1853

nom_produit  prix_vente
Bougie nordique      16.9
Bougie parfumée compact 27.9
Bougie nordique      28.9

jour
2025-01-01
2025-01-02
2025-01-03
```

DATE_TRUNC('month', ...) ramène chaque date au premier jour de son mois (on retrouve 1 509 commandes en novembre et 1 853 en décembre) ; ILIKE retrouve les bougies **sans se soucier**

de la casse (alors que LIKE n'y suffirait pas dans PostgreSQL) ; GENERATE_SERIES produit en une ligne le calendrier que nous avons fabriqué avec une CTE récursive en 3.4.5. DuckDB offre même des raccourcis que PostgreSQL n'a pas, comme QUALIFY, qui filtre sur le résultat d'une fonction fenêtre sans passer par une requête extérieure :

```
print(dk.sql("""SELECT categorie, nom_produit, prix_vente FROM produits
               QUALIFY ROW_NUMBER() OVER (PARTITION BY categorie ORDER BY prix_vente DESC) = 1
               ORDER BY categorie""").df().to_string(index=False))
print(dk.sql("SELECT 7 / 2 AS division, 7 // 2 AS division_entiere").df().to_string(index=False))
```

categorie	nom_produit	prix_vente
Bien-être	Tisane nordique	68.9
Cuisine	Casserole nordique	57.9
Décoration	Horloge nordique	96.9
Jardin	Transat nordique	152.9
Maison	Lampe design	126.9
Papeterie	Trousse mat	21.9
division	division_entiere	
3.5	3	

Le premier résultat reprend le produit le plus cher de chaque catégorie, comme en 3.5.1 mais en **une seule requête** sans sous-requête. Le second illustre la différence de division : DuckDB renvoie 3,5 pour 7 / 2 (il réserve // à la division entière), tandis que SQLite renvoie 3. Un même texte SQL, deux résultats : c'est exactement le genre de piège dont ce chapitre vous protège.

3.6.5 Écrire du SQL portable

Si votre requête doit tourner sur plusieurs systèmes, ou si vous changerez de base d'ici un an, quelques règles limitent les dégâts :

- **Rester dans le cœur du langage** : CASE, COALESCE, CAST, jointures explicites, GROUP BY, CTE, fonctions fenêtres. Ces constructions sont partout.
- **Isoler les fonctions de dialecte** (dates, texte) dans **une seule étape** (une CTE ou une vue) : le jour où l'on change de système, on ne corrige qu'à un endroit.
- **Forcer les types** : CAST(x AS DECIMAL(12, 2)), 100.0 * a / b, pour que les divisions donnent partout le même résultat.
- **Écrire les dates au format ISO** AAAA-MM-JJ, que tous les systèmes comprennent, avec le mot-clé DATE quand le type existe.
- **Tester sur le système cible** avec un jeu de données connu, puis comparer les totaux avec ceux de l'ancien système avant de faire confiance au résultat.

À retenir.

- Le SQL est un standard, mais chaque produit a ses dialectes ; les différences touchent surtout les **dates**, le **texte**, la **limitation du nombre de lignes** (LIMIT, TOP, FETCH FIRST) et les **types** (la division !).
- Le cœur (jointures, agrégats, CASE, CTE, fonctions fenêtres) est portable ; isolez le reste.
- **Ce chapitre n'a exécuté que SQLite et DuckDB** : les exemples PostgreSQL, MySQL, SQL Server et Oracle sont à **vérifier dans la documentation** de votre version.

Pour s'entraîner. Cahier, chapitre 3 : exercice 3.13 (traduire des requêtes d'un dialecte à l'autre).

Bilan du chapitre 3

Vous savez maintenant :

- **lire une base relationnelle** : tables, grain (ce que représente une ligne), clés primaires et étrangères, types, et pourquoi une base bien conçue **déclare** ses clés ;
- **interroger une table** avec SELECT, WHERE (comparaisons, IN, LIKE, BETWEEN, précédence de AND et OR), ORDER BY, LIMIT et DISTINCT, calculer des colonnes avec CASE et des fonctions de date ;
- **agréger** avec COUNT, SUM, AVG, MIN, MAX, GROUP BY et HAVING, connaître l'**ordre logique d'exécution** (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT) et la règle d'or du regroupement ;
- **gérer les valeurs absentes** (NULL, logique à trois valeurs, COALESCE, NULLIF) et éviter quatre pièges silencieux : la division entière, la moyenne des moyennes, BETWEEN sur des horodateurs, LIMIT sans ORDER BY ;
- **joindre des tables** (INNER, LEFT, FULL, CROSS, auto-jointure), écrire une **anti-jointure** pour trouver ce qui manque, placer un filtre au bon endroit (ON ou WHERE), joindre sur une clé composée et **empiler ou comparer des ensembles** (UNION ALL, INTERSECT, EXCEPT) ;
- **repérer la multiplication des lignes** après une jointure un-à-plusieurs et l'éviter (agréger avant de joindre, compter des éléments distincts, appliquer le test des trois comptes) ;
- **calculer sans perdre le détail** avec les fonctions fenêtres : parts, classements (ROW_NUMBER, RANK, DENSE_RANK), comparaisons avec la ligne précédente (LAG, LEAD), cumuls, moyennes mobiles, tranches (NTILE) ;
- **structurer une requête complexe** en CTE nommées, par une méthode en six étapes (question, grain, sources, jointures, filtres et agrégats, vérification), fabriquer un calendrier par une CTE récursive, et **vérifier un résultat par un second outil** ;
- (en option) **utiliser sous-requêtes, index, vues, transactions, déclencheurs**, lire un plan d'exécution, se protéger de l'injection SQL, et **situer les dialectes** de PostgreSQL, MySQL, SQL Server et Oracle.

Le chapitre a mis des chiffres sur les questions que la gérante posait en passant dans le couloir. Tous viennent de requêtes réellement exécutées sur la base de la boutique :

Question	Résultat mesuré
Quel chiffre d'affaires en 2025 ?	1 324 764 € (retrouvé par quatre chemins : somme par catégorie, cumul d'une fenêtre, CTE, pandas)
Que pèsent nos dix meilleurs clients ?	24 886 €, soit 1,88 % du chiffre d'affaires de 2025
La clientèle est-elle concentrée ?	les 10 % de clients qui dépensent le plus : 32,9 % du chiffre d'affaires ; les 20 % premiers : 51,9 %
Combien de clients n'ont jamais commandé ?	1 194 sur 6 000 (un sur cinq)
Fidélité en 2025	3 133 clients fidèles, 742 nouveaux, 931 perdus depuis 2024
Quel est l'écart typique entre deux commandes d'un client ?	86,4 jours en moyenne
D'où viennent les retours ?	site : 8,8 % des lignes ; boutique : 3,3 %
Quand vend-on le plus ?	décembre (+27,8 % sur novembre) ; le samedi
Que coûte une jointure mal écrite ?	le budget publicitaire de 75 995 € devient 2 991 731 € après jointure sur la date : près de quarante fois trop

Le fil conducteur du chapitre tient en une phrase : **une requête est une description précise du tableau que l'on veut, et la précision commence par le grain**. Toutes les erreurs sérieuses vues

ici (doubles comptages, filtres qui disparaissent, divisions entières, jours manquants) sont des erreurs de grain : on croyait compter des commandes, on comptait des lignes ; on croyait garder tous les clients, on les filtrait ; on croyait moyenner des paniers, on moyennait des moyennes. Avant chaque requête, écrivez en une phrase ce que représentera **une ligne du résultat** ; après chaque requête, **vérifiez un total** par un autre chemin.

Le chapitre 4 reprend ces mêmes questions avec deux autres langages d'analyse, **Python (pandas)** et **R (tidyverse)**. Vous y retrouverez, sous une autre écriture, les filtres, les regroupements, les jointures et les fenêtres de ce chapitre ; le SQL reste l'outil de choix pour **extraire et résumer** les données à la source, et pandas ou R prennent le relais pour **explorer, modéliser et dessiner**.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 à 3.9 (découvrir la base, ventes par canal, meilleurs clients, paniers et doubles comptages, retours, évolution mensuelle, fidélité et cohortes, segmentation RFM, index et plans d'exécution) et exercices 3.1 à 3.14, tous corrigés.

Chapitre 4 : Python et R pour l'analyse

« Un calcul que l'on sait refaire d'un clic vaut mieux qu'un calcul parfait que l'on ne refera jamais. »

La gérante vous arrête dans le couloir, un lundi matin :

— Chaque lundi, je veux le même tableau : les ventes de la semaine par canal, comparées à la même semaine de l'an dernier. Jusqu'ici, une collègue (qui a quitté la boutique) le refaisait à la main dans un classeur, en une heure, avec une chance sur trois de se tromper de ligne. Peux-tu l'automatiser ?

Vous avez déjà deux outils pour répondre : le tableur (chapitre 2) et le langage SQL (chapitre 3). Ce chapitre ajoute le troisième, celui qui fait le plus gagner de temps quand une analyse **se répète** ou **grossit** : un langage de programmation pensé pour les données. Deux sont couramment utilisés par les analystes : **Python**, avec la bibliothèque **pandas**, et **R**, avec l'ensemble de paquets appelé **tidyverse**. Nous apprenons les deux côte à côte, sur le même problème, avec les mêmes chiffres à l'arrivée.

Le chemin de ce chapitre

- **4.1 Les fondamentaux de pandas** : une table de données dans Python (le *DataFrame*), la lire, la sélectionner, la filtrer, la compléter.
- **4.2 Les fondamentaux du tidyverse** : les mêmes gestes en R, avec les verbes de *dplyr* et le tube `|>`.
- **4.3 Lire, filtrer, regrouper, restructurer** : un export de caisse désordonné remis en état, des agrégats, des jointures, des tableaux croisés, et enfin **le tableau du lundi** de la gérante.
- **4.4 Notebooks d'analyse** : un document qui mêle texte, code et résultats, et les précautions à prendre pour qu'il soit reproductible.
- **4.5 ⊕ R : ggplot2 et Shiny** : la grammaire des graphiques et une application interactive.
- **4.6 ⊕ Python : NumPy, polars, rapports avec Jupyter** : le moteur de calcul sous pandas, une alternative plus rapide, et des rapports automatiques.

Parcours essentiel. Les sections 4.1 à 4.4 suffisent pour la suite du volume. Les sections 4.5 et 4.6, marquées ⊕, élargissent la boîte à outils : lisez-les quand le besoin se présente.

Pourquoi un langage, alors qu'Excel et SQL existent ?

Le tableur est excellent pour regarder les données, essayer une idée, produire un tableau que quelqu'un d'autre retouchera. SQL est excellent pour interroger de **gros volumes** rangés dans une base. Un langage de programmation apporte trois choses de plus.

- **La répétition sans effort.** Un programme est une recette écrite : on la rejoue le lundi suivant, ou sur un autre fichier, sans refaire les gestes. Une erreur corrigée dans la recette est corrigée pour toujours.
- **La traçabilité.** Le programme **dit** ce qui a été fait, ligne par ligne : quel fichier lu, quelles lignes écartées, quelles colonnes calculées. Un collègue, ou vous dans six mois, peut le relire et le contester. Un clic dans un menu ne laisse aucune trace.
- **L'étendue.** Nettoyer un texte, ajuster un modèle, tracer cent graphiques, lire dix fichiers d'un coup : ce qui est pénible dans un tableur tient en quelques lignes.

Le prix à payer est une marche d'entrée : il faut apprendre une syntaxe. Ce chapitre la rend la plus douce possible, en n'apprenant du langage que ce dont un analyste se sert pour **manipuler des tables**. Il ne remplace pas un cours de programmation, et n'en a pas l'ambition.

Python ou R ?

La question revient toujours, et la réponse honnête est : **les deux font très bien le travail**, et la différence tient plus aux habitudes de l'entourage qu'aux capacités.

	Python (pandas)	R (tidyverse)
Origine	langage généraliste, devenu langage de la donnée	langage conçu par des statisticiens
Points forts	un seul langage pour l'analyse, l'automatisation, le web, l'apprentissage automatique	statistique, graphiques (ggplot2), rapports, très bonne lisibilité des enchaînements
Syntaxe pour manipuler une table	des méthodes enchaînées avec des points : <code>table.query(...).groupby(...)</code>	des verbes reliés par un tube : <code>filter(...), group_by(...), summarise(...)</code>
Écosystème rencontré	équipes de données, informatique, industrie	recherche, santé, enquêtes, statistique publique

Dans un poste d'analyste, vous rencontrerez les deux. Savoir lire l'un quand on écrit l'autre est un vrai atout, et les concepts sont les mêmes : **sélectionner, filtrer, calculer, regrouper, joindre, restructurer**. Vous verrez qu'une fois un geste compris dans un langage, on le retrouve dans l'autre en quelques minutes.

Comment lire les blocs de ce chapitre

Chaque notion est montrée **deux fois** : un bloc Python (avec pandas), puis un bloc R (avec le tidyverse). Les deux blocs répondent à la même question, et **leurs résultats coïncident** : c'est la **vérification croisée**, un réflexe d'analyste que nous cultivons tout au long du volume. Quand deux outils indépendants donnent le même chiffre, on a une bonne raison de lui faire confiance ; quand ils diffèrent, on a trouvé quelque chose à comprendre. Pour que cette comparaison ne soit pas qu'une affirmation, un bloc caché la fait réellement, et le livre en affiche le verdict : « identique au résultat de pandas : TRUE ».

Une première illustration, avant d'entrer dans le détail : le chiffre d'affaires de 2025, calculé par chaque langage. Python lit deux fichiers, les relie par le numéro de commande et additionne les montants de l'année.

```
lignes = pd.read_csv("donnees/lignes_commande.csv")
commandes = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"])
ventes = lignes.merge(commandes, on="id_commande")
ca_2025 = ventes.loc[ventes["date_commande"].dt.year == 2025, "montant"].sum()
print(round(ca_2025, 2))
```

1324763.72

R fait la même chose, avec un tube `|>` qui se lit « et ensuite » : on lit les deux fichiers, on les relie, on garde 2025, on additionne.

```
ventes <- read_csv("donnees/lignes_commande.csv", show_col_types = FALSE) |>
  left_join(read_csv("donnees/commandes.csv", show_col_types = FALSE), by = "id_commande")
ca <- ventes |> filter(year(date_commande) == 2025) |> summarise(ca = sum(montant)) |> pull(ca)
cat(sprintf("%.2f", ca), "\n")
```

1324763.72

identique au résultat de pandas : TRUE

Les deux langages donnent le même chiffre d'affaires pour 2025, à l'euro et au centime près. Ce petit programme contient déjà l'essentiel de ce chapitre : lire, relier, filtrer, additionner. Il suffit maintenant de **comprendre chaque pas** et de savoir l'enchaîner.

Les données du chapitre. Elles viennent de la boutique, **simulée** (les données sont fictives, générées avec des graines fixes) : `clients.csv` (6 000 clients), `produits.csv` (120 produits), `commandes.csv` (36 395 commandes de 2023 à 2025), `lignes_commande.csv` (une ligne par produit commandé, avec la quantité, le prix et la remise), `retours.csv`, `jours_exploitation.csv` (un jour par ligne : commandes, chiffre d'affaires, météo, promotion) et `export_caisse_brut.csv`, un export de caisse **désordonné** que nous remettrons en état en 4.3. Les fichiers sont dans le dossier `donnees/` ; la description complète est en tête du volume.

4.1 Les fondamentaux de pandas

pandas est la bibliothèque de Python qui sait manipuler des tables : lire un fichier, trier, filtrer, calculer de nouvelles colonnes, regrouper. Cette section en donne les gestes de base, un par un, sur les commandes de la boutique. Elle commence par quelques mots de Python, juste ce qu'il faut pour lire les exemples sans se perdre.

4.1.1 Python en dix minutes

Python est un langage où l'on **donne des noms à des valeurs** (des *variables*), puis où l'on calcule avec ces noms. Le signe `=` n'est pas une égalité mathématique : il range une valeur sous un nom. Tout ce qui suit un `#` est un commentaire, ignoré par l'ordinateur.

```
prix = 19.9           # un nombre décimal
quantite = 3          # un nombre entier
total = prix * quantite
print(total)
print(round(total, 2))
```

```
59.699999999999996
59.7
```

Le premier résultat étonne : l'ordinateur calcule en binaire, et la plupart des décimaux ne s'y écrivent pas exactement, d'où un résidu minuscule. Pour **afficher** un montant, on arrondit avec `round`. Ce détail explique pourquoi, dans tout le chapitre, nos montants sont arrondis à deux décimales avant d'être affichés ou comparés.

Deux structures reviennent sans cesse. Une **liste** range des éléments dans l'ordre, entre crochets ; on en prend un par sa position, **en commençant à zéro**. Un **dictionnaire** range des paires « clé : valeur » entre accolades ; on retrouve une valeur par sa clé. Une **fonction** est une recette à laquelle on donne un nom : `def` la définit, `return` renvoie le résultat, et l'indentation (quatre espaces) délimite son contenu.

```
canaux = ["Boutique", "Site", "Réseaux"]
remises = {"SOLDES": 0.20, "BIENVENUE": 0.10, "FIDELITE": 0.05}

def prix_remise(prix, code):
    return prix * (1 - remises.get(code, 0))    # sans code connu : remise 0

print(canaux[0], len(canaux), remises["SOLDES"])
print(round(prix_remise(49.9, "FIDELITE"), 2), round(prix_remise(49.9, "AUCUN"), 2))
```

```
Boutique 3 0.2
47.4 49.9
```

Enfin, Python ne connaît au départ qu'une petite partie de ce qu'il sait faire : pour le reste, on **importe** des bibliothèques. La ligne `import pandas as pd` charge pandas et lui donne le surnom `pd`, que tout le monde utilise. Chaque fonction de la bibliothèque s'appelle ensuite par `pd.nom_de_la_fonction`.

En pratique. Vous n'avez pas besoin de maîtriser davantage de Python pour la suite : on se sert surtout de pandas, et ses gestes s'apprennent en les pratiquant. Quand un message d'erreur apparaît, lisez sa **dernière ligne** : elle dit presque toujours ce qui ne va pas (un nom mal orthographié, une parenthèse oubliée).

4.1.2 Le DataFrame et la Series

Dans pandas, une table s'appelle un **DataFrame** : des lignes (les observations), des colonnes (les variables), chaque colonne ayant un **seul type**. Une colonne prise isolément est une **Series**. Le gabarit de lecture est toujours le même : `pd.read_csv("fichier.csv")` renvoie un DataFrame.

```
commandes = pd.read_csv("donnees/commandes.csv")
print(commandes.shape)
print(commandes.head(3))
```

```
(36395, 7)
   id_commande  date_commande  heure  id_client  canal  mode_livraison  code_promo
0             1    2023-01-01  15:35     2637    Site      Domicile         NaN
1             2    2023-01-01  16:32        96  Boutique  Retrait magasin         NaN
2             3    2023-01-01  18:07     3490  Boutique  Retrait magasin         NaN
```

`shape` donne le nombre de lignes et de colonnes ; `head(3)` montre les trois premières lignes (sans argument, cinq). Regardez toujours vos données après les avoir lues : ce petit coup d'œil détecte la moitié des problèmes. Le **type** de chaque colonne se lit avec `dtypes`.

```
print(commandes.dtypes)
```

```
id_commande      int64
date_commande     str
heure             str
id_client        int64
canal             str
mode_livraison    str
code_promo        str
dtype: object
```

Les colonnes numériques sont de type `int64` (entiers) ou `float64` (décimaux). Les textes sont de type `str`, depuis la version 3 de pandas ; dans les versions plus anciennes, vous verriez `object`, et un bloc de code écrit pour l'une fonctionne presque toujours avec l'autre. Remarquez que `date_commande` est lue comme **du texte** : pandas ne devine pas qu'il s'agit de dates. Nous allons le lui dire.

4.1.3 Lire un fichier : les arguments qui comptent

`read_csv` accepte des dizaines d'arguments. Une poignée suffit presque toujours.

Argument	À quoi il sert	Exemple
<code>sep</code>	le séparateur de colonnes	<code>sep=";"</code> (fichiers « à la française »)
<code>decimal</code>	le signe décimal	<code>decimal=","</code>
<code>encoding</code>	le codage des caractères accentués	<code>encoding="cp1252"</code> (anciens fichiers Windows)
<code>parse_dates</code>	les colonnes à lire comme des dates	<code>parse_dates=["date_commande"]</code>
<code>dtype</code>	forcer le type d'une colonne	<code>dtype={"id_client": "str"}</code>
<code>usecols, nrows</code>	ne lire que certaines colonnes, ou les premières lignes	<code>usecols=["id_ligne", "montant"], nrows=1000</code>
<code>na_values</code>	les textes à traiter comme « manquant »	<code>na_values=["n/a", "-"]</code>

On range donc les dates dans le bon type dès la lecture. Voici le chargement de toutes les tables

dont nous aurons besoin dans ce chapitre.

```
commandes = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"])
lignes = pd.read_csv("donnees/lignes_commande.csv")
produits = pd.read_csv("donnees/produits.csv")
clients = pd.read_csv("donnees/clients.csv", parse_dates=["date_inscription"])
retours = pd.read_csv("donnees/retours.csv", parse_dates=["date_retour"])
jours = pd.read_csv("donnees/jours_exploitation.csv", parse_dates=["date"])
print({nom: len(t) for nom, t in [("commandes", commandes), ("lignes", lignes), ("clients", clients),
    ("retours", retours)]})
print(commandes["date_commande"].dtype)
```

```
{'commandes': 36395, 'lignes': 83905, 'clients': 6000, 'retours': 5002}
datetime64[us]
```

Le type affiché pour la date, `datetime64`, est celui des **dates et heures** : on peut désormais en extraire l'année, le mois, le jour de la semaine, ou les comparer. Un type bien choisi à la lecture évite des heures de travail ensuite.

Piège : un identifiant n'est pas un nombre. Le numéro d'un client ou d'une commande se lit comme un entier, mais ne se somme pas et ne se moyenne pas : « le numéro moyen des clients » n'a aucun sens. Quand un identifiant commence par des zéros (00123), la lecture comme entier les supprime : forcez alors `dtype="str"`. La section 5.1 reviendra sur cette distinction entre types de données.

4.1.4 Sélectionner : colonnes, lignes, cellules

Il y a trois façons de désigner une partie d'une table. Les **crochets** sélectionnent des **colonnes** : un nom donne une *Series*, une liste de noms donne un *DataFrame*.

```
canal = commandes["canal"] # une colonne : une Series
extrait = commandes[["id_commande", "canal"]] # deux colonnes : un DataFrame
print(type(canal).__name__, extrait.shape)
```

```
Series (36395, 2)
```

`.loc` sélectionne par **étiquette** (le nom des lignes et des colonnes) ; `.iloc` sélectionne par **position** (le rang, en commençant à zéro). Les deux se ressemblent et se comportent **différemment** sur un point : avec `.loc`, la borne de fin est **inclusive** ; avec `.iloc`, elle est **exclue**.

```
print(commandes.loc[0:2, ["id_commande", "canal"]])
print(commandes.iloc[0:2, [0, 4]])
```

```
id_commande  canal
0           1    Site
1           2  Boutique
2           3  Boutique
id_commande  canal
0           1    Site
1           2  Boutique
```

La première commande renvoie trois lignes (les étiquettes 0, 1 et 2), la seconde deux lignes (les positions 0 et 1). C'est l'une des sources d'erreur les plus fréquentes des débutants : en cas de doute, comptez les lignes obtenues.

4.1.5 Filtrer : garder les lignes qui répondent à une condition

Filtrer, c'est garder les lignes pour lesquelles une condition est vraie. On écrit la condition comme un calcul, qui produit une colonne de vrais et de faux, et on la passe entre crochets. Pour combiner plusieurs conditions, on utilise & (et), | (ou) et ~ (non), **en entourant chaque condition de parenthèses** quand on les écrit en une seule expression.

Combien de commandes passées sur le Site en 2025 avec le code SOLDES ?

```
est_site = commandes["canal"] == "Site"
en_2025 = commandes["date_commande"].dt.year == 2025
avec_soldes = commandes["code_promo"] == "SOLDES"
print(len(commandes[est_site & en_2025 & avec_soldes]))
```

495

Trois conditions, nommées une à une, se relisent mieux qu'une seule ligne interminable. La méthode query permet d'écrire le même filtre sous forme de texte, souvent plus lisible ; isin teste l'appartenance à une liste ; between teste l'appartenance à un intervalle (bornes incluses).

```
n_query = len(commandes.query("canal == 'Site' and date_commande >= '2025-01-01' and code_promo == 'SOLDES'))
n_isin = commandes["canal"].isin(["Boutique", "Réseaux"]).sum()
n_nov = commandes["date_commande"].between("2025-11-01", "2025-11-30").sum()
print(n_query, n_isin, n_nov)
```

495 20932 1509

On retrouve les 495 commandes du premier filtre (par une autre écriture), puis 20 932 commandes passées en boutique ou sur les réseaux, et 1 509 commandes en novembre 2025. Les mêmes comptages en R sont faits en section 4.2.

4.1.6 Colonnes calculées et tri

Créer une colonne, c'est lui donner un nom et un calcul qui porte sur d'autres colonnes, ligne par ligne. La méthode assign le fait sans modifier la table d'origine : elle renvoie une **nouvelle** table. Sur les lignes de commande, calculons le montant avant remise, la remise en euros, puis vérifions que le fichier est cohérent.

```
l2 = lignes.assign(
    brut=lambda d: d["quantite"] * d["prix_unitaire"],
    remise_eur=lambda d: d["brut"] * d["remise_pct"] / 100,
)
print((l2["brut"] - l2["remise_eur"] - l2["montant"]).abs().max().round(2))
```

0.01

La notation lambda d: ... se lit « pour une table d, calculer ... » : elle permet à la seconde colonne d'utiliser la première, tout juste créée. Le plus grand écart vaut un centime : montant = quantité × prix × (1 – remise) pour **toutes** les lignes, à l'arrondi du montant au centime près. Le fichier est cohérent, et nous pourrions nous fier à montant.

Trier se fait par sort_values, en précisant la colonne et le sens. Les cinq lignes les plus chères :

```
print(l2.sort_values("montant", ascending=False).head(5)[["id_ligne", "quantite", "prix_unitaire",
    ↪ "remise_pct", "montant"]])
```

	id_ligne	quantite	prix_unitaire	remise_pct	montant
68711	68712	4	157.49	0	629.96

78316	78317	4	157.49	0	629.96
70637	70638	4	157.49	0	629.96
64160	64161	4	157.49	0	629.96
23327	23328	4	152.90	0	611.60

4.1.7 Valeurs manquantes et comptages

Une cellule vide est lue comme **NaN** (« not a number »), la valeur manquante de pandas. `isna()` la détecte, et la somme des vrais donne le nombre de manquants par colonne.

```
print(commandes.isna().sum())
```

```
id_commande      0
date_commande    0
heure            0
id_client        0
canal            0
mode_livraison   0
code_promo       30652
dtype: int64
```

Seul `code_promo` a des manquants, pour une raison **métier** : une cellule vide signifie « pas de code promotionnel ». Ce n'est pas une donnée perdue. Pour compter proprement, on le dit explicitement avec `fillna`, puis `value_counts` donne la fréquence de chaque valeur ; avec `normalize=True`, des proportions.

```
codes = commandes["code_promo"].fillna("aucun")
print((codes.value_counts(normalize=True) * 100).round(1))
```

```
code_promo
aucun      84.2
SOLDES     8.3
FIDELITE   6.7
BIENVENUE  0.9
Name: proportion, dtype: float64
```

Piège : NaN n'est égal à rien, pas même à lui-même. `commandes["code_promo"] == np.nan` ne renvoie que des faux : pour tester un manquant, utilisez toujours `isna()` (ou `notna()`). Par ailleurs, la plupart des calculs (somme, moyenne) **ignorent** les NaN : une moyenne sur une colonne à moitié vide est une moyenne sur la moitié des lignes, sans le dire. Interrogez toujours le nombre de manquants **avant** de calculer.

4.1.8 La méthode chaînée

Chaque méthode de pandas renvoie une table, sur laquelle on peut appliquer la suivante : on peut donc **enchaîner** les étapes, une par ligne, dans des parenthèses. Le résultat se lit de haut en bas comme une recette. Quelle est, selon le canal, la part des commandes de 2025 passées avec un code promotionnel ?

```
part_promo = (
    commandes
    .loc[commandes["date_commande"].dt.year == 2025]
    .assign(avec_code=lambda d: d["code_promo"].notna())
    .groupby("canal")["avec_code"].mean()
    .mul(100).round(1)
)
print(part_promo)
```

```
canal
Boutique    15.1
Réseaux     15.0
Site        15.8
Name: avec_code, dtype: float64
```

La part est d'environ 15 % dans les trois canaux : les codes ne sont pas l'affaire d'un canal en particulier. Chaque ligne du programme est une étape : on garde 2025, on ajoute un indicateur « a un code », on regroupe par canal, on prend la moyenne de l'indicateur (la moyenne de vrais et de faux est la proportion de vrais), on exprime en pourcentage. Ce style est précisément celui que le tidyverse rend plus naturel encore, comme on le voit en 4.2.

Une dernière propriété mérite d'être connue : dans pandas 3, un extrait de table est toujours **une copie indépendante**. Modifier l'extrait ne touche jamais la table d'origine, ce qui supprime une classe entière d'erreurs qui hantait les versions précédentes.

```
extrait = commandes.loc[commandes["canal"] == "Site"].copy()
extrait["canal"] = "X"
print(commandes["canal"].unique().tolist())
```

```
['Site', 'Boutique', 'Réseaux']
```

À retenir. Une table pandas est un *DataFrame* ; on **lit** en précisant séparateur, décimale, encodage et colonnes de dates ; on **sélectionne** avec `[], .loc` (étiquettes, fin incluse) et `.iloc` (positions, fin exclue) ; on **filtre** avec des conditions combinées par `&, |, ~` ; on **calcule** avec `assign` ; on **compte** avec `value_counts` ; on **enchaîne** les étapes entre parenthèses. Un manquant se détecte par `isna()`, jamais par `==`.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 et 4.2, exercices 4.1 à 4.5.

4.2 Les fondamentaux du tidyverse

Le **tidyverse** est un ensemble de paquets R qui partagent une même philosophie : des tables bien rangées (une ligne par observation, une colonne par variable) et des **verbes** simples, qui se lisent comme des phrases et s'enchaînent avec un **tube**. Cette section reprend, en R, les gestes de la section 4.1, sur les mêmes données et avec les mêmes résultats.

4.2.1 R, les paquets et le tube

R est, comme Python, un langage où l'on range des valeurs sous des noms. Deux différences se remarquent tout de suite : l'affectation s'écrit avec une flèche `<-` (`x <- 5`), et les positions **commencent à 1** (le premier élément d'une liste est le numéro 1, pas le numéro 0). Les structures de base sont les **vecteurs** `c(...)`, qui peuvent être nommés) et les **fonctions**.

```
canaux <- c("Boutique", "Site", "Réseaux")
remises <- c(SOLDES = 0.20, BIENVENUE = 0.10, FIDELITE = 0.05)
prix_remise <- function(prix, code) prix * (1 - ifelse(code %in% names(remises), remises[code], 0))
cat(canaux[1], length(canaux), remises["SOLDES"], "\n")
cat(round(prix_remise(49.9, "FIDELITE"), 2), round(prix_remise(49.9, "AUCUN"), 2), "\n")
```

```
Boutique 3 0.2
47.4 49.9
```

On retrouve exactement les valeurs du programme Python de 4.1.1 (la fonction `ifelse` joue le rôle de `get` avec une valeur par défaut). Pour le reste, R vient avec des **paquets** que l'on installe une fois (`install.packages("tidyverse")`) et que l'on charge à chaque session avec `library`. Le paquet `tidyverse` en charge plusieurs d'un coup : `readr` pour lire, `dplyr` pour manipuler les tables, `tidyr` pour les restructurer, `ggplot2` pour les graphiques, `stringr` pour le texte, `lubridate` pour les dates, `forcats` pour les catégories.

```
library(tidyverse)
commandes <- read_csv("donnees/commandes.csv", show_col_types = FALSE)
commandes |> head(3)
```

```
# A tibble: 3 × 7
  id_commande date_commande heure id_client canal mode_livraison code_promo
    <dbl>    <date>      <time>    <dbl> <chr>      <chr>      <chr>
1         1 2023-01-01 15:35      2637 Site      Domicile    <NA>
2         2 2023-01-01 16:32         96 Boutique Retrait magasin <NA>
3         3 2023-01-01 18:07      3490 Boutique Retrait magasin <NA>
```

(Les messages de chargement des paquets ne sont pas reproduits dans ce livre.) Une table R s'appelle un **tibble** : un data frame dont l'affichage est plus lisible, qui indique le type de chaque colonne sous son nom (`<dbl>` pour un décimal, `<chr>` pour un texte, `<date>` pour une date) et qui n'imprime que ce qui tient à l'écran. Deux remarques sur ce résultat. D'abord, **readr reconnaît les dates au format ISO** (2023-01-01) dès la lecture, alors que `pandas` demande `parse_dates`. Ensuite, un tibble n'a pas de numéros de ligne : la table n'a pas d'« index » comme celle de `pandas`.

Le **tube** `|>` est l'outil central de R pour l'analyse. Il prend ce qui est à sa gauche et le donne comme premier argument à la fonction de droite : `commandes |> head(3)` équivaut à `head(commandes, 3)`. Un enchaînement de verbes se lit alors du haut vers le bas, « **et ensuite** » : « prends les commandes, **et ensuite** garde celles de 2025, **et ensuite** regroupe par canal... ».

4.2.2 Lire avec readr

`read_csv` lit un fichier séparé par des virgules ; `read_csv2` lit un fichier « à la française » (séparateur point-virgule, virgule décimale). Pour un contrôle plus fin, l'argument `locale` règle le signe décimal et l'encodage, et `col_types` force le type des colonnes. Chargeons toutes les tables du chapitre, puis jetons un coup d'œil avec `glimpse`, l'équivalent de `dtypes` plus quelques valeurs.

```
lignes <- read_csv("donnees/lignes_commande.csv", show_col_types = FALSE)
produits <- read_csv("donnees/produits.csv", show_col_types = FALSE)
clients <- read_csv("donnees/clients.csv", show_col_types = FALSE)
retours <- read_csv("donnees/retours.csv", show_col_types = FALSE)
jours <- read_csv("donnees/jours_exploitation.csv", show_col_types = FALSE)
cat(nrow(commandes), nrow(lignes), nrow(clients), nrow(retours), "\n")
glimpse(commandes)
```

```
36395 83905 6000 5002
Rows: 36,395
Columns: 7
$ id_commande    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, ...
$ date_commande  <date> 2023-01-01, 2023-01-01, 2023-01-01, 2023-01-01, 2023-01-01, 2023-01-01, 20...
$ heure          <time> 15:35:00, 16:32:00, 18:07:00, 17:10:00, 12:00:00, 19:32:00, 18:10:00, 15:5...
$ id_client      <dbl> 2637, 96, 3490, 743, 2931, 3017, 1418, 1038, 2689, 995, 3722, 4054, 495, 65...
$ canal          <chr> "Site", "Boutique", "Boutique", "Boutique", "Boutique", "Boutique", "Boutiq...
$ mode_livraison <chr> "Domicile", "Retrait magasin", "Retrait magasin", "Retrait magasin", "Retra...
$ code_promo     <chr> NA, NA, NA, NA, NA, NA, "FIDELITE", NA, "FIDELITE", NA, NA, "SOLDES", NA, N...
```

Les quatre nombres de lignes sont ceux que `pandas` a donnés en 4.1.3. Le résumé de `glimpse` liste chaque colonne avec son type et ses premières valeurs : c'est le réflexe de « regarder ses données » avant toute chose.

identique au résultat de pandas : TRUE

4.2.3 Les verbes de dplyr

Cinq verbes couvrent l'essentiel du travail quotidien sur une table, auxquels s'ajoutent `count` et `left_join`.

Verbe	Rôle	Équivalent pandas
<code>select()</code>	choisir des colonnes	<code>df[["a", "b"]]</code>
<code>filter()</code>	garder des lignes	<code>df[condition]</code> , <code>df.query()</code>
<code>mutate()</code>	créer ou modifier des colonnes	<code>df.assign()</code>
<code>arrange()</code>	trier	<code>df.sort_values()</code>
<code>summarise()</code> avec <code>group_by()</code>	résumer, par groupes	<code>df.groupby().agg()</code>
<code>count()</code>	compter les lignes par valeur	<code>df.value_counts()</code>
<code>left_join()</code>	relier deux tables	<code>df.merge(how="left")</code>

Commençons par `filter`. Les conditions séparées par une virgule sont **toutes** exigées (un « et ») ; `%in%` teste l'appartenance à un vecteur et `between` l'appartenance à un intervalle. Voici les trois comptages de 4.1.5.

```
n1 <- commandes |> filter(canal == "Site", year(date_commande) == 2025, code_promo == "SOLDES") |>
  nrow()
n2 <- commandes |> filter(canal %in% c("Boutique", "Réseaux")) |> nrow()
n3 <- commandes |> filter(between(date_commande, as.Date("2025-11-01"), as.Date("2025-11-30"))) |>
  nrow()
cat(n1, n2, n3, "\n")
```

495 20932 1509

identique au résultat de pandas : TRUE

Trois nombres identiques à ceux de pandas (495, 20 932 et 1 509). Notez deux différences d'écriture. R sait qu'une valeur manquante dans une condition donne ni vrai ni faux et **écarte** la ligne, tandis que pandas traite `NaN == "SOLDES"` comme faux : le résultat est le même. Et `mutate` permet à une colonne d'utiliser **immédiatement** celle qui vient d'être créée, sans la notation lambda de pandas.

```
l2 <- lignes |> mutate(brut = quantite * prix_unitaire, remise_eur = brut * remise_pct / 100)
cat(round(max(abs(l2$brut - l2$remise_eur - l2$montant)), 2), "\n")
l2 |> arrange(desc(montant)) |> select(id_ligne, quantite, prix_unitaire, remise_pct, montant) |>
  head(5)
```

```
0.01
# A tibble: 5 × 5
  id_ligne quantite prix_unitaire remise_pct montant
  <dbl>      <dbl>      <dbl>      <dbl>    <dbl>
1   64161         4        157.         0    630.
2   68712         4        157.         0    630.
3   70638         4        157.         0    630.
4   78317         4        157.         0    630.
5   10446         4        153.         0    612.
```

Le tri décroissant s'écrit `desc()`, et `select` choisit les colonnes à afficher : on retrouve les cinq lignes les plus chères de 4.1.6 (le tibble n'affiche que trois chiffres significatifs, d'où 157. pour 157,49 : l'**affichage** est arrondi, pas la valeur). Reste le couple `group_by` et `summarise`, qui reproduit

le groupement de 4.1.8 : la part des commandes de 2025 passées avec un code promotionnel, par canal.

```
part_promo_r <- commandes |>
  filter(year(date_commande) == 2025) |>
  group_by(canal) |>
  summarise(part_promo = round(100 * mean(!is.na(code_promo)), 1))
part_promo_r
```

```
# A tibble: 3 × 2
  canal    part_promo
  <chr>      <dbl>
1 Boutique    15.1
2 Réseaux     15
3 Site       15.8
```

identique au résultat de pandas : TRUE

Le ! signifie « non » : `!is.na(code_promo)` est vrai quand un code existe, et la moyenne de vrais et de faux est la proportion de vrais, comme en Python. Le verbe `count`, enfin, compte les lignes par valeur ; c'est la version courte de `group_by` suivi de `summarise(n = n())`.

```
commandes |> count(canal, sort = TRUE)
```

```
# A tibble: 3 × 2
  canal      n
  <chr>  <int>
1 Boutique 16975
2 Site    15463
3 Réseaux  3957
```

4.2.4 Dates, texte et catégories : lubridate, stringr, forcats

Trois paquets du tidyverse rendent les opérations courantes sur les dates, les textes et les catégories plus agréables. **lubridate** extrait les composantes d'une date (`year`, `month`, `isoweek`) et arrondit une date à son début de mois (`floor_date`). **stringr** manipule les textes (`str_to_title`, `str_detect`, `str_replace`). **forcats** ordonne et regroupe les catégories (`fct_infreq` classe par fréquence).

```
commandes |> mutate(mois = floor_date(date_commande, "month")) |> count(mois) |> head(3)
str_to_title(c("BOÎTE RUSTIQUE", "tapis nordique"))
levels(fct_infreq(commandes$code_promo))
```

```
# A tibble: 3 × 2
  mois      n
  <date>  <int>
1 2023-01-01 785
2 2023-02-01 642
3 2023-03-01 801
[1] "Boîte Rustique" "Tapis Nordique"
[1] "SOLDES"         "FIDELITE"       "BIENVENUE"
```

Le premier résultat donne le nombre de commandes des trois premiers mois ; le deuxième remet des majuscules aux initiales (y compris sur une lettre accentuée) ; le troisième liste les codes promotionnels, du plus fréquent au plus rare. On retrouve ces gestes en Python avec `dt.to_period("M")`, `str.title()` et `value_counts()` : la logique est la même, seule la syntaxe diffère.

4.2.5 L'équivalence entre pandas et le tidyverse

Une fois la logique comprise, passer d'un langage à l'autre revient à traduire. Le tableau suivant sert d'aide-mémoire.

Geste	pandas (Python)	tidyverse (R)
Lire un CSV	<code>pd.read_csv("f.csv")</code>	<code>read_csv("f.csv")</code>
Voir la structure	<code>df.dtypes, df.head()</code>	<code>glimpse(df), head(df)</code>
Choisir des colonnes	<code>df[["a", "b"]]</code>	<code>select(df, a, b)</code>
Filtrer	<code>df[df["a"] > 3], df.query("a > 3")</code>	<code>filter(df, a > 3)</code>
Créer une colonne	<code>df.assign(c=lambda d: d.a * 2)</code>	<code>mutate(df, c = a * 2)</code>
Trier	<code>df.sort_values("a", ascending=False)</code>	<code>arrange(df, desc(a))</code>
Résumer par groupe	<code>df.groupby("g").agg(m=("a", "mean"))</code>	<code>group_by(df, g) puis summarise(m = mean(a))</code>
Compter	<code>df["g"].value_counts()</code>	<code>count(df, g)</code>
Joindre	<code>df.merge(t, on="k", how="left")</code>	<code>left_join(df, t, by = "k")</code>
Manquant ?	<code>df["a"].isna()</code>	<code>is.na(df\$a)</code>
Large vers long	<code>df.melt(...)</code>	<code>pivot_longer(df, ...)</code>
Long vers large	<code>df.pivot_table(...)</code>	<code>pivot_wider(df, ...)</code>

4.2.6 Ce qui surprend quand on passe de l'un à l'autre

Trois différences causent la plupart des erreurs de traduction. La première, déjà vue : Python compte les positions **à partir de zéro**, R **à partir de un**. La deuxième : l'**égalité** se teste par `==` dans les deux langages, mais `=` n'est pas la même chose partout (affectation en R et en Python, argument nommé dans un appel de fonction). La troisième est plus surnoise : le traitement **par défaut** des valeurs manquantes.

```
x = pd.Series([1.0, 2.0, np.nan])
print(x.sum(), x.mean())
```

```
3.0 1.5
```

```
x <- c(1, 2, NA)
sum(x); mean(x); mean(x, na.rm = TRUE)
```

```
[1] NA
[1] NA
[1] 1.5
```

Python (pandas) **ignore** les manquants et renvoie 3 pour la somme et 1,5 pour la moyenne. R **propage** le manquant : la somme d'un vecteur qui contient un NA est NA, ce qui oblige à écrire `na.rm = TRUE` pour les ignorer. Aucun des deux choix n'est faux, mais ils conduisent à des bugs inverses : dans pandas, on oublie qu'un calcul a écarté des lignes ; dans R, on s'aperçoit tout de suite qu'un NA traîne. La leçon est valable partout : **comptez toujours vos manquants avant de calculer**.

À retenir. Le tidyverse range les verbes par fonction : `select` (colonnes), `filter` (lignes), `mutate` (calculs), `arrange` (tri), `group_by` + `summarise` (résumés), `count`, `left_join` ; le tube

|> les relie, comme les points de la méthode chaînée de pandas. Les résultats sont ceux de pandas, à la syntaxe près ; les pièges sont l'indexation (à partir de 1) et les manquants (propagés par défaut).

Pour s'entraîner. Cahier, chapitre 4 : application 4.3, exercice 4.6.

4.3 Lire, filtrer, regrouper, restructurer

Les sections précédentes ont posé les gestes de base. Celle-ci les met au travail sur des cas d'analyste : remettre en état un export désordonné, résumer par groupes, relier des tables, changer la forme d'un tableau, calculer des moyennes glissantes et comparer des semaines d'une année à l'autre. Elle se termine par la réponse à la gérante : **le tableau du lundi**.

4.3.1 Lire un export désordonné

Le fichier `export_caisse_brut.csv` est un export de caisse de la boutique physique pour la semaine du 3 au 9 novembre 2025. Il a l'allure de ce que produisent beaucoup de logiciels de gestion : fait pour être **lu par un humain**, pas par un programme. Avant de le lire avec pandas, regardons-le tel quel, ligne par ligne.

```
with open("donnees/export_caisse_brut.csv", encoding="cp1252") as f:
    lignes_brutes = f.read().splitlines()
print("\n".join(lignes_brutes[:6]))
print("...")
print(lignes_brutes[-1])
```

```
Export caisse - Boutique;;;;;;
Période du 03/11/2025 au 09/11/2025;;;;;;
;;;;;;
N° ticket;Date;Heure;Article;Catégorie;Qté;Prix unitaire;Montant
T33133;03/11/2025;09:00;BOÎTE RUSTIQUE;Maison;2;52,43;104,86
T33137;03/11/2025;10:35;Tapis nordique;Maison;1;64,79;64,79
...
;;;;;;Total;11561,47
```

On y voit cinq défauts typiques. Trois lignes de **titre** précèdent l'en-tête des colonnes. Le séparateur est le **point-virgule** et les décimaux s'écrivent avec une **virgule** (52,43). Les dates sont au format **jour/mois/année**. La dernière ligne est un **total**, qui n'est pas une vente. Et, comme on le verra, l'en-tête **se répète** à plusieurs endroits (le logiciel le réimprime à chaque « page »). Un sixième défaut ne se voit pas : le fichier est encodé en cp1252, l'ancien codage des fichiers Windows, et non en UTF-8. Lu avec le mauvais codage, il produit une erreur ou des accents mutilés.

```
try:
    pd.read_csv("donnees/export_caisse_brut.csv", sep=";")
except UnicodeDecodeError as e:
    print(type(e).__name__, "-", e.reason)
```

```
UnicodeDecodeError - invalid continuation byte
```

Avec les bons arguments, la lecture réussit. On saute les trois lignes de titre avec `skiprows=3`.

```
brut = pd.read_csv("donnees/export_caisse_brut.csv", sep=";", encoding="cp1252", skiprows=3)
print(brut.shape)
print(brut["Qté"].dtype, brut["Montant"].dtype)
```

```
(285, 8)
str str
```

Il y a 285 lignes, mais le résultat est décevant : même la quantité est lue comme **du texte**. La cause est que la colonne contient des mots (la ligne d'en-tête répétée, et le mot « Total ») : une seule cellule non numérique suffit à faire lire toute la colonne comme du texte. Il faut donc écarter ces lignes parasites **avant** de convertir. On garde les lignes dont le numéro de ticket est renseigné et n'est pas le mot « N° ticket », puis on convertit les dates (au format %d/%m/%Y) et les nombres (en remplaçant la virgule décimale par un point).

```
caisse = brut[brut["N° ticket"].notna() & (brut["N° ticket"] != "N° ticket")].copy()
print(len(brut) - len(caisse), "lignes écartées :", brut["N° ticket"].isna().sum(), "total,",
      ↪ (brut["N° ticket"] == "N° ticket").sum(), "en-têtes répétés")
total_fichier = float(brut.iloc[-1]["Montant"].replace(",", "."))
caisse["Date"] = pd.to_datetime(caisse["Date"], format="%d/%m/%Y")
for col in ["Qté", "Prix unitaire", "Montant"]:
    caisse[col] = pd.to_numeric(caisse[col].str.replace(",", "."))
print(caisse[["Date", "Qté", "Prix unitaire", "Montant"]].dtypes)
```

```
5 lignes écartées : 1 total, 4 en-têtes répétés
Date          datetime64[us]
Qté           int64
Prix unitaire  float64
Montant        float64
dtype: object
```

Reste à traiter les **valeurs manquantes** : huit lignes n'ont pas de montant. On peut le **reconstituer** (quantité × prix unitaire), mais avant de le faire, vérifions que ce calcul est légitime et mesurons son effet en le comparant au total que le fichier annonce lui-même. Cette comparaison est un réflexe : **toute transformation doit se réconcilier avec un chiffre connu**.

```
print(caisse["Montant"].isna().sum())
manquants = caisse[caisse["Montant"].isna()]
tickets = manquants["N° ticket"].str[1:].astype(int)
print(commandes.loc[commandes["id_commande"].isin(tickets), ["id_commande", "code_promo"]].dropna())
print(manquants.loc[manquants["N° ticket"] == "T33353", ["Article", "Qté", "Prix unitaire"]])
caisse["Montant"] = caisse["Montant"].fillna((caisse["Qté"] * caisse["Prix unitaire"]).round(2))
print(round(caisse["Montant"].sum(), 2), total_fichier, round(caisse["Montant"].sum() -
      ↪ total_fichier, 2))
```

```
8
      id_commande code_promo
33352      33353  FIDELITE
      Article  Qté  Prix unitaire
205 Boîte rustique    1      52.43
11564.09 11561.47 2.62
```

Le total reconstitué est de 11 564,09 €, et le fichier annonce 11 561,47 € : **2,62 € d'écart**. D'où vient-il ? L'un des huit tickets sans montant (le numéro 33353) a été payé avec le code FIDELITE, qui accorde 5 % de remise ; or le calcul « quantité × prix » ignore les remises. Cinq pour cent de 52,43 €, le prix de la ligne concernée, font 2,62 €. L'écart est expliqué au centime : on **sait** que les 280 lignes restantes sont fiables, et que la reconstitution surestime d'une remise que la caisse ne nous donne pas. En pratique, on garderait la reconstitution en la documentant (le chapitre 4 du volume II traite de la documentation des transformations).

Dernier défaut : le **texte**. Des articles sont en majuscules (BOÎTE RUSTIQUE) et des catégories en minuscules (bien-être), ce qui crée de faux doublons : pour pandas, Maison et maison sont deux catégories.

```
print(caisse["Catégorie"].nunique(), caisse["Article"].nunique())
caisse["Catégorie"] = caisse["Catégorie"].str.capitalize()
caisse["Article"] = caisse["Article"].str.capitalize()
print(caisse["Catégorie"].nunique(), caisse["Article"].nunique())
```

```
10 71
6 58
```

La mise en forme normalisée ramène les 10 catégories apparentes à 6, qui sont les vraies, et les 71 articles apparents à 58. Faisons le même travail en R. Le paquet `readr` lit tout en texte avec `col_types = cols(.default = col_character())`, puis `dmy` convertit les dates jour-mois-année et `parse_number` les nombres, avec la virgule décimale précisée dans `locale`. La fonction `coalesce` remplace un manquant par une autre valeur, et `str_to_sentence` met une majuscule initiale.

```
brut <- read_delim("donnees/export_caisse_brut.csv", delim = ";", skip = 3, col_types = cols(.default
  ↳ = col_character()),
  locale = locale(encoding = "cp1252", decimal_mark = ","))
nombre <- function(x) parse_number(x, locale = locale(decimal_mark = ","))
caisse_r <- brut |>
  filter(!is.na(`N° ticket`), `N° ticket` != "N° ticket") |>
  mutate(Date = dmy(Date), across(c(Qté, `Prix unitaire`, Montant), nombre)) |>
  mutate(Montant = coalesce(Montant, round(Qté * `Prix unitaire`, 2)),
         across(c(Catégorie, Article), str_to_sentence))
cat(nrow(caisse_r), round(sum(caisse_r$Montant), 2), n_distinct(caisse_r$Catégorie),
  ↳ n_distinct(caisse_r$Article), "\n")
```

```
280 11564.09 6 58
```

```
identique au résultat de pandas : TRUE
```

Même nombre de lignes (280), même total, mêmes comptes de catégories et d'articles. Remarquez comme le programme R, avec ses verbes, se lit presque comme la liste des défauts que nous venons d'énumérer : c'est l'intérêt du style « et ensuite ».

En pratique : la routine d'un fichier étranger. (1) Regarder le texte brut avant de lire. (2) Lire **tout en texte** si l'on a un doute sur les types. (3) Écarter les lignes parasites (titres, totaux, en-têtes répétés). (4) Convertir types, dates et nombres. (5) Traiter les manquants, en sachant pourquoi ils manquent. (6) Normaliser les textes. (7) **Réconcilier** avec un total connu. Le volume II consacre un chapitre entier à ces tâches ; retenez dès maintenant que chaque étape doit être écrite, pas faite à la main.

4.3.2 Regrouper : une question, un groupe, un calcul

Presque toute question d'analyse revient à : « combien, **par** quoi ? » Le regroupement (*split-apply-combine*) découpe la table en groupes, calcule un résumé dans chacun, puis rassemble les résultats. Pour poser cette base, il nous faut une table de **ventes** qui relie, ligne par ligne, la commande (date, canal), le produit (catégorie, coût) et le montant. C'est le travail d'une **jointure**, dont la section 4.3.3 détaillera le mécanisme ; admettons-la pour l'instant.

```
ventes = (lignes.merge(commandes, on="id_commande")
  .merge(produits[["id_produit", "categorie", "cout_achat"]], on="id_produit",
  ↳ validate="m:1")
  .assign(annee=lambda d: d["date_commande"].dt.year))
resume = (ventes.groupby("canal")
  .agg(ca=("montant", "sum"), commandes=("id_commande", "nunique"),
  ↳ montant_moyen_ligne=("montant", "mean"))
  .assign(panier_moyen=lambda d: d["ca"] / d["commandes"]).round(1))
```

```
print(resume)
```

	ca	commandes	montant_moyen_ligne	panier_moyen
canal				
Boutique	1712728.8	16975	43.5	100.9
Réseaux	397741.7	3957	43.8	100.5
Site	1542686.8	15463	43.5	99.8

`groupby("canal")` découpe par canal ; `agg` calcule **plusieurs** résumés en une fois, chacun avec le nom de la colonne résultat (`ca=("montant", "sum")`) se lit « ca est la somme de montant ». `nunique` compte les valeurs **distinctes** : une commande compte plusieurs lignes, il faut compter chaque numéro de commande une fois. Ce petit tableau contient une leçon importante : la gérante a demandé le « panier moyen », mais **de quoi** la moyenne ? Le montant moyen **d'une ligne** est d'environ 43,5 € dans les trois canaux ; le montant moyen **d'une commande** (le panier) est d'environ 100 €. Les deux sont corrects, ils répondent à deux questions différentes, et confondre l'un avec l'autre fausse la décision. Le panier est ici de 100,9 € en boutique, 100,5 € sur les réseaux et 99,8 € sur le Site : presque identique, malgré des chiffres d'affaires très différents.

Le regroupement sert aussi à calculer des **parts**. La méthode `transform` renvoie, pour chaque ligne, le résultat du groupe auquel elle appartient (ici, le chiffre d'affaires total de l'année) : on peut alors calculer la part de chaque canal sans perdre le détail des lignes.

```
ca_an_canal = ventes.groupby(["annee", "canal"], as_index=False)["montant"].sum()
total_annee = ca_an_canal.groupby("annee")["montant"].transform("sum")
ca_an_canal["part_pct"] = (ca_an_canal["montant"] / total_annee * 100).round(1)
print(ca_an_canal.pivot(index="annee", columns="canal", values="part_pct"))
```

canal	Boutique	Réseaux	Site
annee			
2023	52.1	10.8	37.1
2024	46.9	10.8	42.2
2025	42.3	11.0	46.6

La lecture est immédiate : en 2023, la boutique réalise 52,1 % du chiffre d'affaires et le Site 37,1 % ; en 2025, le Site (46,6 %) a dépassé la boutique (42,3 %), tandis que les réseaux restent stables, autour de 11 %. La gérante demandait si le Site faisait « perdre des clients à la boutique » : la question est plus fine (le chiffre d'affaires de la boutique a baissé de 2023 à 2024 puis s'est stabilisé), mais la **tendance** de fond est là, et elle vient de ce premier regroupement.

Voici le même travail en R. Le `group_by` suivi de `summarise` calcule les résumés ; `n_distinct` est l'équivalent de `nunique`.

```
ventes_r <- lignes |>
  left_join(commandes, by = "id_commande") |>
  left_join(select(produits, id_produit, categorie, cout_achat), by = "id_produit") |>
  mutate(annee = year(date_commande))
resume_r <- ventes_r |> group_by(canal) |>
  summarise(ca = sum(montant), commandes = n_distinct(id_commande), montant_moyen_ligne =
    <- mean(montant)) |>
  mutate(panier_moyen = ca / commandes, across(where(is.numeric), \ (x) round(x, 1)))
resume_r
```

```
# A tibble: 3 × 5
  canal      ca commandes montant_moyen_ligne panier_moyen
  <chr>    <dbl>    <dbl>          <dbl>          <dbl>
1 Boutique 1712729.    16975          43.5           101.
2 Réseaux  397742.      3957           43.8           100.
3 Site    1542687.    15463           43.5           99.8
```

identique au résultat de pandas : TRUE

Pour la part par année, `mutate` appliqué à une table **groupée** joue le rôle de `transform` : la somme du `mutate` est calculée dans chaque groupe.

```
ventes_r |> group_by(annee, canal) |> summarise(ca = sum(montant), .groups = "drop_last") |>
  mutate(part_pct = round(100 * ca / sum(ca), 1)) |> select(-ca) |> pivot_wider(names_from = canal,
  ↪ values_from = part_pct)
```

```
# A tibble: 3 × 4
# Groups:   annee [3]
  annee Boutique Réseaux Site
  <dbl>   <dbl>   <dbl> <dbl>
1  2023     52.1    10.8  37.1
2  2024     46.9    10.8  42.2
3  2025     42.3     11   46.6
```

Les parts sont celles de pandas (et, ici encore, l'affichage d'un tibble arrondi : 101. pour un panier de 100,9 € dans le tableau précédent, alors que la valeur n'est pas arrondie). Le résultat s'affiche avec une mention « Groups: annee » : la table est restée **groupée** par année après le `summarise`, ce qui change le comportement des verbes suivants. L'argument `.groups = "drop"` ou la fonction `ungroup()` retire ce regroupement : c'est un piège classique de `dplyr`, qui explique des résultats surprenants quand on oublie de dégroupier.

4.3.3 Joindre : relier deux tables par une clé

Une **jointure** relie les lignes de deux tables qui partagent une valeur, la **clé** : le numéro de commande relie lignes à commandes, le numéro de produit relie lignes à produits. Trois variantes couvrent la plupart des cas.

Jointure	Lignes conservées	pandas	dplyr
gauche	toutes celles de la table de gauche ; les colonnes de droite sont manquantes quand il n'y a pas de correspondance	<code>how="left"</code>	<code>left_join</code>
interne	seulement celles qui ont une correspondance des deux côtés	<code>how="inner"</code>	<code>inner_join</code>
complète	toutes celles des deux tables	<code>how="outer"</code>	<code>full_join</code>

La jointure **gauche** est la plus utile : on part de la table qui nous intéresse (les lignes de vente) et on lui **ajoute** des informations, sans en perdre. Le danger principal est la **multiplication des lignes** : si la clé n'est pas unique dans la table de droite, chaque ligne de gauche est dupliquée autant de fois qu'elle a de correspondances, et tous les totaux sont faussés **sans qu'aucune erreur n'apparaisse**. Simulons-le en ajoutant au catalogue de produits un doublon du produit numéro 1.

```
doublon = pd.concat([produits, produits.head(1)])
print(len(lignes.merge(produits, on="id_produit")), len(lignes.merge(doublon, on="id_produit")))
try:
  lignes.merge(doublon, on="id_produit", validate="m:1")
except Exception as e:
```



```
print(type(e).__name__, "-", str(e).split(";")[0])
```

```
83905 85345
MergeError - Merge keys are not unique in right dataset
```

Avec le catalogue correct, la jointure garde les 83 905 lignes ; avec le doublon, elle en produit 85 345, soit 1 440 de plus : le nombre de lignes de commande du produit 1, **comptées deux fois**. Le chiffre d'affaires de ce produit serait doublé sans alerte. L'argument `validate="m:1"` (« plusieurs à un ») dit à pandas ce que l'on attend : plusieurs lignes à gauche pour une seule à droite. Si ce n'est pas le cas, pandas **refuse** et lève une erreur. C'est une assurance qui coûte quelques caractères : prenez l'habitude de l'écrire.

Dans dplyr, l'argument équivalent est `relationship = "many-to-one"`.

```
doublon_r <- bind_rows(produits, head(produits, 1))
cat(nrow(suppressWarnings(left_join(lignes, produits, by = "id_produit"))),
    nrow(suppressWarnings(left_join(lignes, doublon_r, by = "id_produit"))), "\n")
tryCatch(left_join(lignes, doublon_r, by = "id_produit", relationship = "many-to-one"),
  error = function(e) cat("erreur :", conditionMessage(e) |> str_split_1("\n") |> head(1),
    "\n"))
```

```
83905 85345
erreur : Each row in `x` must match at most 1 row in `y`.
```

Mêmes nombres, même refus. (Par défaut, dplyr se contente d'un **avertissement** quand il détecte une relation « plusieurs à plusieurs » ; nous l'avons masqué ici, mais en situation réelle ne l'ignorez jamais.) Reste à utiliser nos jointures. Deux questions que la gérante se pose depuis longtemps : quelle est la **marge** par catégorie, et quelle part des lignes est **retournée** ? On reconnaît un retour par la présence de la ligne dans retours.

```
retourne = lignes[[ "id_ligne" ]].assign(retourne=lignes[ "id_ligne" ].isin(retours[ "id_ligne" ]))
taux_retour = (ventes.merge(retourne, on="id_ligne").groupby("categorie")["retourne"].mean() *
  100).round(1)
marge = ventes.assign(marge=lambda d: d[ "montant" ] - d[ "quantite" ] *
  d[ "cout_achat" ]).groupby("categorie")["marge", "montant"].sum()
cat_tab = pd.DataFrame({"taux_retour_pct": taux_retour, "taux_marge_pct": (marge[ "marge" ] /
  marge[ "montant" ] * 100).round(1)})
print(cat_tab)
```

	taux_retour_pct	taux_marge_pct
categorie		
Bien-être	6.1	45.0
Cuisine	6.1	46.9
Décoration	6.0	49.0
Jardin	6.2	47.5
Maison	5.9	47.4
Papeterie	5.4	46.3

Les taux de retour sont presque identiques d'une catégorie à l'autre (entre 5,4 % et 6,2 %), et le taux de marge varie de 45,0 % pour le bien-être à 49,0 % pour la décoration. Voici le même calcul en R.

```
retourne_r <- lignes |> mutate(retourne = id_ligne %in% retours$id_ligne) |> select(id_ligne,
  retourne)
cat_tab_r <- ventes_r |> left_join(retourne_r, by = "id_ligne") |> group_by(categorie) |>
  summarise(taux_retour_pct = round(100 * mean(retourne), 1),
    taux_marge_pct = round(100 * sum(montant - quantite * cout_achat) / sum(montant), 1))
cat_tab_r
```

```
# A tibble: 6 × 3
  categorie  taux_retour_pct  taux_marge_pct
  <chr>          <dbl>          <dbl>
1 Bien-être      6.1             45
2 Cuisine        6.1            46.9
3 Décoration     6              49
4 Jardin         6.2            47.5
5 Maison         5.9            47.4
6 Papeterie      5.4            46.3
```

identique au résultat de pandas : TRUE

Le taux de retour, presque uniforme **par catégorie**, ne l'est pas du tout **par canal**. Changer le regroupement change l'histoire : c'est la raison d'être de l'analyse.

```
tx_canal = ventes.merge(retourne,
  ↪ on="id_ligne").groupby("canal")["retourne"].mean().mul(100).round(1)
print(tx_canal.to_dict(), "| remboursé :", round(retours["montant_rembourse"].sum()), "€")
```

```
{'Boutique': 3.1, 'Réseaux': 6.7, 'Site': 9.0} | remboursé : 221010 €
```

Un retour sur 11 lignes vendues sur le Site (9,0 %), contre une sur 32 en boutique (3,1 %) et 6,7 % sur les réseaux. Le montant total remboursé sur trois ans est de 221 010 €, soit 6,05 % du chiffre d'affaires : voilà la réponse à la question « combien nous coûtent les retours ? », et l'indication que le **canal**, bien plus que la catégorie, est le levier à examiner.

4.3.4 Restructurer : le format large et le format long

Un même tableau peut s'écrire de deux façons. Le **format large** a une colonne par modalité : une colonne Boutique, une Réseaux, une Site. C'est le format de lecture, celui d'un tableau de synthèse. Le **format long** a une ligne par observation : une ligne par couple (année, canal), avec une colonne canal et une colonne ca. C'est le format de calcul, celui que les graphiques, les modèles et les regroupements préfèrent.

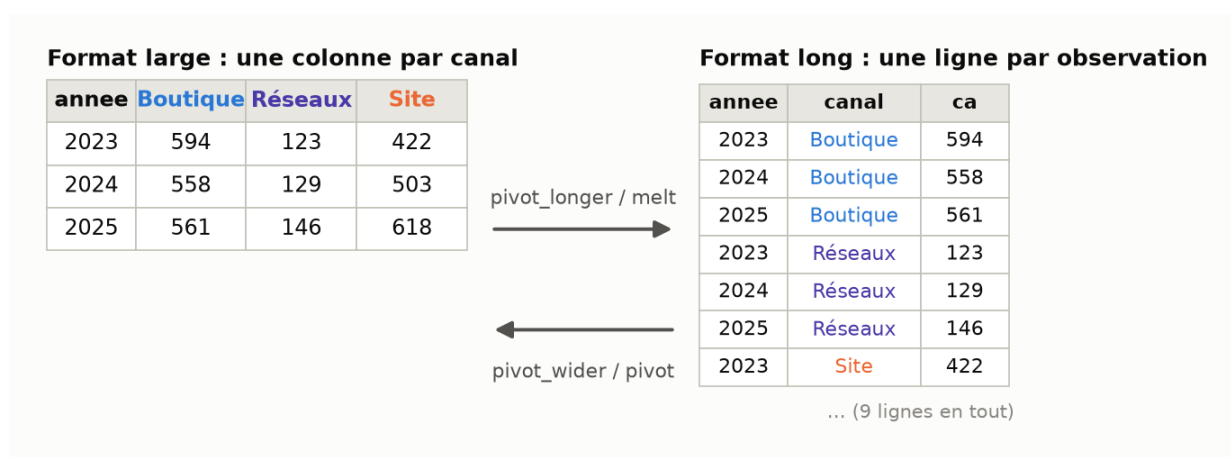


Figure 33: Le même tableau en format large (à gauche) et en format long (à droite) : les noms de colonnes Boutique, Réseaux, Site deviennent des valeurs d'une colonne canal. Chiffres d'affaires en milliers d'€.

On passe de l'un à l'autre avec deux opérations : **pivoter** vers le large (pivot_table dans pandas, pivot_wider dans tidyr) et **dépivoter** vers le long (melt dans pandas, pivot_longer dans tidyr).

```
large = ca_an_canal.pivot_table(index="annee", columns="canal", values="montant",
    ↪ aggfunc="sum").round(0)
long = large.reset_index().melt(id_vars="annee", var_name="canal", value_name="ca")
print(large)
print(long.head(4))
```

```
canal  Boutique  Réseaux  Site
annee
2023   593612.0  122880.0  422440.0
2024   558143.0  128787.0  502531.0
2025   560974.0  146074.0  617715.0
  annee  canal    ca
0  2023  Boutique  593612.0
1  2024  Boutique  558143.0
2  2025  Boutique  560974.0
3  2023  Réseaux  122880.0
```

Le premier résultat est le tableau de synthèse que l'on mettrait dans un rapport ; le second est la même information, rangée pour qu'un graphique ou un calcul s'en empare (une colonne pour l'axe, une pour la couleur, une pour la valeur). Le même aller-retour en R :

```
large_r <- ventes_r |> group_by(annee, canal) |> summarise(ca = round(sum(montant)), .groups =
    ↪ "drop") |>
  pivot_wider(names_from = canal, values_from = ca)
long_r <- large_r |> pivot_longer(-annee, names_to = "canal", values_to = "ca")
large_r
head(long_r, 3)
```

```
# A tibble: 3 × 4
  annee Boutique Réseaux Site
<dbl>   <dbl>   <dbl> <dbl>
1  2023   593612  122880 422440
2  2024   558143  128787 502531
3  2025   560974  146074 617715
# A tibble: 3 × 3
  annee canal    ca
<dbl> <chr>   <dbl>
1  2023 Boutique 593612
2  2023 Réseaux 122880
3  2023 Site    422440
```

Intuition. Si les en-têtes de colonnes sont des **valeurs** (des canaux, des mois, des années), la table est large ; si ce sont des **noms de variables** (le canal, le mois, le montant), elle est longue. Un graphique réclame presque toujours une table longue ; un lecteur humain préfère presque toujours une table large.

4.3.5 Fenêtres glissantes et comparaisons dans le temps

Un chiffre quotidien est bruité : un samedi de pluie, un jour de soldes. Pour voir la tendance, on le **lisse** par une **moyenne glissante** : à chaque jour, la moyenne des sept derniers jours. En pandas, `rolling(7).mean()` la calcule ; avec `center=True`, la fenêtre est centrée sur le jour au lieu de le suivre.

```
jours = jours.assign(ca_7j=jours["chiffre_affaires"].rolling(7).mean(),
                    ca_7j_centre=jours["chiffre_affaires"].rolling(7, center=True).mean())
print(jours.iloc[5:9][["chiffre_affaires", "ca_7j",
    ↪ "ca_7j_centre"]].round(0).assign(date=jours["date"].dt.date))
print(pd.Timestamp("2025-12-31").isocalendar())
```

	chiffre_affaires	ca_7j	ca_7j_centre	date
5	2691.0	NaN	2431.0	2023-01-06
6	2391.0	2027.0	2328.0	2023-01-07
7	1732.0	2162.0	2169.0	2023-01-08
8	3091.0	2431.0	2172.0	2023-01-09

`datetime.IsoCalendarDate(year=2026, week=1, weekday=3)`

Les six premiers jours n'ont **pas** de moyenne glissante « suivante » : il faut sept jours d'historique pour la calculer (d'où le NaN du 6 janvier), alors que la version centrée existe dès le quatrième jour mais nécessite trois jours de **futur**, ce qui l'exclut pour une prévision. Le 7 janvier 2023, le chiffre d'affaires du jour était de 2 391 € alors que la moyenne des sept derniers jours était de 2 027 € : le lissage change la lecture.

Pour comparer une semaine à la même semaine de l'an dernier, il faut une définition de la « semaine ». On utilise la **semaine ISO** : elle commence le lundi et appartient à l'année qui contient son jeudi. La conséquence est surprenante : le **31 décembre 2025** appartient à la **semaine 1 de 2026**. Pour cette raison, une comparaison d'année à année doit utiliser l'**année ISO** (et non l'année du calendrier), faute de quoi les premiers et derniers jours de l'année seraient rangés dans la mauvaise semaine. Calculons le chiffre d'affaires par semaine ISO et par canal.

```
iso = ventes["date_commande"].dt.isocalendar()
ventes = ventes.assign(annee_iso=iso["year"].astype(int), semaine=iso["week"].astype(int))
sem = ventes.groupby(["annee_iso", "semaine", "canal"], as_index=False)["montant"].sum()
print(sem[(sem["semaine"] == 45) & (sem["annee_iso"] >= 2024)])
```

	annee_iso	semaine	canal	montant
290	2024	45	Boutique	12783.90
291	2024	45	Réseaux	2856.28
292	2024	45	Site	12089.19
446	2025	45	Boutique	11561.47
447	2025	45	Réseaux	4581.86
448	2025	45	Site	14096.67

Deux lignes par canal pour la semaine 45 : celle de 2024 et celle de 2025. On a les ingrédients du tableau du lundi.

Piège : décaler de 52 semaines. Pour « la même semaine l'an dernier », on pourrait décaler la série hebdomadaire de 52 lignes. Mais certaines années ISO ont **53** semaines : le décalage dérive alors d'une semaine. Joindre sur le couple (année ISO moins un, numéro de semaine), comme nous allons le faire, est plus sûr.

4.3.6 Le tableau du lundi

Nous pouvons maintenant répondre à la gérante. On écrit une **fonction** qui prend un numéro de semaine et une année, et renvoie le tableau : les ventes par canal de cette semaine, de la même semaine l'année précédente, et l'évolution en pourcentage. Une fonction est une recette réutilisable : c'est ce qui fait passer du calcul **ponctuel** à l'**automatisation**.

```
def tableau_du_lundi(semaine, annee):
    cur = sem[(sem.annee_iso == annee) & (sem.semaine == semaine)].set_index("canal")["montant"]
    pre = sem[(sem.annee_iso == annee - 1) & (sem.semaine == semaine)].set_index("canal")["montant"]
    t = pd.DataFrame({"cette_semaine": cur, "meme_semaine_an_dernier": pre})
    t.loc["Total"] = t.sum()
    t["evolution_pct"] = (t["cette_semaine"] / t["meme_semaine_an_dernier"] - 1) * 100
    return t.round(1)

lundi = tableau_du_lundi(45, 2025)
print(lundi)
```

	cette_semaine	meme_semaine_an_dernier	evolution_pct
canal			
Boutique	11561.5	12783.9	-9.6
Réseaux	4581.9	2856.3	60.4
Site	14096.7	12089.2	16.6
Total	30240.0	27729.4	9.1

La semaine du 3 au 9 novembre 2025 a rapporté 30 240,0 € au total, contre 27 729,4 € la semaine correspondante de 2024 : **+9,1 %**. Mais la moyenne cache des mouvements opposés : la boutique recule de 9,6 % (11 561,5 € contre 12 783,9 €), le Site progresse de 16,6 % et les réseaux de 60,4 %. Ce dernier chiffre impressionne, mais il part d'une **petite base** (2 856,3 € l'an dernier) : +1 725,6 € suffisent, soit quelques commandes de plus. Un analyste ne livre jamais un pourcentage sans le montant qui le porte. Écrivons la même fonction en R.

```
sem_r <- ventes_r |> mutate(annee_iso = isoyear(date_commande), semaine = isoweek(date_commande)) |>
  group_by(annee_iso, semaine, canal) |> summarise(montant = sum(montant), .groups = "drop")
sem_r <- bind_rows(sem_r, sem_r |> group_by(annee_iso, semaine) |> summarise(montant = sum(montant),
  <- canal = "Total", .groups = "drop"))
tableau_du_lundi_r <- function(s, a) {
  sem_r |> filter(semaine == s, annee_iso %in% c(a, a - 1)) |>
    mutate(periode = if_else(annee_iso == a, "cette_semaine", "meme_semaine_an_dernier")) |>
    select(canal, periode, montant) |> pivot_wider(names_from = periode, values_from = montant) |>
    relocate(cette_semaine, .after = canal) |>
    mutate(evolution_pct = (cette_semaine / meme_semaine_an_dernier - 1) * 100,
  <- across(where(is.numeric), \(x) round(x, 1)))
}
lundi_r <- tableau_du_lundi_r(45, 2025)
lundi_r
```

```
# A tibble: 4 × 4
  canal   cette_semaine meme_semaine_an_dernier evolution_pct
<chr>      <dbl>          <dbl>          <dbl>
1 Boutique  11562.         12784.         -9.6
2 Réseaux   4582.          2856.          60.4
3 Site     14097.         12089.          16.6
4 Total    30240          27729.          9.1
```

identique au résultat de pandas : TRUE

Les deux langages produisent le même tableau. Une dernière vérification croisée, d'une autre nature : le chiffre d'affaires de la boutique en semaine 45 de 2025 est de **11 561,47 €** dans la table `sem`, et c'est **exactement** le total que l'export de caisse de la section 4.3.1 annonçait. Le fichier désordonné, remis en état à la main, et les données de la base racontent la même histoire : les

chiffres sont fiables.

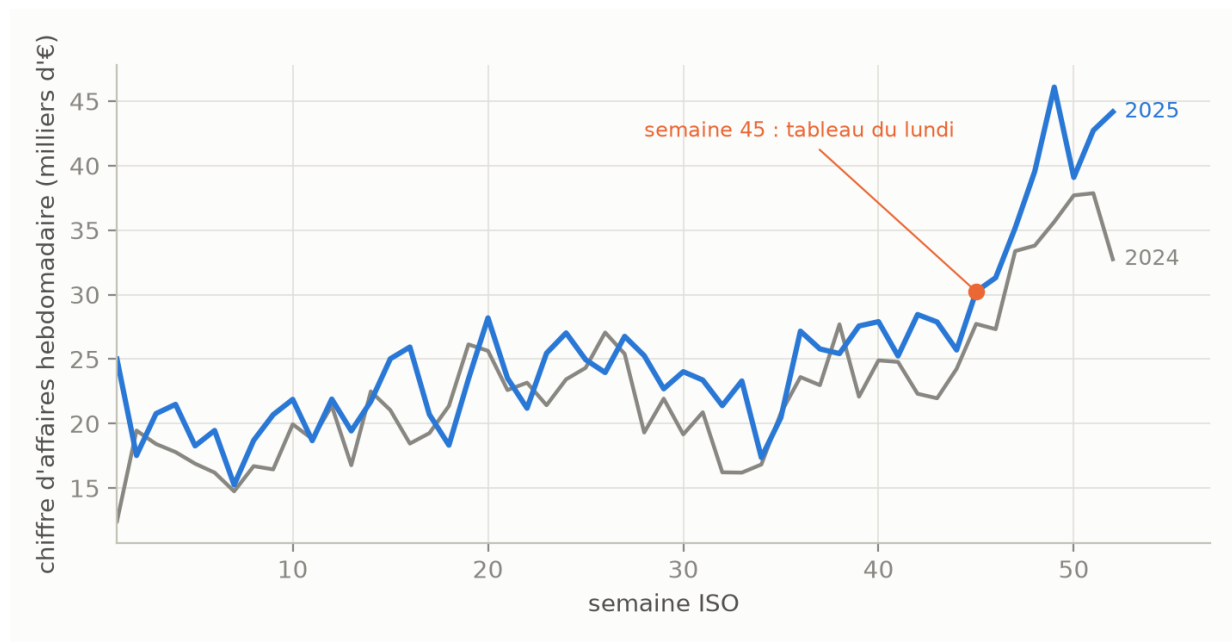


Figure 34: Chiffre d'affaires hebdomadaire total (milliers d'€) par semaine ISO, 2025 contre 2024. Le point orange est la semaine 45, celle du tableau du lundi.

Il reste à « automatiser » : rendre la fonction capable de produire, sans intervention, le tableau de la **semaine qui vient de s'achever**. À partir de la date du jour, `isocalendar()` donne l'année et le numéro de la semaine ISO ; on retranche une semaine pour avoir la semaine **terminée**, puis on appelle la fonction. Le programme complet tient dans un fichier de script, que l'ordinateur lance de lui-même chaque lundi matin grâce à un planificateur de tâches (cron sur Linux, le Planificateur de tâches de Windows).

```
# crontab : chaque lundi à 7 h, exécuter le script qui écrit le tableau dans le dossier partagé
0 7 * * 1 cd /chemin/vers/le/projet && python tableau_du_lundi.py
```

À retenir. Un fichier étranger se lit en trois temps : **regarder**, **lire avec les bons arguments**, **nettoyer** ; et toute transformation se **réconcilie** avec un chiffre connu. Regrouper, c'est répondre à « combien **par** quoi ? » : `groupby + agg` (pandas), `group_by + summarise` (dplyr), `transform` ou `mutate` pour les parts. Une jointure gauche **ajoute** des informations ; vérifiez avec `validate / relationship` qu'elle ne **multiplie** pas les lignes. Le format large se lit, le format long se calcule. Pour comparer des semaines, utilisez les **semaines ISO** et l'**année ISO**, et ne livrez jamais un pourcentage sans le montant qui le porte.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.4 à 4.7, exercices 4.7 à 4.10 et 4.12.

4.4 Notebooks d'analyse

Jusqu'ici, nous avons écrit des morceaux de programme. Reste à les **ranger** quelque part, avec leur explication et leurs résultats, de façon qu'un collègue (ou vous dans six mois) puisse les relire, les relancer et en tirer le même chiffre. L'outil le plus répandu pour cela est le **notebook**. Cette section en explique le principe, son principal piège, et les habitudes qui rendent un notebook fiable.

4.4.1 Qu'est-ce qu'un notebook ?

Un **notebook** est un document fait d'une suite de **cellules**. Une cellule de **texte** (en Markdown) contient une explication, un titre, une conclusion. Une cellule de **code** contient quelques lignes de programme ; quand on l'exécute, son **résultat** (un nombre, un tableau, un graphique) s'affiche juste en dessous et est conservé dans le document. Le tout forme un récit : question, méthode, résultat, commentaire. Le notebook de **Jupyter** est le format standard pour Python (il sait aussi exécuter R) ; **R Markdown** et **Quarto** sont les équivalents côté R, avec des documents texte qui se « tricotent » en rapport (nous les voyons en 4.4.4).

Derrière un notebook Jupyter se cache un **noyau** (*kernel*) : un programme Python qui reste allumé et **garde en mémoire** les variables au fil des cellules. C'est ce qui rend l'outil si agréable pour explorer (on charge les données une fois, on les interroge dix fois) ; c'est aussi la source de son principal piège.

4.4.2 Le piège de l'état caché

Le noyau se souvient de ce que vous avez exécuté, **dans l'ordre où vous l'avez exécuté**, pas dans l'ordre où les cellules sont écrites. Rien ne vous empêche d'exécuter la cellule 5, puis la 2, puis de modifier la 3 sans la relancer. À la fin, l'écran montre des résultats qui ne correspondent plus au code affiché : on parle d'**état caché**. Le notebook semble marcher, mais un collègue qui l'exécute de haut en bas obtient un autre résultat, ou une erreur.

Pour le montrer sans rien exagérer, construisons des notebooks **par programme** et exécutons-les pour de bon dans un noyau neuf, avec la bibliothèque `nbformat` (qui construit le fichier) et `nbclient` (qui l'exécute). Une fonction d'aide fait ce travail ; elle prend une liste de cellules et renvoie le notebook exécuté.

```
cellules = [
    ("md", "# Chiffre d'affaires 2025"),
    ("code", "import pandas as pd\nl = pd.read_csv('donnees/lignes_commande.csv')\nnc =
↪ pd.read_csv('donnees/commandes.csv', parse_dates=['date_commande']),
    ("code", "v = l.merge(c, on='id_commande')\nca = v.loc[v['date_commande'].dt.year == 2025,
↪ 'montant'].sum()),
    ("code", "print(round(ca, 2))"),
]
nb, erreur = 0.executer_notebook(cellules, os.getcwd())
print(0.sorties_texte(nb)[-1], erreur)
```

1324763.72 None

Exécuté **de haut en bas** dans un noyau neuf, le notebook donne le chiffre d'affaires de 2025 que nous avons calculé en 4.0, au centime près : c'est le test de reproductibilité de base. Voyons maintenant ce que donne l'ordre d'exécution. Trois cellules : A fixe une remise de 10 %, B calcule le prix après remise, C change la remise à 20 %.


```
A, B, C = ("code", "remise = 0.10"), ("code", "print(round(100 * (1 - remise), 2))"), ("code", "remise
↪ = 0.20")
nb1, _ = 0.executer_notebook([A, B, C], os.getcwd())
nb2, _ = 0.executer_notebook([A, C, B], os.getcwd())
print("ordre A, B, C :", 0.sorties_texte(nb1)[1], "| ordre A, C, B :", 0.sorties_texte(nb2)[2])
```

```
ordre A, B, C : 90.0 | ordre A, C, B : 80.0
```

Le même code, deux ordres d'exécution, deux résultats : 90 et 80. Si vous avez exécuté C avant B sans y penser, l'écran affiche 80 alors que le document, relu de haut en bas, donne 90. Un autre scénario courant est la **variable disparue** : on supprime la cellule qui la définissait, mais le noyau la garde en mémoire, et tout continue de fonctionner... jusqu'au jour où quelqu'un d'autre ouvre le fichier.

```
nb3, erreur = 0.executer_notebook([("code", "print(taux_tva)"), ("code", "taux_tva = 0.2")],
↪ os.getcwd())
print(erreur)
```

```
NameError: name 'taux_tva' is not defined
```

Piège : un notebook n'est reproductible que s'il s'exécute de haut en bas, dans un noyau neuf. C'est le seul test qui compte. Avant de partager ou d'utiliser un notebook, faites toujours « **Redémarrer le noyau et tout exécuter** » (*Restart and Run All*). Si une erreur apparaît, ou si un chiffre change, c'est qu'un état caché vous cachait un défaut.

4.4.3 Les habitudes d'un notebook fiable

Quelques règles simples évitent la plupart des ennuis. Elles valent aussi pour les scripts, mais le notebook, plus souple, a besoin qu'on se les impose.

À faire	À éviter
Une question par notebook , annoncée en tête, avec la conclusion écrite en clair	Un notebook fourre-tout où l'on a empilé trois ans d'explorations
Les données en entrée (un chemin relatif, <code>donnees/ventes.csv</code>) et les résultats en sortie (un fichier écrit)	Un chemin absolu qui n'existe que sur votre poste (<code>C:\Users\...</code>)
Importer toutes les bibliothèques et lire toutes les données en haut	Un <code>import</code> caché au milieu, qui plante pour qui n'a pas la bibliothèque
Fixer les graines aléatoires et noter les versions des bibliothèques	Un résultat qui change à chaque exécution sans que personne ne sache pourquoi
Sortir les fonctions réutilisables dans un fichier <code>.py</code> que le notebook importe	Copier-coller la même fonction dans dix notebooks
Aucun secret dans un notebook (mot de passe, clé d'accès) : variables d'environnement	Un mot de passe écrit dans une cellule, puis partagé par courriel
Effacer les sorties avant de partager si elles contiennent des données sensibles	Envoyer un notebook dont les sorties montrent les noms de clients

Un dernier point pratique : un notebook est, en dessous, un fichier au format JSON, mélange de code et de résultats. Deux versions d'un même notebook se **comparent mal** avec un outil de gestion de versions comme Git. Une bonne pratique est de n'archiver que la version avec les sorties effacées, ou d'utiliser un outil qui tient le notebook synchronisé avec un script texte (par exemple Jupyter, non installé ici : à vérifier selon votre environnement).

4.4.4 R Markdown et Quarto : écrire le rapport comme un programme

Côté R, la tradition est différente : on écrit un **document texte** (Markdown) où l'on insère des morceaux de code R, et un programme (knitr, ou Quarto qui le généralise) **exécute** le document et fabrique le rapport final : le code est exécuté, ses résultats sont inclus. Le document est **toujours** exécuté de haut en bas dans une session neuve : le piège de l'état caché disparaît. Voici à quoi ressemble un tel document (le texte entre accolades, {r}, ouvre un bloc de code R) :

```
---
title: "Ventes 2025"
---
Le chiffre d'affaires de 2025 vaut `r round(ca, 2)` €.

```{r}
ventes |> filter(year(date_commande) == 2025) |> count(canal)
```
```

Nous pouvons faire fabriquer un tel rapport par knitr. Le bloc suivant construit le document dans un dossier temporaire, le fait « tricoter », et nous montre le texte obtenu.

```
Le chiffre d'affaires de 2025 vaut 1324763.72 €.
## # A tibble: 3 × 2
##   canal      n
##   <chr>   <int>
## 1 Boutique 12611
## 2 Réseaux  3288
## 3 Site    13928
```

Le texte est devenu un rapport (nous n'en montrons que les lignes utiles : la mise en forme du code et des tableaux a été retirée) : la phrase d'ouverture contient le chiffre d'affaires **calculé**, 1 324 763,72 €, et le tableau des commandes de 2025 par canal est celui que produit le code (12 611 commandes en boutique, 3 288 sur les réseaux, 13 928 sur le Site). Si les données changent, il suffit de relancer : le rapport se met à jour tout seul, sans recopier un seul nombre. C'est la forme la plus aboutie de la **reproductibilité** : le rapport **est** le programme. **Quarto** (quarto render rapport.qmd) offre le même principe pour Python, R et d'autres langages, avec des sorties en HTML, PDF ou Word ; il n'est pas installé sur la machine qui a produit ce livre, et la commande n'est donc pas exécutée ici.

```
quarto render rapport.qmd --to html      # non exécuté ici : Quarto n'est pas installé
```

4.4.5 Quand quitter le notebook ?

Le notebook est le bon outil pour **explorer** et pour **raconter** une analyse. Il est moins bon pour **automatiser** : une tâche qui doit tourner chaque lundi sans personne devant l'écran s'écrit plutôt en **script** (un fichier .py ou .R) que l'on appelle depuis un planificateur, comme au 4.3.6. La démarche classique : on explore dans un notebook, on **extraît** les parties stables dans des fonctions (un fichier de code), puis on garde le notebook pour la narration. Garder cette séparation évite les deux écueils : le script illisible et le notebook fragile.

À retenir. Un notebook mêle texte, code et résultats ; son noyau garde les variables **dans l'ordre d'exécution**, pas dans l'ordre d'écriture : c'est l'**état caché**. Le seul test de fiabilité est « **redémarrer et tout exécuter** ». Un bon notebook répond à **une** question, lit ses données en entrée par un chemin relatif, n'embarque aucun secret et sort ses fonctions réutilisables. Côté R, R Markdown et Quarto exécutent le document **de haut en bas** et font du rapport le

programme lui-même.

Pour s'entraîner. Cahier, chapitre 4 : application 4.8, exercice 4.11.

4.5 ⊕ R : ggplot2 et Shiny

Section complémentaire. Elle présente les deux paquets R qui font la réputation du langage pour la communication : **ggplot2** pour les graphiques et **Shiny** pour les applications interactives. Rien de ce qui suit n'est nécessaire à la suite du volume. Le volume IV de cette série est consacré à la visualisation et à la communication ; cette section en donne un avant-goût côté R.

4.5.1 La grammaire des graphiques

ggplot2 repose sur une idée : un graphique n'est pas un « type » à choisir dans un menu (histogramme, courbe, camembert), c'est un **assemblage de couches**, que l'on décrit avec les mêmes briques à chaque fois.

| Brique | Rôle | Exemple |
|---|--|--|
| Données | la table à représenter
(idéalement en format long , voir 4.3.4) | <code>ggplot(mensuel, ...)</code> |
| Esthétiques (<code>aes</code>) | relier des colonnes à des propriétés visuelles : position, couleur, taille | <code>aes(x = mois, y = ca, colour = canal)</code> |
| Géométries (<code>geom_*</code>) | la forme dessinée : ligne, barre, point | <code>geom_line()</code> , <code>geom_col()</code> |
| Échelles (<code>scale_*</code>) | comment traduire une valeur en couleur ou en position | <code>scale_colour_manual(values = ...)</code> |
| Facettes (<code>facet_*</code>) | découper en petits graphiques, un par modalité | <code>facet_wrap(~categorie)</code> |
| Thème (<code>theme_*</code>) | l'habillage : fond, grille, police | <code>theme_minimal()</code> |

On assemble les couches avec le signe `+`. Représentons le chiffre d'affaires mensuel de chaque canal. Les données sont en format long (une ligne par mois et par canal), et la couleur est reliée à la colonne canal.

```
mensuel <- ventes_r |> mutate(mois = floor_date(date_commande, "month")) |>
  group_by(mois, canal) |> summarise(ca = sum(montant) / 1000, .groups = "drop")
palette <- c(Boutique = "#2a78d6", Réseaux = "#4a3aa7", Site = "#eb6834")
g <- ggplot(mensuel, aes(x = mois, y = ca, colour = canal)) +
  geom_line(linewidth = 1) +
  scale_colour_manual(values = palette) +
  labs(x = NULL, y = "chiffre d'affaires (milliers d'€)", colour = NULL) +
  theme_minimal(base_size = 11) + theme(legend.position = "top")
ggsave("figures/ch04-ggplot-mensuel.png", g, width = 8, height = 3.6, dpi = 200)
```

Chaque ligne du code correspond à une décision lisible : `aes` relie le mois à l'axe horizontal, le montant à l'axe vertical, le canal à la couleur ; `geom_line` dessine des lignes ; `scale_colour_manual`

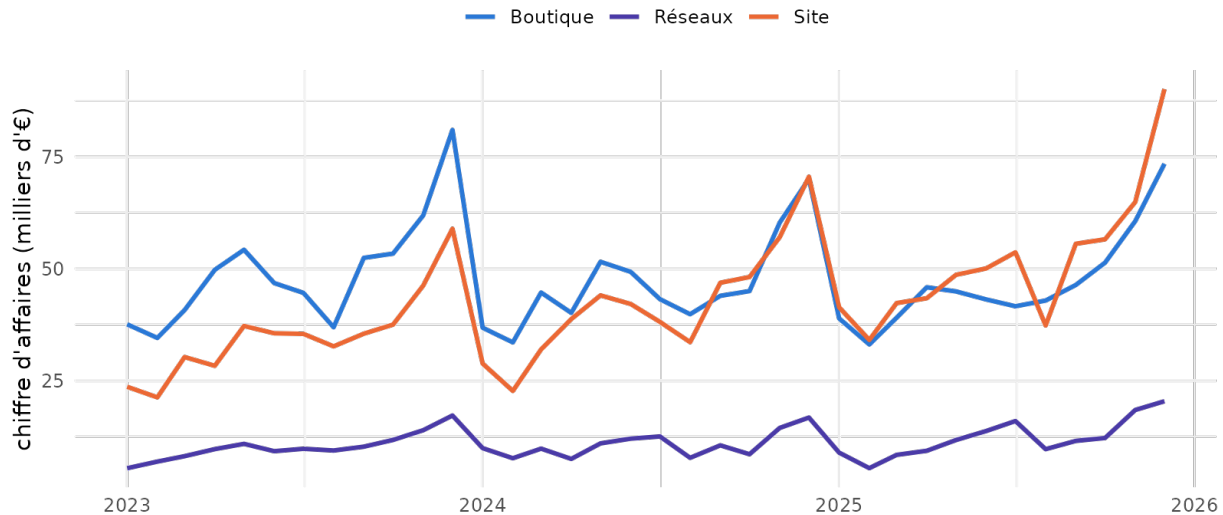


Figure 35: Chiffre d'affaires mensuel (milliers d'€) par canal, de 2023 à 2025, tracé avec ggplot2. Le pic de fin d'année est visible chaque année ; le Site rattrape puis dépasse la boutique.

impose notre palette ; labs nomme les axes ; theme_minimal allège le fond. Le graphique se lit en quelques secondes : une saison très marquée, avec un pic chaque fin d'année, et un Site qui rejoint la boutique. Vérifions ce dernier point par le calcul plutôt que par l'œil : en quel mois le Site a-t-il dépassé la boutique pour la première fois, et combien de mois sur 36 est-il devant ?

```
devant <- mensuel |> pivot_wider(names_from = canal, values_from = ca) |> filter(Site > Boutique)
cat(format(devant$mois[1], "%Y-%m"), nrow(devant), "\n")
```

```
2024-09 13
```

Le Site passe devant en septembre 2024 pour la première fois, et il est en tête 13 mois sur 36. Un graphique ne remplace pas le chiffre : il **oriente** vers celui qu'il faut vérifier.

4.5.2 Facettes : plusieurs petits graphiques

Quand on veut comparer plusieurs groupes, plutôt que d'empiler six courbes sur le même graphique, on les sépare en **petits multiples** : une facette par catégorie, tous à la même échelle. Le code ne change presque pas : une couche facet_wrap suffit. Nous représentons le chiffre d'affaires de chaque catégorie en 2025, mois par mois.

```
par_cat <- ventes_r |> filter(annee == 2025) |> mutate(mois = month(date_commande)) |>
  group_by(categorie, mois) |> summarise(ca = sum(montant) / 1000, .groups = "drop")
g2 <- ggplot(par_cat, aes(x = mois, y = ca)) +
  geom_col(fill = "#2a78d6", width = 0.8) +
  facet_wrap(~categorie, ncol = 3) +
  scale_x_continuous(breaks = c(1, 4, 7, 10)) +
  labs(x = "mois de 2025", y = "milliers d'€") +
  theme_minimal(base_size = 10)
ggsave("figures/ch04-ggplot-facettes.png", g2, width = 8, height = 4.2, dpi = 200)
```

On voit, d'un coup d'œil, des profils différents : le jardin culmine en été (juillet), toutes les autres catégories culminent en décembre, et la décoration avec un pic particulièrement marqué. Cette comparaison par petits multiples est l'un des gestes les plus utiles de la visualisation : elle garde la même échelle partout, et évite à l'œil de comparer des couleurs.

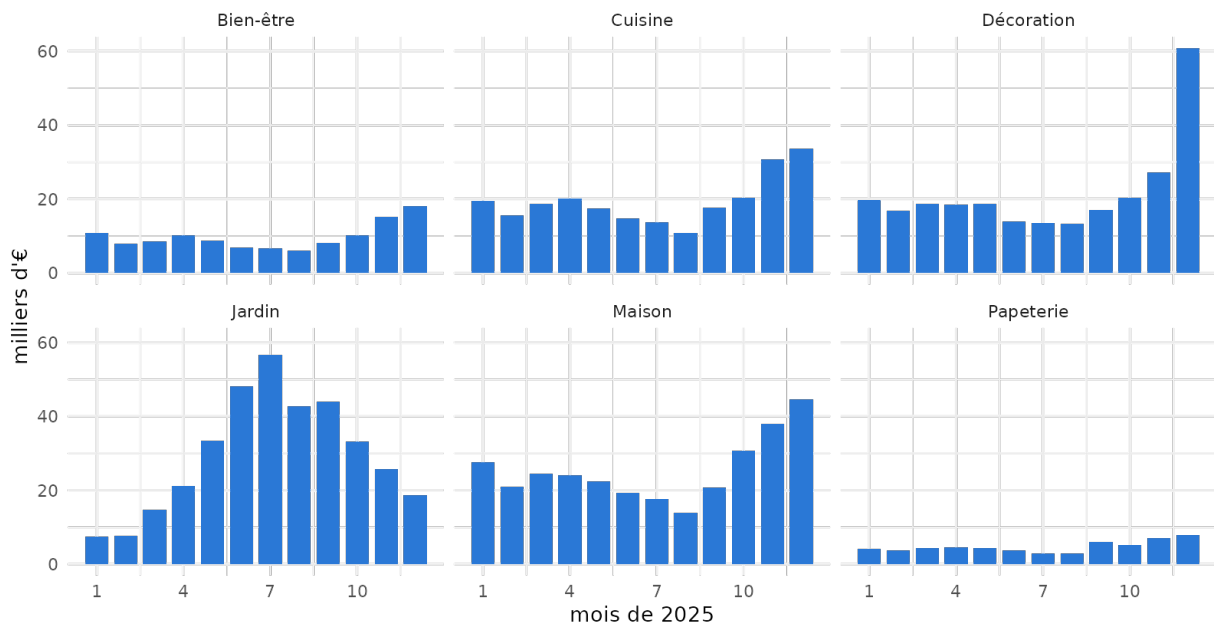


Figure 36: Chiffre d'affaires mensuel de 2025 par catégorie (milliers d'€), une facette par catégorie, tracé avec ggplot2.

Intuition. La grammaire des graphiques ressemble à la grammaire d'une langue : une fois les briques apprises (données, esthétiques, géométries, échelles, facettes, thème), on peut dire des choses nouvelles sans apprendre de nouveaux mots. C'est ce qui fait la force de ggplot2, et la raison pour laquelle des équivalents ont été écrits en Python (plotnine, altair). En Python, la voie habituelle passe par matplotlib et seaborn, dont le fonctionnement est plus **impératif** (on dessine étape par étape).

4.5.3 Shiny : une application interactive

Un graphique figé répond à une question. Une **application** laisse l'utilisateur poser lui-même ses questions : choisir un canal, une période, une catégorie, et voir le résultat changer. **Shiny** permet de construire ces applications en R, sans connaître le web. Une application Shiny a deux parties : une **interface** (ui), qui décrit ce que l'utilisateur voit (une liste déroulante, un texte), et un **serveur** (server), qui décrit comment les résultats se calculent à partir de ce que l'utilisateur choisit. Entre les deux, la **réactivité** : quand l'utilisateur change son choix, seuls les résultats qui en dépendent sont recalculés.

```
library(shiny)
app <- shinyApp(
  ui = fluidPage(selectInput("canal", "Canal", c("Boutique", "Site", "Réseaux")), textOutput("ca")),
  server = function(input, output, session) {
    ca <- reactive(sum(ventes_r$montant[ventes_r$canal == input$canal & ventes_r$annee == 2025]))
    output$ca <- renderText(paste0("CA 2025 : ", format(round(ca()), big.mark = " "), " €"))
  })
class(app)
```

```
[1] "shiny.appobj"
```

La dernière ligne confirme que l'on a bien construit un objet « application », qui n'est pas lancé. L'interface déclare une liste déroulante `canal` et une zone de texte `ca`. Le serveur déclare un calcul **réactif** (`ca`, qui dépend de `input$canal`) et une sortie (`output$ca`) qui l'affiche. La commande

`runApp(app)` démarrerait un serveur local et ouvrirait l'application dans un navigateur ; nous ne le faisons pas ici. Mais la **logique** de l'application peut se tester sans navigateur, avec `testServer`, qui joue le rôle de l'utilisateur : il choisit un canal, puis lit ce qui s'affiche.

```
testServer(app, {
  session$setInputs(canal = "Site");   cat(output$ca, "\n")
  session$setInputs(canal = "Boutique"); cat(output$ca, "\n")
})
```

```
CA 2025 : 617 715 €
CA 2025 : 560 974 €
```

Les deux chiffres sont ceux de 4.3.4 : 617 715 € pour le Site et 560 974 € pour la boutique. Tester la logique d'une application sans l'ouvrir est une bonne habitude : on y vérifie les **calculs**, qui sont l'essentiel de la responsabilité de l'analyste. Ce test ne voit pas l'**aspect** de l'application ni sa rapidité perçue, qui se jugent à l'œil.

En pratique. Pour partager une application Shiny, il faut l'héberger sur un serveur (service payant ou serveur de l'entreprise : à vérifier selon votre organisation). Si le besoin est seulement de **montrer** des résultats, un rapport HTML (section 4.4.4) ou un tableau de bord (volume IV) suffit souvent. Pour une application en Python, des outils comme Streamlit jouent le même rôle ; le volume IV de la série 1 (chapitre 7) en détaille le fonctionnement.

À retenir. `ggplot2` décrit un graphique comme un **assemblage de couches** (données, esthétiques, géométries, échelles, facettes, thème) reliées par `+` ; les données doivent être en **format long**. Les **facettes** comparent des groupes à la même échelle. Shiny sépare l'**interface** et le **serveur** et recalcule uniquement ce qui dépend d'un choix de l'utilisateur ; la logique se teste sans navigateur avec `testServer`. Un graphique **oriente** vers un chiffre, mais ne le remplace pas.

Pour s'entraîner. Cahier, chapitre 4 : application 4.9.

4.6 ⊕ Python : NumPy, polars, rapports avec Jupyter

Section complémentaire. Elle regarde sous le capot de pandas (NumPy, le moteur de calcul), présente une alternative plus rapide (polars) et montre comment un notebook devient un rapport automatique. Rien de ce qui suit n'est nécessaire à la suite du volume.

4.6.1 NumPy : les tableaux et la vectorisation

Chaque colonne d'un DataFrame pandas repose sur un **tableau NumPy** : une suite de nombres de même type, rangée de façon compacte en mémoire. NumPy est la bibliothèque de base du calcul scientifique en Python, et on l'utilise directement pour des calculs numériques purs. Son idée-clé est la **vectorisation** : une opération s'applique à **tout le tableau d'un coup**, sans écrire de boucle.

```
q = lignes["quantite"].to_numpy()
p = lignes["prix_unitaire"].to_numpy()
r = lignes["remise_pct"].to_numpy()
```

```
montants = q * p * (1 - r / 100)
print(montants.shape, round(montants.sum(), 2), round(lignes["montant"].sum(), 2))
```

```
(83905,) 3653161.35 3653157.28
```

Les trois colonnes sont multipliées **ligne par ligne** en une seule expression, et le résultat est un nouveau tableau de 83 905 valeurs. La somme obtenue (3 653 161,35 €) est celle des montants du fichier (3 653 157,28 €) à 4 € près, soit un millionième : l'effet cumulé de l'arrondi au centime de chaque montant dans le fichier. Pourquoi la vectorisation compte-t-elle ? Parce qu'une boucle Python traite les nombres **un par un**, alors qu'une opération vectorisée délègue le travail à du code compilé, beaucoup plus rapide. Comparons, sur un million de nombres, une boucle et son équivalent vectorisé ; nous ne citons pas de durées (elles dépendent de la machine), seulement un test.

```
import time
x = np.random.default_rng(0).random(1_000_000)
def meilleur_temps(f, n=3):
    essais = []
    for _ in range(n):
        t = time.perf_counter(); f(); essais.append(time.perf_counter() - t)
    return min(essais)
somme_boucle, somme_vect = sum(v * 2 for v in x), (x * 2).sum()
print(np.isclose(somme_boucle, somme_vect), meilleur_temps(lambda: sum(v * 2 for v in x)) > 5 *
      meilleur_temps(lambda: (x * 2).sum()))
```

```
True True
```

Les deux sommes sont égales, et la version vectorisée est plus de cinq fois plus rapide (en pratique, de l'ordre de vingt à cent fois). Le message pratique est simple : **quand vous écrivez une boucle sur les lignes d'une table, cherchez l'opération vectorisée équivalente**. pandas et NumPy en ont pour presque tous les besoins.

NumPy fournit aussi des fonctions statistiques directes (mean, std, percentile, corrcoef) et where, qui est le « si » vectorisé. Utilisons-le pour repérer les **jours exceptionnels** : ceux dont le chiffre d'affaires dépasse la moyenne d'un écart-type.

```
ca = jours["chiffre_affaires"].to_numpy()
seuil = ca.mean() + ca.std()
fort = np.where(ca > seuil, "fort", "normal")
print(round(ca.mean(), 1), round(ca.std(), 1), round(seuil, 1), (fort == "fort").sum())
print(round(jours.loc[fort == "fort", "date"].dt.month.isin([11, 12]).mean() * 100))
```

```
3333.2 1329.2 4662.4 172
58
```

Le chiffre d'affaires moyen d'un jour est de 3 333,2 €, avec un écart-type de 1 329,2 € ; 172 jours sur 1 096 dépassent 4 662,4 €. Plus de la moitié d'entre eux (58 %) tombent en novembre et en décembre : c'est la saison de fin d'année que la courbe hebdomadaire de la section 4.3.6 faisait déjà apparaître. (Le volume suivant de cette série, sur la préparation des données, reprend la question : un jour « exceptionnel » est-il une **erreur** ou un **événement** ?)

Piège : random sans graine. Tout calcul qui tire des nombres au hasard doit fixer une **graine** (`np.random.default_rng(0)`) si l'on veut retrouver le même résultat à la prochaine exécution. Sans graine, deux exécutions du même notebook donneraient deux chiffres différents, et personne ne saurait lequel croire.

4.6.2 polars : une alternative rapide à pandas

polars est une bibliothèque plus récente que **pandas**, écrite pour la vitesse : elle utilise tous les cœurs du processeur, et peut optimiser un calcul **avant** de l'exécuter. Elle se distingue par une syntaxe fondée sur des **expressions** : on décrit ce que l'on veut calculer avec `pl.col("nom")`, et **polars** décide comment l'exécuter. Voici le calcul du chiffre d'affaires par année et par canal, qui est celui de 4.3.2.

```
import polars as pl
pl.Config.set_tbl_formatting("MARKDOWN")
pl.Config.set_tbl_hide_dataframe_shape(True)
lig_pl = pl.read_csv("donnees/lignes_commande.csv")
cmd_pl = pl.read_csv("donnees/commandes.csv", try_parse_dates=True)
res_pl = (lig_pl.join(cmd_pl, on="id_commande")
          .with_columns(annee=pl.col("date_commande").dt.year())
          .group_by("annee", "canal").agg(ca=pl.col("montant").sum().round(0))
          .sort("annee", "canal"))
print(res_pl.head(4))
```

| annee | canal | ca |
|-------|----------|----------|
| --- | --- | --- |
| i32 | str | f64 |
| ----- | ----- | ----- |
| 2023 | Boutique | 593612.0 |
| 2023 | Réseaux | 122880.0 |
| 2023 | Site | 422440.0 |
| 2024 | Boutique | 558143.0 |

On reconnaît chaque étape : jointure (`join`), colonne calculée (`with_columns`), regroupement (`group_by` puis `agg`), tri (`sort`). Contrairement à **pandas**, **polars** n'a **pas d'index** de ligne, et affiche le **type** de chaque colonne sous son nom. Les chiffres sont ceux de **pandas**, ce que l'on peut vérifier plutôt que d'en croire la parole :

```
pd_ca = large.reset_index().melt(id_vars="annee", var_name="canal",
    ↪ value_name="ca").sort_values(["annee", "canal"])
print(np.allclose(res_pl["ca"].to_numpy(), pd_ca["ca"].to_numpy()))
```

```
True
```

Le grand intérêt de **polars** est le mode **paresseux** (*lazy*). Au lieu de lire le fichier et de calculer tout de suite, `scan_csv` ne fait que **décrire** le calcul : rien n'est exécuté avant l'appel à `collect`. **Polars** peut alors regarder tout le plan, n'en lire que les colonnes utiles et supprimer les étapes inutiles. Sur un fichier de plusieurs gigaoctets, cette optimisation fait la différence entre un calcul qui tient en mémoire et un qui n'y tient pas.

```
plan = (pl.scan_csv("donnees/lignes_commande.csv")
        .group_by("id_produit").agg(ca=pl.col("montant").sum().round(2))
        .sort("ca", descending=True).head(3))
print(plan.collect())
```

| id_produit | ca |
|------------|-----------|
| --- | --- |
| i64 | f64 |
| ----- | ----- |
| 86 | 175236.24 |
| 87 | 128420.16 |
| 82 | 127162.57 |

Les trois produits qui rapportent le plus (les numéros 86, 87 et 82) sont obtenus sans avoir chargé les colonnes dont le calcul n'a pas besoin. Les deux bibliothèques ont leur place : **pandas** est partout, très bien documenté, et suffit pour la grande majorité des analyses ; **polars** brille sur de

gros volumes ou quand la vitesse compte. Savoir les lire toutes deux est un atout.

| Geste | pandas | polars |
|----------------------------|---------------------------------------|---|
| Lire | <code>pd.read_csv("f.csv")</code> | <code>pl.read_csv("f.csv"),</code>
<code>pl.scan_csv(paresseux)</code> |
| Filtrer | <code>df[df["a"] > 3]</code> | <code>df.filter(pl.col("a") > 3)</code> |
| Créer une colonne | <code>df.assign(c=...)</code> | <code>df.with_columns(c=...)</code> |
| Regrouper | <code>df.groupby("g").agg(...)</code> | <code>df.group_by("g").agg(...)</code> |
| Trier | <code>df.sort_values("a")</code> | <code>df.sort("a")</code> |
| Exécuter un plan paresseux | (sans objet) | <code>.collect()</code> |

4.6.3 Des rapports avec Jupyter

Un notebook exécuté contient déjà le texte, le code et les résultats : il suffit de le **convertir** pour obtenir un rapport lisible par quelqu'un qui n'a ni Python ni Jupyter. La bibliothèque `nbconvert` transforme un notebook en page HTML. Reprenons le notebook de 4.4.2, exécuté, et fabriquons son rapport.

```
from nbconvert import HTMLExporter
nb_rapport, _ = 0.executer_notebook(cellules, os.getcwd())
html, _ = HTMLExporter().from_notebook_node(nb_rapport)
print(len(html) > 10_000, "Chiffre d'affaires 2025" in html, "1324763.72" in html)
```

```
True True True
```

La page HTML produite contient le titre du notebook, et le chiffre calculé par la cellule de code. En ligne de commande, la même opération s'écrit en une ligne, qui exécute le notebook **puis** le convertit. Associée à un planificateur de tâches (voir 4.3.6), elle produit un rapport à jour chaque lundi sans que personne n'ouvre Jupyter.

```
jupyter nbconvert --to html --execute rapport.ipynb      # non exécuté ici : l'écriture du fichier se
↳ fait hors du livre
```

Pour qu'un même notebook serve à plusieurs semaines ou plusieurs canaux, on lui donne des **paramètres** : une cellule de tête fixe la semaine et l'année, qu'un outil (par exemple `papermill`, non installé ici : à vérifier selon votre environnement) remplace avant l'exécution. C'est le principe du tableau du lundi : une seule recette, des paramètres qui changent, des rapports qui se fabriquent seuls.

À retenir. NumPy est le moteur de calcul de pandas : une opération **vectorisée** s'applique à tout un tableau d'un coup, bien plus vite qu'une boucle ; fixez toujours une **graine** pour les tirages au hasard. polars est une alternative rapide, fondée sur des **expressions** et un mode **paresseux** qui optimise le calcul avant de l'exécuter. `nbconvert` transforme un notebook exécuté en rapport HTML, que l'on peut automatiser.

Pour s'entraîner. Cahier, chapitre 4 : application 4.10, exercice 4.13.

Bilan du chapitre 4

Vous savez maintenant :

- **lire** un fichier avec le bon séparateur, la bonne décimale, le bon encodage et les bonnes dates, en Python (`read_csv`) comme en R (`read_csv`, `read_delim`) ; **remettre en état** un export désordonné (titres, total, en-têtes répétés, manquants, textes) et **réconcilier** le résultat avec un total connu ;
- **sélectionner, filtrer, calculer, trier et compter** avec pandas (`[]`, `.loc`, `.iloc`, `query`, `assign`, `value_counts`) et avec le tidyverse (`select`, `filter`, `mutate`, `arrange`, `count`), et traduire un langage dans l'autre ;
- **regrouper** (`groupby` et `agg`, `group_by` et `summarise`, `transform` pour les parts), **joindre** sans multiplier les lignes (`validate`, `relationship`) et **restructurer** entre format large et format long (`pivot_table`, `melt`, `pivot_wider`, `pivot_longer`) ;
- **comparer** des périodes avec des moyennes glissantes et des semaines ISO, et **écrire une fonction** qui produit le tableau du lundi ;
- **ranger** une analyse dans un notebook **reproductible** (état caché, « tout exécuter »), produire un rapport avec R Markdown ou Quarto, et reconnaître le moment où passer du notebook au script ;
- (en option) **dessiner** avec ggplot2 (couches, facettes), tester la logique d'une application Shiny, **vectoriser** un calcul avec NumPy, lire un fichier avec polars, convertir un notebook en rapport HTML.

Le chapitre a mis des chiffres sur la boutique, et chacun a été **obtenu de deux façons** :

| Question | Ce que nous avons mesuré |
|---|---|
| Chiffre d'affaires de 2025 | 1 324 763,72 €, identique en pandas et en R |
| Export de caisse | 285 lignes lues, 280 ventes retenues, 8 montants à reconstituer ; écart de 2,62 € avec le total, expliqué par une remise de 5 % |
| Panier moyen par canal | 100,9 € (boutique), 100,5 € (réseaux), 99,8 € (Site) ; le montant moyen d'une ligne est de 43,5 € |
| Part du Site dans le chiffre d'affaires | de 37,1 % en 2023 à 46,6 % en 2025 ; le Site passe devant la boutique en septembre 2024 |
| Taux de retour par canal | 3,1 % (boutique), 6,7 % (réseaux), 9,0 % (Site) ; 221 010 € remboursés, soit 6,05 % du chiffre d'affaires |
| Tableau du lundi, semaine 45 de 2025 | 30 240,0 € contre 27 729,4 € (+9,1 %) ; boutique -9,6 %, Site +16,6 %, réseaux +60,4 % sur une petite base |
| Notebook | le même code donne 90 ou 80 selon l'ordre d'exécution des cellules |

Trois idées dépassent ce chapitre. **D'abord, écrire une analyse comme un programme la rend refaisable** : la gérante obtient son tableau chaque lundi, vous gagnez une heure, et l'erreur de ligne de la collègue disparaît. **Ensuite, la vérification croisée est un réflexe** : un chiffre obtenu par deux outils indépendants (pandas et R, la base et l'export de caisse) mérite confiance ; un écart est une information. **Enfin, un pourcentage sans le montant qui le porte, ou une moyenne sans la précision de ce qui est moyenné, trompe** : la moyenne d'une ligne n'est pas celle d'une commande, et +60 % sur une petite base n'est pas une tendance.

En pratique : liste de contrôle avant de livrer un chiffre.

1. Les données ont été **lues avec les bons arguments**, et on les a **regardées** (head, dtypes, glimpse).
2. Les **manquants** ont été comptés, et l'on sait pourquoi ils manquent.
3. Chaque **jointure** a été validée (validate, relationship) : le nombre de lignes n'a pas bougé sans raison.
4. Un **total connu** a été retrouvé (un export, un tableau de bord, un autre outil).
5. Le calcul a été **refait** dans un second outil, ou par une seconde méthode.
6. Le notebook ou le script s'exécute **de haut en bas** dans une session neuve.
7. Le chiffre est accompagné de **ce qu'il mesure** (moyenne de quoi, sur quelle période, avec quelle base).

Le chapitre 5 clôt ce volume en remontant à la source : **d'où viennent les données** ? Il décrit les types de données et leurs niveaux de mesure, les grandes familles de sources, et la conception d'une enquête, de l'enquête de satisfaction de la boutique à ses biais.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 à 4.10 (premier contact avec pandas, filtres, traduction en tidyverse, export de caisse, jointure et marge, restructuration, tableau du lundi, notebook reproductible, ggplot2, polars et NumPy) et exercices 4.1 à 4.13.

Chapitre 5 : Types de données, collecte et conception d'enquêtes

« Avant de demander ce que disent les données, demandez ce qu'elles mesurent, d'où elles viennent, et qui a décidé qu'elles existeraient. »

La gérante de la boutique vous tend une feuille. En 2025, l'équipe a invité par courriel les clients qui avaient commandé dans l'année à répondre à une enquête de satisfaction. Environ un client sur quatre a répondu, et la note moyenne est de **3,64 sur 5**. « C'est un beau chiffre, dit-elle. Est-ce que je peux le croire ? Est-ce que je peux le montrer à mon banquier ? »

Cette question n'est pas une question de statistique au sens des chapitres précédents : il ne s'agit ni de calculer une moyenne ni de tracer une courbe, mais de savoir **ce que ce chiffre représente**. Que veut dire « satisfait » quand on répond en cochant une case de 1 à 5 ? Peut-on calculer une moyenne de cases cochées ? Les clients qui ont répondu ressemblent-ils à ceux qui n'ont pas répondu ? Qui a rempli deux fois le formulaire, et qui a coché cinq fois « 5 » en huit secondes pour en finir ? Comment aurait-on dû poser les questions, et à qui ?

Tout le reste du volume suppose que l'on sache répondre à ces questions. Les chapitres de statistique, d'Excel, de SQL et de programmation vous donnent des outils pour **calculer** ; ce chapitre vous apprend à **regarder ce que l'on calcule**. Il est volontairement le dernier du volume : vous avez désormais assez de pratique pour que des exemples chiffrés, tirés des tables de la boutique, éclairent chaque idée.

Ce que le chapitre suppose. Les notions de moyenne, de médiane, d'écart-type et d'échantillon (chapitre 1) et l'usage de pandas (chapitre 4). Aucune autre notion n'est nécessaire ; les sections complémentaires (⊕) utilisent un peu plus de code.

Données simulées. Tous les fichiers sont **simulés** (graines fixes, générateur `build/donnees_a1.py`) : la boutique est fictive, ses clients aussi. Comme nous avons écrit le simulateur, nous connaissons la **vérité programmée** et la révélons quand elle éclaire une analyse : c'est un luxe que la vie réelle n'offre jamais.

- `clients.csv` : 6 000 clients, avec leur ville, leur année de naissance, leur canal d'acquisition, leur carte de fidélité (sections 5.1, 5.2, 5.4).
- `commandes.csv` et `lignes_commande.csv` : 36 395 commandes et 83 905 lignes de commande de 2023 à 2025 (sections 5.1 à 5.4).
- `produits.csv` : le catalogue de 120 produits (section 5.5).
- `retours.csv` : les lignes de commande retournées (section 5.4).
- `enquete_satisfaction.csv` : les **958 réponses** reçues à l'enquête de 2025 (sections 5.1 et 5.3). L'enquête a été envoyée à **3 875 clients invités**, soit tous ceux qui avaient commandé en 2025.
- Pour la section 5.5, un **mini-serveur local** (écrit dans `build/outils_ch05.py`) joue le rôle d'un site et d'une API : aucun accès réseau externe n'est nécessaire, et rien de ce qui est collecté ne sort de votre machine.

5.1 Types de données et niveaux de mesure

Une **colonne** d'un tableau n'est pas seulement une suite de nombres ou de mots : c'est la trace d'une **mesure**, faite d'une certaine manière, sur une certaine unité. Cette section pose le vocabulaire qui sépare ce que l'on peut calculer de ce que l'on ne devrait pas calculer, puis ce qui fait qu'un tableau est « propre » : une ligne, une observation, et un **grain** que l'on connaît.

5.1.1 Ce que contient une colonne

On distingue d'abord les grandes familles de variables, dont le nom doit vous devenir familier.

- Une variable **qualitative** (ou *catégorielle*) prend des modalités. Elle est **nominale** quand les modalités n'ont pas d'ordre (la ville, le canal de vente, la catégorie d'un produit) et **ordinaire** quand elles sont ordonnées sans que l'écart entre deux modalités soit défini (une satisfaction de « très insatisfait » à « très satisfait », une tranche d'âge).
- Une variable **quantitative** est un nombre qui mesure une quantité. Elle est **discrète** quand elle compte (le nombre de lignes d'une commande, la quantité achetée) et **continue** quand elle peut prendre des valeurs intermédiaires (un montant, une durée, une température).
- Une variable **binaire** n'a que deux modalités (oui/non, 0/1) : la carte de fidélité, un courriel valide. C'est un cas particulier de qualitative, qui se traite comme une qualitative **et** comme un nombre (sa moyenne est une proportion).
- Une **date** ou une **heure** est un point du temps : ce n'est pas un nombre ordinaire (on soustrait deux dates pour obtenir une durée, on n'additionne pas deux dates).
- Un **texte libre** (un commentaire) n'a pas de modalités fixes ; il demande un traitement particulier.
- Un **identifiant** (numéro de client, de commande) est un nom, écrit avec des chiffres. Il ne mesure rien.

Voici les tables de la boutique, lues par pandas, avec le type que celui-ci a reconnu.

```
print(cli.dtypes)
```

```
id_client                int64
date_inscription         datetime64[us]
annee_naissance          int64
ville                    str
canal_acquisition        str
fidelite                 int64
email_valide             int64
consentement_marketing   int64
dtype: object
```

Le tableau suivant range quelques colonnes de la boutique dans ces familles. La dernière colonne est le niveau de mesure, que nous définissons en 5.1.2.

| Colonne | Exemple | Famille | Niveau de mesure |
|--------------------------|-------------|------------------------------|------------------|
| clients.id_client | 2482 | identifiant | nominal (un nom) |
| clients.ville | « Ville K » | qualitative nominale | nominal |
| clients.annee_naissance | 1992 | quantitative discrète (date) | intervalle |
| clients.fidelite | 1 | binaire | nominal |
| clients.date_inscription | 2018-01-02 | date | intervalle |
| commandes.canal | « Site » | qualitative nominale | nominal |
| commandes.heure | « 15:35 » | heure | intervalle |

| Colonne | Exemple | Famille | Niveau de mesure |
|------------------------------|-------------------|------------------------------------|------------------|
| lignes_commande.quantite | | quantitative discrète | rapport |
| lignes_commande.montant | 37,90 | quantitative continue | rapport |
| enquete.satisfaction_globale | | qualitative ordinale | ordinal |
| enquete.recommandation_0_10 | | qualitative ordinale (ou discrète) | ordinal |
| enquete.duree_reponse_s | 70 | quantitative continue | rapport |
| enquete.commentaire | « Colis soigné. » | texte libre | — |

Intuition. Pour classer une variable, posez-vous trois questions : *peut-on les ordonner ? l'écart entre deux valeurs a-t-il un sens ? le zéro veut-il dire « rien » ?* Les réponses (non/non/non, oui/non/non, oui/oui/non, oui/oui/oui) donnent les quatre niveaux de mesure que nous voyons maintenant.

5.1.2 Quatre niveaux de mesure, et ce que l'on a le droit de calculer

Le psychologue Stanley Stevens a proposé, au milieu du XX^e siècle, de ranger les variables en **quatre niveaux de mesure**. Chaque niveau autorise des opérations que le précédent n'autorise pas.

| Niveau | Ce qu'il permet | Exemples | Résumés légitimes |
|-------------------|---------------------------------------|--------------------------------|--------------------------------------|
| Nominal | égal ou différent | ville, canal, identifiant | effectifs, fréquences, mode |
| Ordinal | plus grand ou plus petit | satisfaction, tranche d'âge | + médiane, quantiles |
| Intervalle | différences (mais zéro arbitraire) | température en °C, année, date | + moyenne, écart-type |
| Rapport | différences et rapports (zéro absolu) | montant, durée, quantité | + rapports, coefficient de variation |

Les deux derniers niveaux se distinguent par le zéro. Une température de 20 °C n'est pas « deux fois plus chaude » qu'une température de 10 °C : en kelvins, ces températures valent 293,15 K et 283,15 K, et leur rapport est de 1,035. Le zéro de l'échelle Celsius est une convention. De même, 2024 n'est pas « deux fois » 1012. Au contraire, 60 € est bien deux fois 30 €, parce que le zéro euro signifie qu'il n'y a pas de montant.

Une erreur fréquente est de calculer une moyenne là où elle n'a pas de sens. La moyenne des numéros de client de la boutique vaut 3000,5 : le calcul est correct, le résultat ne veut rien dire, parce qu'un identifiant est un nom. Dans le même esprit, la moyenne des codes postaux d'un fichier de clients n'est pas un code postal.

La moyenne d'une échelle de satisfaction

Le cas qui divise les praticiens est celui de l'**échelle d'opinion** (dite de **Likert**) : un client choisit un chiffre de 1 à 5. La variable est ordinale à coup sûr ; est-elle aussi à intervalles égaux ? Rien ne garantit que l'écart entre « 2 » et « 3 » soit ressenti comme l'écart entre « 4 » et « 5 ». Calculer une moyenne suppose que oui.

Un exemple à la main montre le danger. Deux groupes de dix clients répondent sur l'échelle de 1 à 5. Dans le groupe A, tous répondent 3. Dans le groupe B, quatre répondent 1 et six répondent 4.

- La **moyenne** du groupe A vaut 3, celle du groupe B vaut $(4 \times 1 + 6 \times 4) / 10 = 2,8$: A « gagne ».
- La **médiane** du groupe A vaut 3, celle du groupe B vaut 4 : B « gagne ».

Les deux résumés se contredisent, et aucun n'est faux : la moyenne dépend de l'écart entre les modalités, la médiane seulement de leur ordre. Si l'on recode la modalité 4 en 10 (ce qui préserve l'ordre), la moyenne de B passe à 6,4 et B l'emporte aussi par la moyenne. **Une conclusion qui change selon un recodage qui respecte l'ordre n'est pas une conclusion sur les clients, c'est une conclusion sur le recodage.**

Que faire alors ? Les analystes affichent d'abord la **distribution** (la part de chaque réponse), puis un résumé que l'on peut défendre : la médiane, ou la **part des réponses favorables** (« 4 ou 5 », en anglais *top-2 box*). La moyenne reste utile pour **suivre une évolution** au fil du temps avec la même échelle, à condition de dire ce qu'elle est.

```
s = enq["satisfaction_globale"]
print(s.value_counts(normalize=True).sort_index().round(3).to_string())
print("moyenne", round(s.mean(), 2), "| médiane", s.median(), "| part de 4 ou 5 :", round((s >=
↪ 4).mean(), 3))
```

```
satisfaction_globale
1    0.028
2    0.142
3    0.278
4    0.267
5    0.285
moyenne 3.64 | médiane 4.0 | part de 4 ou 5 : 0.552
```

Sur les 958 réponses brutes, la distribution penche vers les notes favorables : 2,8 % de réponses « 1 », 14,2 % de « 2 », 27,8 % de « 3 », 26,7 % de « 4 » et 28,5 % de « 5 ». La moyenne (3,64), la médiane (4) et la part de réponses favorables (55,2 %) racontent la même histoire sous trois angles. La figure suivante montre la distribution.

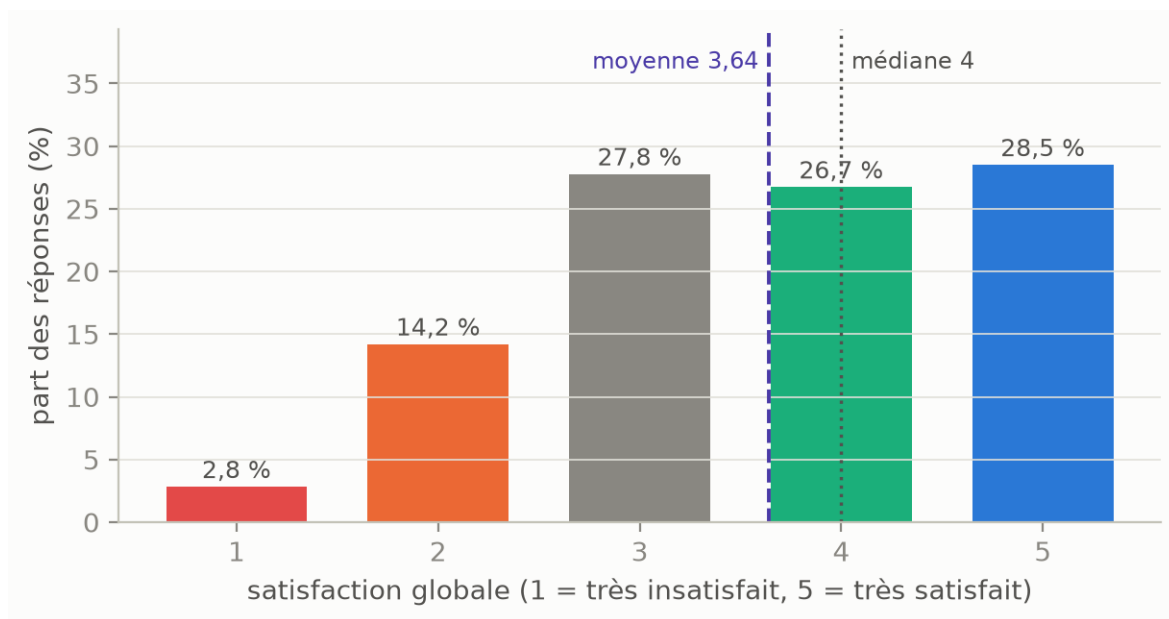


Figure 37: Distribution de la satisfaction globale (958 réponses brutes). La moyenne et la médiane ne disent pas la même chose d'une distribution étalée.

Piège. Deux distributions très différentes peuvent avoir la même moyenne : tout le monde à 3, ou la moitié à 1 et la moitié à 5. Ne résumez jamais une échelle d'opinion par sa seule moyenne.

Pour s'entraîner. Cahier, chapitre 5 : exercices 5.1 et 5.2.

5.1.3 Types techniques et types statistiques

Un logiciel a ses propres types : entier, décimal, chaîne de caractères, booléen, date. Ces **types techniques** décrivent la manière dont la valeur est stockée, pas ce qu'elle mesure. Un entier peut être un identifiant (nominal), un nombre d'articles (rapport) ou une année (intervalle). Le premier travail d'un analyste est de **vérifier que le type technique correspond au type statistique**, et pas seulement de faire confiance à la lecture automatique.

Les pièges les plus fréquents à la lecture d'un fichier sont connus. Voici un petit fichier, comme on en reçoit : un code postal avec un zéro initial, un montant écrit à la française avec un espace pour les milliers et une virgule décimale, une date jour/mois/année et un booléen écrit « oui » ou « non ».

```
import io
brut = "code;montant;date;actif\n01200;1 234,50;03/11/2025;oui\n07000;890,00;04/11/2025;non\n"
d1 = pd.read_csv(io.StringIO(brut), sep=";")
print(d1.dtypes.to_string()); print([str(x) for x in d1.iloc[0]])
```

```
code      int64
montant    str
date       str
actif      str
['1200', '1 234,50', '03/11/2025', 'oui']
```

Lue sans précaution, la table perd presque tout : le code postal est devenu l'entier 1200 (le zéro initial a disparu, le code ne désigne plus la même commune), le montant est resté du texte (à cause de l'espace et de la virgule), la date aussi, et la colonne actif est du texte au lieu d'un booléen. Il faut dire à `read_csv` ce que l'on sait.

```
d2 = pd.read_csv(io.StringIO(brut), sep=";", dtype={"code": "string", "decimal": ",", "thousands": " ",
           parse_dates=["date"], date_format="%d/%m/%Y", true_values=["oui"],
           false_values=["non"]})
print(d2.dtypes.to_string()); print([str(x) for x in d2.iloc[0]])
```

```
code      string
montant    float64
date      datetime64[us]
actif      bool
['01200', '1234.5', '2025-11-03 00:00:00', 'True']
```

Trois règles pratiques en découlent.

1. **Les identifiants se lisent en texte**, même quand ils ne contiennent que des chiffres. On ne les additionne jamais ; on les compare, on les joint, et on ne veut pas perdre un zéro.
2. **Les dates se déclarent** avec leur format (jour/mois ou mois/jour ?) : 03/11/2025 est le 3 novembre pour un lecteur français et le 11 mars pour un lecteur américain, et le logiciel ne peut pas le deviner pour les jours inférieurs ou égaux à 12.
3. **Les nombres se lisent avec leur convention** (séparateur décimal, séparateur de milliers) et leur **unité** : un montant en euros, en centimes ou en milliers d'euros ne se lit pas de la même façon.

En pratique. Après toute lecture, imprimez les types, les valeurs minimale et maximale de chaque colonne numérique et le nombre de valeurs distinctes de chaque colonne texte. Cela prend trois lignes et évite la moitié des erreurs d'analyse.

Une valeur absente n'est pas toujours une valeur manquante

Les tables de la boutique contiennent des cases vides. Toutes ne veulent pas dire la même chose, et la nuance change le traitement.

- Dans `commandes.code_promo`, **84,2 % des cases sont vides**. Ces commandes n'ont tout simplement pas utilisé de code : la case n'est pas manquante, elle signifie « aucun code ». On la remplace par une modalité explicite (« AUCUN ») avant d'analyser, sinon on les perdra dans le premier comptage.
- Dans `enquete.satisfaction_conseil`, la case est vide pour **100,0 % des clients hors boutique et jamais pour les clients de la boutique** : la question n'est posée qu'aux clients qui ont été conseillés en magasin. La case est **non applicable**. La remplir par une moyenne serait une faute.
- Dans `enquete.id_client`, **20,3 % des réponses n'ont pas d'identifiant** : ce sont des réponses anonymes. La valeur existe mais on ne la connaît pas, et le manque est volontaire.
- Une case vide peut aussi être une vraie lacune : une information que l'on aurait dû recevoir. C'est le seul cas où l'on parle de valeur **manquante** au sens strict, et c'est celui du volume suivant.

Les statisticiens distinguent trois mécanismes de lacune véritable. Les données sont **manquantes complètement au hasard** quand la probabilité qu'une valeur manque ne dépend de rien ; **manquantes au hasard** quand elle dépend d'autres variables observées (les jeunes remplissent moins souvent le champ « revenu ») ; **manquantes non au hasard** quand elle dépend de la valeur elle-même (les personnes à hauts revenus refusent de les déclarer). Le troisième cas est le plus dangereux : aucune information du fichier ne permet de le détecter, et l'on ne peut que **raisonner sur le processus de collecte**. Le volume II traite le sujet en détail.

5.1.4 Un tableau propre, et son grain

Un tableau est dit **propre** (*tidy*, en anglais) quand chaque **variable** forme une colonne, chaque **observation** forme une ligne et chaque **type d'unité observée** forme une table. Les tables de la boutique respectent cette règle. Mais une même information peut s'écrire de deux façons.

La table de l'enquête est **large** : une ligne par réponse, trois colonnes pour les trois notes de satisfaction. Pour tracer une figure qui compare les trois notes, il est plus commode d'avoir une table **longue** : une ligne par couple (réponse, thème).

```
large = enq[["id_reponse", "satisfaction_globale", "satisfaction_livraison",
  ↳ "satisfaction_prix"]].head(2)
long = large.melt(id_vars="id_reponse", var_name="theme",
  ↳ value_name="note").sort_values(["id_reponse", "theme"])
print(long.to_string(index=False))
```

| id_reponse | theme | note |
|------------|------------------------|------|
| 1 | satisfaction_globale | 3 |
| 1 | satisfaction_livraison | 3 |
| 1 | satisfaction_prix | 2 |
| 2 | satisfaction_globale | 4 |
| 2 | satisfaction_livraison | 1 |
| 2 | satisfaction_prix | 3 |

Aucune des deux formes n'est la bonne : le format long convient aux graphiques groupés et aux

calculs par thème, le format large aux calculs entre colonnes (la différence entre la satisfaction de livraison et celle du prix). On passe de l'une à l'autre dans les deux sens ; le chapitre 4 en donne les outils.

Le grain d'une table

Le **grain** d'une table est ce que représente une ligne. C'est la propriété la plus importante d'une table, et la plus souvent oubliée. La boutique en fournit trois exemples, représentés sur la figure suivante.

- **clients** : une ligne par **client**.
- **commandes** : une ligne par **commande**.
- **lignes_commande** : une ligne par **produit dans une commande**.

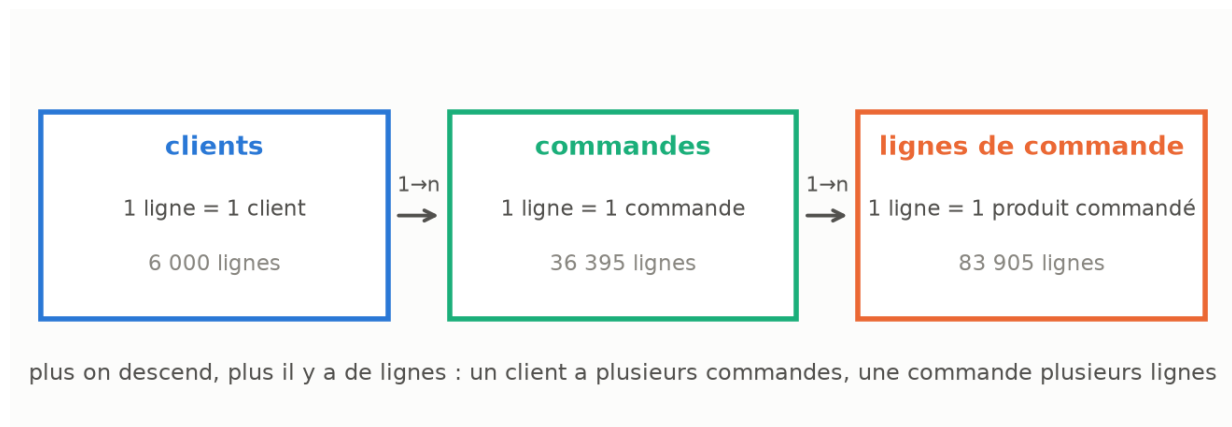


Figure 38: Les trois grains des tables de la boutique : un client a plusieurs commandes, une commande a plusieurs lignes. La flèche se lit « un ... a plusieurs ... ».

Le piège est celui du **double comptage**. Quand on joint une table à un grain fin à une table à un grain plus gros, les informations de la table grossière sont **répétées** sur chaque ligne de la table fine. Compter ou additionner ensuite sans y penser donne des résultats gonflés. Voici deux calculs qui paraissent anodins sur la table des lignes jointe aux commandes.

```
m = lig.merge(cmd[["id_commande", "canal"]], on="id_commande")
print("lignes :", len(m), "| commandes distinctes :", m["id_commande"].nunique())
print("montant moyen d'une ligne :", round(m["montant"].mean(), 2), "| panier moyen (par commande)
↳ :", round(m.groupby("id_commande")["montant"].sum().mean(), 2))
```

```
lignes : 83905 | commandes distinctes : 36395
montant moyen d'une ligne : 43.54 | panier moyen (par commande) : 100.38
```

Un count sur la table jointe renvoie 83 905, le nombre de **lignes**, alors que la question portait sur les **commandes** (36 395). Et le « montant moyen » de 43,54 € est celui d'une ligne, pas celui d'une commande : le **panier moyen** de la boutique est de 100,38 €, plus de deux fois plus. Aucun des deux calculs n'est faux ; l'un répond à une autre question que celle que l'on croyait poser.

À retenir.

- Rangez chaque colonne dans une **famille** (nominale, ordinale, quantitative, binaire, date, texte, identifiant) et un **niveau de mesure** : ils déterminent les résumés légitimes.
- On ne fait pas de moyenne d'un identifiant, d'un code postal ni (sans prudence) d'une échelle d'opinion : affichez d'abord la **distribution**.
- Le type technique lu par le logiciel n'est pas le type statistique : **vérifiez** les types, lisez les identifiants en texte, déclarez les dates et les formats de nombres.

- Une case vide peut être **non applicable**, **volontaire** ou **manquante** : la remplacer sans distinguer est une faute.
- Avant de compter ou de sommer, demandez-vous : **quel est le grain** de cette table, et mon calcul le respecte-t-il ?

Pour s'entraîner. Cahier, chapitre 5 : applications 5.2 et 5.3, exercices 5.3 et 5.4.

5.2 Sources de données

Les données ne tombent pas du ciel : quelqu'un les a **enregistrées**, pour une raison, à un moment, sur une population. Cette section apprend à poser à toute source les questions qui décident de sa valeur : qui est dedans, qui n'y est pas, jusqu'à quand, à qui appartient-elle, et qu'a-t-on le droit d'en faire.

5.2.1 Les sources d'une entreprise

On range les sources selon leur **origine** et leur **mode de production**.

- Les **sources internes** sont produites par l'activité de l'entreprise : le système de **caisse** et le site de vente (les *transactions*), le fichier clients (le *CRM*, pour *customer relationship management*), les retours, le service client, les journaux du site web. Elles ont l'avantage d'être **complètes sur ce qu'elles enregistrent** et gratuites ; elles ont l'inconvénient de n'enregistrer que ce que l'entreprise a jugé utile d'enregistrer.
- Les **sources externes** viennent d'ailleurs : données **ouvertes** publiées par des administrations ou des instituts de statistique, données de **partenaires** (un transporteur, une plateforme), données **achetées** à un fournisseur spécialisé. Elles apportent un contexte que l'entreprise ne peut pas produire (la population d'une ville, la météo), mais leur qualité, leur licence et leur mise à jour échappent à votre contrôle.
- Les **tableurs** « **sauvages** » méritent une mention à part : des fichiers Excel tenus à la main dans un service, sans règle, qui contiennent souvent **le** chiffre dont on a besoin et qui ne figurent dans aucun inventaire. Ils sont précieux, fragiles, et leur propriétaire ne dort pas la veille d'une migration informatique.

On oppose aussi les données **primaires**, collectées **pour répondre à votre question** (une enquête que vous concevez), et les données **secondaires**, collectées pour un autre usage et réutilisées (les commandes servent d'abord à livrer, ensuite à analyser). Les secondes sont peu coûteuses et souvent exhaustives, mais elles ne mesurent pas forcément ce dont vous avez besoin. Enfin, une collecte est **passive** quand elle enregistre le comportement sans intervenir (un clic, un achat) et **active** quand elle sollicite une personne (une enquête, un entretien). Le comportement ment moins que la déclaration, mais il ne dit pas pourquoi.

Voici les fichiers de ce volume rangés ainsi.

| Fichier | Origine | Mode | Question à laquelle il répond le mieux |
|-------------------------------|-------------------------|----------------------|---|
| commandes,
lignes_commande | interne, transactionnel | passif, secondaire | que vend-on, quand, à qui, par quel canal ? |
| clients | interne, CRM | passif (inscription) | qui sont nos clients ? |
| retours | interne | passif | que renvoie-t-on, pourquoi ? |

| Fichier | Origine | Mode | Question à laquelle il répond le mieux |
|-----------------------------------|-----------------------------------|--------------|--|
| jours_exploitation | interne + météo | passif | comment les ventes varient-elles avec le temps, la promotion ? |
| enquete_satisfaction | interne, primaire | actif | que pensent les clients ? |
| export_caisse_brut.csv | interne, export de caisse | passif | que vend-on en magasin, ticket par ticket ? |
| ventes_2025.xlsx | interne, tableur | passif | (même question, sous forme de classeur, pour le chapitre 2) |
| une table de villes (section 5.5) | externe , données ouvertes | passif | combien d'habitants autour de chaque ville ? |

5.2.2 La carte d'identité d'une source

Avant d'analyser une source, établissez sa **carte d'identité** : combien de lignes, quel grain, quelle clé, quelle période, quelle part de cases vides, quelles relations avec les autres tables. Cette vérification prend quelques lignes ; elle est le meilleur investissement d'un début de projet.

```
def fiche(nom, d, cle, date=None):
    return {"table": nom, "lignes": len(d), "colonnes": d.shape[1], "clé unique":
        bool(d[cle].is_unique),
            "vide (%)": round(100 * d.isna().mean().mean(), 1), "du": d[date].min().date() if date
            else "", "au": d[date].max().date() if date else ""}
fiches = [fiche("clients", cli, "id_client", "date_inscription"), fiche("commandes", cmd,
    "id_commande", "date_commande"),
    fiche("lignes", lig, "id_ligne"), fiche("retours", ret, "id_retour", "date_retour"),
    fiche("produits", prod, "id_produit"), fiche("enquête", enq, "id_reponse", "date_reponse")]
print(pd.DataFrame(fiches).to_string(index=False))
```

| table | lignes | colonnes | clé unique | vide (%) | du | au |
|-----------|--------|----------|------------|----------|------------|------------|
| clients | 6000 | 8 | True | 0.0 | 2018-01-01 | 2025-12-30 |
| commandes | 36395 | 7 | True | 12.0 | 2023-01-01 | 2025-12-31 |
| lignes | 83905 | 7 | True | 0.0 | | |
| retours | 5002 | 5 | True | 0.0 | 2023-01-06 | 2026-01-19 |
| produits | 120 | 7 | True | 0.0 | | |
| enquête | 958 | 12 | True | 11.3 | 2025-01-06 | 2026-01-14 |

On lit sur ce tableau que chaque table a une clé unique (aucun doublon d'identifiant), que commandes couvre trois années pleines, que retours se prolonge de quelques jours après la fin des commandes (un retour a lieu après l'achat) et que la table des clients commence en 2018 alors que les commandes ne commencent qu'en 2023 : la boutique a des clients **inscrits avant** la période couverte par les commandes. Un client sans commande n'est pas une erreur : c'est un client dormant. Retenez cette dernière remarque : **la population d'un fichier n'est pas toujours celle que son nom annonce**.

On vérifie ensuite les **relations** entre les tables : chaque ligne doit appartenir à une commande, chaque commande à un client connu.

```
print("lignes sans commande :", int((~lig["id_commande"].isin(cmd["id_commande"])).sum()))
print("commandes d'un client inconnu :", int((~cmd["id_client"].isin(cli["id_client"])).sum()))
print("retours sans ligne :", int((~ret["id_ligne"].isin(lig["id_ligne"])).sum()))
av = cmd.merge(cli[["id_client", "date_inscription"]], on="id_client")
print("commandes antérieures à l'inscription du client :", int((av["date_commande"] <
    av["date_inscription"]).sum()))
```



```
lignes sans commande : 0
commandes d'un client inconnu : 0
retours sans ligne : 0
commandes antérieures à l'inscription du client : 0
```

Aucune anomalie : ces tables ont été fabriquées sans erreur de relation. Dans une vraie entreprise, ce n'est presque jamais le cas, et ce contrôle des **clés étrangères** révèle souvent des commandes dont le client a été supprimé, des retours dont la ligne est introuvable, ou des dates incohérentes. Le volume II consacre un chapitre à ces contrôles de qualité.

5.2.3 Droits, licences et vie privée

Pouvoir lire une donnée ne donne pas le droit de s'en servir. Quatre questions doivent être posées avant d'utiliser une source, et leurs réponses, **écrites**, font partie de l'analyse. Ce qui suit est volontairement général : les règles précises dépendent du pays et du secteur, et sont à vérifier auprès de la personne compétente de l'entreprise.

1. **À qui appartient la donnée ?** Une donnée collectée par un partenaire n'est pas la vôtre ; une donnée achetée est soumise à un contrat.
2. **Quelle licence ?** Une donnée ouverte est publiée sous une licence qui fixe ce qu'on peut en faire (la réutiliser, la modifier, l'utiliser commercialement) et ce qu'on doit mentionner (la source, la date). Une donnée sans licence n'est pas libre de droits par défaut.
3. **S'agit-il de données personnelles ?** Un fichier de clients en contient : nom, adresse électronique, historique d'achats. Dans de nombreux pays, un texte de protection des données impose alors une **finalité** (on collecte pour un usage précis), la **minimisation** (on ne garde que le nécessaire), une **durée de conservation** limitée et le respect du **consentement**.
4. **Que fait-on de l'information sensible ?** L'état de santé, les opinions, l'origine appellent des précautions renforcées.

Le fichier de la boutique donne un exemple concret du consentement : `consentement_marketing` vaut 1 pour 60 % des clients seulement, et 92 % des clients ont une adresse valide. L'ensemble des clients **joignables** pour une campagne est donc l'intersection des deux : 3 353 clients sur 6 000, soit 56 %. Une enquête envoyée par courriel n'atteint que ceux-là : nous retrouverons ce fait en 5.3.

Pseudonymiser n'est pas anonymiser

Remplacer le nom d'un client par son numéro (`id_client`) est une **pseudonymisation** : la personne n'apparaît plus en clair, mais elle reste identifiable par recoupement. Une table **anonymisée** ne permet plus, même avec d'autres informations, de retrouver une personne. La différence compte, parce que la loi traite souvent différemment les deux.

Un exemple le montre. Sur les 6 000 clients de la boutique, retirons les noms et gardons trois informations de pure routine : la ville, l'année de naissance, le canal d'acquisition.

```
for cols in (["ville"], ["ville", "annee_naissance"], ["ville", "annee_naissance",
↳ "canal_acquisition", "fidelite"]):
    taille = cli.groupby(cols)["id_client"].transform("size")
    print(f"{len(cols)} variable(s) : {100 * (taille == 1).mean():.1f} % des clients sont uniques ;
↳ plus petit groupe : {taille.min()}")
```

```
1 variable(s) : 0.0 % des clients sont uniques ; plus petit groupe : 50
2 variable(s) : 3.4 % des clients sont uniques ; plus petit groupe : 1
4 variable(s) : 26.4 % des clients sont uniques ; plus petit groupe : 1
```

Avec la seule ville, aucun client n'est unique. Avec la ville et l'année de naissance, 3,4 % des clients sont seuls dans leur groupe ; avec quatre variables, ils sont 26,4 %, soit **un client sur quatre**

environ. Pour ceux-là, quiconque connaît la ville, l'année de naissance, le canal et la carte de la personne retrouve sa ligne, et son historique d'achats. Ces variables sont des **quasi-identifiants**. La parade classique est la **k-anonymat** : n'autoriser une publication que si chaque combinaison de quasi-identifiants compte au moins k personnes (par exemple cinq), quitte à regrouper les âges en tranches.

Piège. « Il n'y a pas de nom dans le fichier » ne veut pas dire « le fichier est anonyme ». Avant de partager un jeu de données hors de l'équipe, comptez les combinaisons uniques de ses colonnes descriptives.

5.2.4 Le biais de la source

Toute source observe une **partie** du monde, et cette partie n'est pas choisie au hasard. Le biais qui en résulte, dit **biais de sélection**, est invisible dans les données elles-mêmes : un fichier ne vous dit pas ce qu'il ne contient pas. Quatre situations reviennent sans cesse.

- **Les clients qui s'expriment.** Les avis laissés sur un site viennent de ceux qui ont une raison de le faire, très contents ou très mécontents : leur note moyenne n'est pas celle de l'ensemble des clients.
- **Les survivants.** Une analyse des clients « actuels » ne dit rien de ceux qui sont partis : c'est le piège de la **survie**, qui conduit à étudier les caractéristiques des gagnants en oubliant que les perdants les avaient aussi.
- **Les canaux observés.** Un outil d'analyse du site ne voit que les visiteurs du site ; le système de caisse que les clients du magasin.
- **Les clients joignables.** Un fichier de contacts ne contient que ceux qui ont donné leur courriel et leur accord.

Un exemple chiffré avec la boutique : supposons que l'équipe estime le **taux de retour** de l'entreprise à partir du seul système du site, parce que c'est là que les retours sont le plus facilement enregistrés et suivis.

```
l_ret = lig.merge(cmd[["id_commande", "canal"]], on="id_commande").assign(retourne=lambda d:
↳ d["id_ligne"].isin(ret["id_ligne"]))
taux = l_ret.groupby("canal")["retourne"].agg(lignes="size", retournees="sum", taux="mean")
print(taux.assign(taux=(100 * taux["taux"]).round(1)).to_string())
print("taux de retour, tous canaux :", round(100 * l_ret["retourne"].mean(), 1), "%")
```

| | lignes | retournees | taux |
|-------------------------------------|--------|------------|------|
| canal | | | |
| Boutique | 39362 | 1211 | 3.1 |
| Réseaux | 9071 | 605 | 6.7 |
| Site | 35472 | 3186 | 9.0 |
| taux de retour, tous canaux : 6.0 % | | | |

Le Site compte pour 42 % des commandes, et son taux de retour est de **9,0 %** contre 3,1 % à la Boutique et 6,0 % toutes lignes confondues. Estimer le taux de retour de l'entreprise à partir du Site seul le **surestime** de 3,0 points, soit de 51 % en valeur relative : une erreur qui coûterait cher à une décision (revoir la politique de retour, négocier avec un fournisseur).

Le même raisonnement appliqué au **panier moyen** ne produit presque aucun biais : il vaut 99,8 € sur le Site, 100,9 € à la Boutique et 100,4 € toutes commandes confondues, parce que les paniers sont très proches d'un canal à l'autre. La leçon est importante : **la sélection ne fausse un chiffre que si elle est liée à ce que l'on mesure**. On ne le sait qu'en **comparant à une source plus complète**, quand on en a une, et en **disant quelle population la source observe** quand on n'en a pas.

5.2.5 Choisir une source

La bonne source dépend de la question. Le tableau suivant résume le raisonnement ; la dernière colonne est celle qui s'oublie.

| Question | Source adaptée | Alternative | Le piège à écarter |
|---|--|---|---|
| Combien a-t-on vendu au dernier trimestre ? | transactions (caisse, site) | tableau de bord de la comptabilité | confondre commande, ligne et chiffre d'affaires net de retours |
| Qui sont nos meilleurs clients ? | transactions + CRM | enquête | un « bon client » dépend de la période et de la définition |
| Les clients sont-ils satisfaits ? | enquête (primaire) | avis en ligne, service client | non-réponse et clients qui s'expriment |
| Pourquoi un client est-il parti ? | entretiens, enquête ciblée | historique d'achats (le « quand », pas le « pourquoi ») | confondre comportement et motivation |
| Notre implantation est-elle bien placée ? | données ouvertes (population, revenus) + clients | achat de données | la zone administrative n'est pas la zone de chalandise |
| Les prix des concurrents ? | relevés manuels, collecte automatisée (5.5) | données achetées | conditions d'utilisation des sites collectés |

À retenir.

- Rangez toute source selon son **origine** (interne ou externe), son **mode** (passif ou actif, primaire ou secondaire) et sa **population** : qui est observé, et qui ne l'est pas ?
- Établissez une **carte d'identité** (lignes, grain, clé, période, vides) et contrôlez les **relations** entre tables avant toute analyse.
- Documentez les **droits** : propriété, licence, données personnelles, consentement, finalité.
- **Pseudonymiser n'est pas anonymiser** : comptez les combinaisons uniques des quasi-identifiants.
- Une source n'est jamais neutre : la **sélection** de ce qu'elle observe peut fausser un chiffre, et seule une comparaison à une autre source ou un raisonnement sur la collecte permet de le voir.

Pour s'entraîner. Cahier, chapitre 5 : application 5.1 (fiche d'une table), exercices 5.5 et 5.6.

5.3 Bases de la conception d'enquêtes

Quand les données de gestion ne répondent pas à la question (que pensent les clients ? pourquoi achètent-ils ?), il faut **demander**. Une enquête est une mesure active, donc délicate : la réponse dépend de la question posée, de la personne interrogée et de la décision, volontaire, qu'elle a prise de répondre. Cette section suit la démarche d'une enquête, puis ouvre celle de la boutique pour répondre à la gérante.

5.3.1 De l'objectif au questionnaire

Une enquête sérieuse se déroule dans cet ordre, et l'ordre compte : chaque étape fixe ce que l'on pourra dire à la suivante.

1. **L'objectif** : quelle décision l'enquête doit-elle éclairer ? Une enquête « pour mieux connaître nos clients » ne produit rien d'utilisable ; une enquête « pour décider s'il faut garder ou arrêter le service de retrait en magasin » oui.
2. **Les questions de recherche** : ce que l'on veut savoir, en phrases (« les clients livrés à domicile sont-ils moins satisfaits que ceux qui retirent en magasin ? »).
3. **La population cible** : de qui parle-t-on ? Ici, les clients qui ont commandé en 2025.
4. **La base de sondage** : la liste dont on dispose pour joindre la population cible. L'écart entre la population cible et la base de sondage est une première source de biais : on l'appelle **l'erreur de couverture** (un client sans adresse valide n'est jamais invité).
5. **L'échantillon** : tous les clients de la base, ou une partie tirée selon un plan (section 5.4).
6. **Le questionnaire** : les questions et leur ordre (5.3.2).
7. **Le test pilote** : on fait remplir le questionnaire par dix ou vingt personnes, en les écoutant. On repère les mots ambigus, les questions qui n'ont pas de réponse possible, la durée réelle. Un pilote évite presque toujours une catastrophe.
8. **La collecte** et le suivi des réponses (relances, taux de réponse).
9. **L'analyse**, qui commence par le **nettoyage** (5.3.5) et la **mesure de la non-réponse** (5.3.3).

Pour la boutique, l'objectif était de mesurer la satisfaction des clients de 2025 et de savoir si elle diffère selon le canal. La population cible compte **3 875 clients**, et l'enquête leur a été proposée à tous. Chaque invité est donc connu par ses commandes et son fichier client : c'est ce qui va nous permettre, en 5.3.3, de **comparer ceux qui ont répondu à ceux qui n'ont pas répondu**, un luxe que l'on n'a pas quand la base de sondage est anonyme.

5.3.2 Les types de questions

Un questionnaire combine quelques types de questions, qui ne s'analysent pas de la même manière.

- Les questions **fermées** proposent des réponses prédéfinies : choix unique (« par quel canal avez-vous commandé ? »), choix multiple, ou **échelle** ordonnée. Elles se codent et s'analysent facilement, mais enferment le répondant dans les réponses que vous avez prévues.
- Les questions à **échelle de Likert** demandent un degré d'accord ou de satisfaction (de 1 à 5 par exemple). L'enquête de la boutique en contient quatre : `satisfaction_globale`, `satisfaction_livraison`, `satisfaction_prix` et `satisfaction_conseil` (cette dernière réservée aux clients de la boutique, d'où ses cases vides non applicables, voir 5.1.3).
- La question de **recommandation** (`recommandation_0_10`) : « Sur une échelle de 0 à 10, quelle est la probabilité que vous nous recommandiez à un proche ? ». On en tire le **Net Promoter Score** (NPS) : les réponses de 9 et 10 sont les **promoteurs**, celles de 0 à 6 les **détracteurs**, celles de 7 et 8 les **passifs**, et le NPS est la **part de promoteurs moins la part de détracteurs**, en points (de -100 à +100).
- Les questions **ouvertes** laissent le répondant écrire (commentaire). Elles donnent des raisons que vous n'aviez pas prévues, mais elles se dépouillent à la main ou par traitement du texte, et peu de personnes les remplissent : dans notre enquête, **44 %** des réponses ont un commentaire.

```
reco = enq_u["recommandation_0_10"]
cat = pd.cut(reco, [-1, 6, 8, 10], labels=["détracteur (0-6)", "passif (7-8)", "promoteur (9-10)"])
print(cat.value_counts(normalize=True).round(3).to_string())
print("NPS :", round(100 * ((reco >= 9).mean() - (reco <= 6).mean()), 1), "points")
```

```
recommandation_0_10
détracteur (0-6)      0.469
passif (7-8)         0.275
promoteur (9-10)     0.256
NPS : -21.4 points
```

Sur les 931 réponses distinctes, il y a 46,9 % de détracteurs, 27,5 % de passifs et 25,6 % de promoteurs : le NPS vaut **−21,4** points. Un NPS négatif ne signifie pas que les clients sont mécontents (la satisfaction moyenne est de 3,64 sur 5), mais que les détracteurs, qui répondent de 0 à 6 sur une échelle où 6 reste un score moyen, sont plus nombreux que les promoteurs, qui exigent 9 ou 10. Le NPS est un indicateur **sévère** et conventionnel : on le suit dans le temps, on le compare à ses concurrents s'ils publient le leur, mais on ne l'interprète pas comme une opinion.

Piège. Le NPS est une différence de deux proportions, donc il est **incertain** comme n'importe quelle estimation. Annoncer « le NPS est passé de −21 à −19 » sans intervalle de confiance, c'est annoncer du bruit : nous calculerons l'intervalle en 5.3.6.

5.3.3 Le taux de réponse et la non-réponse

Le **taux de réponse** est le nombre de réponses divisé par le nombre d'invitations : ici 931 réponses distinctes pour 3 875 invités, soit **24,0 %**. Un taux de réponse n'est ni bon ni mauvais en soi ; ce qui compte est de savoir si les **76 % qui n'ont pas répondu ressemblent à ceux qui ont répondu**. Si non, la moyenne observée chez les répondants n'est pas celle de la population : c'est le **bias de non-réponse**.

On ne connaît pas l'opinion des non-répondants, par définition. En revanche, **on connaît leurs caractéristiques** (canal, âge, carte de fidélité, ancienneté de leur dernière commande), parce que ce sont des clients de la boutique. On peut donc comparer les deux groupes sur ce que l'on sait d'eux. Seules les réponses identifiées (celles dont `id_client` est connu) se rattachent aux invités ; les 20 % de réponses anonymes ne se comparent que sur le canal et la tranche d'âge, qu'elles déclarent.

```
rep = enq_u.dropna(subset=["id_client"]).astype({"id_client": int}).merge(inv[["id_client",
↳ "fidelite", "jours"]], on="id_client")
comp = pd.DataFrame({"invités": [inv["fidelite"].mean(), (inv["jours"] <= 60).mean(), (inv["canal"]
↳ == "Site").mean()],
                    "répondants": [rep["fidelite"].mean(), (rep["jours"] <= 60).mean(),
↳ (enq_u["canal"] == "Site").mean()],
                    index=["avec carte de fidélité", "dernière commande il y a ≤ 60 jours", "dernière
↳ commande sur le Site"]})
print((100 * comp).round(1).to_string())
```

| | invités | répondants |
|-------------------------------------|---------|------------|
| avec carte de fidélité | 35.6 | 39.8 |
| dernière commande il y a ≤ 60 jours | 54.0 | 59.0 |
| dernière commande sur le Site | 47.3 | 46.2 |

Les répondants identifiés (741 sur 931) comptent plus de clients avec carte (39,8 % contre 35,6 % chez les invités) et plus de clients récents (59,0 % contre 54,0 %). Ils se répartissent à peu près comme les invités entre canaux (le Site compte pour 46,2 % des répondants et 47,2 % des invités). La figure suivante donne le même constat pour les trois grandeurs ; ces écarts, d'ordre de quelques points, sont ceux que l'on attend d'un échantillon de cette taille **et** d'un effet réel, que nous ne pouvons pas départager sans test : le chapitre 1 donne les outils de comparaison, nous nous en tenons ici à l'ordre de grandeur.

Une **composition** différente n'entraîne pas forcément un **bias** sur le chiffre mesuré. Pour qu'il y

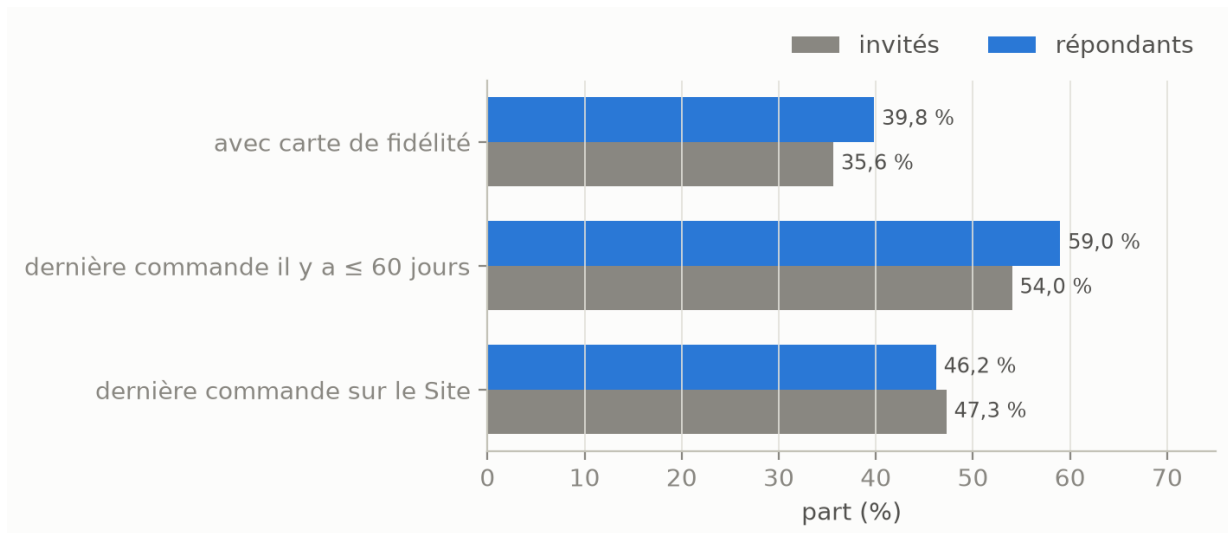


Figure 39: Ce que l'on sait des clients invités à l'enquête et de ceux qui ont répondu : les répondants sont un peu plus souvent des clients récents et des clients avec carte.

ait biais, il faut que ce qui rend les répondants différents soit **lié à ce que l'on mesure** : si les clients avec carte sont plus souvent répondants mais n'ont pas la même satisfaction que les autres, la moyenne des répondants est faussée ; sinon elle ne l'est pas. Voilà pourquoi les praticiens parlent de **mécanisme de réponse**.

Intuition. Imaginez que seuls les clients qui ont reçu leur colis un jour de pluie répondent. La composition des répondants est très particulière, mais si la pluie n'a aucun rapport avec la satisfaction, la moyenne reste juste. Le biais naît du **lien** entre la décision de répondre et la grandeur mesurée.

5.3.4 Peut-on corriger la non-réponse ?

On peut tenter de corriger la composition par une **pondération**. L'idée est simple : une cellule (par exemple « clients du Site de 25 à 34 ans ») sur-représentée chez les répondants reçoit un poids inférieur à 1, une cellule sous-représentée un poids supérieur à 1, de sorte que la pondération rétablisse la composition de la population invitée. Ce procédé s'appelle la **post-stratification**. Il suppose de connaître la composition de la population sur les variables de pondération, ce qui est le cas ici pour le canal et la tranche d'âge.

Un exemple à la main avec deux cellules. Parmi les invités, 60 % sont du Site et 40 % de la Boutique ; parmi les répondants, 50 % du Site et 50 % de la Boutique. Le poids du Site vaut $0,60/0,50 = 1,2$ et celui de la Boutique $0,40/0,50 = 0,8$. Si les répondants du Site donnent en moyenne 3,4 et ceux de la Boutique 3,9, la moyenne brute vaut $0,5 \times 3,4 + 0,5 \times 3,9 = 3,65$ et la moyenne pondérée $0,6 \times 3,4 + 0,4 \times 3,9 = 3,60$.

```
cell_inv = inv.groupby(["canal", "tranche_age"]).size() / len(inv)
cell_rep = enq_u.groupby(["canal", "tranche_age"]).size() / len(enq_u)
poids = (cell_inv / cell_rep).rename("poids")
w_rep = enq_u.join(poids, on=["canal", "tranche_age"])["poids"]
print("moyenne brute :", round(enq_u["satisfaction_globale"].mean(), 3))
print("moyenne pondérée (canal × âge) :", round(np.average(enq_u["satisfaction_globale"],
↪ weights=w_rep), 3))
```



```
moyenne brute : 3.636
moyenne pondérée (canal × âge) : 3.633
```

La pondération déplace la moyenne de 3,636 à 3,633 : presque rien. Ce résultat n'est pas un échec, c'est une information : **le canal et l'âge ne sont pas ce qui distingue les répondants des non-répondants**, ou bien cela n'a pas de lien avec la satisfaction. Il faut garder à l'esprit la limite du procédé : on ne corrige que la différence **qu'expliquent les variables que l'on connaît**.

Et la vérité ? Comme les données sont simulées, nous avons programmé la satisfaction de **tous** les invités, répondants ou non, et nous pouvons la recalculer.

```
v = 0.verite_satisfaction(inv, repetitions=200)
print({k: round(x, 3) for k, x in v.items()})
```

```
{'sat_population': 3.607, 'sat_repondants': 3.615, 'nps_population': -19.701, 'nps_repondants':
↳ -17.859, 'taux_reponse': 0.243}
```

Si **tous** les invités avaient répondu, la satisfaction moyenne attendue serait de **3,607**. Les répondants, eux, donneraient en moyenne **3,615** : le biais de non-réponse programmé est d'environ **0,01** point, négligeable. La moyenne observée dans le fichier, 3,636, s'écarte de la vérité de 0,029, ce qui est de l'ordre de l'erreur d'échantillonnage (l'erreur-type de la moyenne est d'environ 0,037). Autrement dit : **dans cette enquête, la réponse de la gérante est « oui, vous pouvez croire ce chiffre, à quelques centièmes près »**.

Pourquoi le biais est-il si faible ? Parce que, dans le mécanisme programmé, la probabilité de répondre augmente avec la **récence** et avec la **fidélité** (qui n'ont pas de lien avec la satisfaction) et avec le fait d'être **très** satisfait **ou très** mécontent (deux effets qui se compensent presque). Le jour où seuls les mécontents ou seuls les satisfaits répondent, le biais est grand : nous le simulerons en 5.4.3.

Ce que l'on peut dire sans aucune hypothèse

Une dernière approche encadre la vérité **sans rien supposer** des non-répondants. Notons r le taux de réponse, $\bar{y}_r$ la moyenne des répondants. La moyenne de la population est $r\bar{y}_r + (1 - r)\bar{y}_n$, où $\bar{y}_n$ est la moyenne, inconnue, des non-répondants. Elle est forcément comprise entre 1 et 5 : en prenant les cas extrêmes (tous les non-répondants à 1, tous à 5), on obtient un **intervalle de bornes** qui ne dépend d'aucune hypothèse.

```
r_, m_ = len(enq_u) / len(inv), enq_u["satisfaction_globale"].mean()
print("bornes sans hypothèse :", round(r_ * m_ + (1 - r_) * 1, 2), "à", round(r_ * m_ + (1 - r_) * 5,
↳ 2))
```

```
bornes sans hypothèse : 1.63 à 4.67
```

Les bornes vont de 1,63 à 4,67 : **inutilisables**. Ce résultat est instructif : avec 24 % de réponses, **aucune** conclusion sur la satisfaction de la population n'est possible sans une **hypothèse sur les non-répondants**. L'analyse de la non-réponse consiste à rendre cette hypothèse explicite (les non-répondants ressemblent aux répondants, à canal, âge et fidélité donnés) et à la **tester sur ce que l'on sait**, comme nous venons de le faire.

L'écart entre la moyenne des répondants et celle de la population vaut, plus généralement, $(1 - r)(\bar{y}_r - \bar{y}_n)$: il croît avec la **part de non-répondants** et avec l'**écart d'opinion** entre les deux groupes. Réduire le premier terme (relances, courts questionnaires) est la seule action sous votre contrôle.

5.3.5 Nettoyer l'enquête avant de calculer

Une enquête brute contient des réponses à écarter. Les règles de nettoyage s'écrivent **avant** de regarder leur effet sur les résultats : sinon, on est tenté de retenir celles qui arrangent.

- Les **doublons** : un formulaire envoyé deux fois (double clic, rechargement) crée deux lignes identiques, au numéro de réponse près.
- Les réponses en **ligne droite** (*straight-lining*) : le répondant coche la même réponse partout, sans lire, pour terminer vite. Ici : trois notes de 5 en moins de 25 secondes (la durée médiane est de 70 secondes).
- Les réponses **anonymes** ne se retirent pas : elles sont valides, mais ne se rattachent pas à un client.

```
net = enq_u[~ligne_droite]
resume = pd.DataFrame({"réponses": [len(enq), len(enq_u), len(net)],
                        "satisfaction moyenne": [enq["satisfaction_globale"].mean(),
                                                  enq_u["satisfaction_globale"].mean(), net["satisfaction_globale"].mean()],
                        "NPS": [0.nps(d["recommandation_0_10"])[0] for d in (enq, enq_u, net)]},
                      index=["brut", "sans doublons", "sans doublons ni ligne droite"]).round(2)
print(resume.to_string())
```

| | réponses | satisfaction moyenne | NPS |
|-------------------------------|----------|----------------------|--------|
| brut | 958 | 3.64 | -21.71 |
| sans doublons | 931 | 3.64 | -21.37 |
| sans doublons ni ligne droite | 897 | 3.58 | -21.07 |

On retire 27 doublons (2,8 % des lignes) puis 34 réponses en ligne droite. L'effet sur les chiffres est **faible** : la satisfaction passe de 3,64 à 3,58 et le NPS de -21,7 à -21,1. Le nettoyage n'a pas changé la conclusion ; il a changé la **confiance** que l'on peut avoir dans le fichier (un doublon ou une ligne droite ne mesurent rien), et il aurait pesé davantage si ces réponses avaient été plus nombreuses ou plus extrêmes. Si les réponses en ligne droite avaient donné toutes 5, elles auraient poussé la moyenne vers le haut : c'est le cas ici, et c'est pourquoi la moyenne **baisse** légèrement après nettoyage.

En pratique. Notez dans le rapport ce qui a été retiré et pourquoi : « 27 doublons exacts et 34 réponses en ligne droite rapide écartés (6 % du fichier) ». Un lecteur doit pouvoir refaire votre nettoyage, et pouvoir le contester.

5.3.6 Le NPS et son intervalle de confiance

Reste à donner une **fourchette** au NPS. Notons p la proportion de promoteurs, d celle de détracteurs et n le nombre de réponses. Le NPS est $p - d$; chaque répondant vaut +1 (promoteur), -1 (détracteur) ou 0 (passif) ; la variance d'une réponse est $p + d - (p - d)^2$, d'où l'erreur-type de l'estimation

$$ET(p - d) = \sqrt{\frac{p + d - (p - d)^2}{n}}, \quad \text{IC à 95 \%} \approx (p - d) \pm 1,96 ET.$$

Vérifions à la main sur un petit échantillon. Sur $n = 50$ réponses, 10 promoteurs, 25 détracteurs et 15 passifs, $p = 0,20$, $d = 0,50$, le NPS vaut -30 points, la variance d'une réponse est $0,20 + 0,50 - 0,09 = 0,61$, l'erreur-type vaut $\sqrt{0,61/50} \approx 0,110$, soit 11 points, et l'intervalle à 95 % est de $-30 \pm 21,6$ points. Avec 50 réponses, on ne distingue pas un NPS de -30 d'un NPS de -10 : voilà pourquoi il faut quelques centaines de réponses.

```
lignes = []
for nom, d in [("ensemble", net)] + [(c, net[net["canal"] == c]) for c in ["Boutique", "Site",
↪ "Réseaux"]]:
    n_, (valeur, marge) = len(d), 0.nps(d["recommandation_0_10"])
    lignes.append((nom, n_, round(valeur, 1), round(valeur - marge, 1), round(valeur + marge, 1)))
print(pd.DataFrame(lignes, columns=["groupe", "n", "NPS", "borne basse", "borne
↪ haute"]).to_string(index=False))
```

| groupe | n | NPS | borne basse | borne haute |
|----------|-----|-------|-------------|-------------|
| ensemble | 897 | -21.1 | -26.5 | -15.7 |
| Boutique | 369 | -0.5 | -9.0 | 7.9 |
| Site | 416 | -38.7 | -46.2 | -31.2 |
| Réseaux | 112 | -23.2 | -38.1 | -8.4 |

Pour l'ensemble, le NPS est de **-21,1** avec un intervalle de -26,5 à -15,7. Les intervalles de la Boutique (-0,5) et du Site (-38,7) **ne se chevauchent pas** : l'écart de près de quarante points est réel, la Boutique est bien mieux placée. L'intervalle des Réseaux (-23,2, sur 112 réponses seulement) chevauche les deux autres : ce canal est trop incertain pour être distingué de l'un ou de l'autre. Un intervalle large n'est pas une mauvaise nouvelle : c'est le chiffre qui dit **combien on sait**.

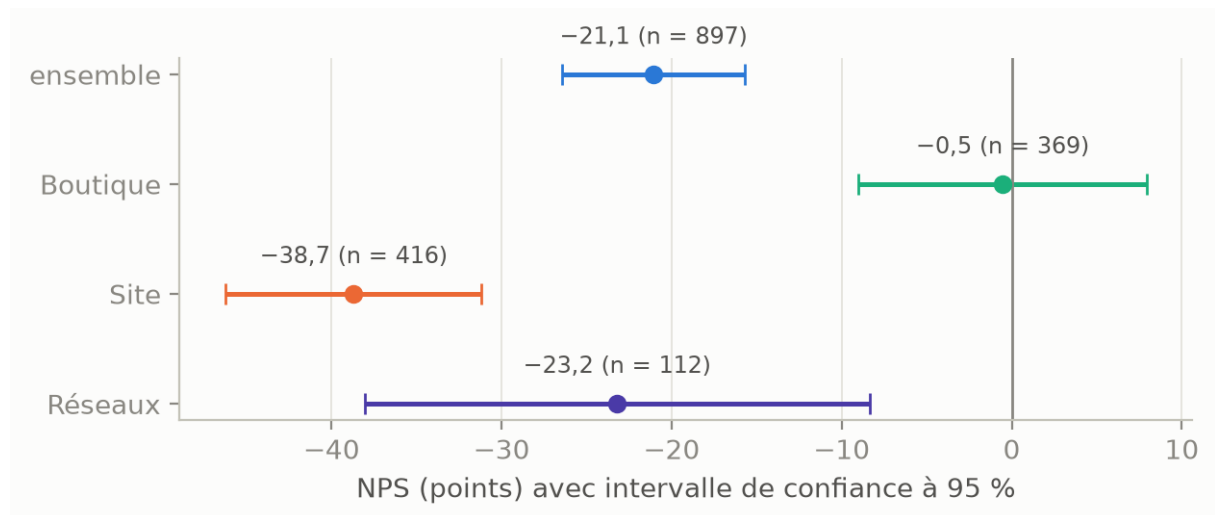


Figure 40: NPS de la boutique et par canal, après nettoyage, avec intervalle de confiance à 95 %. Le canal Réseaux compte peu de réponses : son intervalle est large.

À retenir.

- Une enquête suit une démarche : **objectif, questions de recherche, population cible, base de sondage, échantillon, questionnaire, pilote, collecte, analyse**. Un défaut à une étape se paie à toutes les suivantes.
- La **non-réponse se mesure** (on compare répondants et invités sur ce que l'on connaît), se **corrige** un peu (pondération) et se **borne** (sans hypothèse, les bornes sont trop larges pour conclure).
- Un écart de **composition** ne produit un **biais** que si ce qui différencie les répondants est lié à la **grandeur mesurée**.
- On **nettoie** selon des règles écrites **avant** de voir les résultats, et on **documente** ce qui est retiré.
- Un **NPS** (ou toute moyenne) s'accompagne de son **intervalle de confiance** ; sur quelques dizaines de réponses, il est inexploitable.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.4 et 5.5, exercices 5.7 à 5.9.

5.4 ⊕ Pour aller plus loin : questionnaires, plans de sondage et biais d'enquête

Section optionnelle. Elle approfondit la section 5.3 : comment formuler les questions, comment choisir les personnes à interroger et combien, et quels biais guettent une enquête. Elle contient des simulations ; la vérité y est connue parce que nous l'avons programmée.

5.4.1 Bien poser une question

Une question mal posée produit une réponse que l'on ne peut pas interpréter, même avec dix mille répondants. Les défauts reviennent toujours à quelques familles.

| Défaut | Mauvaise formulation | Meilleure formulation |
|------------------------------|---|---|
| Question orientée | « Comme beaucoup de nos clients, vous trouvez notre livraison rapide, n'est-ce pas ? » | « Comment jugez-vous la rapidité de la livraison ? » (de « très lente » à « très rapide ») |
| Double question | « Le personnel est-il aimable et compétent ? » | deux questions séparées : amabilité, compétence |
| Mot vague | « Achetez-vous souvent chez nous ? » | « Combien de commandes avez-vous passées au cours des 12 derniers mois ? » |
| Échelle déséquilibrée | « excellent, très bon, bon, assez bon, mauvais » (quatre modalités favorables pour une défavorable) | « très insatisfait, insatisfait, ni l'un ni l'autre, satisfait, très satisfait » |
| Réponse impossible | une question sur le conseil en magasin posée à un client du site | une question filtre, ou une modalité « sans objet » |
| Période trop longue | « Combien avez-vous dépensé chez nous depuis trois ans ? » | « Combien avez-vous dépensé le mois dernier ? » (ou, mieux, on lit l'historique des achats) |
| Sujet sensible | « Avez-vous déjà retourné un article en prétendant qu'il était défectueux ? » | formulation indirecte ou donnée de gestion à la place |

L'**ordre** des questions compte aussi : une question générale posée **après** une série de questions détaillées est influencée par elles (« Dans l'ensemble, êtes-vous satisfait ? » juste après trois questions sur la livraison tire la réponse vers la satisfaction de livraison). On pose donc d'abord la question la plus générale, puis les questions précises, et on met les questions personnelles à la fin.

Le moyen le plus sûr de savoir si une formulation change les réponses est de **la tester**, par un **split-ballot** : on tire au hasard la moitié des répondants pour recevoir la formulation A, l'autre moitié pour recevoir la B, et on compare. Simulons une expérience dans laquelle la formulation orientée augmente la note moyenne de 0,25 point (ce que nous programmons, et que l'analyste ne connaît pas).

```

rng = np.random.default_rng(3)
def note(n, effet):
    # échelle 1-5, moyenne 3,5 + effet de formulation
    return np.clip(np.round(rng.normal(3.5 + effet, 1.0, n)), 1, 5)
a_, b_ = note(400, 0.0), note(400, 0.25)
diff = b_.mean() - a_.mean(); se = np.sqrt(a_.var(ddof=1) / 400 + b_.var(ddof=1) / 400)
print(f"écart B - A : {diff:.2f} point, intervalle à 95 % : [{diff - 1.96 * se:.2f} ; {diff + 1.96 * se:.2f}]")

```

```
écart B - A : 0.27 point, intervalle à 95 % : [0.14 ; 0.41]
```

Avec 400 répondants par version, l'écart observé est de 0,27 point et l'intervalle de confiance, de 0,14 à 0,41, exclut zéro : on détecte l'effet. Mais la puissance dépend de l'effectif. En répétant l'expérience mille fois, on détecte un effet de 0,25 point dans **91 %** des cas avec 400 répondants par version, et dans **40 %** seulement avec 100. Un petit test pilote ne verra donc pas une formulation légèrement orientée : l'absence d'écart observé n'est pas la preuve de l'absence d'effet.

5.4.2 Plans de sondage : qui interroger ?

Quand on ne peut pas (ou ne veut pas) interroger tout le monde, on **tire un échantillon**. La manière de le tirer, ou **plan de sondage**, détermine à la fois la **précision** de l'estimation et son éventuel biais. On en distingue quatre.

- Le **tirage aléatoire simple** : chaque personne a la même chance d'être tirée. C'est la référence.
- Le tirage **stratifié** : on divise d'abord la population en **strates** (par exemple, selon les achats de l'année précédente) et on tire dans chaque strate, proportionnellement à sa taille. Il améliore la précision **si les strates diffèrent entre elles sur la grandeur mesurée**.
- Le tirage **en grappes** : on tire des groupes (une ville, un magasin) et on interroge tout le groupe, ce qui coûte moins cher sur le terrain, au prix d'une précision souvent moindre quand les membres d'un groupe se ressemblent.
- Les **quotas** et les échantillons de **commodité** : on interroge les personnes les plus faciles à joindre jusqu'à remplir des quotas (autant d'hommes que de femmes, par exemple). Il n'y a pas de tirage au sort : **on ne peut pas calculer de marge d'erreur honnête**.

Comparons ces plans sur la boutique. La grandeur à estimer est la **dépense moyenne de 2025** des 3 875 clients actifs, que nous connaissons ici (c'est la vérité : 341,9 €). On tire des échantillons de 300 clients selon chaque plan, 500 fois, et l'on regarde la moyenne et la dispersion des estimations. La commodité reprend les clients les plus actifs de 2024, et que l'on peut joindre (adresse valide et consentement).

```

pop = 0.depense_2025(cmd, lig, cli)
sim = 0.simuler_plans(pop, n=300, repetitions=500)
bilan = pd.DataFrame({"moyenne des estimations": sim.mean(), "écart-type": sim.std(), "biais":
    ↳ sim.mean() - pop["depense_2025"].mean()}).round(1)
print(bilan.to_string())

```

| | moyenne des estimations | écart-type | biais |
|------------------|-------------------------|------------|-------|
| aleatoire_simple | 342.2 | 19.3 | 0.3 |
| stratifie | 341.0 | 16.9 | -0.9 |
| grappes | 344.5 | 20.6 | 2.6 |
| commodite | 488.3 | 19.4 | 146.5 |

Le tirage aléatoire simple est **sans biais** (sa moyenne est la vérité à moins d'un euro) avec une dispersion de 19,3 €. Le tirage **stratifié** par la dépense 2024 est aussi sans biais et un peu plus précis (16,9 €, soit 13 % de moins) : la dépense passée prédit modérément la dépense de l'année, donc les strates ne séparent pas beaucoup les gros des petits clients. Le tirage **en grappes** (quatre villes tirées avec la même probabilité) est un peu plus dispersé (20,6 €) et **légèrement biaisé** (2,6

€ de plus que la vérité, moins de 1 %) : donner la même chance à une petite et à une grande ville avantage les clients des petites, qui dépensent un peu plus dans ce fichier. Un tirage des villes proportionnel à leur taille corrigerait ce défaut ; en pratique, les grappes réelles se ressemblent davantage, et la perte de précision est plus forte. Enfin l'échantillon de **commodité** est **massivement biaisé** : il estime 488 € au lieu de 342 €, parce qu'il ne retient que les clients actifs et joignables, et sa dispersion est faible : **il se trompe avec beaucoup d'assurance**.

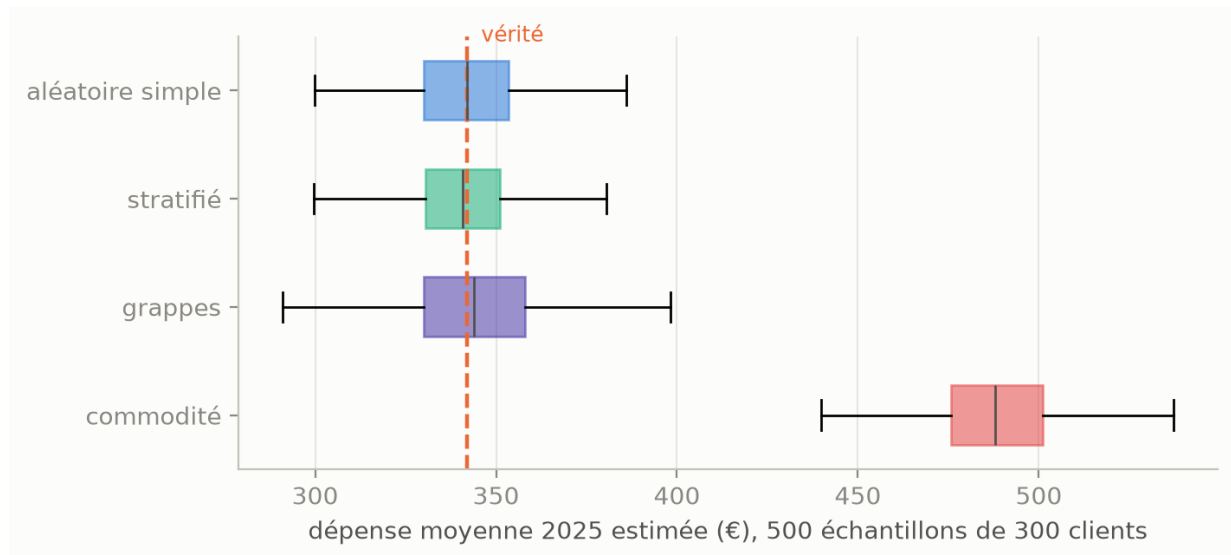


Figure 41: Quatre plans de sondage comparés par simulation : les trois plans aléatoires encadrent la vérité, l'échantillon de commodité s'en écarte et ne le sait pas.

Intuition. Un échantillon de commodité répond à la question « comment sont les gens que j'ai sous la main ? », pas à la question « comment sont mes clients ? ». Un grand échantillon biaisé donne seulement une estimation fausse **plus précise**.

5.4.3 Combien de personnes faut-il interroger ?

Pour estimer une **proportion** p avec une marge d'erreur e (la demi-largeur de l'intervalle de confiance à 95 %), il faut

$$n_0 = \frac{z^2 p(1-p)}{e^2}, \quad z = 1,96.$$

Le produit $p(1-p)$ est maximal pour $p = 0,5$, qui donne la formule prudente $n_0 \approx 0,96/e^2$. Pour une marge de ± 5 points, il faut 384 réponses ; pour ± 3 points, 1 067 ; pour ± 2 points, 2 401 ; pour ± 1 point, 9 604. L'effectif augmente comme **l'inverse du carré** de la marge : diviser la marge par deux demande quatre fois plus de réponses.

Quand la population est de taille N connue et que l'échantillon en représente une part sensible, on **corrige** : $n = n_0 / (1 + (n_0 - 1)/N)$, c'est la correction de **population finie**. Pour les 3 875 clients de la boutique, une marge de ± 3 points demande 837 réponses au lieu de 1 067, et une marge de ± 5 points, 350. Avec un taux de réponse de 24 %, il faudrait **inviter** 3 483 clients pour la première marge (presque toute la population) et 1 455 pour la seconde.

Vérifions la formule : nous tirons 2 000 échantillons de 837 clients et nous regardons dans quelle proportion des cas l'intervalle de confiance contient la vraie part de clients avec carte de fidélité (35,6 %).

```
rng = np.random.default_rng(4)
N, n, vraie = len(inv), int(0.03 * len(inv)), inv["fidelite"].mean()
couvert = 0
for _ in range(2000):
    ph = inv["fidelite"].values[rng.choice(N, n, replace=False)].mean()
    couvert += abs(ph - vraie) <= 1.96 * np.sqrt(ph * (1 - ph) / n * (1 - n / N))
print("couverture de l'intervalle :", couvert / 2000)
```

couverture de l'intervalle : 0.9525

L'intervalle contient la vérité dans **95,2 %** des cas, très près des 95 % annoncés : la formule tient, à **condition que le tirage soit aléatoire et que tous les invités répondent**. Si seuls 24 % répondent, la formule donne la précision **statistique** mais ne dit rien du **biais** de non-réponse (5.3.3).

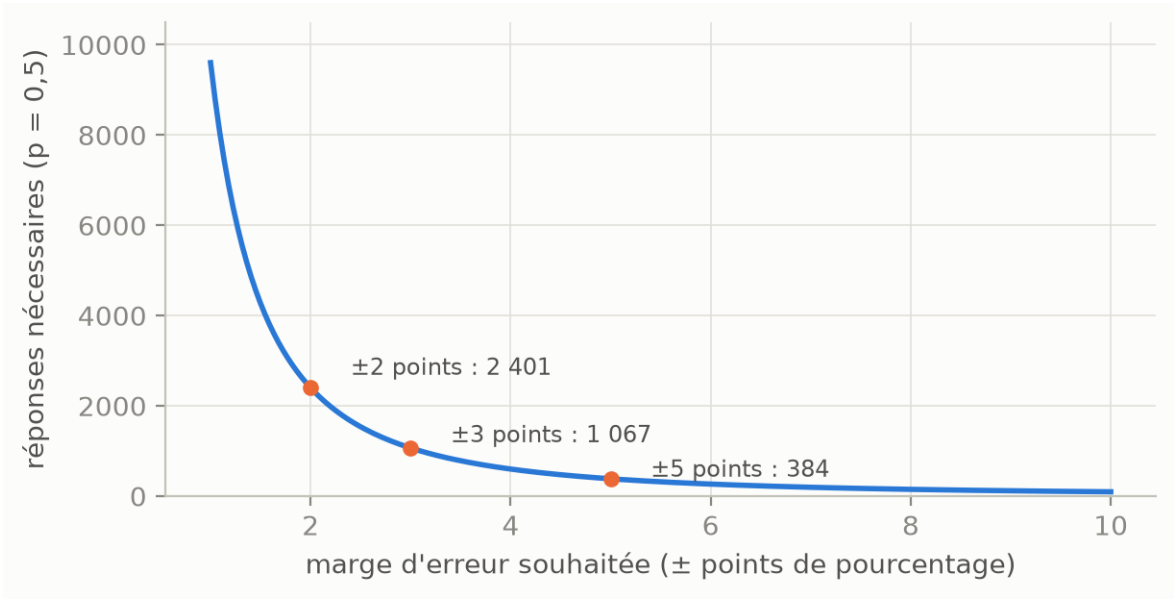


Figure 42: Nombre de réponses nécessaires pour une proportion, selon la marge d'erreur souhaitée : diviser la marge par deux coûte quatre fois plus de réponses.

5.4.4 Les biais d'enquête

Un **biais** est une erreur qui ne disparaît pas quand on augmente l'effectif : elle va toujours dans le même sens. Voici les principaux, et ce qu'il est possible d'y faire.

| Biais | Mécanisme | Symptôme dans les données | Parade |
|----------------------|---|---|--|
| Sélection | l'échantillon n'est pas tiré au hasard dans la population | l'échantillon diffère de la population connue | plan de sondage aléatoire, comparaison à un fichier de référence |
| Couverture | des gens sont absents de la base de sondage | population cible ≠ base de sondage | élargir la base, ou le dire |
| Non-réponse | répondre dépend de ce que l'on mesure | répondants différents des invités | relances, pondération, bornes |
| Désirabilité sociale | on donne la réponse qui fait bonne figure | sous-déclaration des comportements mal vus | questions indirectes, données de gestion |
| Mémoire | on se rappelle mal, surtout les périodes longues | arrondis, oublis, télescopage des dates | période courte, aides à la mémoire, données de gestion |

| Biais | Mécanisme | Symptôme dans les données | Parade |
|-----------------------------|--------------------------------|-----------------------------------|----------------------|
| Formulation et ordre | la question oriente la réponse | écart entre deux versions testées | split-ballot, pilote |

Deux simulations montrent l'ampleur possible.

Quand le mécanisme de réponse dépend de l'opinion

En 5.3.4, la réponse dépendait peu de la satisfaction et le biais était négligeable. Changeons le mécanisme : les clients **mécontents** (note 1 ou 2) répondent trois fois plus souvent que les autres (45 % contre 15 %), parce qu'ils ont quelque chose à dire. La population est simulée comme dans le chapitre ; le seul changement est la probabilité de répondre.

```
rng = np.random.default_rng(11)
nn = len(inv)
lat = rng.normal(3.6, 0.9, nn) + 0.3 * (inv["canal"] == "Boutique").values - 0.25 *
    (inv["mode_livraison"] == "Point relais").values
sat = np.clip(np.round(lat + rng.normal(0, 0.7, nn)), 1, 5)
repond = rng.random(nn) < np.where(sat <= 2, 0.45, 0.15)
r = repond.mean()
print(f"taux de réponse {r:.2f} | moyenne population {sat.mean():.2f} | répondants
    ↳ {sat[repond].mean():.2f} | non-répondants {sat[~repond].mean():.2f}")
print("biais observé :", round(sat[repond].mean() - sat.mean(), 2), "| (1 - r) × (écart répondants -
    ↳ non-répondants) :", round((1 - r) * (sat[repond].mean() - sat[~repond].mean()), 2))
```

```
taux de réponse 0.20 | moyenne population 3.62 | répondants 3.24 | non-répondants 3.72
biais observé : -0.39 | (1 - r) × (écart répondants - non-répondants) : -0.39
```

Le taux de réponse n'est que de 20 %, du même ordre que dans l'enquête réelle, mais la moyenne des répondants (3,24) est **0,39 point sous** celle de la population (3,62) : le biais est environ 52 fois celui de la section 5.3. La seconde ligne vérifie la formule $(1 - r)(\bar{y}_r - \bar{y}_n)$ donnée en 5.3.4, qui retrouve exactement le biais. Le taux de réponse, voisin dans les deux situations, ne les distingue pas : **c'est le mécanisme qui compte, pas le taux**.

La désirabilité sociale

Prenons la question « Avez-vous retourné un article en 2025 ? ». La vérité se lit dans le fichier des retours : 34,0 % des clients actifs en ont retourné au moins un. Supposons qu'une **personne sur trois** qui a retourné un article répond « non » par gêne ou par oubli (c'est un paramètre de la simulation, pas une mesure).

```
l25 = lig.merge(cmd[["id_commande", "id_client", "date_commande"]],
    ↳ on="id_commande").query("date_commande >= '2025-01-01'")
vrai_ret = l25.assign(ret=l25["id_ligne"].isin(ret["id_ligne"])).groupby("id_client")["ret"].any()
ech = vrai_ret.sample(800, random_state=5)
dit_oui = ech & (np.random.default_rng(5).random(800) > 1 / 3)
print("part réelle :", round(vrai_ret.mean(), 3), "| dans l'échantillon :", round(ech.mean(), 3), "|
    ↳ déclarée :", round(dit_oui.mean(), 3))
```

```
part réelle : 0.34 | dans l'échantillon : 0.336 | déclarée : 0.226
```

La part réelle est de 34,0 % ; l'échantillon de 800 en contient 33,6 %, et **22,6 %** le déclarent. L'enquête **sous-estime** d'un tiers un comportement qui est dans les fichiers de la boutique. La leçon n'est pas de renoncer aux enquêtes, mais de **réserver l'enquête à ce que les données de gestion ne donnent pas** (les opinions, les raisons) et de lire le comportement dans les fichiers de gestion.

À retenir.

- Posez des questions **neutres, simples, à une seule idée, avec une échelle équilibrée** ; testez deux formulations par un **split-ballot** quand l'enjeu le justifie, avec assez de monde pour que le test ait de la puissance.
- Le **tirage aléatoire** (simple, stratifié) permet de calculer une marge d'erreur ; **les quotas et la commodité ne le permettent pas**. Stratifier ne sert que si les strates séparent la grandeur mesurée.
- Marge d'erreur d'une proportion : $e \approx 1,96\sqrt{p(1-p)/n}$; **diviser la marge par deux coûte quatre fois plus de réponses** ; corrigez pour une population finie.
- Un **biais** ne se réduit pas en agrandissant l'échantillon. Le **mécanisme de réponse** compte plus que le taux de réponse.
- Quand une donnée de **gestion** existe, elle vaut mieux qu'une déclaration pour un comportement ; l'enquête sert aux **opinions**.

Pour s'entraîner. Cahier, chapitre 5 : application 5.6, exercices 5.10 et 5.11.

5.5 ⊕ Pour aller plus loin : sources ouvertes, API et web scraping

Section optionnelle. Elle montre comment **collecter** des données qui ne sont pas dans vos fichiers : un jeu de données ouvertes, une API, une page web. Tout ce qui est « collecté » ici provient d'un **mini-serveur local** (build/outils_ch05.py, adresse 127.0.0.1) qui joue le rôle d'un site : aucun accès réseau externe n'est nécessaire et rien ne sort de votre machine. Les outils sont ceux que l'on utilise sur un vrai site ; les règles de politesse et de droit s'appliquent de la même manière.

5.5.1 Les données ouvertes

Les **données ouvertes** (*open data*) sont des jeux de données publiés, en général par une administration, un institut de statistique ou une collectivité, **que n'importe qui peut réutiliser**, sous une licence qui précise comment. Elles complètent utilement les fichiers de l'entreprise : population des villes, revenus, météo, calendrier des jours fériés, cartographie. Trois précautions s'imposent avant de s'en servir.

1. **Lire la fiche** du jeu : qui l'a produit, quand il a été mis à jour pour la dernière fois, sur quelle population et quelle période, avec quelle **licence**.
2. **Vérifier la définition** des variables : un « revenu médian » peut être avant ou après impôts, par ménage ou par personne.
3. **Citer la source** et la date de téléchargement dans votre rapport, comme pour toute référence.

Le serveur de démonstration publie un jeu fictif sur les vingt villes de la région, avec sa fiche de métadonnées, en trois formats.

```
meta = requests.get(url + "/ouvert/villes.meta.json").json()
print(meta["licence"], "|", meta["source"], "| mise à jour :", meta["mise_a_jour"])
csv_ = pd.read_csv(url + "/ouvert/villes.csv")
json_ = pd.DataFrame(requests.get(url + "/ouvert/villes.json").json())
print(csv_.head(3).to_string(index=False)); print("même contenu en CSV et en JSON :",
↵ csv_.equals(json_))
```

```
Licence ouverte fictive v1 | Service statistique fictif | mise à jour : 2025-06-30
ville  population  revenu_median
Ville A      84500      19500
Ville B      82500      23000
Ville C      59600      21300
même contenu en CSV et en JSON : True
```

Les trois formats courants se distinguent ainsi.

- Le **CSV** (valeurs séparées par des virgules ou des points-virgules) est le plus simple : une ligne par observation, lisible par un tableau. Il ne porte ni les types ni la structure imbriquée.
- Le **JSON** est un format de texte pour des données **structurées** (objets, listes), très répandu dans les API ; il permet d'imbriquer des listes dans des objets.
- Le **XML** est un ancêtre plus verbeux, à balises, encore fréquent dans l'administration.

Le même contenu se lit des trois façons, et le résultat doit être identique ; vérifier cette égalité est un bon réflexe.

```
import xml.etree.ElementTree as ET
racine = ET.fromstring(requests.get(url + "/ouvert/villes.xml").text)
xml_ = pd.DataFrame([{"ville": e.get("nom"), "population": int(e.findtext("population")),
    ↪ "revenu_median": int(e.findtext("revenu_median"))} for e in racine])
print("même contenu en XML :", xml_.equals(csv_))
```

```
même contenu en XML : True
```

Enrichissons maintenant les clients de la boutique avec ces données : combien de clients la boutique compte-t-elle pour mille habitants dans chaque ville ?

```
par_ville = cli.groupby("ville").size().rename("clients").reset_index().merge(csv_, on="ville")
par_ville["clients_pour_1000_hab"] = (1000 * par_ville["clients"] / par_ville["population"]).round(1)
print(par_ville.sort_values("clients_pour_1000_hab", ascending=False).iloc[[0, 1, 2, -3, -2,
    ↪ -1]][["ville", "clients", "population", "clients_pour_1000_hab"]].to_string(index=False))
```

| ville | clients | population | clients_pour_1000_hab |
|---------|---------|------------|-----------------------|
| Ville E | 483 | 36100 | 13.4 |
| Ville D | 557 | 45400 | 12.3 |
| Ville F | 381 | 32600 | 11.7 |
| Ville R | 71 | 11300 | 6.3 |
| Ville T | 50 | 8000 | 6.2 |
| Ville S | 59 | 10600 | 5.6 |

La boutique compte entre 5,6 et 13,4 clients pour mille habitants selon les villes : la **pénétration** varie d'un facteur 2,4 entre la meilleure et la moins bonne. Voilà une information que ni les ventes ni la population ne donnaient seules, et qui dit où la boutique est installée, où elle est absente, et donc où une campagne a du potentiel. La jointure se fait sur le **nom de la ville** : dans la pratique, elle exige de vérifier que les deux fichiers écrivent les noms de la même façon (accents, majuscules, tirets), problème que le volume II traite en détail.

5.5.2 Interroger une API

Une **API** (*application programming interface*) est une porte d'entrée **prévue pour les programmes** : au lieu de décrire une page pour un humain, un site expose des données dans un format régulier. Une API **REST** repose sur quelques conventions.

- Chaque **ressource** (les produits, les commandes) a une **adresse** (URL), par exemple `/api/v1/produits`.
- On la lit avec la méthode **GET**, accompagnée de **paramètres** dans l'adresse (`?page=2&par_page=25`).
- La réponse est un **code de statut** et un **corps**, en général du JSON.

- Les codes à connaître : **200** (succès), **400** (requête mal formée), **401** (non autorisé : clé absente ou invalide), **404** (ressource introuvable), **429** (trop de requêtes), **500** (erreur du serveur).
- Une **clé d'API**, envoyée dans un en-tête, identifie le demandeur ; une **limite de débit** protège le serveur en refusant les requêtes trop fréquentes ; la **pagination** découpe les gros résultats en pages.

La figure suivante résume le dialogue que nous allons mener.

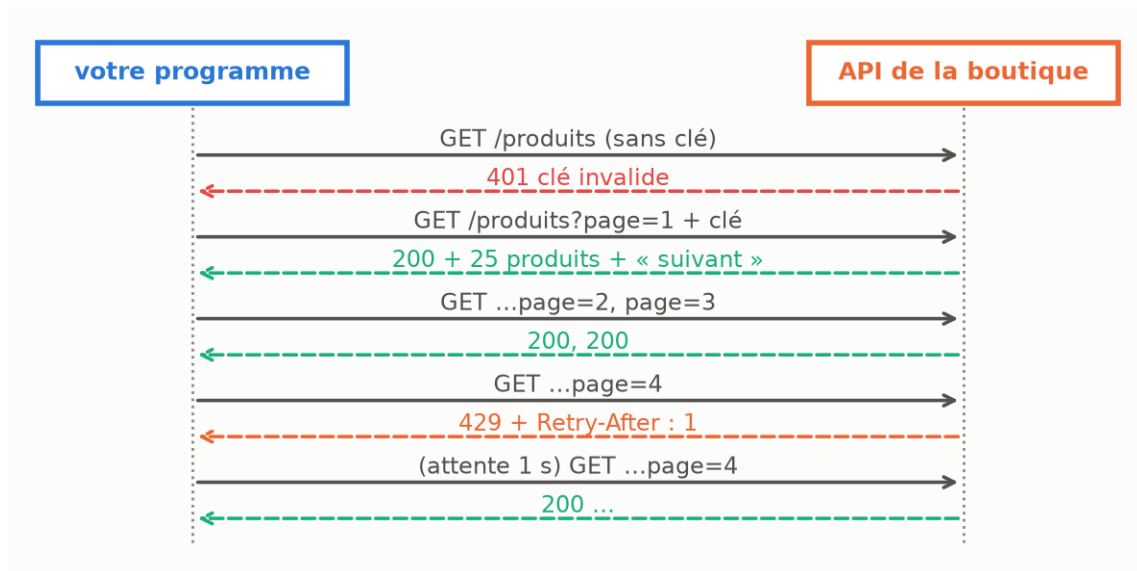


Figure 43: Le dialogue avec l'API : une requête sans clé est refusée (401), une requête valide renvoie une page de résultats, une requête trop rapide reçoit un 429 et doit attendre.

Commençons par un appel manuel, sans clé, puis avec.

```

r1 = requests.get(url + "/api/v1/produits")
print(r1.status_code, r1.json())
cle = {"X-API-Key": "cle-demo-123"}
r2 = requests.get(url + "/api/v1/produits", headers=cle, params={"page": 1, "per_page": 25})
corps = r2.json()
print(r2.status_code, {k: corps[k] for k in ("page", "par_page", "total", "pages", "suivant")});
↪ print(corps["data"][0])
  
```

```

401 {'erreur': 'clé d'API absente ou invalide'}
200 {'page': 1, 'par_page': 25, 'total': 120, 'pages': 5, 'suivant':
↪ '/api/v1/produits?page=2&per_page=25'}
{'id_produit': 1, 'nom': 'Casserole nordique', 'categorie': 'Cuisine', 'prix_vente': 42.9}
  
```

Sans clé, le serveur répond **401** et un message d'erreur en JSON ; avec la clé, **200** et un corps qui contient la première page : 25 produits sur 120 au total, répartis sur 5 pages, avec l'adresse de la page suivante. Pour tout récupérer, il faut **parcourir les pages**. Le serveur limite le débit : il accepte trois requêtes par seconde, puis répond **429** avec l'en-tête `Retry-After` (le nombre de secondes à attendre). Un client correct **obéit** ; un client qui insiste ou qui contourne la limite risque d'être banni.

```

def toutes_les_pages(chemin, entetes, par_page=25):
    produits_api, page, essais_429 = [], 1, 0
    while page:
        r = requests.get(url + chemin, headers=entetes, params={"page": page, "per_page": par_page})
        if r.status_code == 429:
            essais_429 += 1; time.sleep(int(r.headers["Retry-After"])); continue
  
```

```

        r.raise_for_status()
        corps = r.json(); produits_api += corps["data"]
        page = page + 1 if corps["suivant"] else None
    return pd.DataFrame(produits_api), essais_429
avant = len(serveur.journal)
df_api, n429 = toutes_les_pages("/api/v1/produits", cle, par_page=10)
print(len(df_api), "produits en", Counter(s for _, s in serveur.journal[avant:])[200], "requêtes
↳ réussies ; au moins un refus 429 rencontré :", n429 > 0)

```

```
120 produits en 12 requêtes réussies ; au moins un refus 429 rencontré : True
```

Avec dix produits par page, il faut 12 requêtes réussies pour récupérer les 120 produits, et le serveur nous a opposé **au moins un refus 429** en chemin (le nombre exact dépend du moment où l'on commence, puisque la limite se mesure en secondes) : à chaque refus, le programme a attendu le délai annoncé, puis il a repris à la même page. La boucle fait aussi deux choses importantes : elle **arrête** la pagination quand le champ suivant est vide, et elle **lève une erreur** (`raise_for_status`) devant tout code inattendu, au lieu de continuer en silence sur des données incomplètes.

Reste à **vérifier** ce que l'on a reçu. Une collecte non vérifiée est une source d'erreurs invisibles : on contrôle le nombre de lignes, l'unicité de la clé et, ici, l'accord avec le catalogue que l'entreprise possède déjà.

```

ctrl = df_api.merge(prod[["id_produit", "prix_vente"]], on="id_produit", suffixes=("_api",
↳ "_fichier"))
print("lignes :", len(df_api), "| identifiants uniques :", df_api["id_produit"].is_unique, "| prix
↳ identiques au fichier :", bool((ctrl["prix_vente_api"] == ctrl["prix_vente_fichier"]).all()))

```

```
lignes : 120 | identifiants uniques : True | prix identiques au fichier : True
```

En pratique. Quatre habitudes évitent la plupart des incidents : **lire la documentation** de l'API (limites, authentification, versions) ; **ne jamais écrire la clé dans le code partagé** (la lire dans une variable d'environnement) ; **enregistrer la date et la version** de ce que l'on a collecté ; **conserver les réponses brutes** avant de les transformer, pour pouvoir refaire l'analyse si l'API change.

5.5.3 Le web scraping

Quand il n'existe ni API ni fichier à télécharger, on peut parfois **lire la page web** qu'un humain verrait : c'est le **web scraping** (ou moissonnage). On télécharge le code **HTML** de la page, puis on y repère les balises qui contiennent les informations voulues. C'est puissant, mais fragile et encadré : avant d'écrire une ligne de code, il faut se demander si l'on **a le droit** et si l'on **peut le faire poliment**.

Les règles du jeu

Un site publie dans un fichier `robots.txt`, à sa racine, les règles qu'il demande aux robots de respecter : les chemins interdits, et parfois un délai minimal entre deux requêtes. Ce fichier n'est pas une loi, mais c'est la **convention** : l'ignorer est un manque de politesse, et parfois la porte ouverte à un litige. Le module `urllib.robotparser` de Python le lit.

```

from urllib.robotparser import RobotFileParser
rp = RobotFileParser(url + "/robots.txt"); rp.read()
print("catalogue autorisé :", rp.can_fetch("*", url + "/catalogue"), "| stock interne autorisé :",
↳ rp.can_fetch("*", url + "/prive/stock"), "| délai demandé :", rp.crawl_delay("*"), "s")

```

```
catalogue autorisé : True | stock interne autorisé : False | délai demandé : 1 s
```

Le catalogue est autorisé, le chemin /prive/stock est interdit, et le site demande une seconde entre deux requêtes. Nous respecterons ces règles. Les **conditions d'utilisation** d'un site (qui ne se lisent pas dans robots.txt) peuvent interdire la collecte automatique ou la réutilisation commerciale : on les lit **avant**, et l'on garde une trace de cette lecture.

Lire une page

Voici le code d'une carte de produit dans le catalogue de démonstration, tel que l'affiche requests.

```
html = requests.get(url + "/catalogue?page=1").text
print(html.split("\n")[2])
```

```
<div class="produit" data-id="1"><h2 class="nom">Casserole nordique</h2><span
  class="categorie">Cuisine</span><span class="prix">42,90 €</span></div>
```

La bibliothèque **BeautifulSoup** analyse le HTML et permet de chercher des balises par leur nom ou leur **classe**. L'extraction de la page tient en quelques lignes.

```
from bs4 import BeautifulSoup
def lire_page(html):
    soupe = BeautifulSoup(html, "html.parser")
    return [{"id_produit": int(c["data-id"]), "nom": c.select_one(".nom").text, "categorie":
        c.select_one(".categorie").text,
        "prix_vente": float(c.select_one(".prix").text.replace("€", "").replace(",", ""))
        .strip()} for c in soupe.select("div.produit")]
print(lire_page(html)[2])
```

```
[{'id_produit': 1, 'nom': 'Casserole nordique', 'categorie': 'Cuisine', 'prix_vente': 42.9},
 {'id_produit': 2, 'nom': 'Poêle mat', 'categorie': 'Cuisine', 'prix_vente': 38.9}]
```

On parcourt ensuite les pages **en respectant le délai demandé**, puis on contrôle le résultat contre l'API.

```
def moissonner(chemin, pages=12, delai=1.0):
    lignes_ = []
    for p in range(1, pages + 1):
        lignes_ += lire_page(requests.get(url + chemin, params={"page": p}).text); time.sleep(delai)
    return pd.DataFrame(lignes_)
df_web = moissonner("/catalogue")
print(len(df_web), "produits lus sur la page |", "identiques à l'API :",
      bool(df_web.sort_values("id_produit").reset_index(drop=True)[["id_produit",
      "prix_vente"]].equals(df_api.sort_values("id_produit").reset_index(drop=True)[["id_produit",
      "prix_vente"]])))
```

```
120 produits lus sur la page | identiques à l'API : True
```

Les 120 produits lus sur les pages sont identiques à ceux de l'API (mêmes identifiants, mêmes prix) : la collecte est **vérifiée**. Dans cet exemple, l'API est plus simple, plus rapide et plus stable que la lecture des pages : **quand une API existe, c'est elle qu'il faut utiliser**.

La fragilité

Un robot de lecture dépend de la **structure** de la page, qu'un webmestre peut modifier à tout moment, sans prévenir. Le serveur de démonstration publie une seconde version du catalogue, catalogue-v2, qui contient les mêmes produits avec des balises différentes.

```
v2 = requests.get(url + "/catalogue-v2", params={"page": 1}).text
print("produits trouvés par le même code sur la nouvelle version :", len(lire_page(v2)))
```

produits trouvés par le même code sur la nouvelle version : 0

Le programme ne plante pas : il trouve **0 produit** et continue. C'est le pire cas, un échec **silencieux** : une analyse construite sur ce résultat serait vide, et personne ne s'en apercevrait. D'où la règle d'or : **tout moissonnage s'accompagne de contrôles** (nombre de lignes attendu, valeurs plausibles, comparaison à la collecte précédente) qui arrêtent le programme quand quelque chose a changé.

```
def lire_page_sure(html, attendu=10):
    lignes_ = lire_page(html)
    if len(lignes_) != attendu:
        raise ValueError(f"structure de page modifiée ? {len(lignes_)} produits trouvés, {attendu}
        ↳ attendus")
    return lignes_
try:
    lire_page_sure(v2)
except ValueError as e:
    print("Alerte :", e)
```

Alerte : structure de page modifiée ? 0 produits trouvés, 10 attendus

5.5.4 Éthique et droit

La collecte automatisée soulève des questions que le code ne résout pas, et que l'on traite **avant** de coder. Les principes qui suivent sont généraux ; les règles précises dépendent du pays et du texte applicable, à faire vérifier par la personne compétente de l'entreprise.

- **Respecter les conditions d'utilisation et robots.txt.** Une interdiction explicite de collecte automatique ou de réutilisation commerciale se respecte, même si l'on **peut** techniquement la contourner.
- **Ne pas surcharger** le serveur : un délai entre les requêtes, des horaires creux, pas de collecte parallèle sauvage. Un robot trop rapide ressemble à une attaque.
- **Éviter les données personnelles.** Collecter des noms, des avis signés ou des profils tombe sous les règles de protection des données, même si ces informations sont publiques : elles gardent leur finalité d'origine.
- **S'identifier.** Un robot honnête déclare qui il est (un en-tête User-Agent explicite avec un contact) ; les conditions peuvent exiger une clé.
- **Citer la source et la date,** et ne pas redistribuer des données protégées par un droit d'auteur ou un droit sur les bases de données.
- **Préférer la voie officielle** : une API, un fichier ouvert, un partenariat. C'est plus stable, plus rapide, et c'est légitime.

À retenir.

- Une **donnée ouverte** se lit avec sa **fiche** (producteur, date, licence, définitions) et se cite ; le même contenu se lit en **CSV, JSON ou XML**.
- Une **API** s'appelle avec une **clé**, des **paramètres** et une **pagination** ; les codes **200, 401, 404, 429, 500** se gèrent explicitement ; on **obéit** à Retry-After.
- **Vérifiez** toute collecte (nombre de lignes, clés uniques, accord avec une autre source) et conservez la **réponse brute**.
- Le **moissonnage** est le dernier recours : on lit robots.txt et les conditions d'utilisation, on espace les requêtes, et l'on ajoute des **contrôles** qui arrêtent le programme quand la page change.
- **Préférez toujours la voie officielle** (API, fichier ouvert) au moissonnage de pages.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.7 et 5.8, exercice 5.12.

Bilan du chapitre 5

Vous savez maintenant :

- **classer** une variable (nominale, ordinale, quantitative discrète ou continue, binaire, date, texte, identifiant), lui donner un **niveau de mesure** (nominal, ordinal, intervalle, rapport) et en déduire les **résumés légitimes** ; savoir pourquoi la moyenne d'un identifiant n'a pas de sens et pourquoi celle d'une échelle d'opinion demande de la prudence ;
- **vérifier** qu'un type technique correspond au type statistique, lire un fichier sans perdre un zéro ni une date, et distinguer une case **non applicable**, **volontaire** ou **manquante** ;
- reconnaître le **grain** d'une table et éviter le **double comptage** (un nombre de lignes n'est pas un nombre de commandes) ;
- ranger une source selon son **origine**, son **mode de production** et sa **population**, établir sa **carte d'identité** et contrôler les relations entre tables, **documenter les droits** (propriété, licence, données personnelles, consentement) et se méfier d'une pseudonymisation qu'on prendrait pour une anonymisation ;
- suivre la **démarche d'une enquête** (objectif, population, base de sondage, échantillon, questionnaire, pilote, collecte, analyse) et distinguer les **types de questions** ;
- **mesurer** la non-réponse en comparant répondants et invités, **corriger** par pondération, **borner** sans hypothèse, **nettoyer** une enquête (doublons, ligne droite), et donner à un NPS son **intervalle de confiance** ;
- (en option) **formuler** des questions neutres et **tester** une formulation par split-ballot, **comparer** des plans de sondage, **dimensionner** un échantillon, nommer les **biais d'enquête** et mesurer leur ampleur ;
- (en option) **lire** des données ouvertes avec leur fiche, **interroger une API** paginée et limitée en débit, **moissonner** une page web **poliment** et la contrôler contre une autre source.

Le chapitre a mis des chiffres sur des idées que l'on répète volontiers sans les mesurer :

Question	Ce que nous avons mesuré
Moyenne de satisfaction, 958 réponses brutes	3,64 sur 5 ; après nettoyage, 3,58
Taux de réponse	24,0 % (931 réponses distinctes pour 3 875 invités)
Biais de non-réponse programmé	environ 0,01 point (vérité : 3,61 ; répondants attendus : 3,61)
Bornes de la satisfaction sans hypothèse	de 1,63 à 4,67 : inutilisables
NPS après nettoyage	−21,1 points, intervalle de −26,5 à −15,7
Clients uniques sur ville + année de naissance	3,4 % : une pseudonymisation n'est pas une anonymisation
Échantillon de commodité contre vérité	488 € estimés au lieu de 342 €
Réponses pour une marge de ± 3 points (population de 3 875)	837
Part déclarée de clients ayant retourné un article, avec une sous-déclaration d'un tiers	22,6 % pour 34,0 % réels

L'idée du chapitre tient en une phrase : **un chiffre ne vaut que par ce qui l'a produit** : la mesure (type, niveau, grain), la source (population, droits), la collecte (qui a répondu, comment on a demandé). La gérante a obtenu sa réponse : oui, la note de 3,64 est digne de confiance **ici**, parce

que nous avons pu comparer les répondants aux invités et que nous connaissons la vérité programmée. Elle n'aurait pas pu le dire sans cette comparaison. Dans la vie réelle, vous n'aurez pas la vérité : il vous restera la comparaison à ce que vous savez des non-répondants, la pondération, les bornes et l'honnêteté sur les limites.

En pratique : avant de croire un chiffre, cinq questions.

1. **Que mesure-t-il**, sur quelle échelle, et ce calcul a-t-il un sens à ce niveau de mesure ?
2. **À quel grain** est la table sur laquelle je l'ai calculé, et ai-je compté deux fois ?
3. **D'où viennent les données**, qui n'y figure pas, et ai-je le droit de les utiliser ?
4. **Qui a répondu** (ou été observé), et en quoi les absents diffèrent-ils ?
5. **Quelle incertitude** : intervalle de confiance, biais possible, et hypothèses écrites ?

Ce chapitre clôt les fondations. Le volume II, *Préparation des données*, prend ces données telles qu'elles arrivent, imparfaites, et enseigne à les **nettoyer, transformer et fiabiliser** avant l'analyse.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.1 à 5.8 (fiche d'une table, types à la lecture, piège du grain, nettoyage de l'enquête et NPS, répondants contre invités, plans de sondage, API paginée, moissonnage poli) et exercices 5.1 à 5.12.

Points clés

« Un chiffre sans sa provenance est une opinion. »

Ce court chapitre fixe l'essentiel du volume, chapitre par chapitre, puis les idées qui les relient. Le **projet du volume** (une première analyse, du fichier brut au tableau de synthèse) et une **auto-évaluation** de quarante questions se trouvent dans le cahier d'exercices.

Pour s'entraîner. Cahier, projet du volume et auto-évaluation : neuf étapes, de la lecture d'un export désordonné au message pour la gérante, une variante sur l'enquête de satisfaction, puis quarante questions avec corrigés.

Les points clés, chapitre par chapitre

Chapitre	Ce que vous devez emporter
1. Les essentiels de la statistique	Un résumé se fait par le centre , la dispersion et la forme ; sur des montants asymétriques, la médiane et les quartiles disent plus que la moyenne. La normale est un outil, pas une loi de la nature. Un échantillon donne une estimation entourée d'une erreur (erreur type, intervalle de confiance) que la taille réduit, mais que le biais ne quitte pas. Une corrélation n'est pas une cause : la saison explique la plus grande part du lien entre publicité et ventes. ⊕ Pourcentages et points, taux de croissance, moyennes pondérées, marge, TVA, arrondis.
2. Excel	Un tableur est un outil de calcul fiable s'il est structuré : données propres (une ligne, une observation), calculs séparés, hypothèses à part, contrôles. Maîtriser les références (\$), les fonctions conditionnelles, RECHERCHEX/INDEX+EQUIV, le texte et les dates ; le tableau croisé dynamique résume en quelques gestes mais se rafraîchit ; Power Query enregistre un nettoyage rejouable . ⊕ Power Pivot et DAX, VBA, Google Sheets. Les formules de ce volume sont vérifiées avec LibreOffice, pas avec Excel.

Chapitre	Ce que vous devez emporter
3. SQL	Une requête se lit dans l' ordre logique : FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY. Le grain d'une table gouverne tout ; une jointure un-à-plusieurs puis une somme double le compte . Les fonctions fenêtres classent, comparent et cumulent sans perdre le détail ; les CTE nomment les étapes. Toute requête importante se recoupe avec un second outil. ☯ Index, plans d'exécution, injection SQL ; dialectes (PostgreSQL, MySQL, SQL Server, Oracle).
4. Python et R	Un analyste décrit un calcul plutôt que de le refaire à la main : lire (types, séparateurs, encodage), filtrer, regrouper, joindre, restructurer (large/long). pandas et le tidyverse disent la même chose avec une autre syntaxe ; un notebook n'est fiable que s'il se rejoue de haut en bas. ☯ ggplot2, Shiny, NumPy, polars, Jupyter.
5. Types de données, collecte, enquêtes	Une colonne a un type et un niveau de mesure qui décident de ce qu'on peut calculer ; un tableau propre a un grain . Chaque source a une carte d'identité (fraîcheur, droits, biais). Une enquête se conçoit (objectif, population, questions, pilote) ; le taux de réponse et le mécanisme de non-réponse comptent plus que le nombre de réponses ; un NPS s'accompagne d'un intervalle. ☯ Plans de sondage, taille d'échantillon, API et web scraping dans le respect des règles.
Projet (cahier)	Comprendre les tables → lire un fichier brut → réconcilier → calculer la synthèse par trois outils → indicateurs (panier moyen, retours, marge) → classeur vérifiable → graphique → message de cinq lignes avec ses limites.

Six idées qui traversent tout le volume

Les six idées.

1. **Une question avant un outil.** Excel, SQL, Python et R répondent à la même question ; on choisit selon la taille des données, la répétition du travail et le public.
2. **Le grain d'abord.** Savoir ce que représente une ligne évite le double comptage, qui est l'erreur la plus fréquente et la plus silencieuse.
3. **Deux outils valent mieux qu'un.** Un total recoupé par SQL et pandas (ou par un tableur) vaut davantage qu'un total unique, même juste : un écart est un signal.
4. **Une moyenne se justifie.** Médiane, pondération, moyenne des moyennes, panier sur le total : chaque agrégat répond à une question précise.
5. **Ce qu'on a observé n'est pas ce qui s'est passé.** Biais d'échantillonnage, non-réponse, source partielle : on demande toujours *qui* est dans les données.
6. **Refaire, documenter, douter.** Un résultat qu'on ne peut pas refaire ne vaut rien ; un résultat sans ses limites est un slogan.

Et maintenant ?

Vous savez maintenant **lire des données, les interroger avec quatre outils, résumer et mesurer l'incertitude, collecter proprement et livrer une première analyse vérifiée**. Le **volume II** se consacre à ce qui précède toute analyse sérieuse : **préparer les données** (valeurs manquantes, aberrantes, doublons, fusions, qualité, réconciliation, documentation). Pour vous entraîner d'ici là, reprenez le projet du volume avec un autre trimestre ou un autre canal.

À retenir, tout simplement. Un analyste vend de la **confiance** autant que des chiffres : savoir d'où vient le chiffre, ce qu'il mesure, comment on l'a recoupé et ce qu'il ne dit pas.